

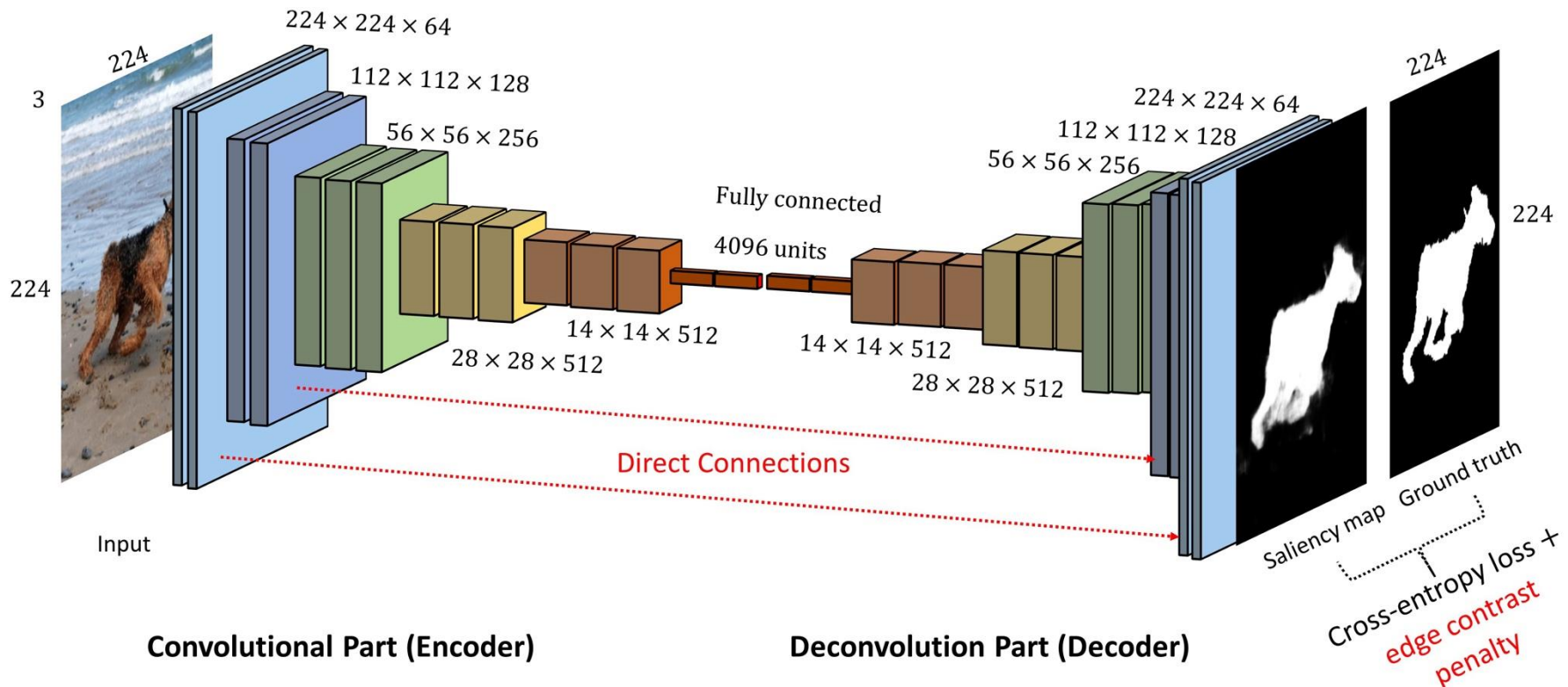
# FPGA Verification of a Verilog-Based ConvTranspose2D Accelerator Design

---

**Presenter: JeongYun Seo**

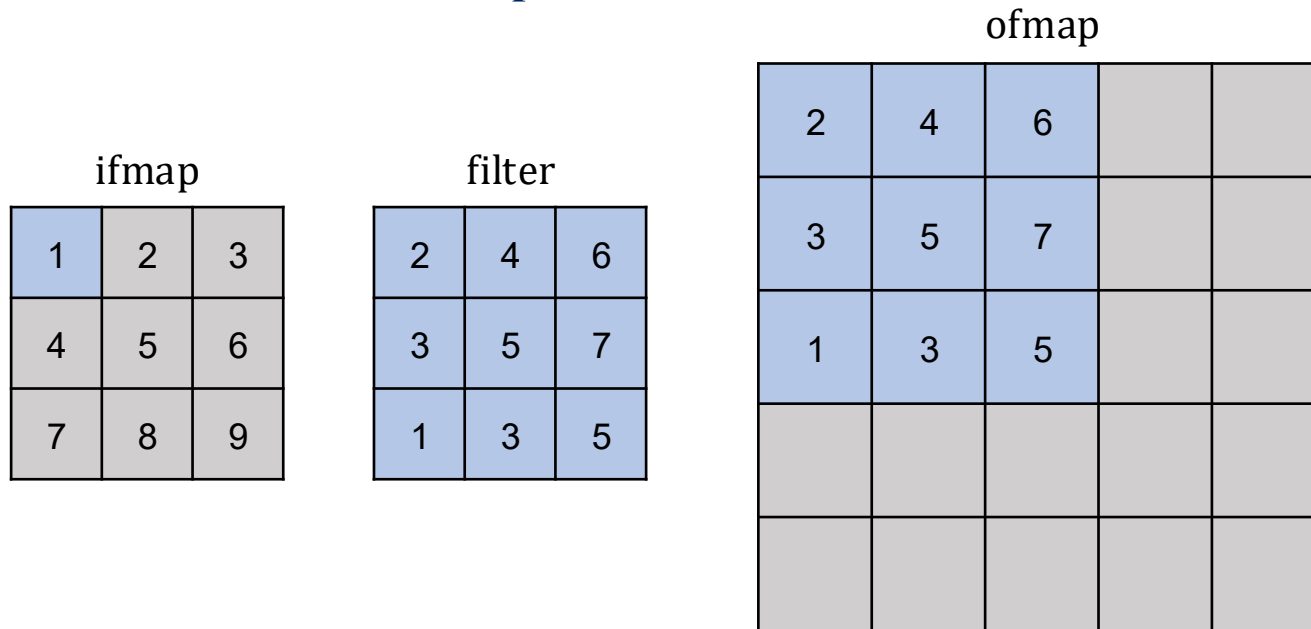
# 1. What is ConvTranspose2D ?

- The lower image presents convolutional and deconvolution parts of an autoencoder.
- While the convolutional part reduces the spatial resolution of the image, the deconvolutional part increases it.
- ConvTranspose2D is one of the methods used in the deconvolutional part.



## 2. How does ConvTranspose2D work?

### Operation Process



### Configuration

| IW | IH | FW | FH | IN_C | OUT_C | Stride | Pad | OW | OH |
|----|----|----|----|------|-------|--------|-----|----|----|
| 3  | 3  | 3  | 3  | 1    | 1     | 1      | 0   | 5  | 5  |

## 2. How does ConvTranspose2D work?

### Operation Process

| ifmap |   |   | filter |   |   | ofmap |    |    |    |  |
|-------|---|---|--------|---|---|-------|----|----|----|--|
| 1     | 2 | 3 | 2      | 4 | 6 | 2     | 8  | 14 | 12 |  |
| 4     | 5 | 6 | 3      | 5 | 7 | 3     | 11 | 17 | 14 |  |
| 7     | 8 | 9 | 1      | 3 | 5 | 1     | 5  | 11 | 10 |  |
|       |   |   |        |   |   |       |    |    |    |  |
|       |   |   |        |   |   |       |    |    |    |  |

### Configuration

| IW | IH | FW | FH | IN_C | OUT_C | Stride | Pad | OW | OH |
|----|----|----|----|------|-------|--------|-----|----|----|
| 3  | 3  | 3  | 3  | 1    | 1     | 1      | 0   | 5  | 5  |

## 2. How does ConvTranspose2D work?

### Operation Process

| ifmap |   |   | filter |   |   | ofmap |    |    |    |    |
|-------|---|---|--------|---|---|-------|----|----|----|----|
| 1     | 2 | 3 | 2      | 4 | 6 | 2     | 8  | 20 | 24 | 18 |
| 4     | 5 | 6 | 3      | 5 | 7 | 3     | 11 | 26 | 29 | 21 |
| 7     | 8 | 9 | 1      | 3 | 5 | 1     | 5  | 14 | 19 | 15 |
|       |   |   |        |   |   |       |    |    |    |    |
|       |   |   |        |   |   |       |    |    |    |    |

### Configuration

| IW | IH | FW | FH | IN_C | OUT_C | Stride | Pad | OW | OH |
|----|----|----|----|------|-------|--------|-----|----|----|
| 3  | 3  | 3  | 3  | 1    | 1     | 1      | 0   | 5  | 5  |

## 2. How does ConvTranspose2D work?

### Operation Process

| ifmap |   |   | filter |   |   | ofmap |    |    |    |    |
|-------|---|---|--------|---|---|-------|----|----|----|----|
| 1     | 2 | 3 | 2      | 4 | 6 | 2     | 8  | 20 | 24 | 18 |
| 4     | 5 | 6 | 3      | 5 | 7 | 11    | 27 | 50 | 29 | 21 |
| 7     | 8 | 9 | 1      | 3 | 5 | 13    | 25 | 42 | 19 | 15 |
|       |   |   |        |   |   | 4     | 12 | 20 |    |    |
|       |   |   |        |   |   |       |    |    |    |    |

### Configuration

| IW | IH | FW | FH | IN_C | OUT_C | Stride | Pad | OW | OH |
|----|----|----|----|------|-------|--------|-----|----|----|
| 3  | 3  | 3  | 3  | 1    | 1     | 1      | 0   | 5  | 5  |

## 2. How does ConvTranspose2D work?

### Operation Process

| ifmap |   |   | filter |   |   | ofmap |    |    |    |    |
|-------|---|---|--------|---|---|-------|----|----|----|----|
| 1     | 2 | 3 | 2      | 4 | 6 | 2     | 8  | 20 | 24 | 18 |
| 4     | 5 | 6 | 3      | 5 | 7 | 11    | 37 | 70 | 59 | 21 |
| 7     | 8 | 9 | 1      | 3 | 5 | 13    | 40 | 67 | 54 | 15 |
|       |   |   |        |   |   | 4     | 17 | 35 | 25 |    |
|       |   |   |        |   |   |       |    |    |    |    |

### Configuration

| IW | IH | FW | FH | IN_C | OUT_C | Stride | Pad | OW | OH |
|----|----|----|----|------|-------|--------|-----|----|----|
| 3  | 3  | 3  | 3  | 1    | 1     | 1      | 0   | 5  | 5  |

## 2. How does ConvTranspose2D work?

### Operation Process

| ifmap |   |   | filter |   |   | ofmap |    |    |    |    |
|-------|---|---|--------|---|---|-------|----|----|----|----|
| 1     | 2 | 3 | 2      | 4 | 6 | 2     | 8  | 20 | 24 | 18 |
| 4     | 5 | 6 | 3      | 5 | 7 | 11    | 37 | 82 | 83 | 57 |
| 7     | 8 | 9 | 1      | 3 | 5 | 13    | 40 | 85 | 84 | 57 |
|       |   |   |        |   |   | 4     | 17 | 41 | 43 | 30 |
|       |   |   |        |   |   |       |    |    |    |    |

### Configuration

| IW | IH | FW | FH | IN_C | OUT_C | Stride | Pad | OW | OH |
|----|----|----|----|------|-------|--------|-----|----|----|
| 3  | 3  | 3  | 3  | 1    | 1     | 1      | 0   | 5  | 5  |

## 2. How does ConvTranspose2D work?

### Operation Process

| ifmap |   |   | filter |   |   | ofmap |    |     |    |    |
|-------|---|---|--------|---|---|-------|----|-----|----|----|
| 1     | 2 | 3 | 2      | 4 | 6 | 2     | 8  | 20  | 24 | 18 |
| 4     | 5 | 6 | 3      | 5 | 7 | 11    | 37 | 82  | 83 | 57 |
| 7     | 8 | 9 | 1      | 3 | 5 | 27    | 68 | 127 | 84 | 57 |
|       |   |   |        |   |   | 25    | 52 | 90  | 43 | 30 |
|       |   |   |        |   |   | 7     | 21 | 35  |    |    |

### Configuration

| IW | IH | FW | FH | IN_C | OUT_C | Stride | Pad | OW | OH |
|----|----|----|----|------|-------|--------|-----|----|----|
| 3  | 3  | 3  | 3  | 1    | 1     | 1      | 0   | 5  | 5  |

## 2. How does ConvTranspose2D work?

### Operation Process

| ifmap |   |   | filter |   |   | ofmap |    |     |     |    |
|-------|---|---|--------|---|---|-------|----|-----|-----|----|
| 1     | 2 | 3 | 2      | 4 | 6 | 2     | 8  | 20  | 24  | 18 |
| 4     | 5 | 6 | 3      | 5 | 7 | 11    | 37 | 82  | 83  | 57 |
| 7     | 8 | 9 | 1      | 3 | 5 | 27    | 84 | 159 | 132 | 57 |
|       |   |   |        |   |   | 25    | 76 | 130 | 99  | 30 |
|       |   |   |        |   |   | 7     | 29 | 59  | 40  |    |

### Configuration

| IW | IH | FW | FH | IN_C | OUT_C | Stride | Pad | OW | OH |
|----|----|----|----|------|-------|--------|-----|----|----|
| 3  | 3  | 3  | 3  | 1    | 1     | 1      | 0   | 5  | 5  |

## 2. How does ConvTranspose2D work?

### Operation Process

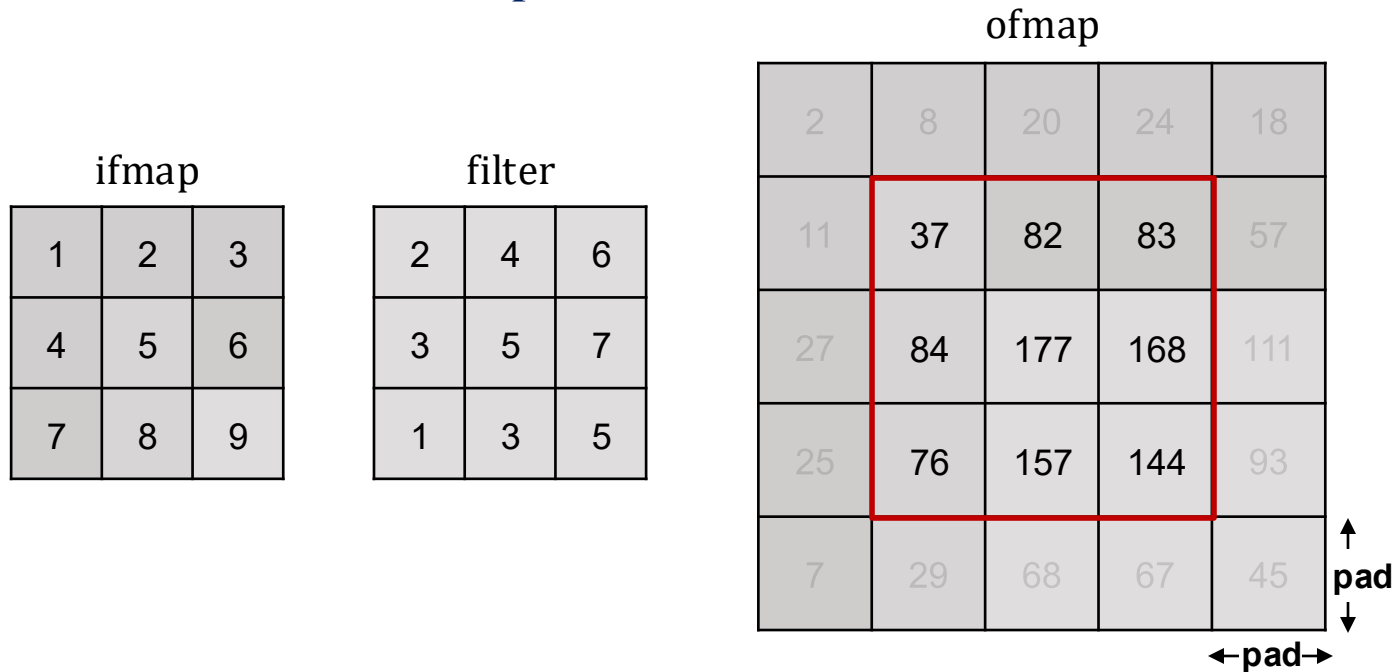
| ifmap |   |   | filter |   |   | ofmap |    |     |     |     |
|-------|---|---|--------|---|---|-------|----|-----|-----|-----|
| 1     | 2 | 3 | 2      | 4 | 6 | 2     | 8  | 20  | 24  | 18  |
| 4     | 5 | 6 | 3      | 5 | 7 | 11    | 37 | 82  | 83  | 57  |
| 7     | 8 | 9 | 1      | 3 | 5 | 27    | 84 | 177 | 168 | 111 |
|       |   |   |        |   |   | 25    | 76 | 157 | 144 | 93  |
|       |   |   |        |   |   | 7     | 29 | 68  | 67  | 45  |

### Configuration

| IW | IH | FW | FH | IN_C | OUT_C | Stride | Pad | OW | OH |
|----|----|----|----|------|-------|--------|-----|----|----|
| 3  | 3  | 3  | 3  | 1    | 1     | 1      | 0   | 5  | 5  |

## 2. How does ConvTranspose2D work?

### Operation Process

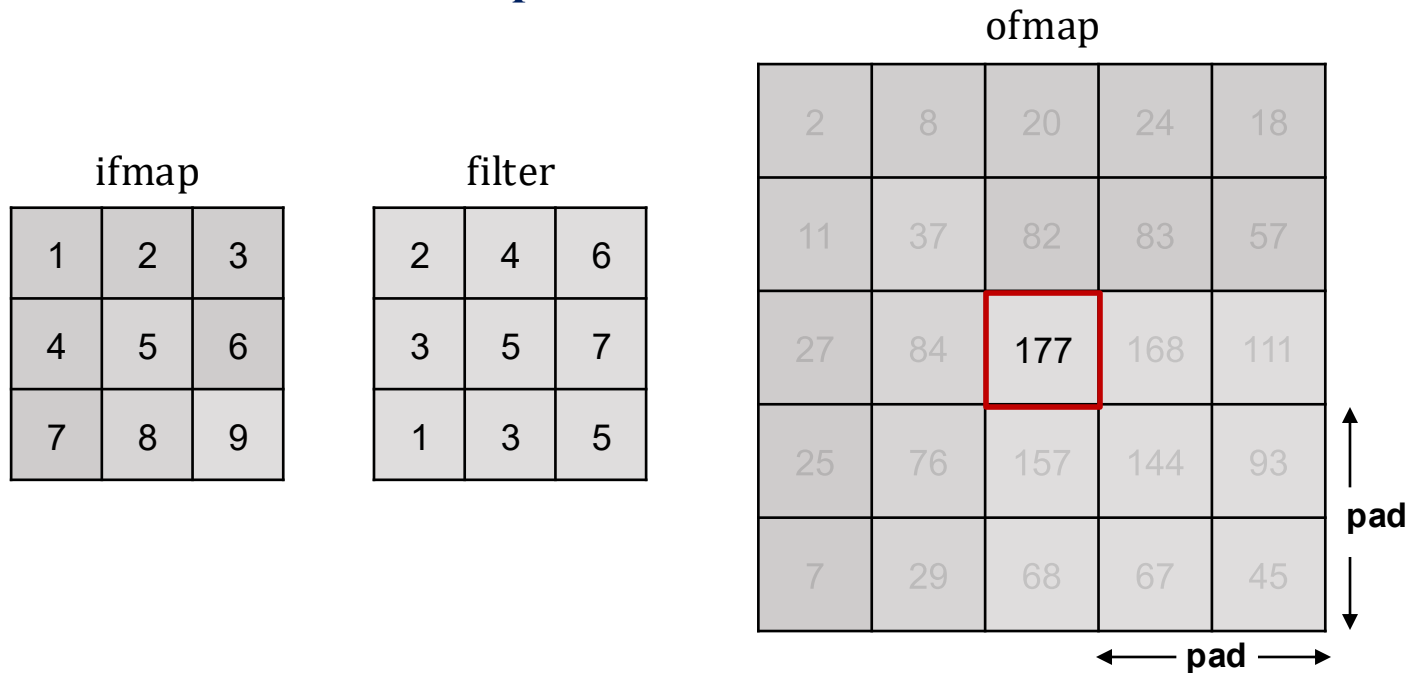


### Configuration

| IW | IH | FW | FH | IN_C | OUT_C | Stride | Pad | OW | OH |
|----|----|----|----|------|-------|--------|-----|----|----|
| 3  | 3  | 3  | 3  | 1    | 1     | 1      | 1   | 3  | 3  |

## 2. How does ConvTranspose2D work?

### Operation Process



### Configuration

| IW | IH | FW | FH | IN_C | OUT_C | Stride | Pad | OW | OH |
|----|----|----|----|------|-------|--------|-----|----|----|
| 3  | 3  | 3  | 3  | 1    | 1     | 1      | 2   | 1  | 1  |

## 2. How does ConvTranspose2D work?

- An equation for finding the size of ofmap:

$$(OW, OH) = ((FW + S(IW - 1) - 2P), (FH + S(IH - 1) - 2P))$$

$$= (3 + 1(1), 3 + 1(1)) = (4, 4)$$

|   |   |
|---|---|
| 1 | 2 |
| 3 | 4 |

x

|   |   |   |
|---|---|---|
| 1 | 2 | 3 |
| 2 | 2 | 1 |
| 3 | 2 | 1 |

=

|   |   |   |  |
|---|---|---|--|
| 1 | 2 | 3 |  |
| 2 | 2 | 1 |  |
| 3 | 2 | 1 |  |
|   |   |   |  |

|   |   |
|---|---|
| 1 | 2 |
| 3 | 4 |

x

|   |   |   |
|---|---|---|
| 1 | 2 | 3 |
| 2 | 2 | 1 |
| 3 | 2 | 1 |

=

|   |     |     |   |
|---|-----|-----|---|
| 1 | 2+2 | 3+4 | 6 |
| 2 | 2+4 | 1+4 | 2 |
| 3 | 2+6 | 1+4 | 2 |
|   |     |     |   |

|   |   |
|---|---|
| 1 | 2 |
| 3 | 4 |

x

|   |   |   |
|---|---|---|
| 1 | 2 | 3 |
| 2 | 2 | 1 |
| 3 | 2 | 1 |

=

|     |     |     |   |
|-----|-----|-----|---|
| 1   | 4   | 7   | 6 |
| 2+3 | 6+6 | 5+9 | 2 |
| 3+6 | 8+6 | 5+2 | 2 |
| 9   | 6   | 3   |   |

|   |   |
|---|---|
| 1 | 2 |
| 3 | 4 |

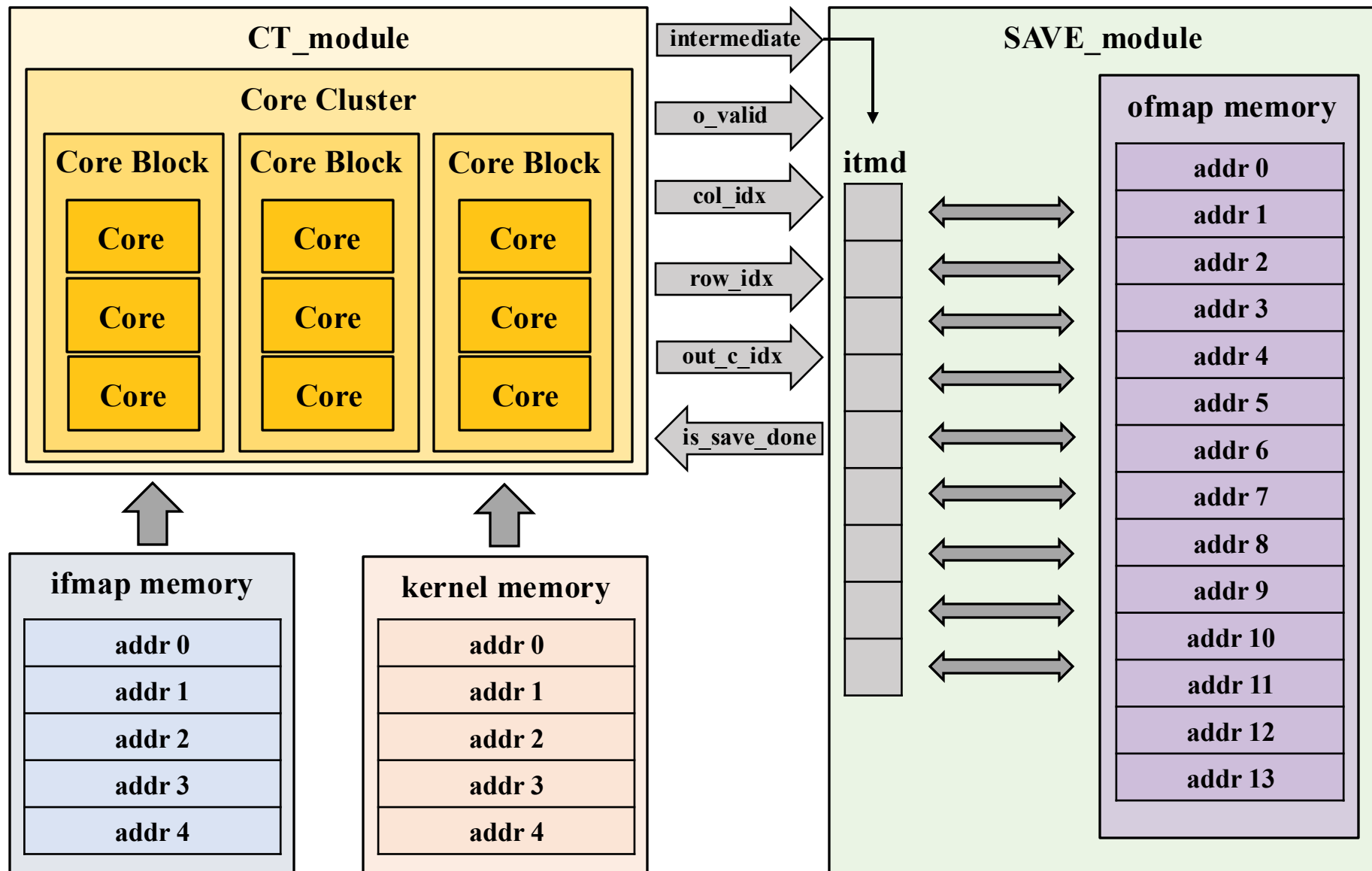
x

|   |   |   |
|---|---|---|
| 1 | 2 | 3 |
| 2 | 2 | 1 |
| 3 | 2 | 1 |

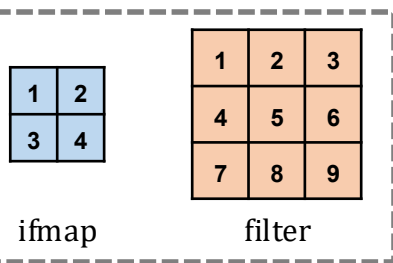
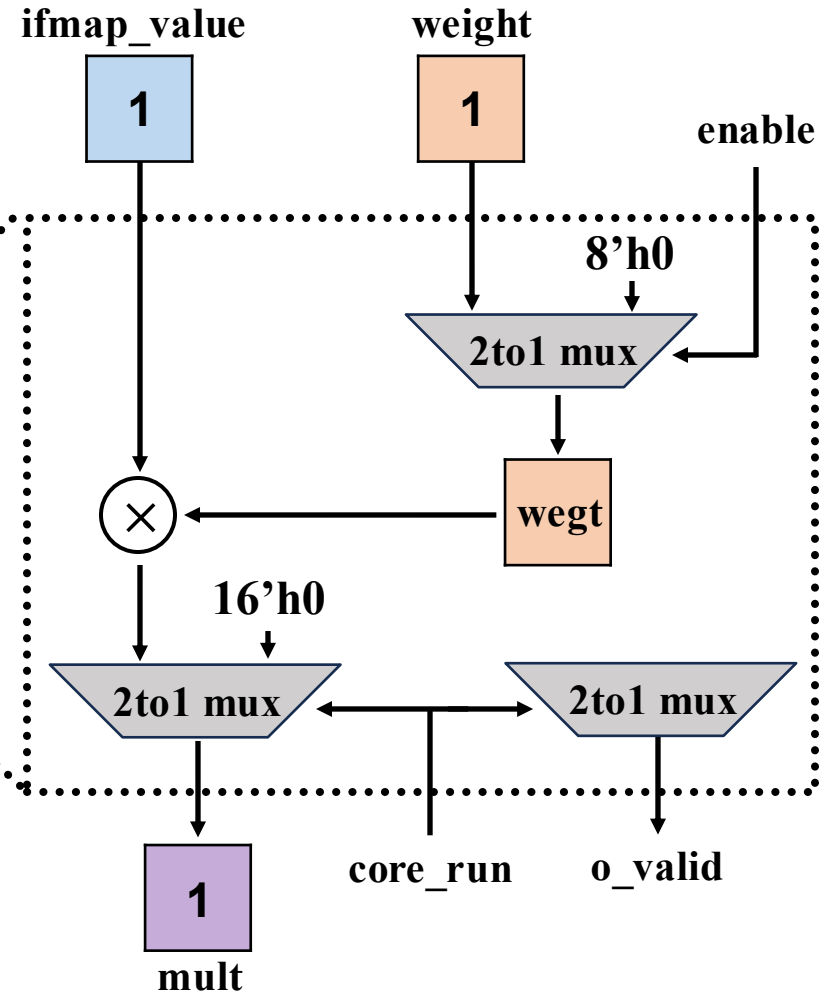
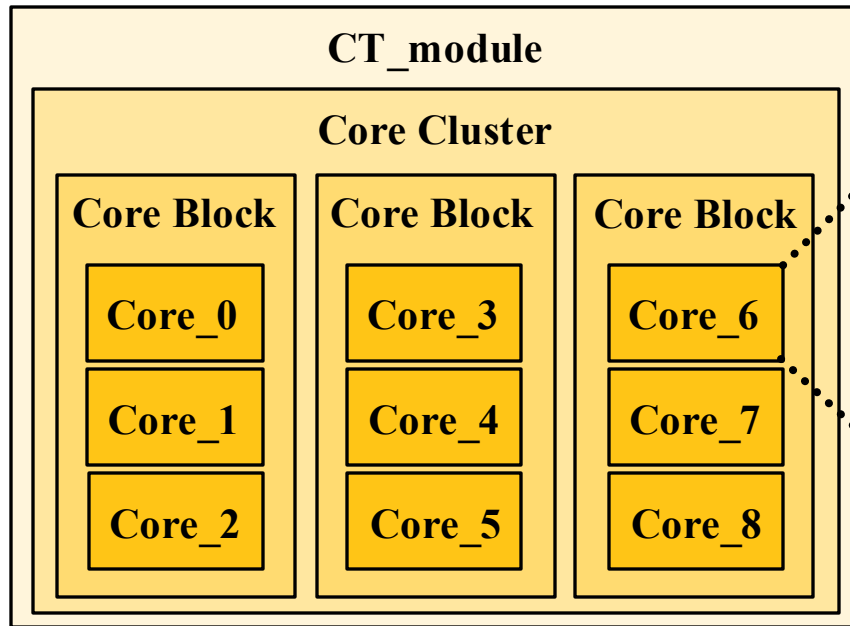
=

|   |      |      |      |
|---|------|------|------|
| 1 | 4    | 7    | 6    |
| 5 | 12+4 | 14+8 | 2+12 |
| 9 | 14+8 | 7+8  | 2+4  |
| 9 | 6+12 | 3+8  | 4    |

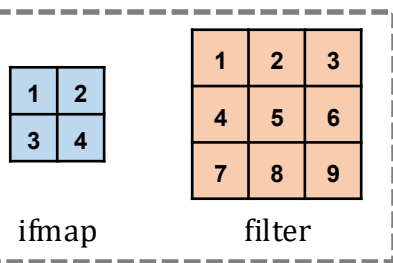
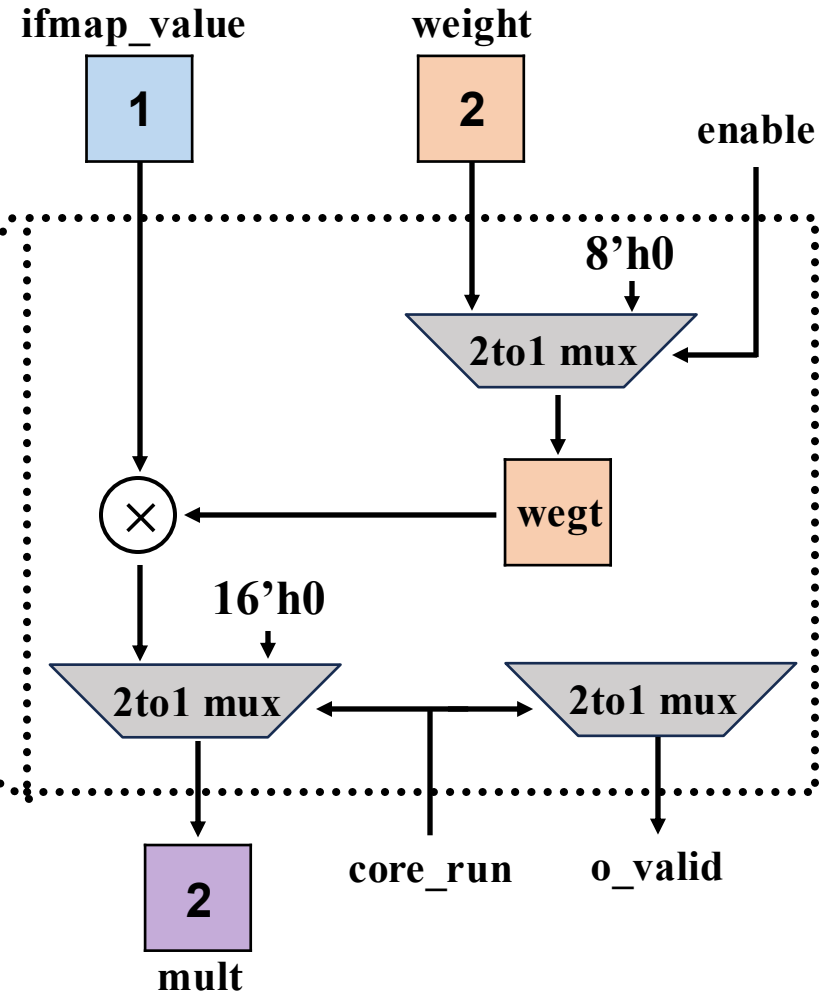
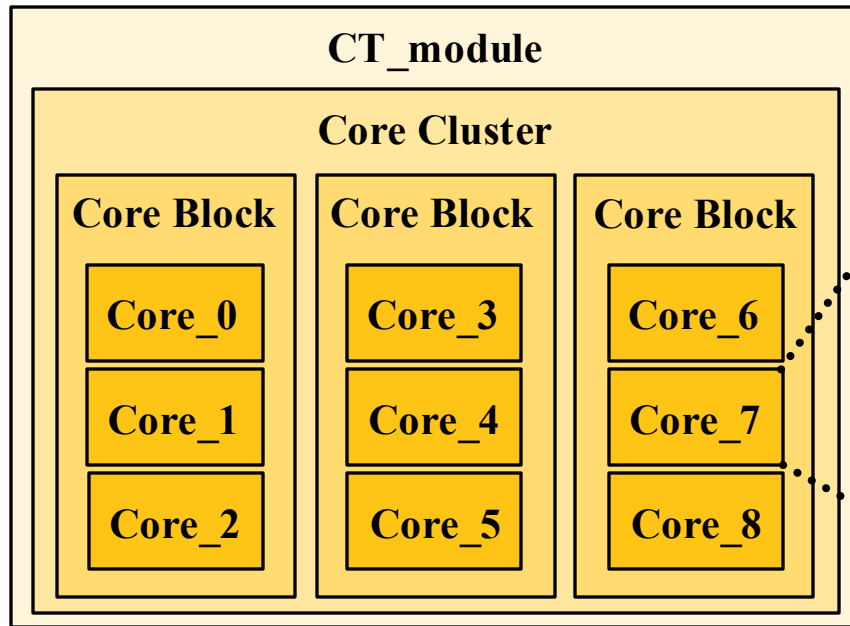
### 3. Full Architecture of CT Accelerator



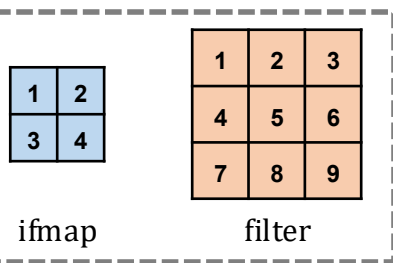
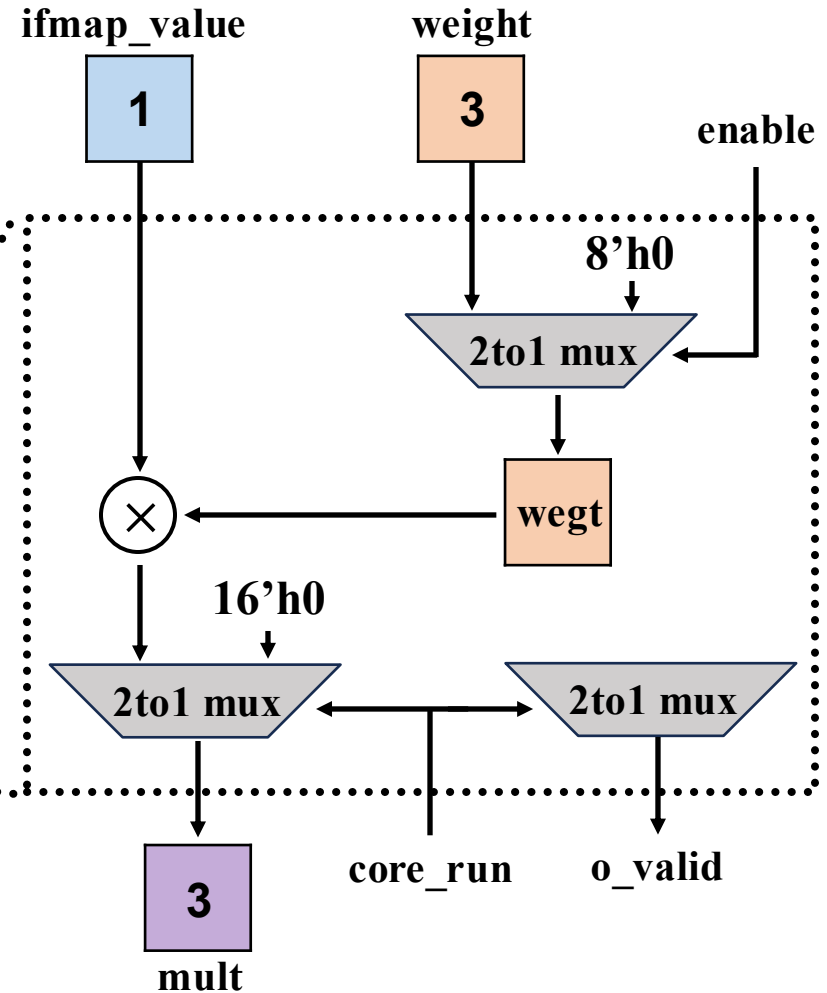
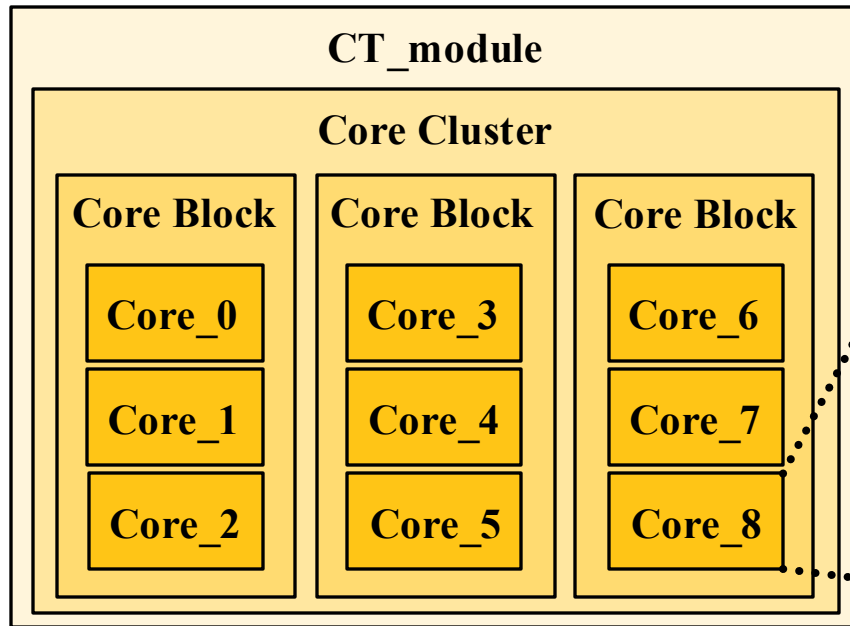
# 4. Core Architecture



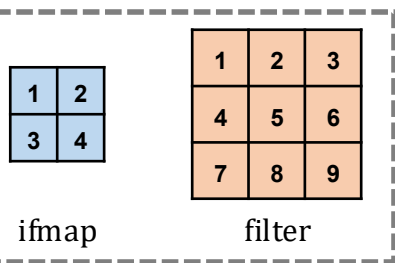
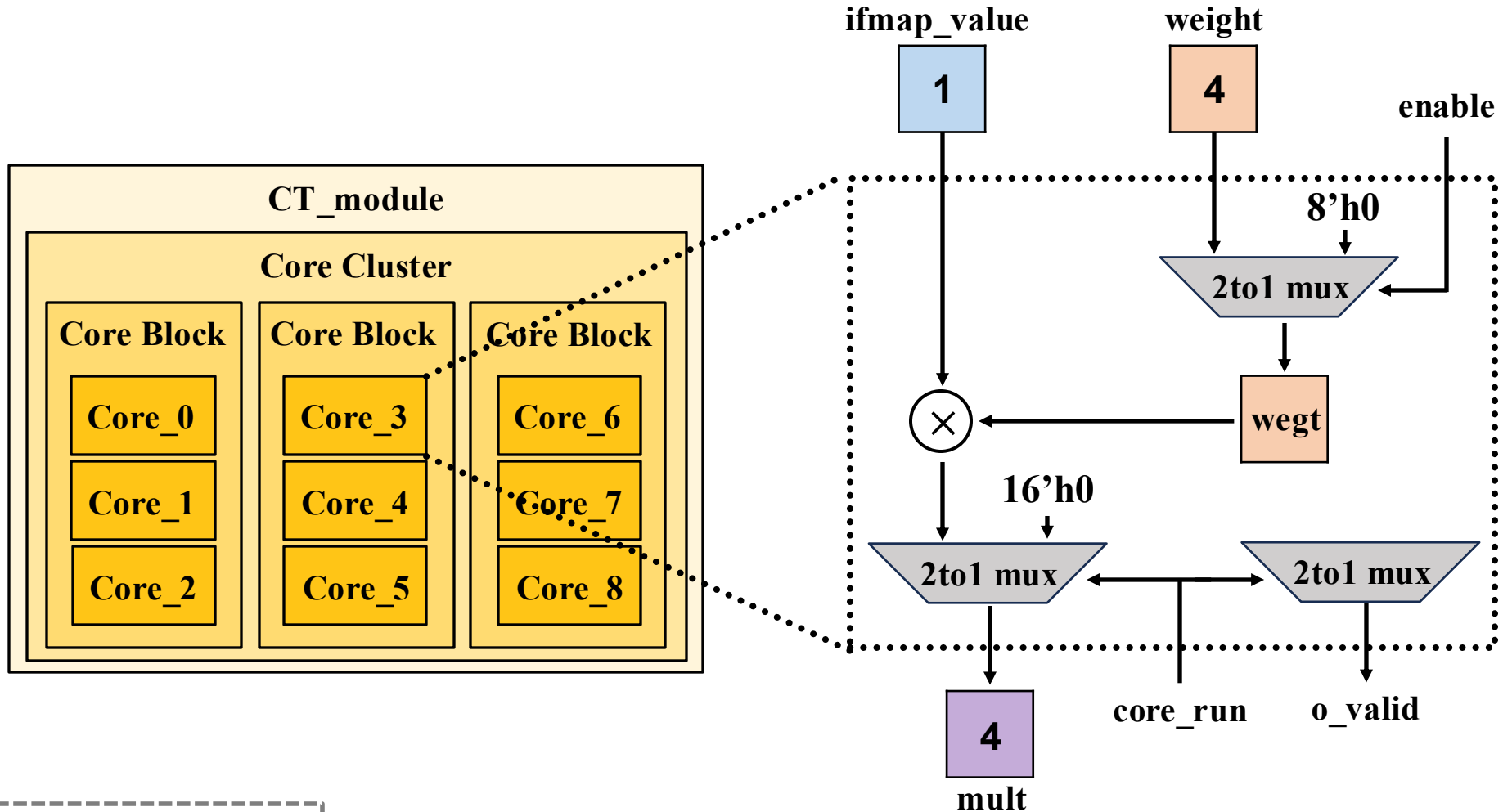
# 4. Core Architecture



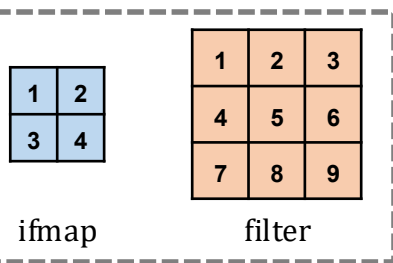
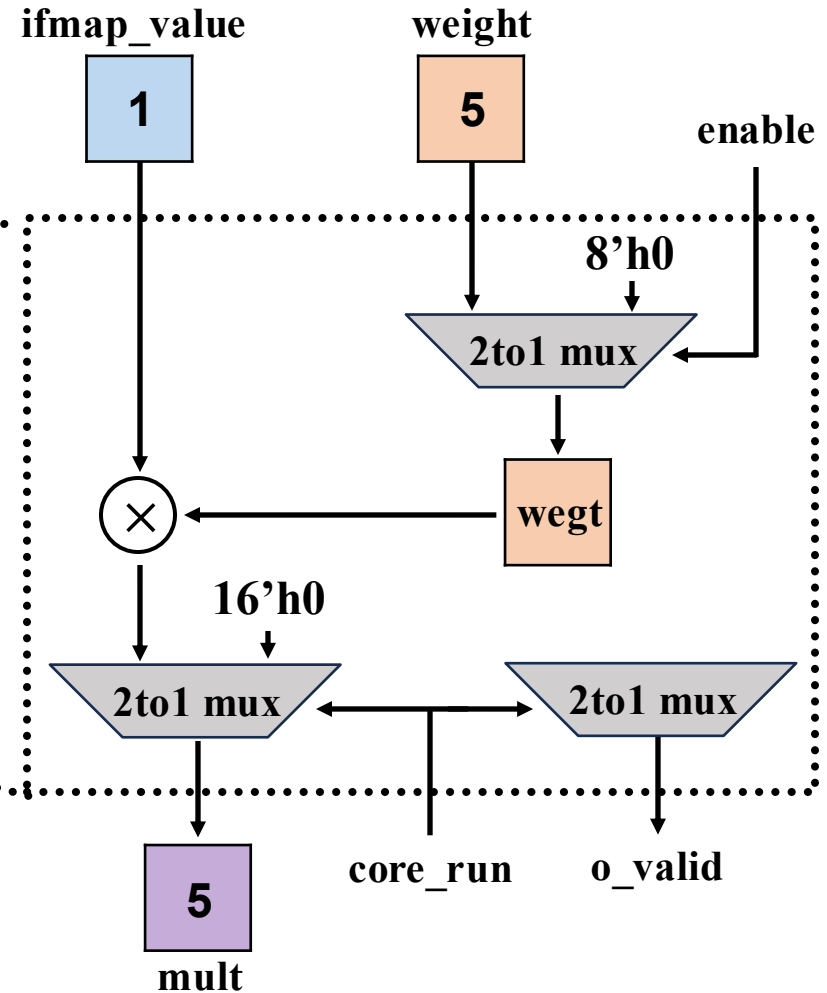
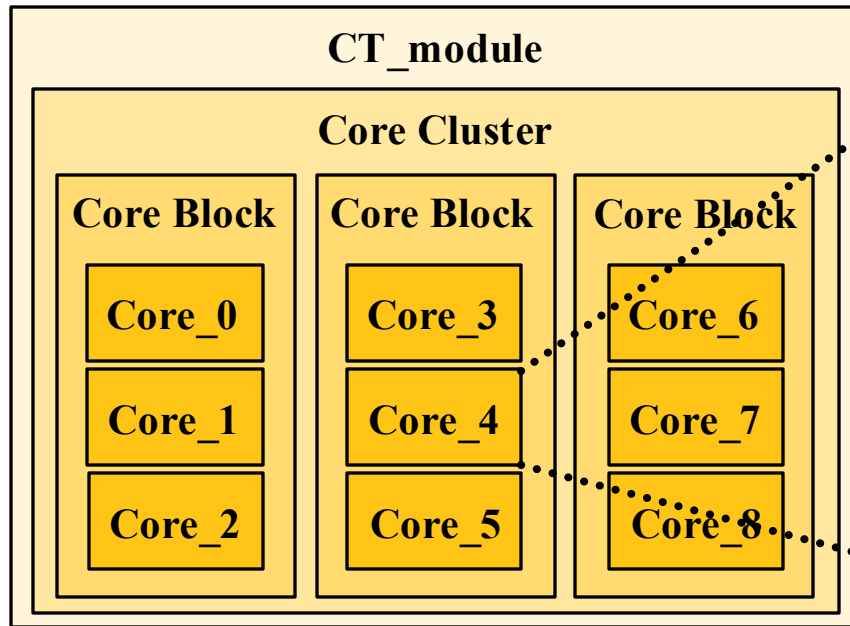
# 4. Core Architecture



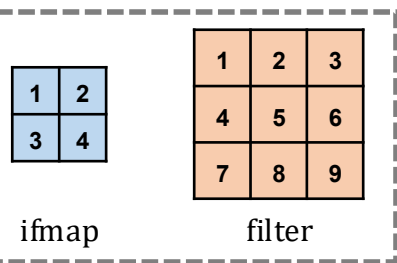
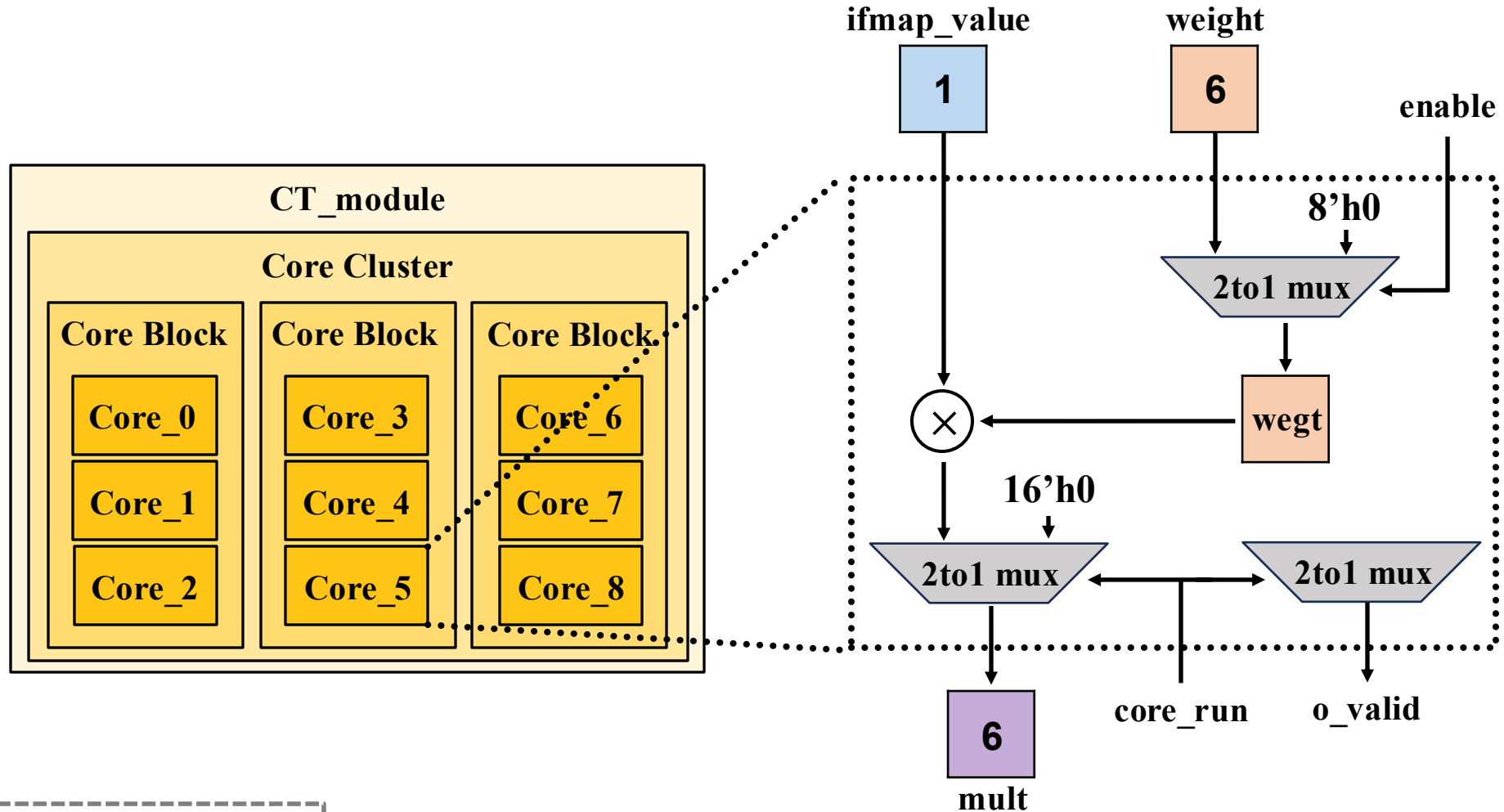
# 4. Core Architecture



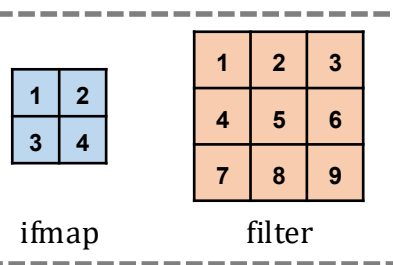
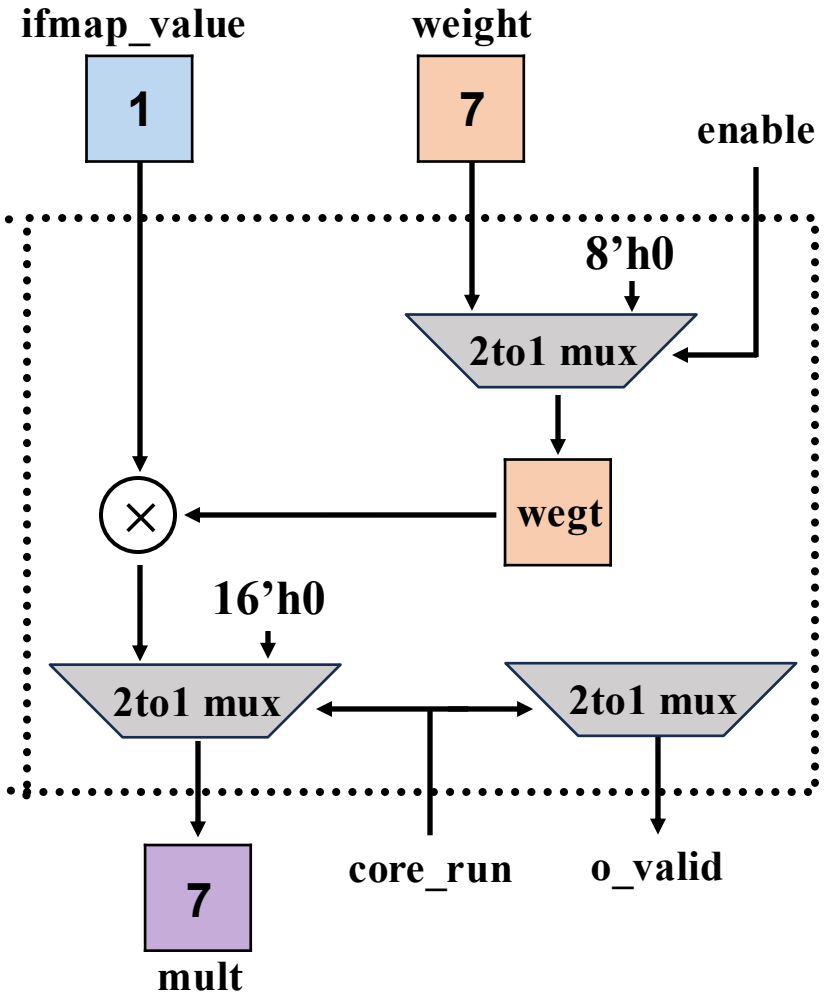
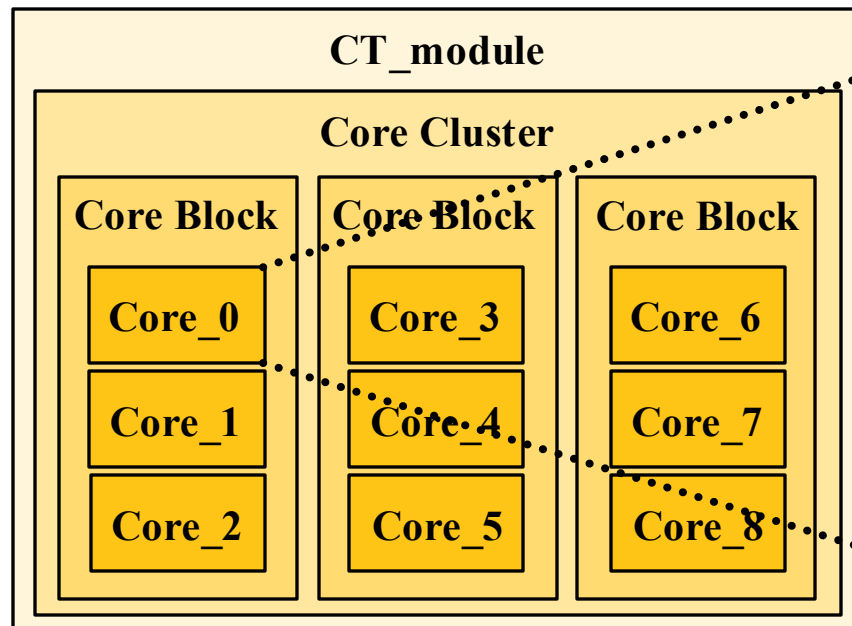
# 4. Core Architecture



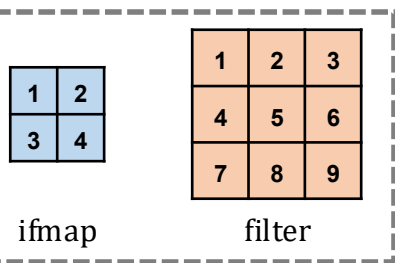
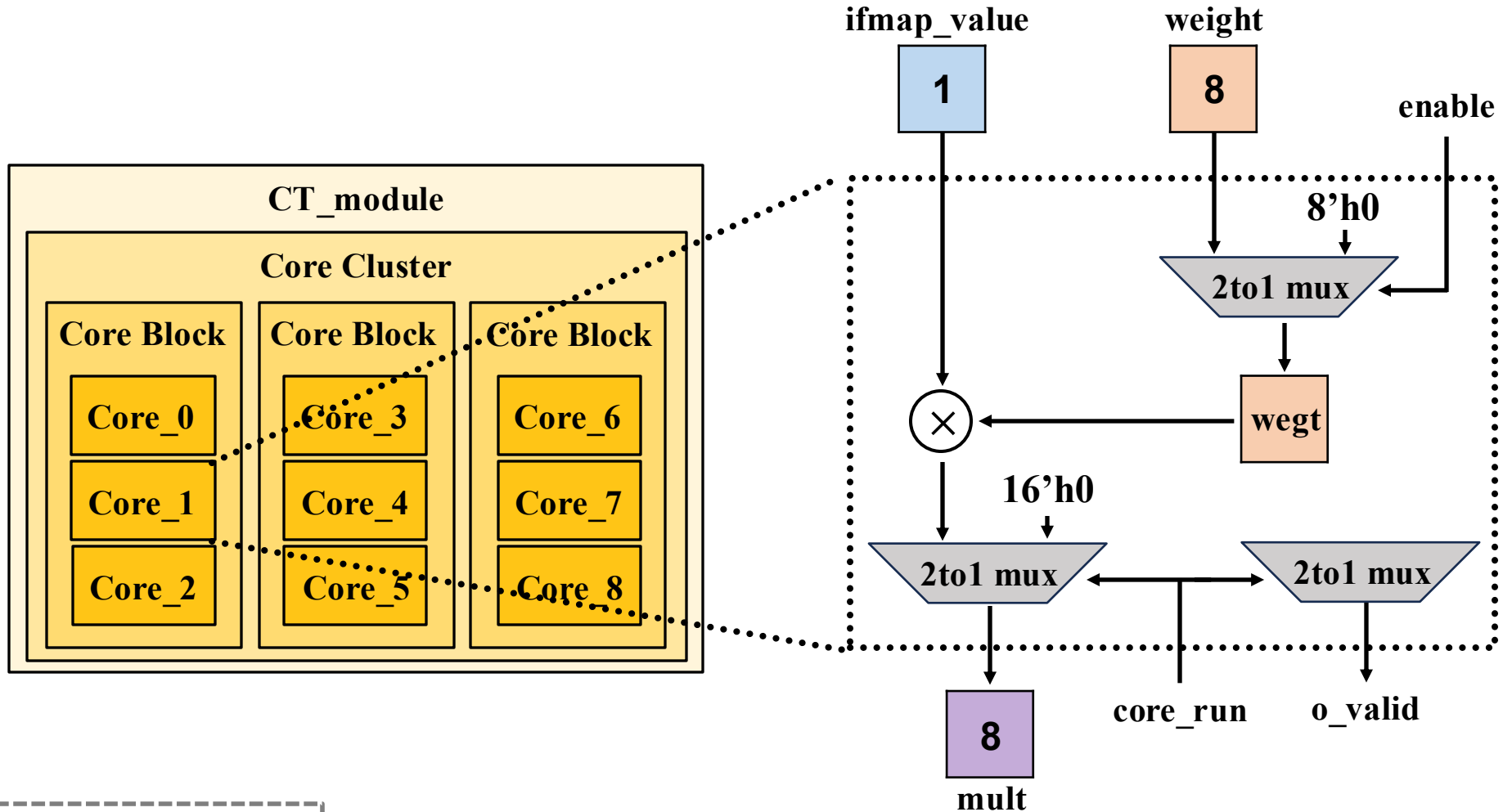
# 4. Core Architecture



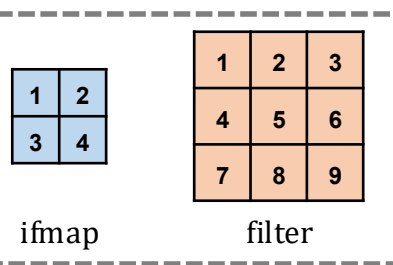
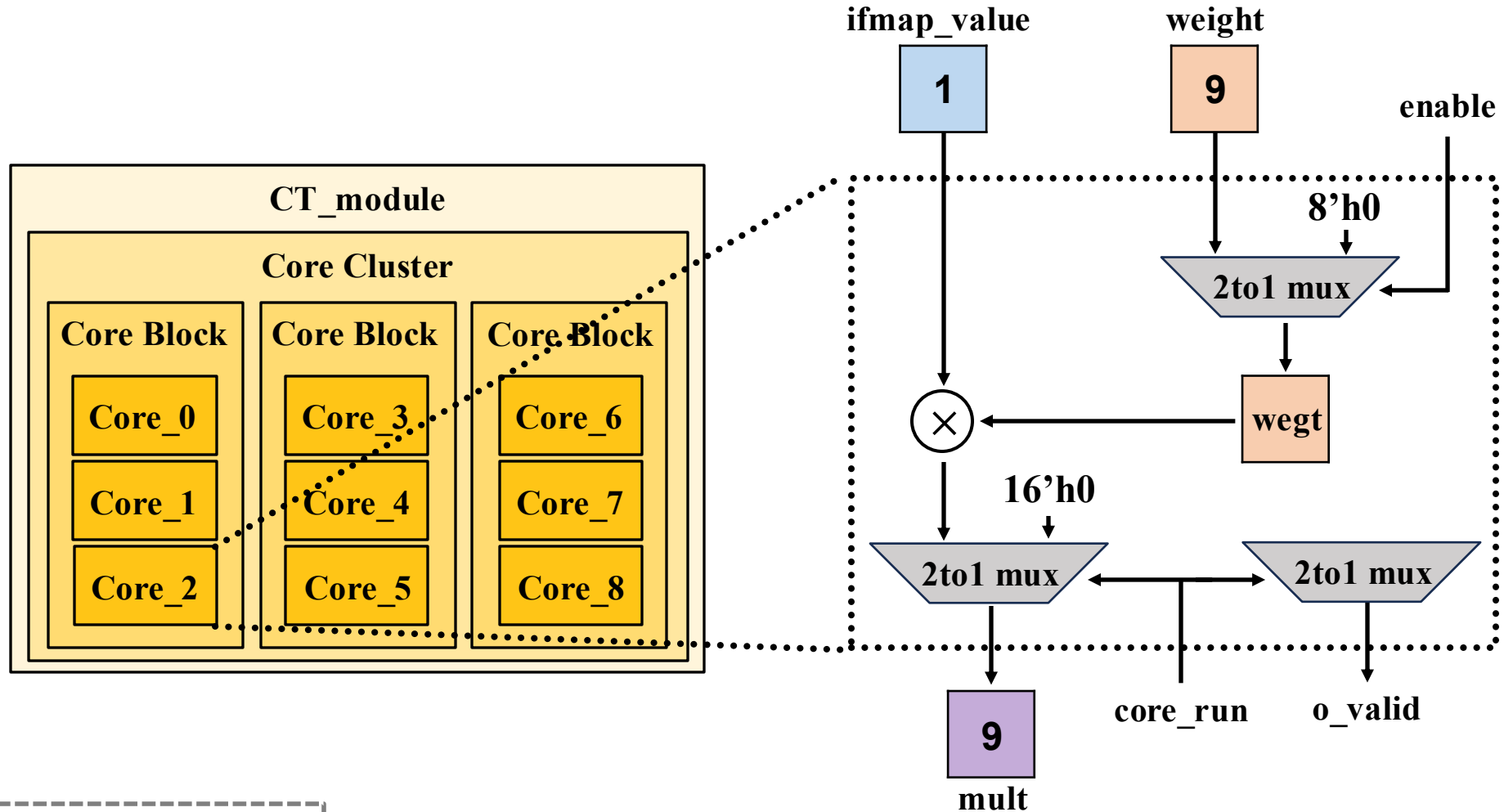
# 4. Core Architecture



# 4. Core Architecture



# 4. Core Architecture

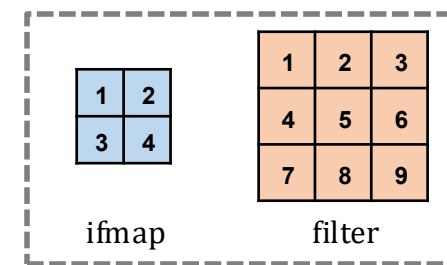
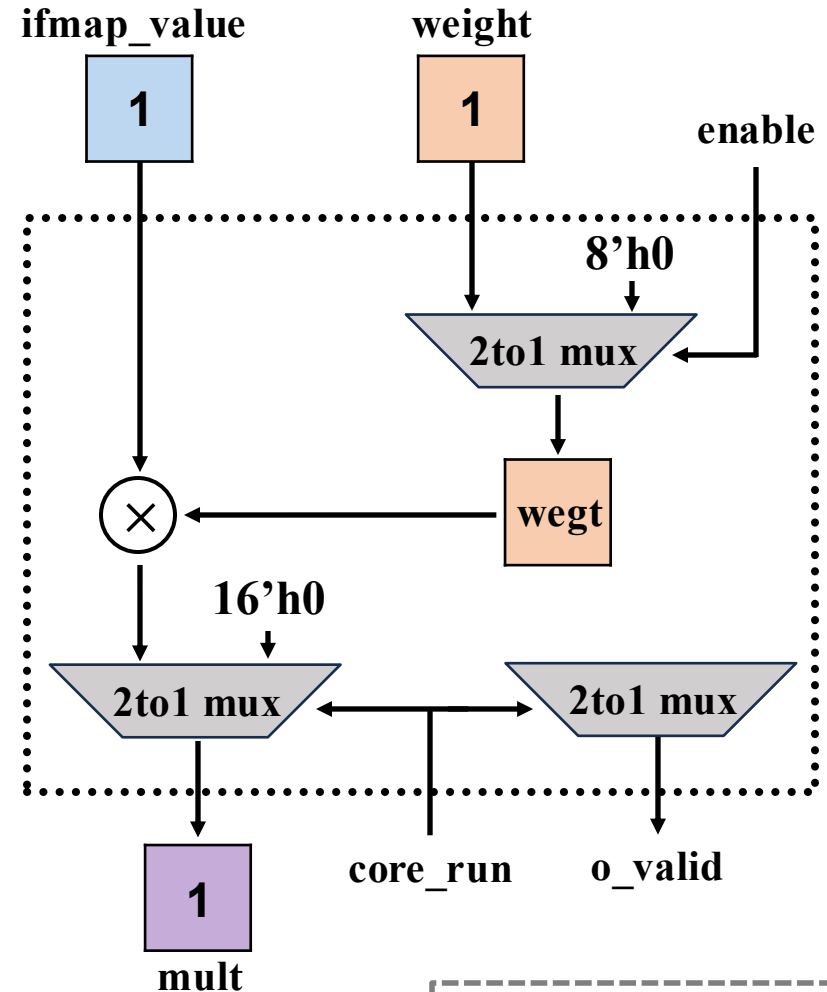


# 4. Core Architecture

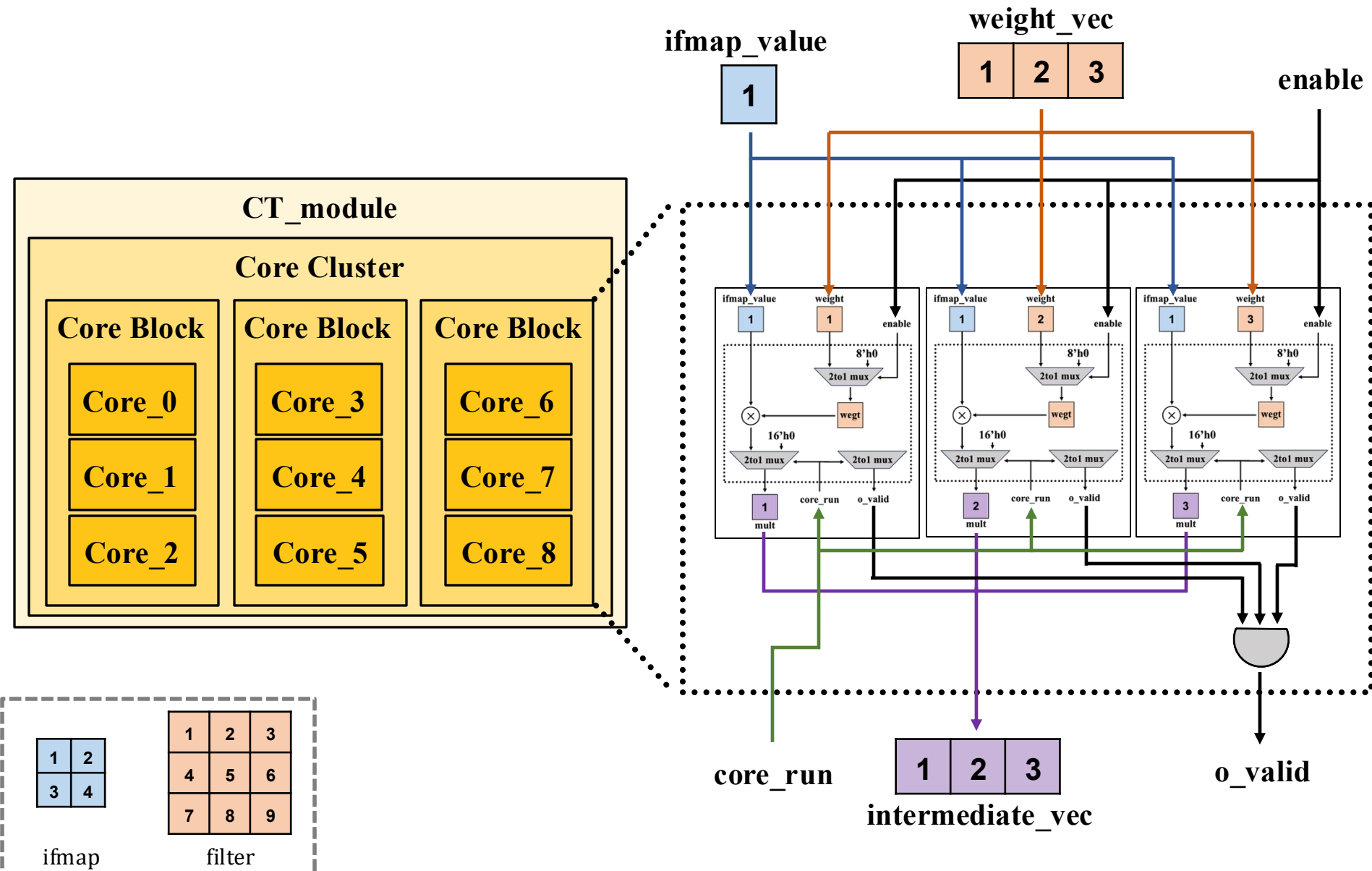
```

module core#
(
  parameter BITS = 8,
  parameter SIGNED_BITS = 9,
  parameter MULT_BITS = 16
)
(
  input clk,
  input reset_n,
  //
  input core_run,
  input enable,
  //
  input signed [SIGNED_BITS - 1 : 0] ifmap_value,
  input signed [SIGNED_BITS - 1 : 0] weight,
  //
  output reg o_valid,
  output reg signed [MULT_BITS - 1 : 0] mult
);

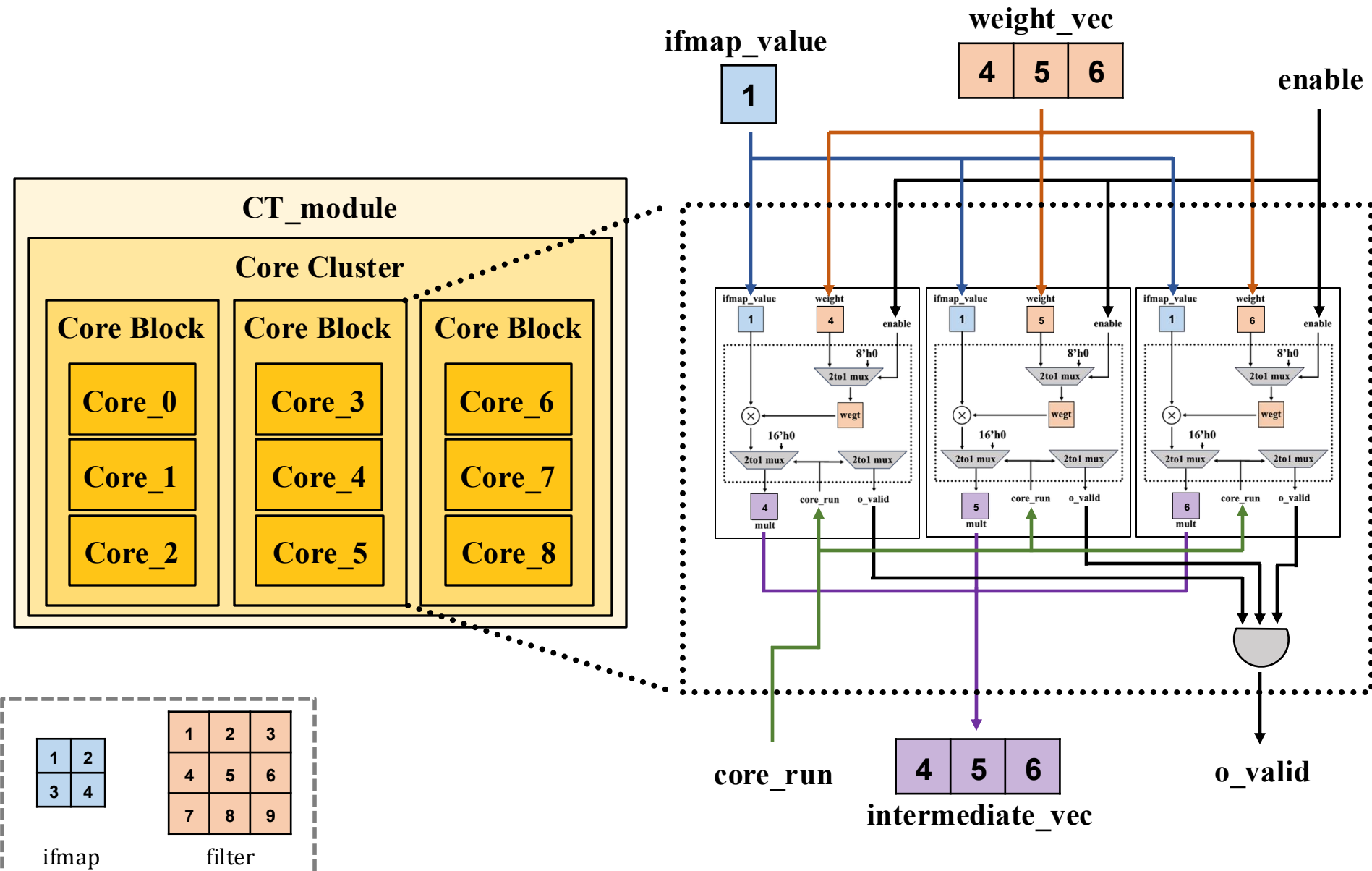
reg signed [BITS - 1 : 0] wegt;
//
always @(posedge clk or negedge reset_n) begin
  if(!reset_n) begin
    mult <= {MULT_BITS{1'b0}};
    wegt <= {BITS{1'b0}};
    o_valid <= 1'b0;
  end
  else if(enable) wegt <= weight;
  else begin
    if(core_run) begin
      mult <= ifmap_value * wegt;
      o_valid <= 1'b1;
    end
  end
end
end
endmodule
  
```



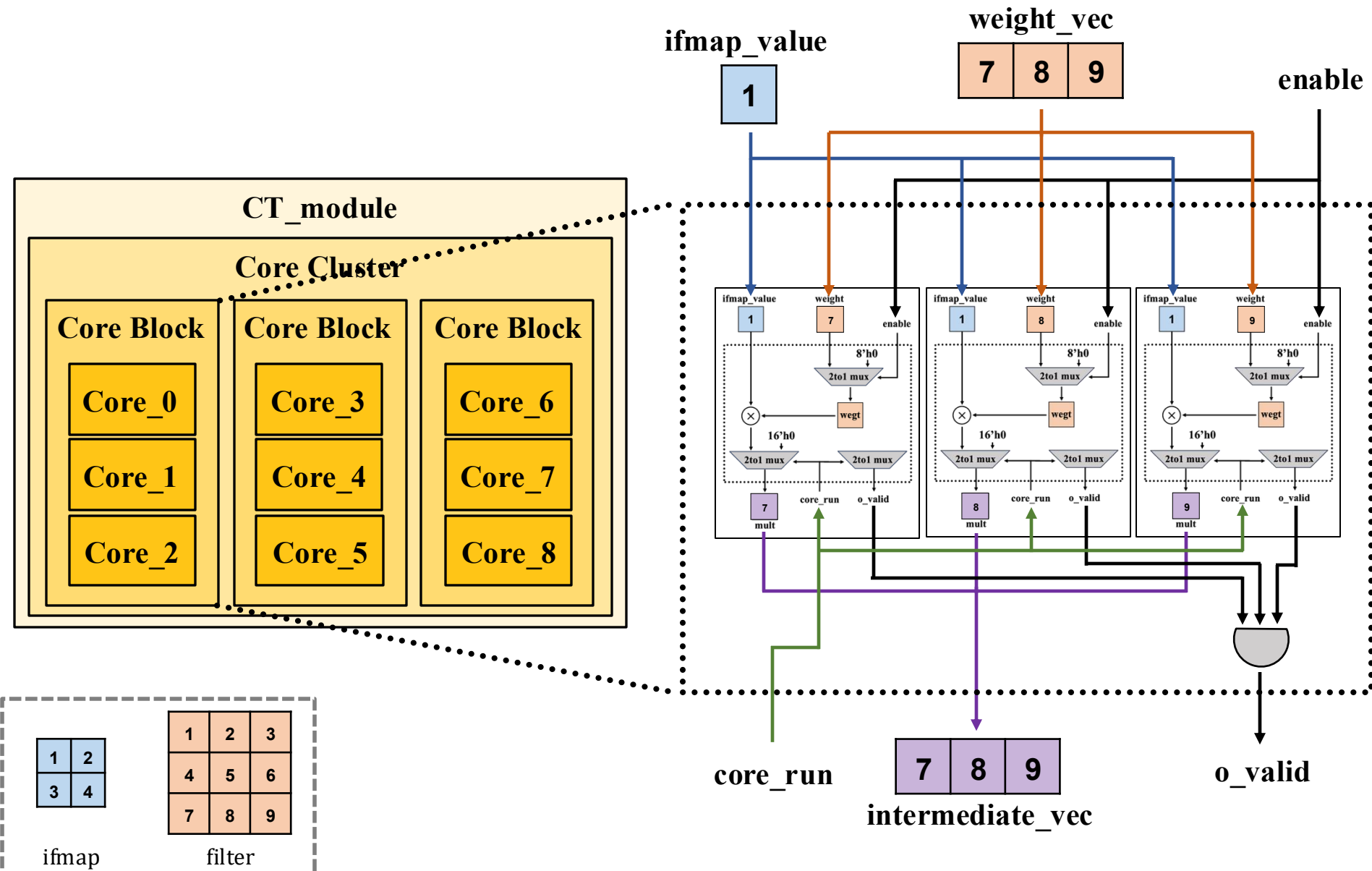
# 5. Core Block Architecture



# 5. Core Block Architecture



# 5. Core Block Architecture

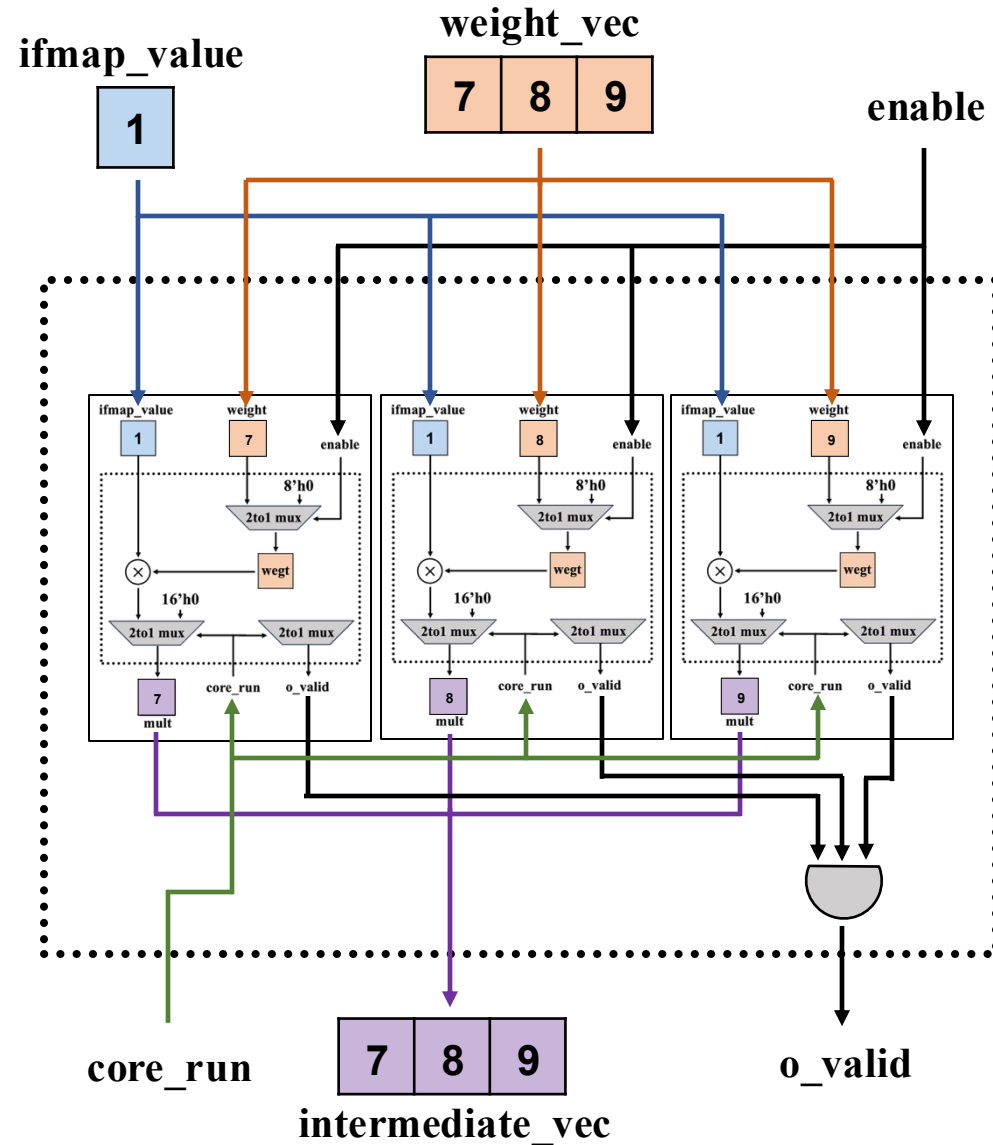


# 5. Core Block Architecture

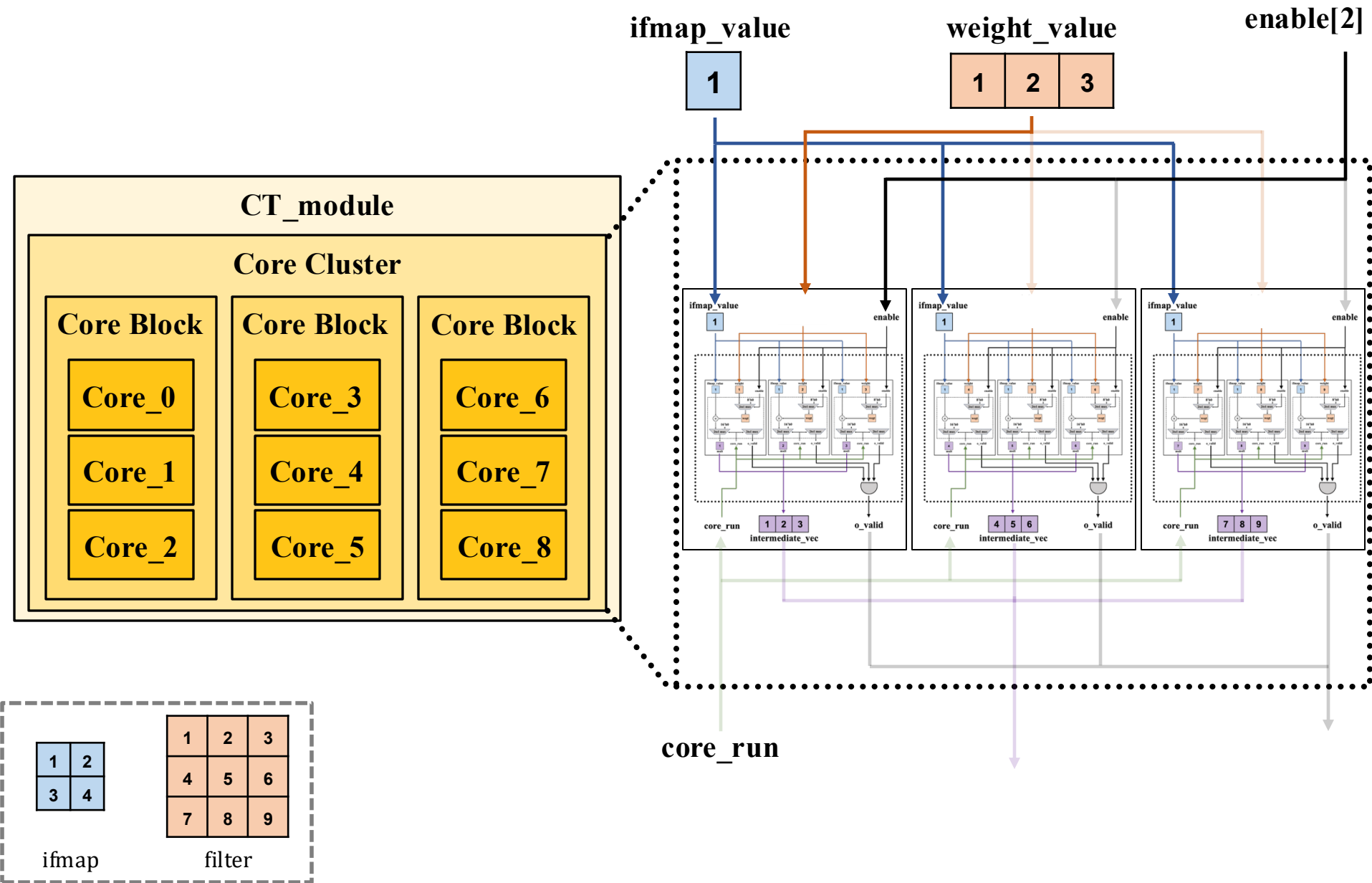
```

module core_block#
(
    parameter FW = 3,
    parameter FH = 3,
    //
    parameter BITS = 8,
    parameter SIGNED_BITS = 9,
    parameter MULT_BITS = 16
)
(
    input clk,
    input reset_n,
    //
    input core_run,
    input enable,
    //
    input signed [SIGNED_BITS - 1 : 0] ifmap_value,
    input signed [BITS * FW - 1 : 0] weight_vec,
    //
    output o_valid,
    output signed [FW * MULT_BITS - 1 : 0] intermediate_vec
);
// *_*_*_*_*_ instantiation of cores *_*_*_*_*_
wire [FW - 1 : 0] o_valid_;

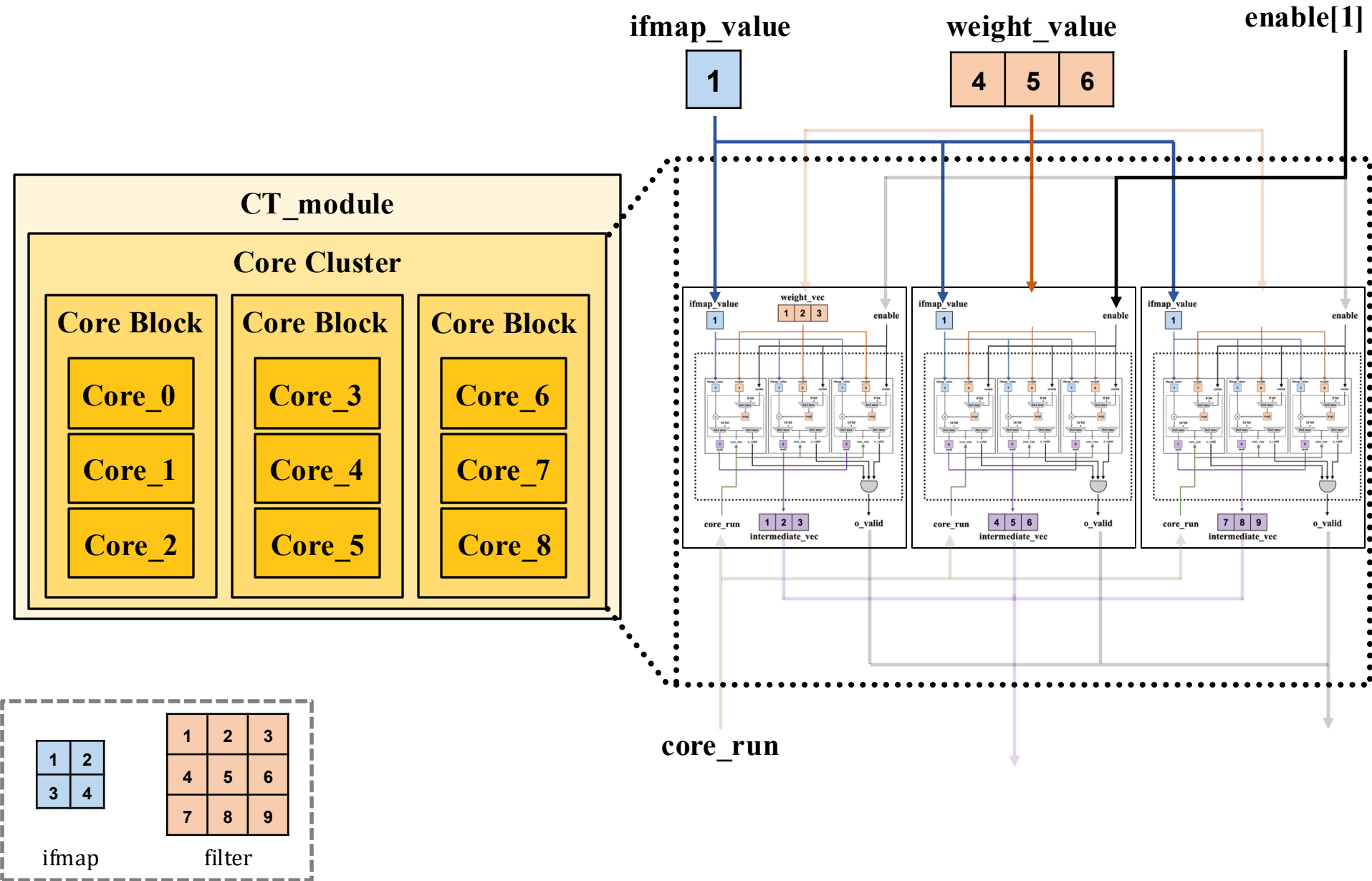
genvar i;
generate
    for(i = 1; i <= FW; i = i + 1) begin
        core#
        (
            .BITS (BITS),
            .SIGNED_BITS (SIGNED_BITS),
            .MULT_BITS (MULT_BITS)
        ) u0_core
        (
            .clk (clk),
            .reset_n (reset_n),
            //
            .core_run (core_run),
            .enable (enable),
            //
            .ifmap_value (ifmap_value),
            .weight (weight_vec[BITS * i - 1 : BITS]),
            // output
            .o_valid (o_valid_[i-1]),
            .mult (intermediate_vec[(FW - i) * MULT_BITS +: MULT_BITS])
        );
    end
endgenerate
assign o_valid = &o_valid_;
endmodule
    
```



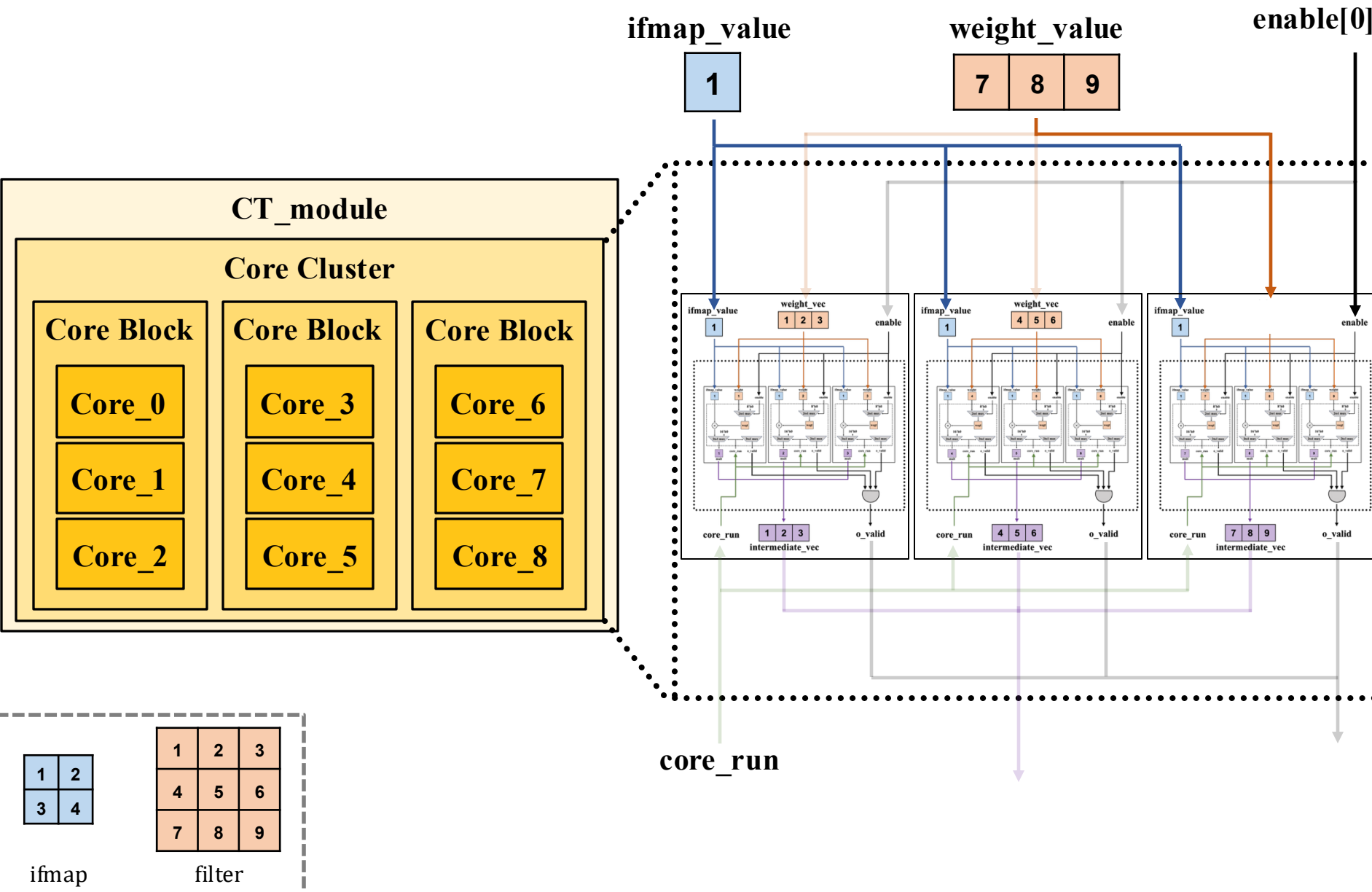
# 6. Core Cluster Architecture



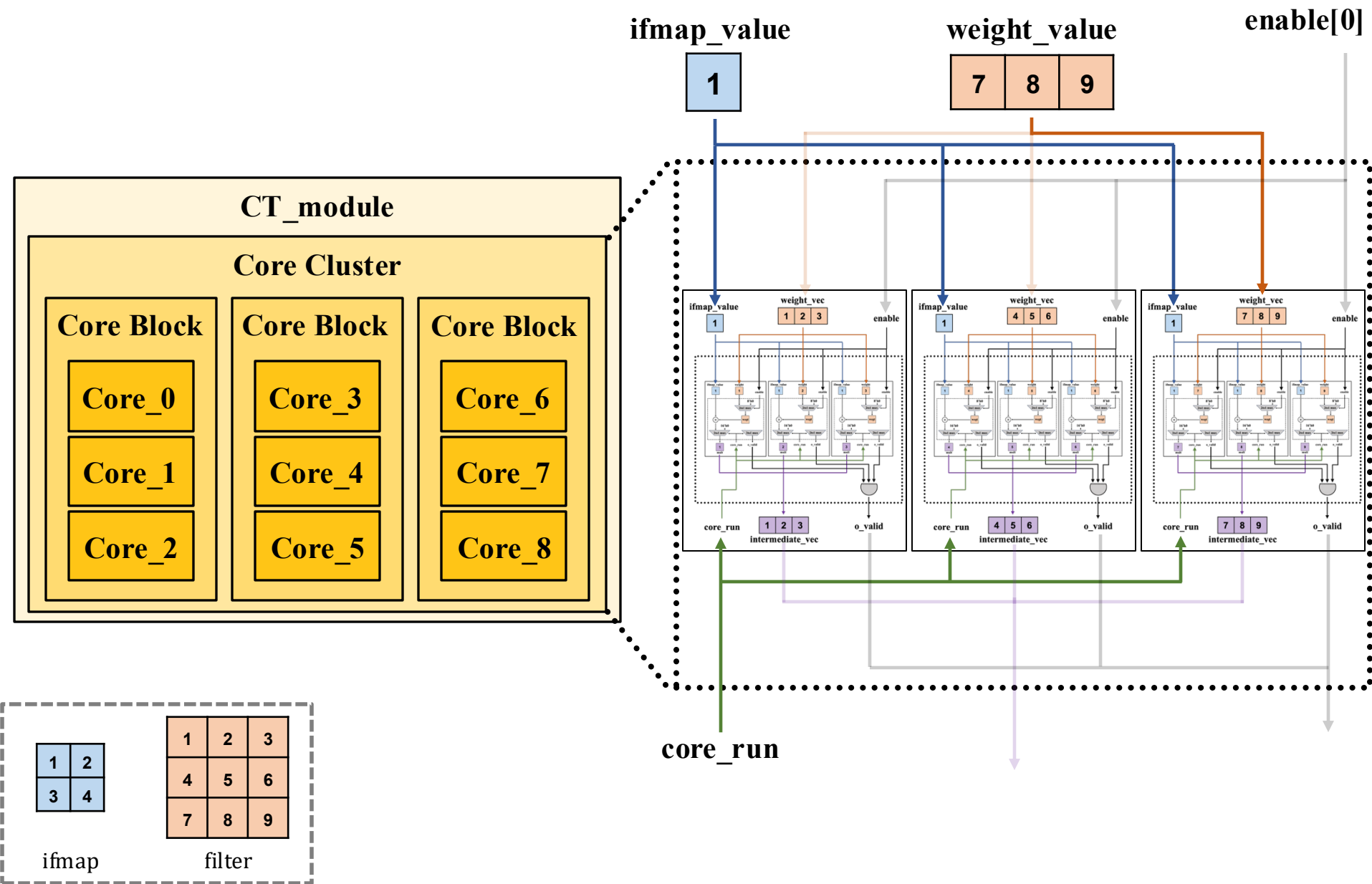
# 6. Core Cluster Architecture



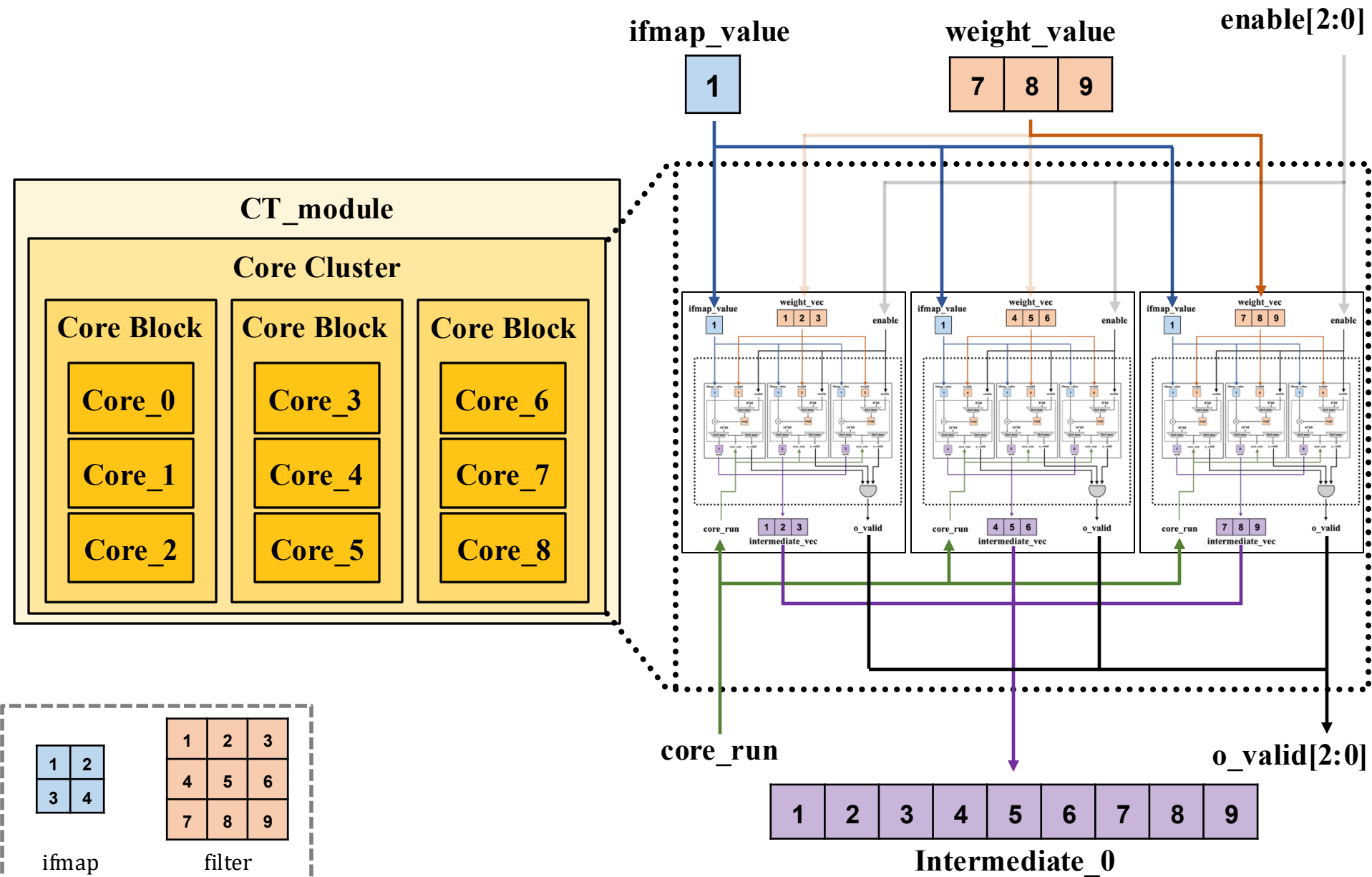
# 6. Core Cluster Architecture



# 6. Core Cluster Architecture



# 6. Core Cluster Architecture



# 6. Core Cluster Architecture

ifmap\_value

enable[2:0]

2

CT\_module

Core Cluster

Core Block

Core\_0

Core\_1

Core\_2

Core Block

Core\_3

Core\_4

Core\_5

Core Block

Core\_6

Core\_7

Core\_8

|   |   |
|---|---|
| 1 | 2 |
| 3 | 4 |

ifmap

|   |   |   |
|---|---|---|
| 1 | 2 | 3 |
| 4 | 5 | 6 |
| 7 | 8 | 9 |

filter

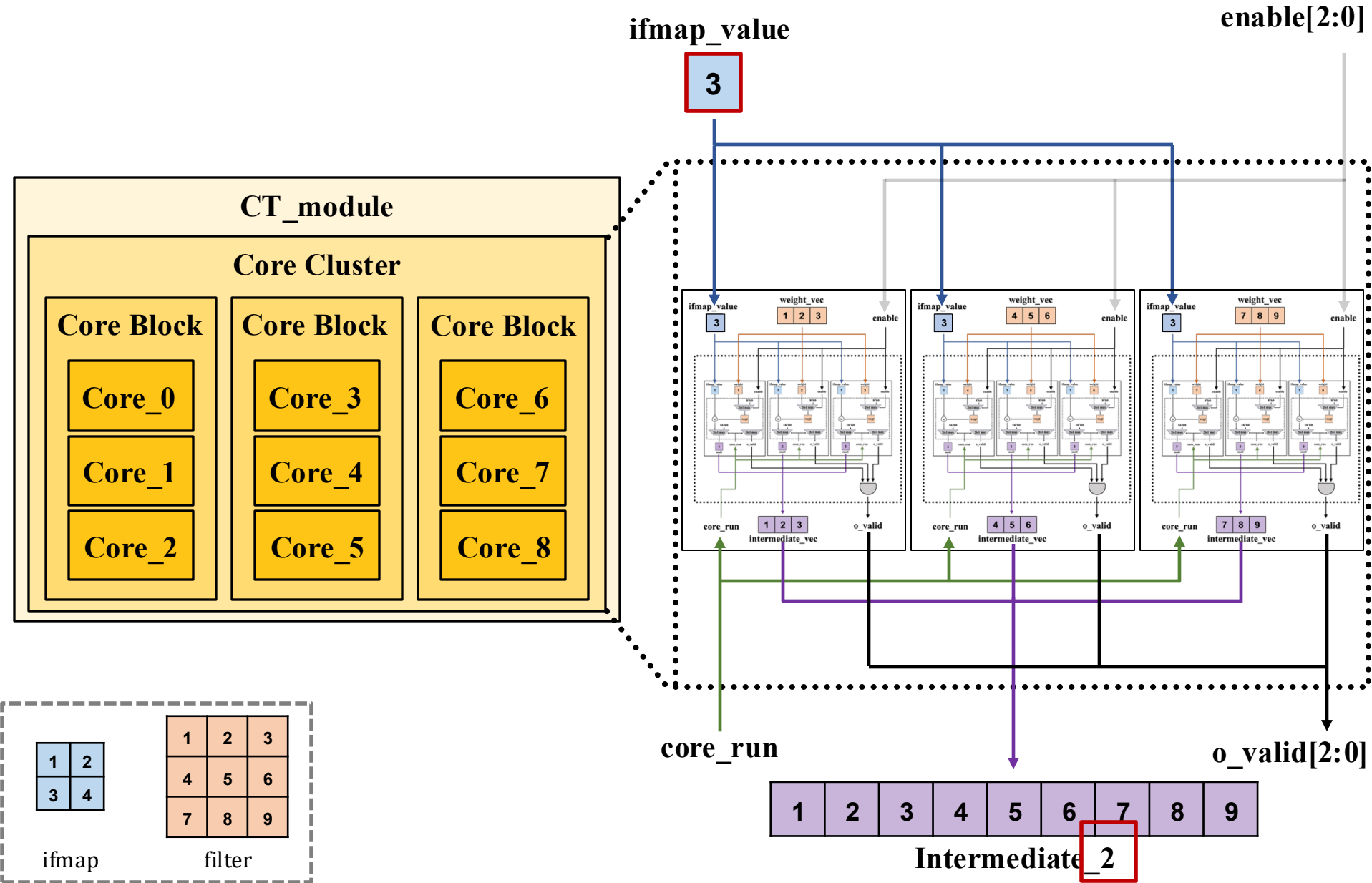
core\_run

o\_valid[2:0]

|   |   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|---|
| 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 |
|---|---|---|---|---|---|---|---|---|

Intermediate\_1

# 6. Core Cluster Architecture



# 6. Core Cluster Architecture

ifmap\_value

enable[2:0]

4

CT\_module

Core Cluster

Core Block

Core Block

Core Block

Core\_0

Core\_3

Core\_6

Core\_1

Core\_4

Core\_7

Core\_2

Core\_5

Core\_8

|   |   |
|---|---|
| 1 | 2 |
| 3 | 4 |

ifmap

|   |   |   |
|---|---|---|
| 1 | 2 | 3 |
| 4 | 5 | 6 |
| 7 | 8 | 9 |

filter

core\_run

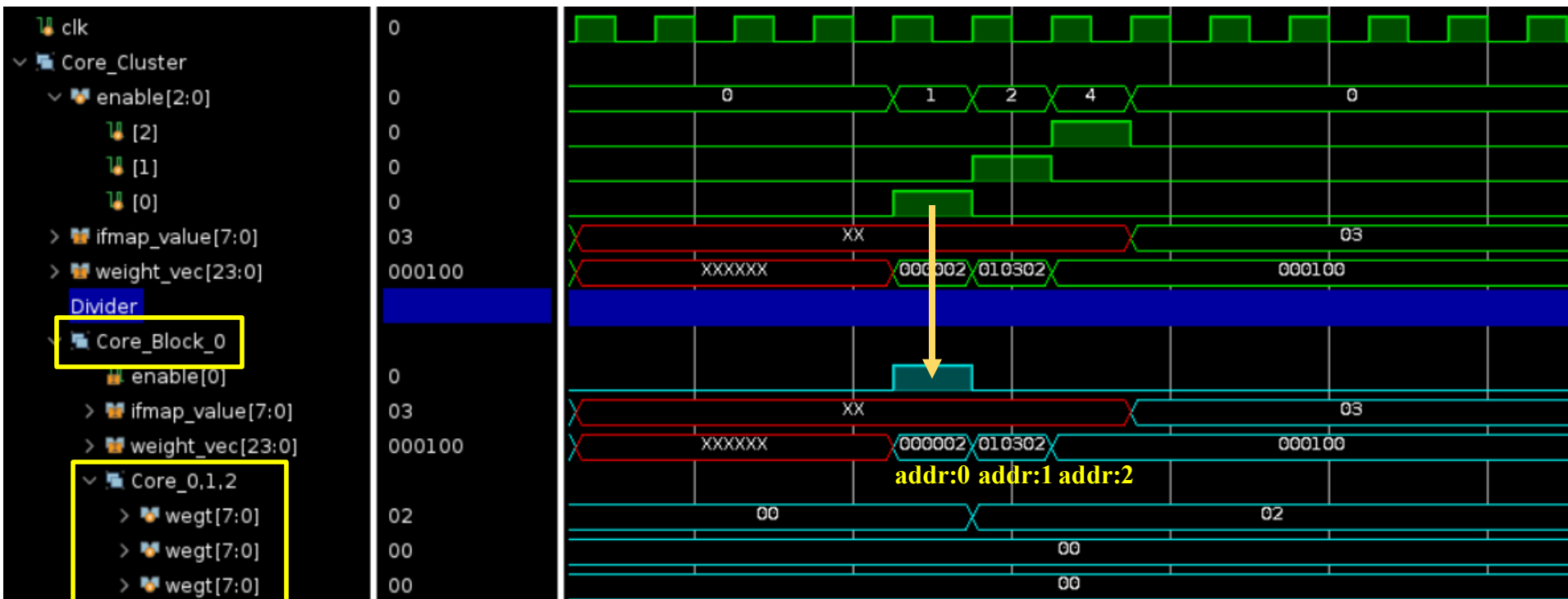
o\_valid[2:0]

|   |   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|---|
| 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 |
|---|---|---|---|---|---|---|---|---|

Intermediate\_3



# 6. Core Cluster Architecture



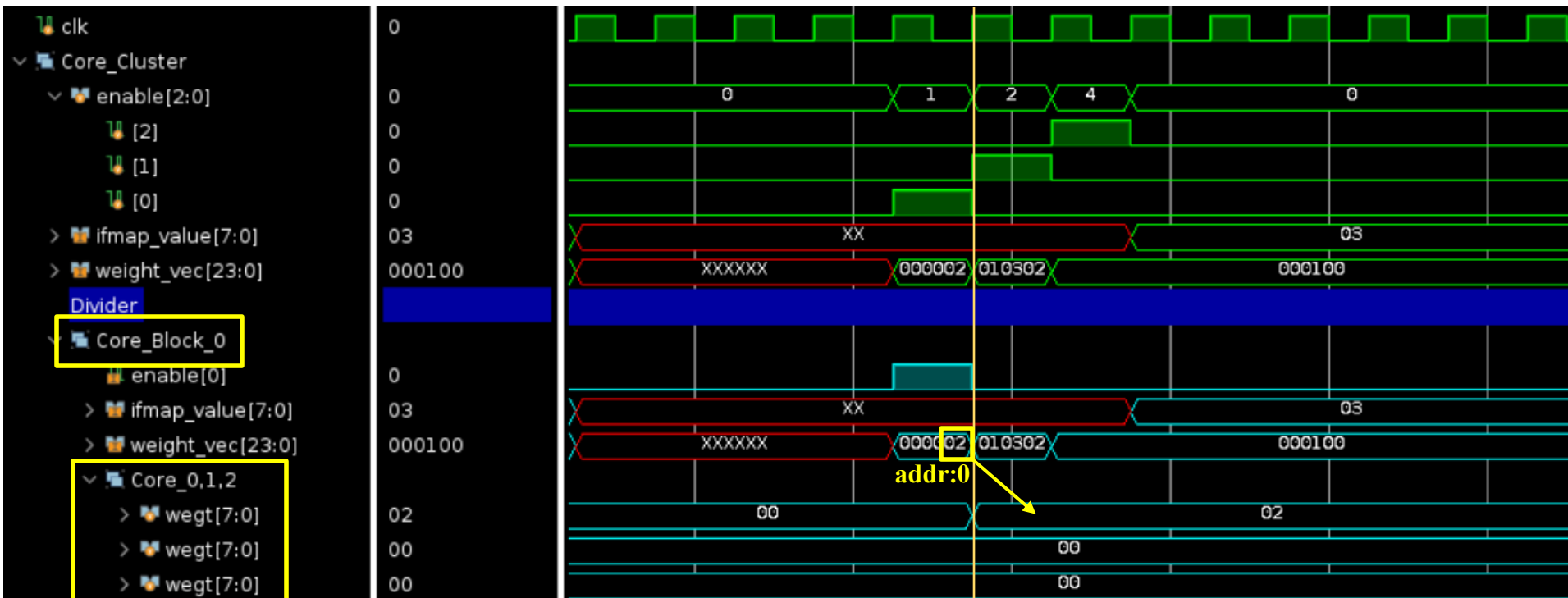
|   |   |
|---|---|
| 3 | 1 |
| 1 | 2 |

ifmap

|   |   |   |
|---|---|---|
| 2 | 0 | 0 |
| 2 | 3 | 1 |
| 0 | 1 | 0 |

filter

# 6. Core Cluster Architecture



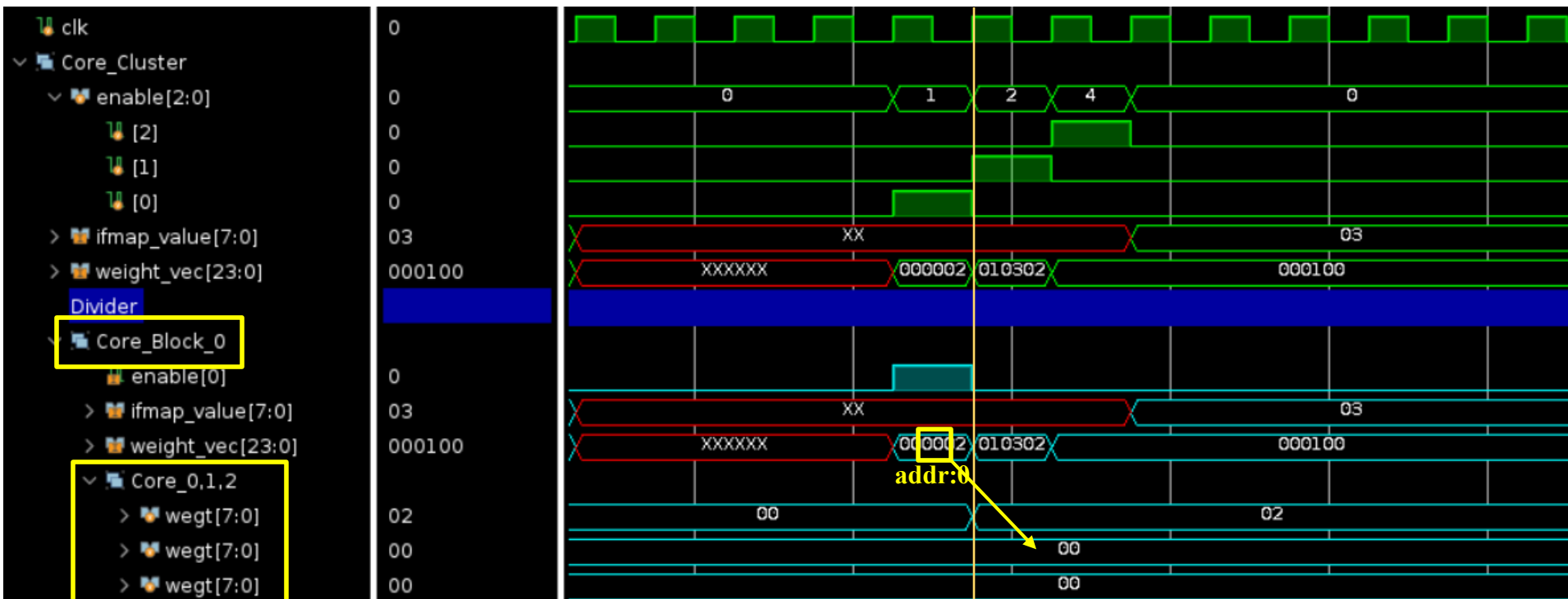
|   |   |
|---|---|
| 3 | 1 |
| 1 | 2 |

ifmap

|   |   |   |
|---|---|---|
| 2 | 0 | 0 |
| 2 | 3 | 1 |
| 0 | 1 | 0 |

filter

# 6. Core Cluster Architecture



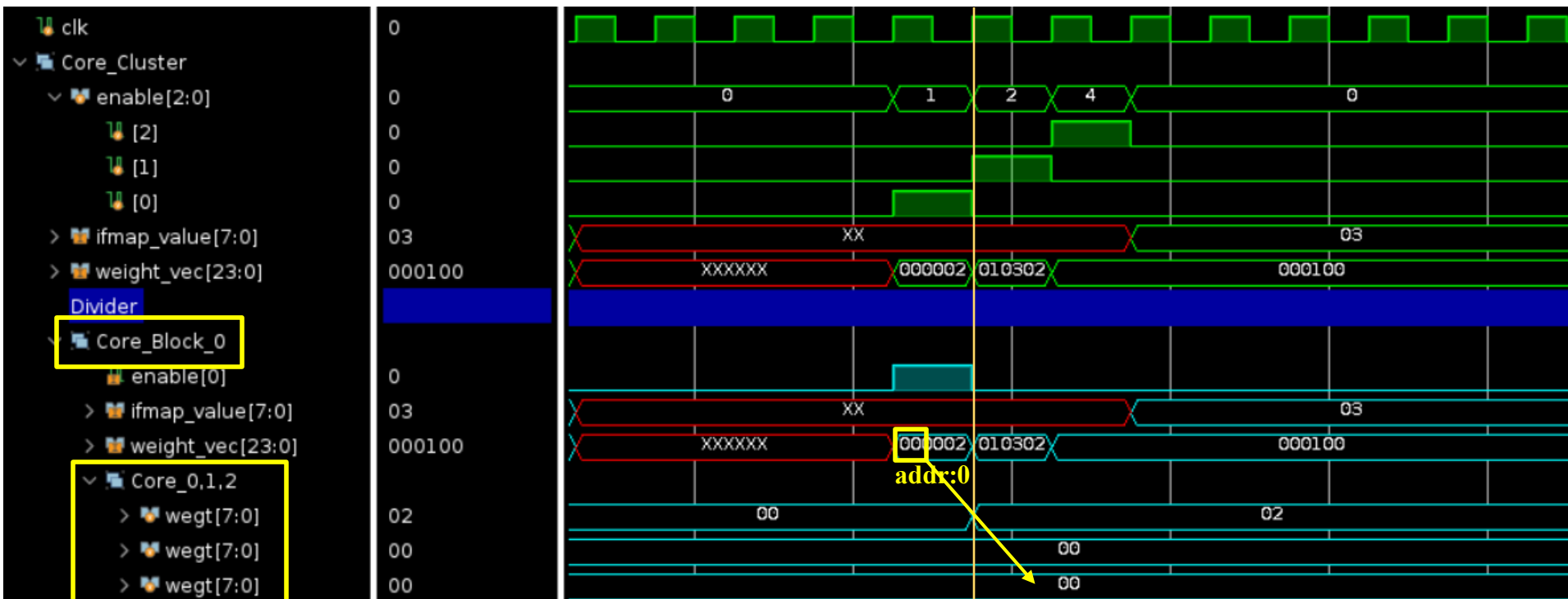
|   |   |
|---|---|
| 3 | 1 |
| 1 | 2 |

ifmap

|   |   |   |
|---|---|---|
| 2 | 0 | 0 |
| 2 | 3 | 1 |
| 0 | 1 | 0 |

filter

# 6. Core Cluster Architecture

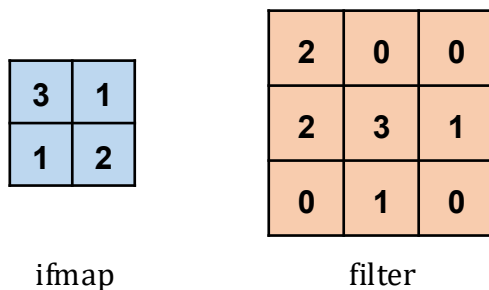
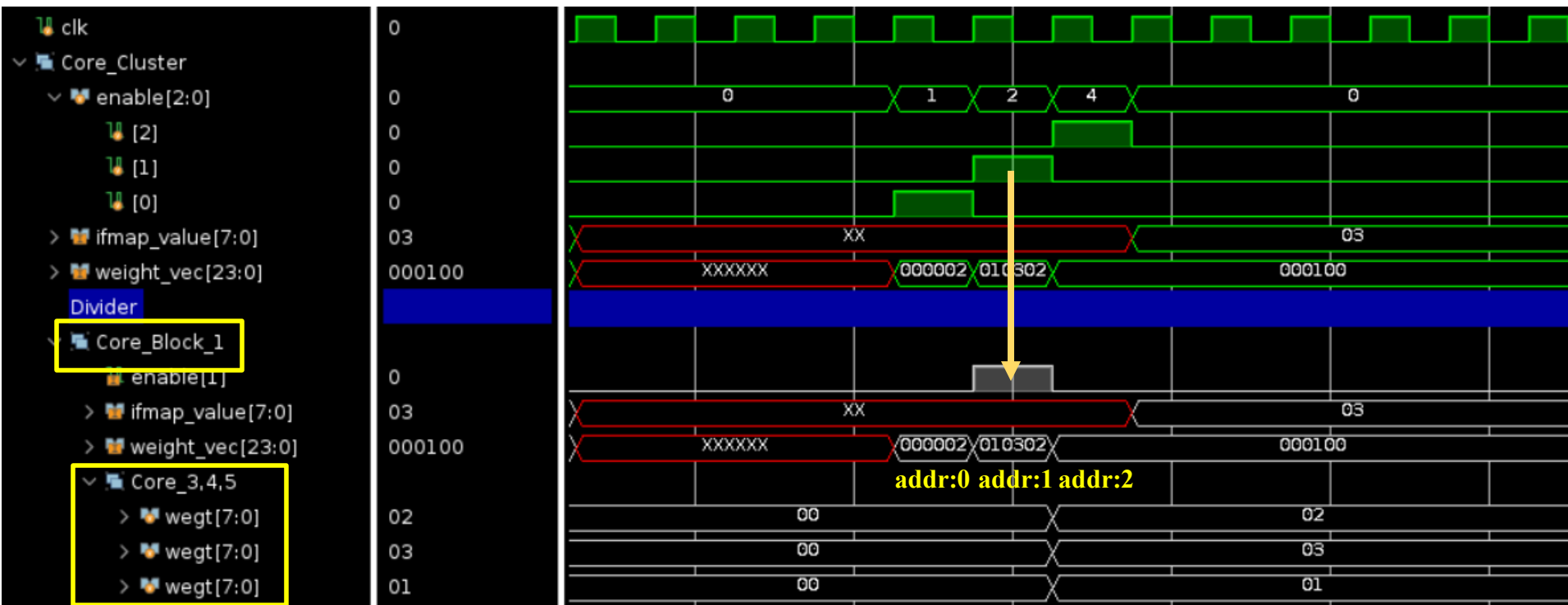


|   |   |
|---|---|
| 3 | 1 |
| 1 | 2 |

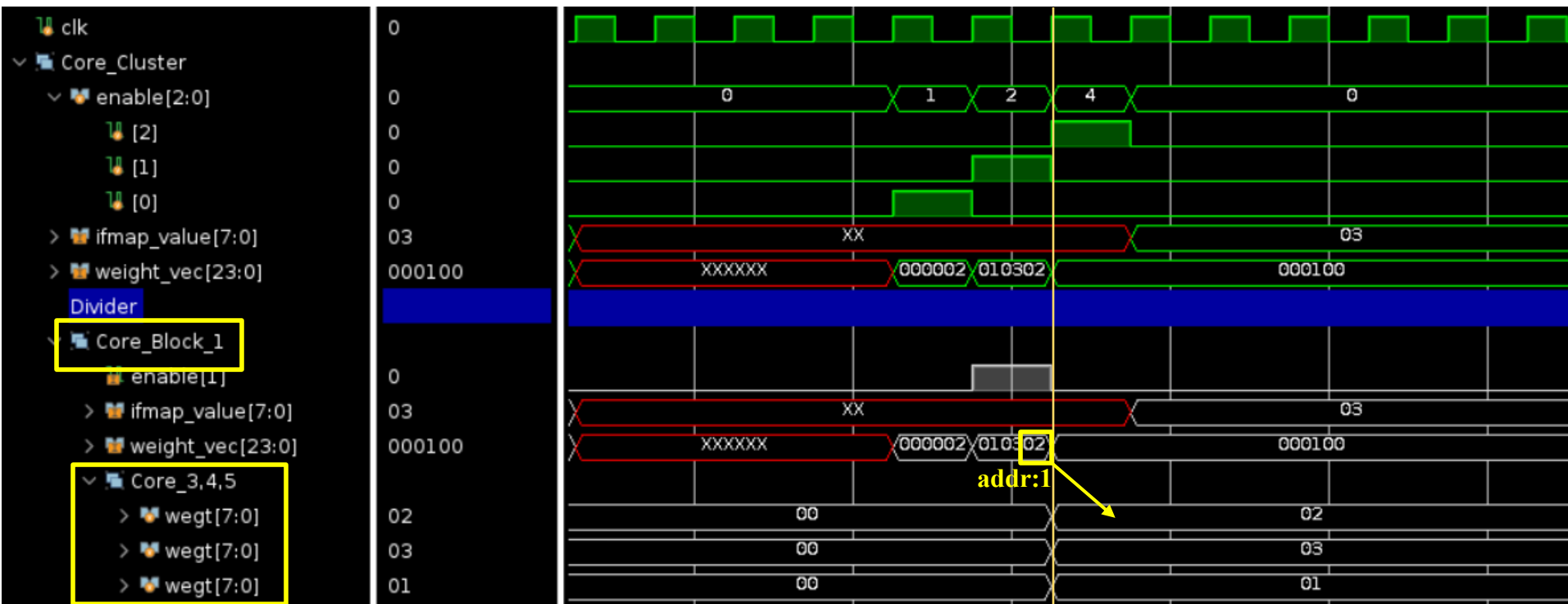
ifmap

|   |   |   |
|---|---|---|
| 2 | 0 | 0 |
| 2 | 3 | 1 |
| 0 | 1 | 0 |

filter



# 6. Core Cluster Architecture



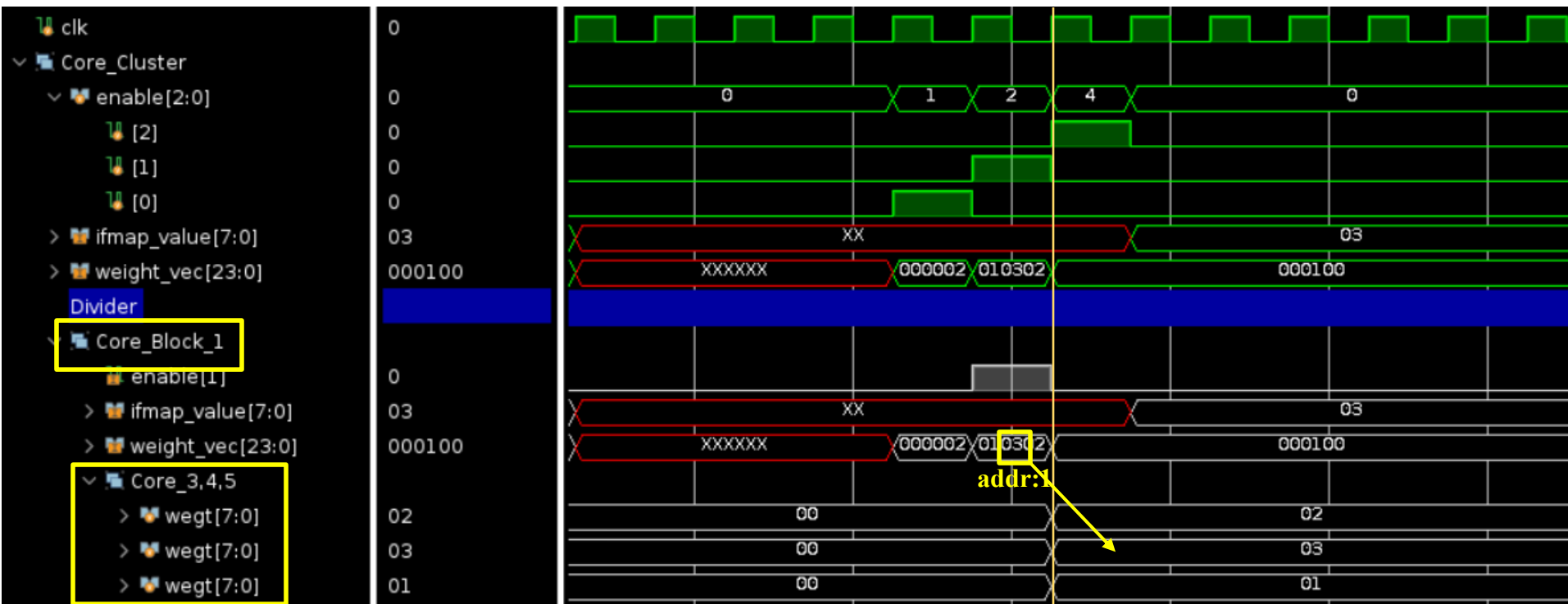
|   |   |
|---|---|
| 3 | 1 |
| 1 | 2 |

ifmap

|   |   |   |
|---|---|---|
| 2 | 0 | 0 |
| 2 | 3 | 1 |
| 0 | 1 | 0 |

filter

# 6. Core Cluster Architecture



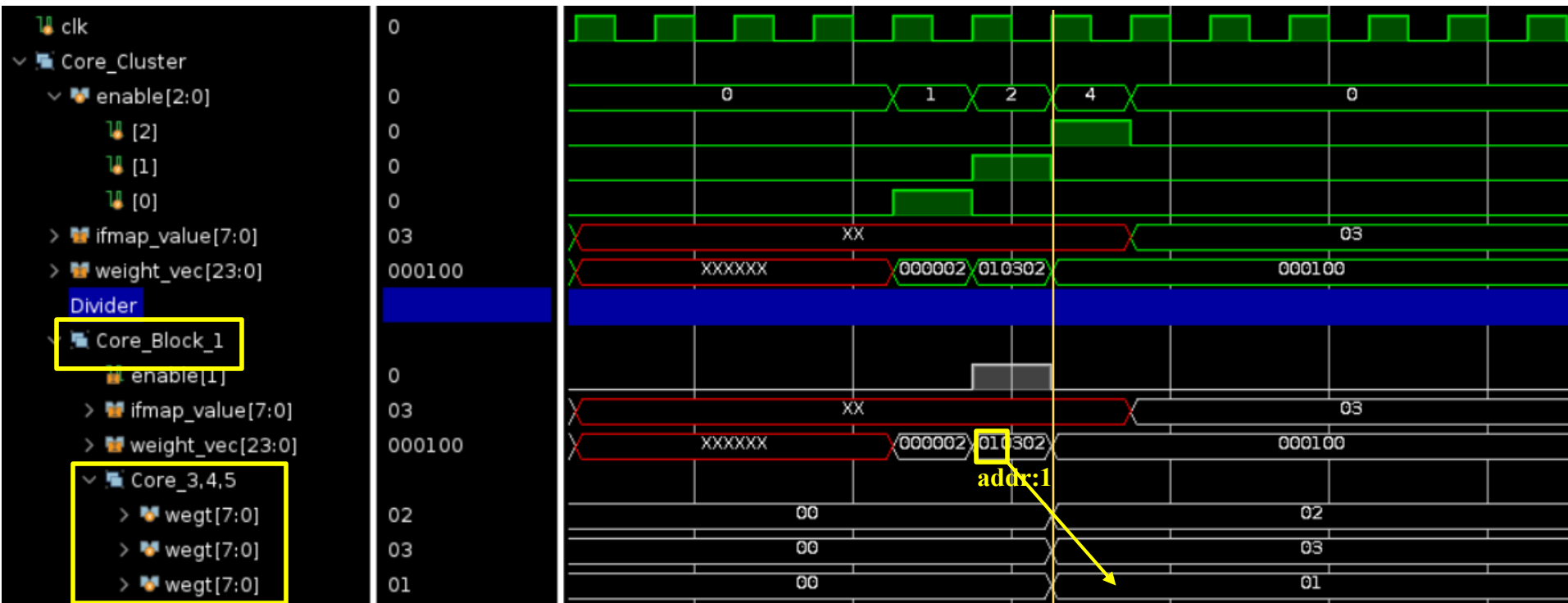
|   |   |
|---|---|
| 3 | 1 |
| 1 | 2 |

ifmap

|   |   |   |
|---|---|---|
| 2 | 0 | 0 |
| 2 | 3 | 1 |
| 0 | 1 | 0 |

filter

# 5. Core Cluster Architecture



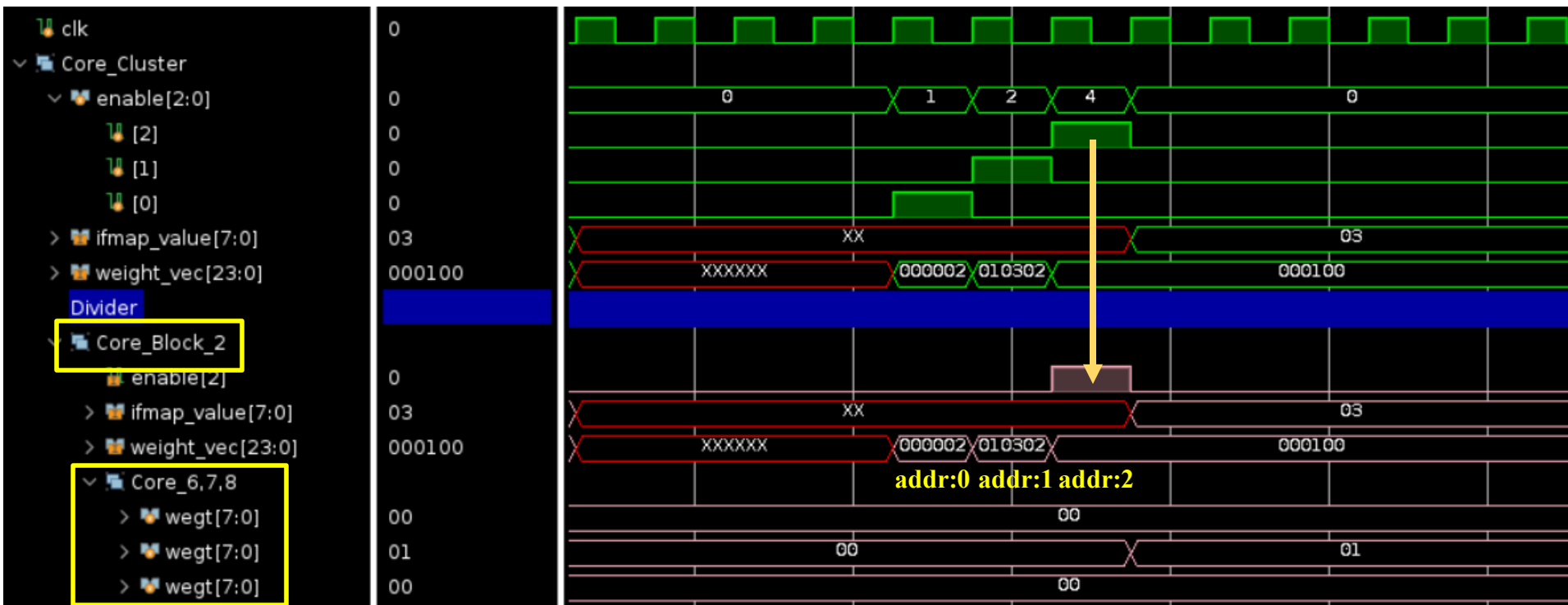
|   |   |
|---|---|
| 3 | 1 |
| 1 | 2 |

ifmap

|   |   |   |
|---|---|---|
| 2 | 0 | 0 |
| 2 | 3 | 1 |
| 0 | 1 | 0 |

filter

# 6. Core Cluster Architecture



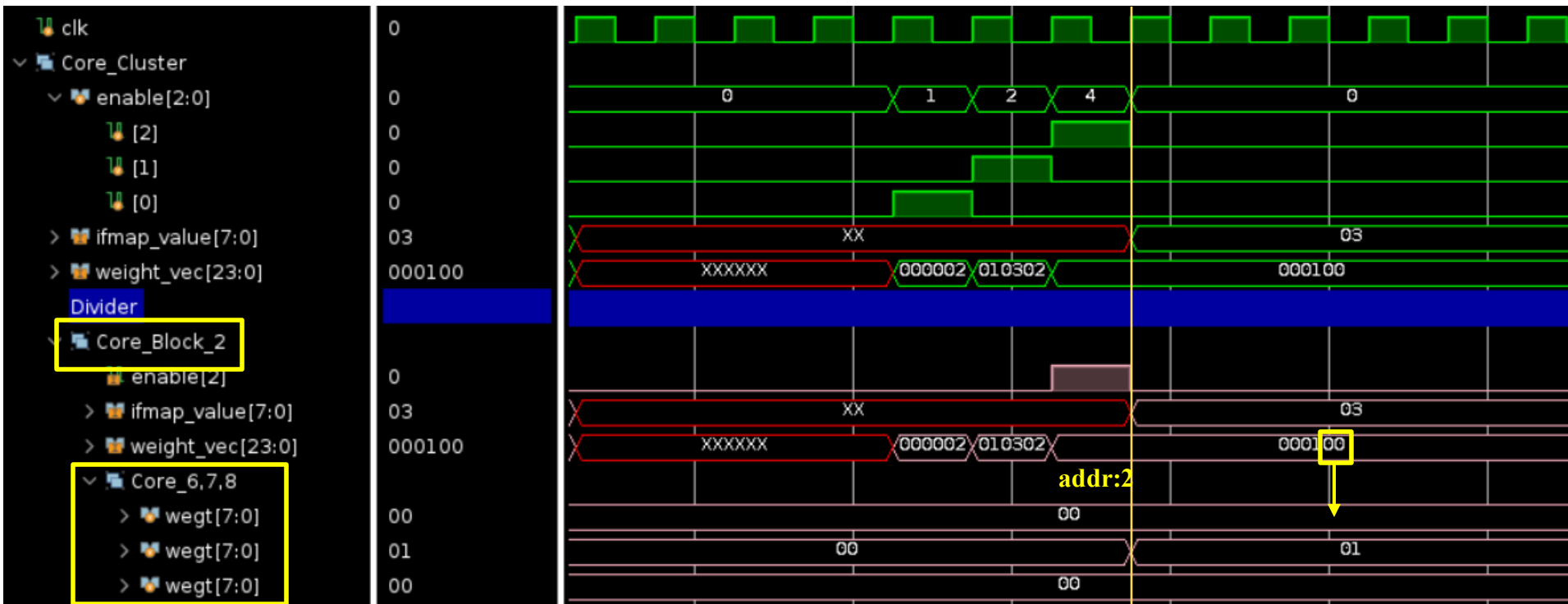
|   |   |
|---|---|
| 3 | 1 |
| 1 | 2 |

ifmap

|   |   |   |
|---|---|---|
| 2 | 0 | 0 |
| 2 | 3 | 1 |
| 0 | 1 | 0 |

filter

# 6. Core Cluster Architecture



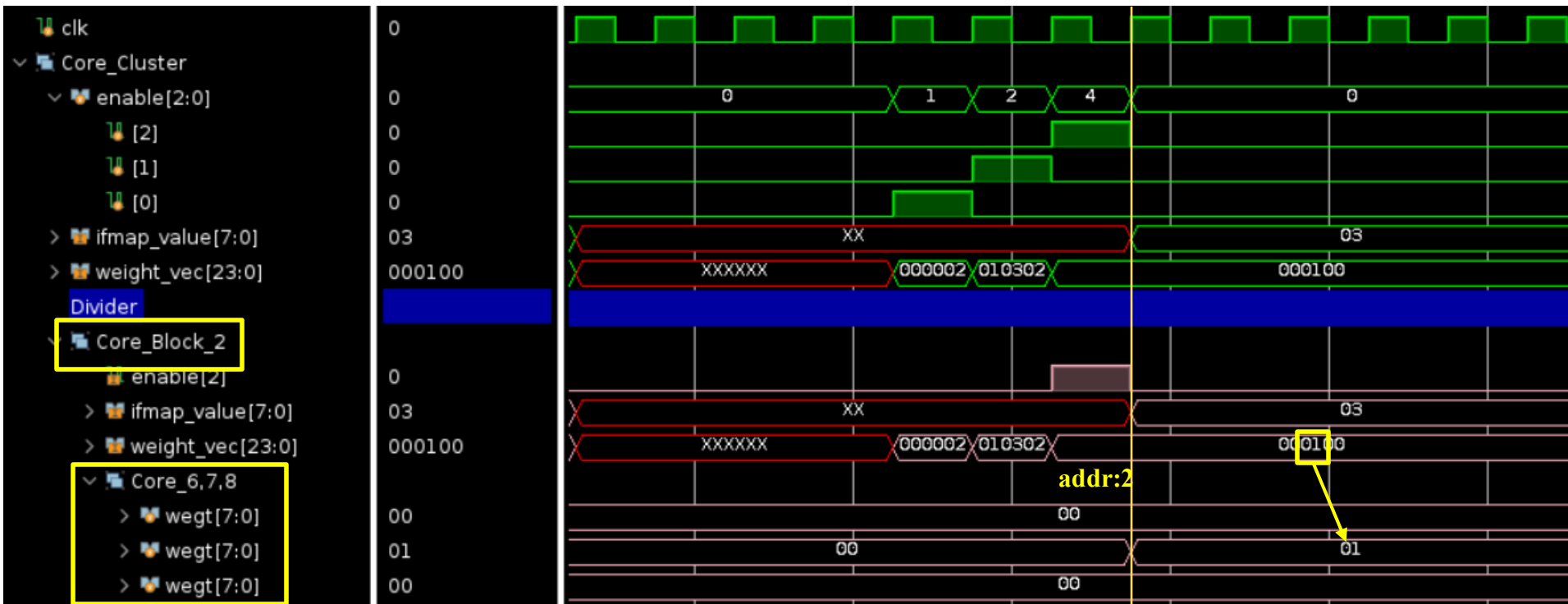
|   |   |
|---|---|
| 3 | 1 |
| 1 | 2 |

ifmap

|   |   |   |
|---|---|---|
| 2 | 0 | 0 |
| 2 | 3 | 1 |
| 0 | 1 | 0 |

filter

# 6. Core Cluster Architecture



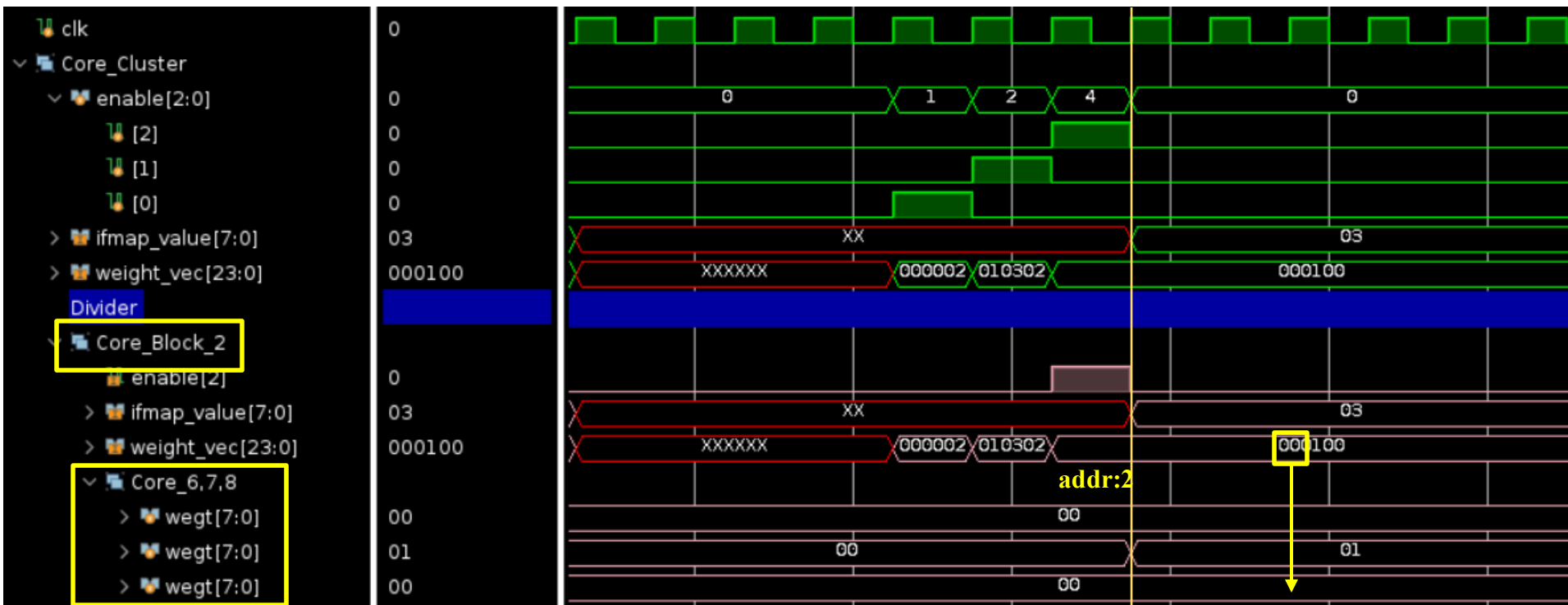
|   |   |
|---|---|
| 3 | 1 |
| 1 | 2 |

ifmap

|   |   |   |
|---|---|---|
| 2 | 0 | 0 |
| 2 | 3 | 1 |
| 0 | 1 | 0 |

filter

# 6. Core Cluster Architecture



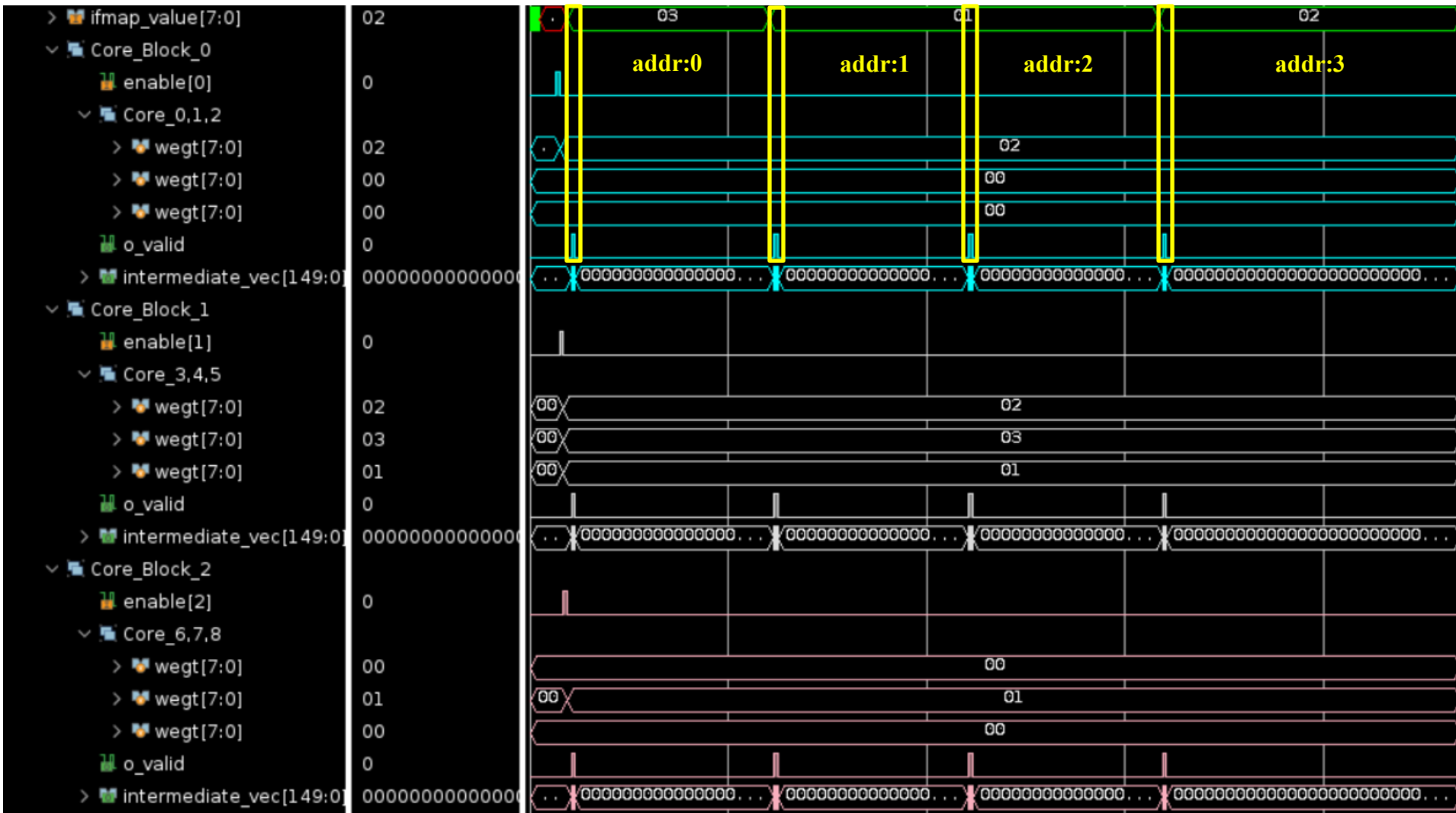
|   |   |
|---|---|
| 3 | 1 |
| 1 | 2 |

ifmap

|   |   |   |
|---|---|---|
| 2 | 0 | 0 |
| 2 | 3 | 1 |
| 0 | 1 | 0 |

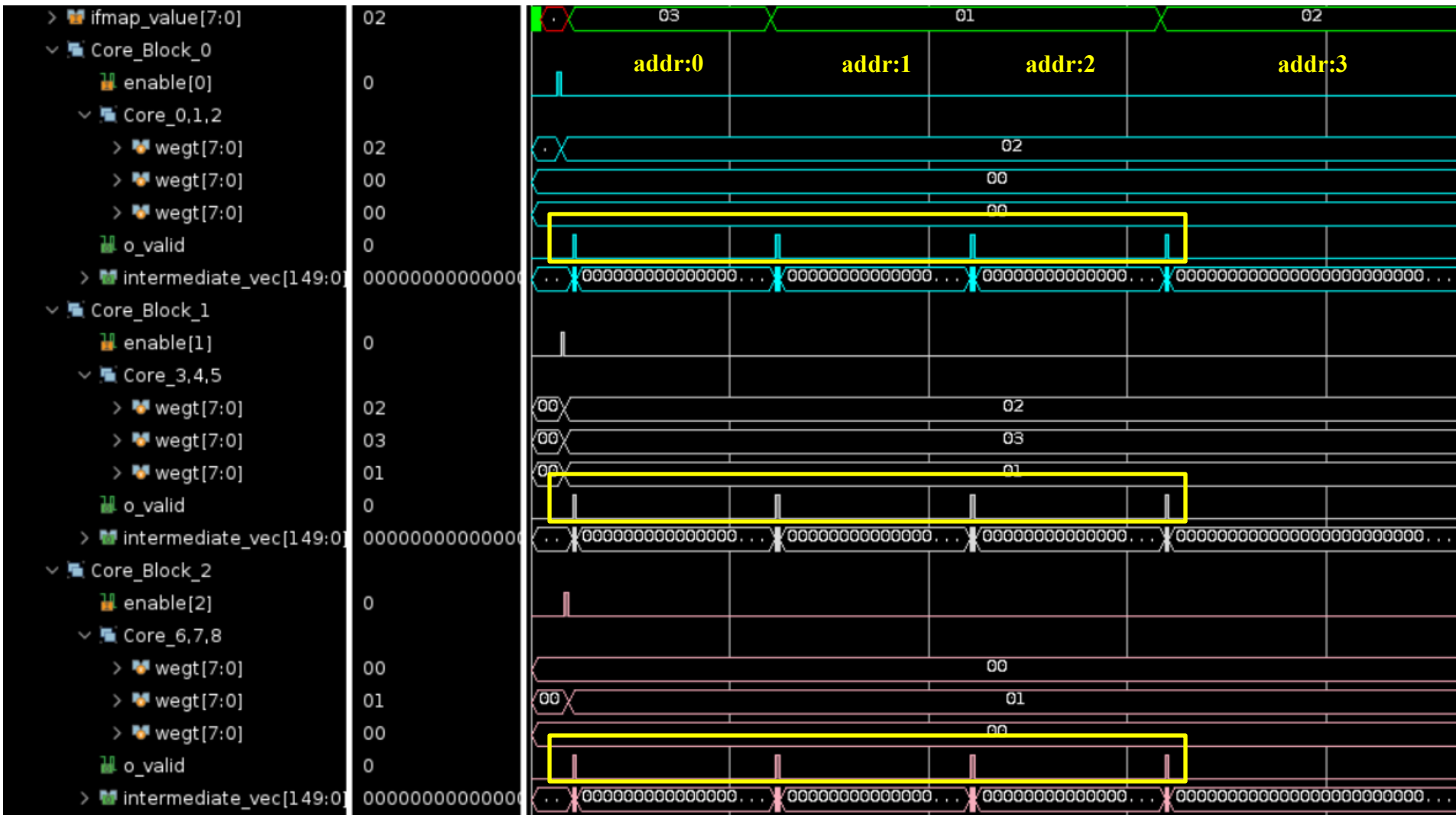
filter

## 6. Core Cluster Architecture



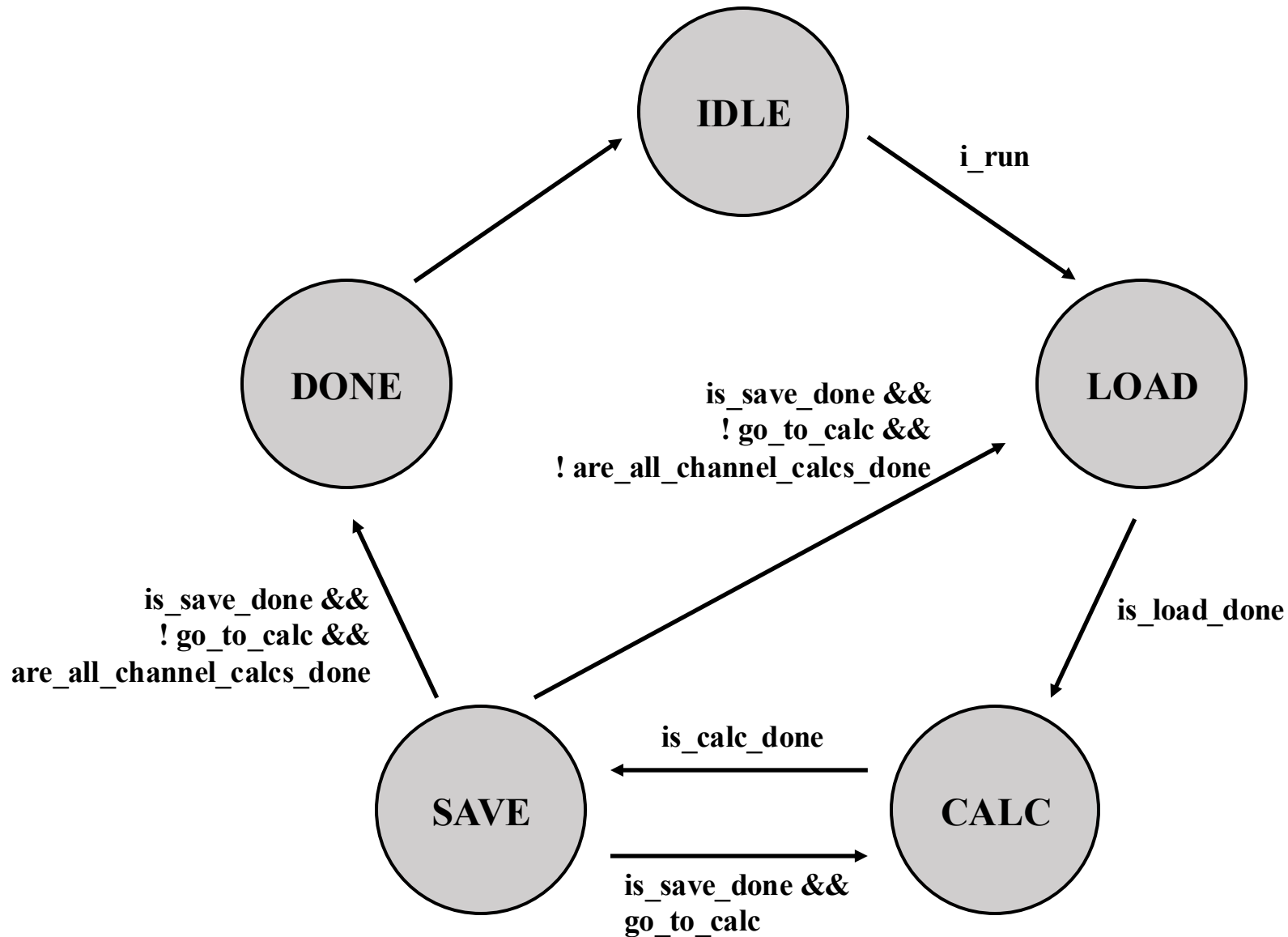
- The weight stored in each core remains unchanged, while only the ifmap values vary to generate the intermediate vectors.

# 6. Core Cluster Architecture



- The weight stored in each core remains unchanged, while only the ifmap values vary to generate the intermediate vectors.
- Each Core Block generates  $IH \times IW$  intermediate vectors.

## 7. CT\_module: FSM





Kumoh National Institute of Technology

```
// *_*_*_*_*_ STATE CONTROL SIGNAL *_*_*_*_*_
wire is_load_done;
wire is_calc_done;
wire go_to_calc;
wire is_single_out_channel;
wire are_all_channel_calcs_done;
//
assign is_load_done = enable[FW-1];
assign is_calc_done = o_calc && &o_valid;
assign go_to_calc = o_save && is_save_done && save_cnt_for_a_channel != (IW * IH) - 1;
assign is_single_out_channel = OUT_C == 1;
assign are_all_channel_calcs_done = \
o_save && is_save_done && (save_cnt_for_all_channels == IW * IH * IN_C * OUT_C - 1);
```



Kumoh National Institute of Technology

The diagram illustrates the hardware architecture of the proposed system, divided into two main components: the **CT\_module** and the **SAVE\_module**.

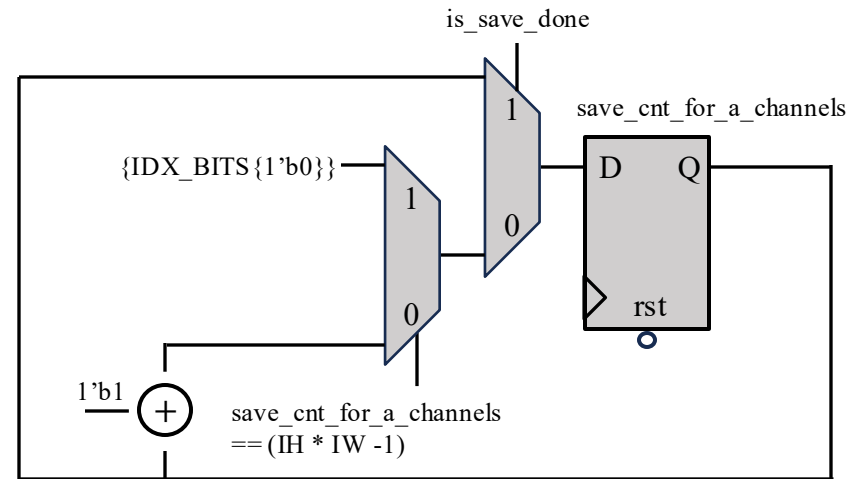
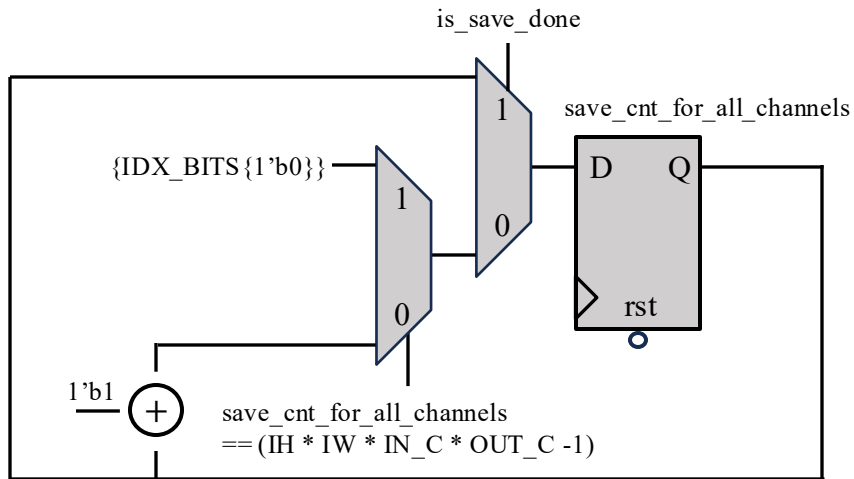
**CT\_module:** This module contains a **Core Cluster**, which is composed of three **Core Block**s. Each **Core Block** contains three **Core**s, for a total of nine cores.

**SAVE\_module:** This module is connected to the CT\_module and consists of two main parts:

- itmd (Instruction Trace Memory):** A vertical stack of memory cells that receive data from the CT\_module. It is connected to the **ofmap memory** via a series of horizontal bidirectional arrows.
- ofmap memory (Output Map Memory):** A memory structure with 14 entries, labeled **addr 0** through **addr 13**. It is connected to the **itmd** via a series of horizontal bidirectional arrows.

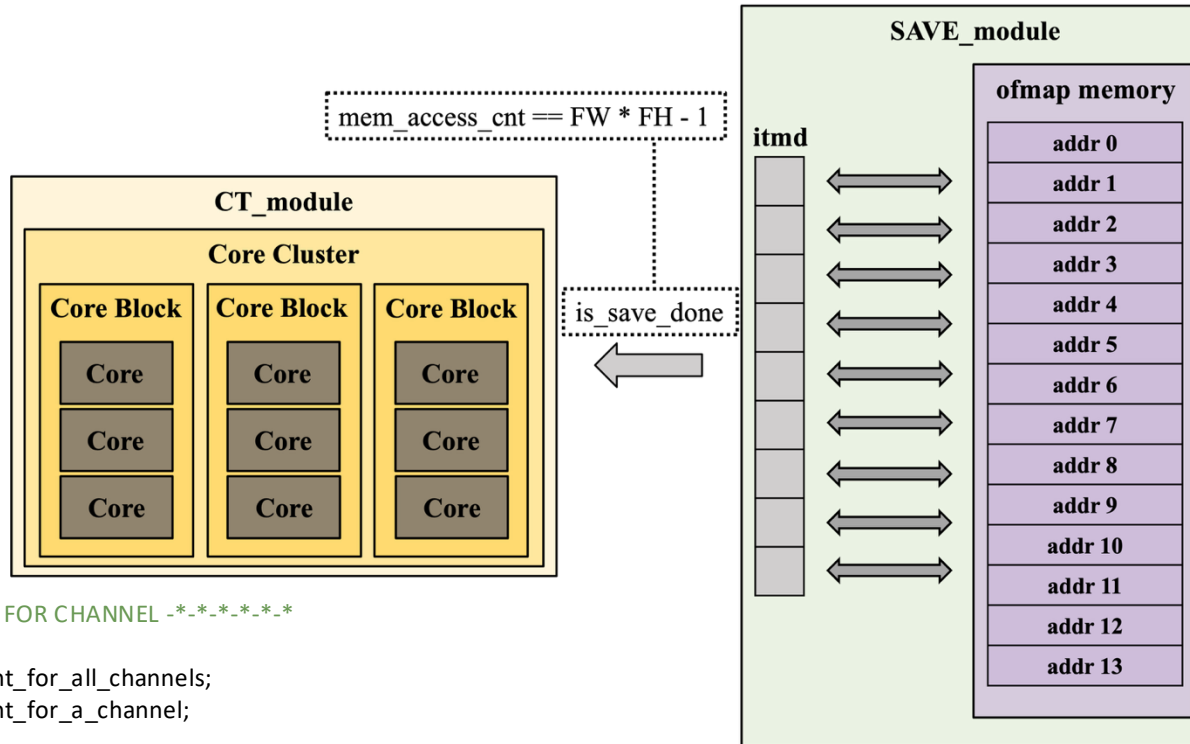
A dashed box indicates the calculation  $\text{mem\_access\_cnt} = \text{FW} * \text{FH} - 1$ , which is used to determine the **is\_save\_done** signal, represented by a large arrow pointing from the CT\_module to the SAVE\_module.

# 7. CT\_module: save\_counter

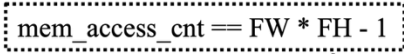


```
// *_*_*_*_* SAVE COUNT FOR CHANNEL *_*_*_*_*
//
reg [IDX_BITS-1 : 0] save_cnt_for_all_channels;
reg [IDX_BITS-1 : 0] save_cnt_for_a_channel;
//
always @(posedge clk or negedge reset_n) begin
    if(!reset_n) begin
        save_cnt_for_all_channels <= {IDX_BITS{1'b0}};
        save_cnt_for_a_channel <= {IDX_BITS{1'b0}};
    end
//
    else if(is_save_done) begin
        if(save_cnt_for_all_channels == (IH * IW * IN_C * OUT_C) - 1) save_cnt_for_all_channels <= {IDX_BITS{1'b0}};
        else save_cnt_for_all_channels <= save_cnt_for_all_channels + 1'b1;
//
        if(save_cnt_for_a_channel == (IH * IW) - 1) save_cnt_for_a_channel <= {IDX_BITS{1'b0}};
        else save_cnt_for_a_channel <= save_cnt_for_a_channel + 1'b1;
    end
end
```

# 7. CT\_module: save\_counter



```
// *_*_*_*_*_* SAVE COUNT FOR CHANNEL *_*_*_*_*_*
//
reg [IDX_BITS-1 : 0] save_cnt_for_all_channels;
reg [IDX_BITS-1 : 0] save_cnt_for_a_channel;
//
always @(posedge clk or negedge reset_n) begin
    if(!reset_n) begin
        save_cnt_for_all_channels <= {IDX_BITS{1'b0}};
        save_cnt_for_a_channel <= {IDX_BITS{1'b0}};
    end
//
    else if(is_save_done) begin
        if(save_cnt_for_all_channels == (IH * IW * IN_C * OUT_C) - 1) save_cnt_for_all_channels <= {IDX_BITS{1'b0}};
        else save_cnt_for_all_channels <= save_cnt_for_all_channels + 1'b1;
    end
//
    if(save_cnt_for_a_channel == (IH * IW) - 1) save_cnt_for_a_channel <= {IDX_BITS{1'b0}};
    else save_cnt_for_a_channel <= save_cnt_for_a_channel + 1'b1;
end
end
```



```
is_save_done
```

//

```
reg [IDX_BITS-1 : 0] save_cnt_for_a_channel;
```

//

end

//

```
done) begin
```

```
else save cnt for all channels <= save cnt for all channels + 1'b1;
```

//

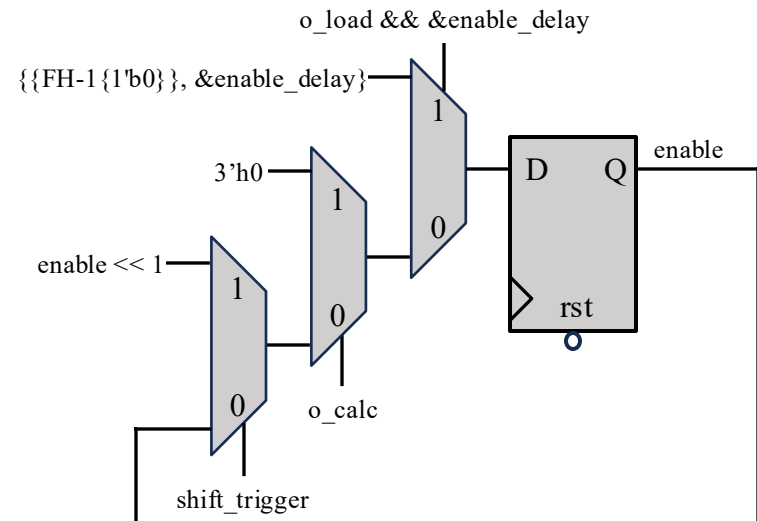
```
else save cnt for a channel <= save cnt for a channel + 1'b1;
```

end

end



```
// *_*_*_*_*_* ENABLE GENERATOR *_*_*_*_*_*  
reg [FH -1 : 0] enable;  
reg [ENABLE_CNT_BITS -1 : 0] enable_cnt; // this cnt uses for wegt_ce0.  
reg [ENABLE_DELAY -1 : 0] enable_delay;  
reg shift_trigger; // this signal uses for shift of enable.  
//  
//  
  
always @(posedge clk or negedge reset_n) begin  
    if(!reset_n) begin  
        enable <= {FH{1'b0}};  
        enable_cnt <= {ENABLE_CNT_BITS{1'b0}};  
    end  
    else if(o_load && &enable_delay) enable <= {{FH-1{1'b0}}, &enable_delay};  
    else if(o_calc) begin  
        enable <= {FH{1'b0}};  
        enable_cnt <= {ENABLE_CNT_BITS{1'b0}};  
    end  
    else if(shift_trigger)begin  
        enable <= enable << 1;  
    end  
    //  
    if(&enable_delay || enable!=0) enable_cnt <= enable_cnt + 1'b1;  
end  
//  
  
always @(posedge clk or negedge reset_n) begin  
    if(!reset_n) begin  
        enable_delay <= {ENABLE_DELAY{1'b0}};  
        shift_trigger <= 1'b0;  
    end  
    else if(o_load && &enable_delay) begin  
        enable_delay <= {ENABLE_DELAY{1'b0}};  
        shift_trigger <= 1'b1;  
    end  
    else if(!o_load) begin  
        enable_delay <= {ENABLE_DELAY{1'b0}};  
        shift_trigger <= 1'b0;  
    end  
    else if(o_load && !shift_trigger) enable_delay <= {enable_delay[FH-1 : 0], o_load};  
end
```

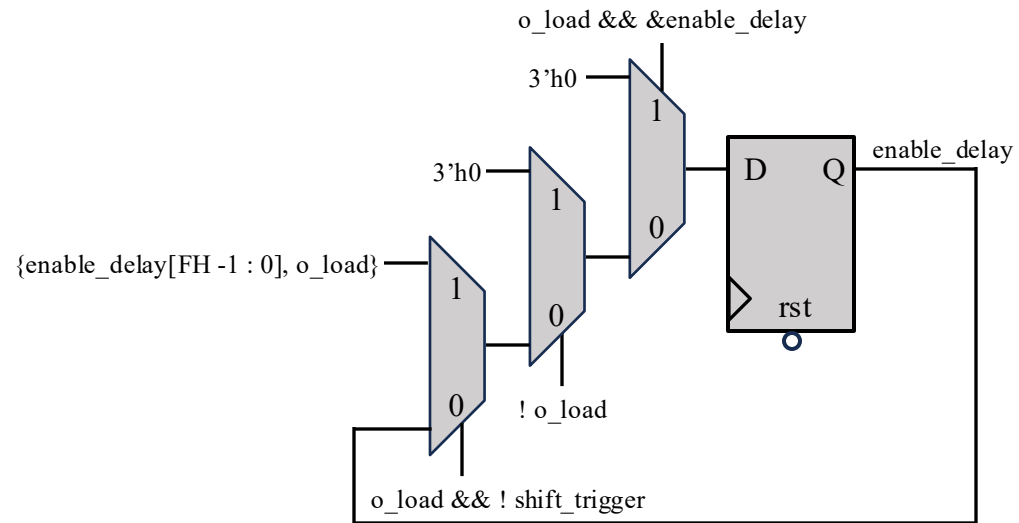




```
// *_*_*_*_*_* ENABLE GENERATOR *_*_*_*_*_*
reg [FH -1 : 0] enable;
reg [ENABLE_CNT_BITS -1 : 0] enable_cnt; // this cnt uses for wegt_ce0.
reg [ENABLE_DELAY -1 : 0] enable_delay;
reg shift_trigger; // this signal uses for shift of enable.
//
//

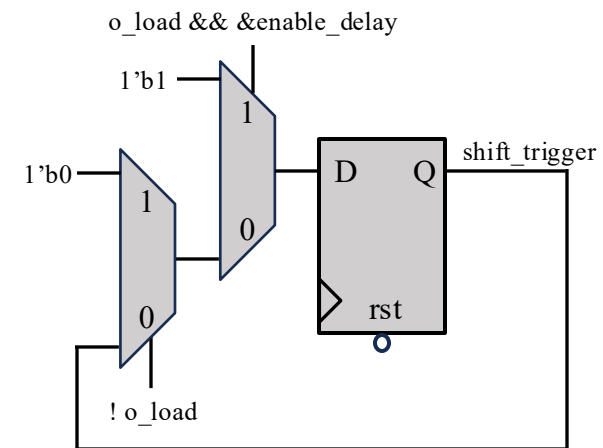
always @(posedge clk or negedge reset_n) begin
    if(!reset_n) begin
        enable <= {FH{1'b0}};
        enable_cnt <= {ENABLE_CNT_BITS{1'b0}};
    end
    else if(o_load && &enable_delay) enable <= {{FH-1{1'b0}}, &enable_delay};
    else if(o_calc) begin
        enable <= {FH{1'b0}};
        enable_cnt <= {ENABLE_CNT_BITS{1'b0}};
    end
    else if(shift_trigger)begin
        enable <= enable << 1;
    end
    //
    if(&enable_delay || enable!=0) enable_cnt <= enable_cnt + 1'b1;
end
//

always @(posedge clk or negedge reset_n) begin
    if(!reset_n) begin
        enable_delay <= {ENABLE_DELAY{1'b0}};
        shift_trigger <= 1'b0;
    end
    else if(o_load && &enable_delay) begin
        enable_delay <= {ENABLE_DELAY{1'b0}};
        shift_trigger <= 1'b1;
    end
    else if(!o_load) begin
        enable_delay <= {ENABLE_DELAY{1'b0}};
        shift_trigger <= 1'b0;
    end
    else if(o_load && !shift_trigger) enable_delay <= {enable_delay[FH-1 : 0], o_load};
end
```



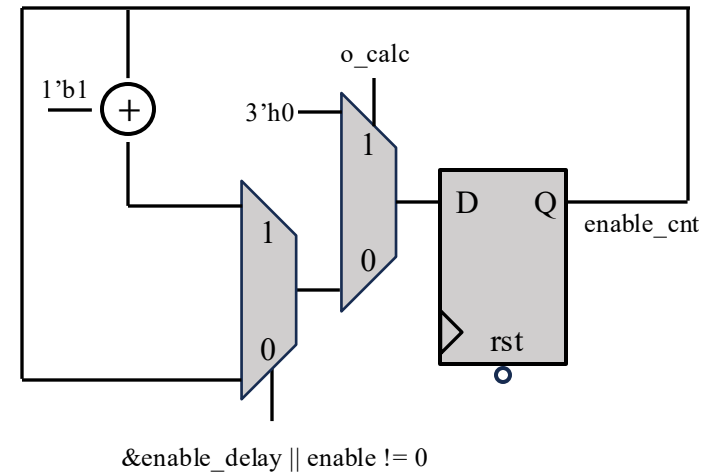


```
// *_*_*_*_*_* ENABLE GENERATOR *_*_*_*_*_*  
reg [FH -1 : 0] enable;  
reg [ENABLE_CNT_BITS -1 : 0] enable_cnt; // this cnt uses for wegt_ce0.  
reg [ENABLE_DELAY -1 : 0] enable_delay;  
reg shift_trigger; // this signal uses for shift of enable.  
//  
//  
  
always @(posedge clk or negedge reset_n) begin  
    if(!reset_n) begin  
        enable <= {FH{1'b0}};  
        enable_cnt <= {ENABLE_CNT_BITS{1'b0}};  
    end  
    else if(o_load && &enable_delay) enable <= {{FH-1{1'b0}}, &enable_delay};  
    else if(o_calc) begin  
        enable <= {FH{1'b0}};  
        enable_cnt <= {ENABLE_CNT_BITS{1'b0}};  
    end  
    else if(shift_trigger)begin  
        enable <= enable << 1;  
    end  
    //  
    if(&enable_delay || enable!=0) enable_cnt <= enable_cnt + 1'b1;  
end  
//  
  
always @(posedge clk or negedge reset_n) begin  
    if(!reset_n) begin  
        enable_delay <= {ENABLE_DELAY{1'b0}};  
        shift_trigger <= 1'b0;  
    end  
    else if(o_load && &enable_delay) begin  
        enable_delay <= {ENABLE_DELAY{1'b0}};  
        shift_trigger <= 1'b1;  
    end  
    else if(!o_load) begin  
        enable_delay <= {ENABLE_DELAY{1'b0}};  
        shift_trigger <= 1'b0;  
    end  
    else if(o_load && !shift_trigger) enable_delay <= {enable_delay[FH-1 : 0], o_load};  
end
```



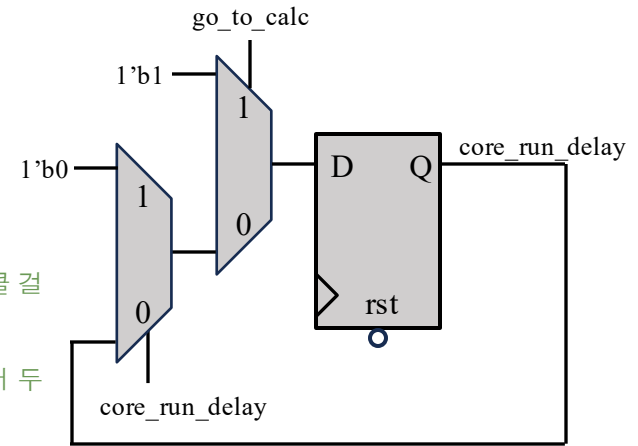


```
assign wegt_ce0 = o_load && (enable_delay[0] || (shift_trigger && enable_cnt < FW-1));
```

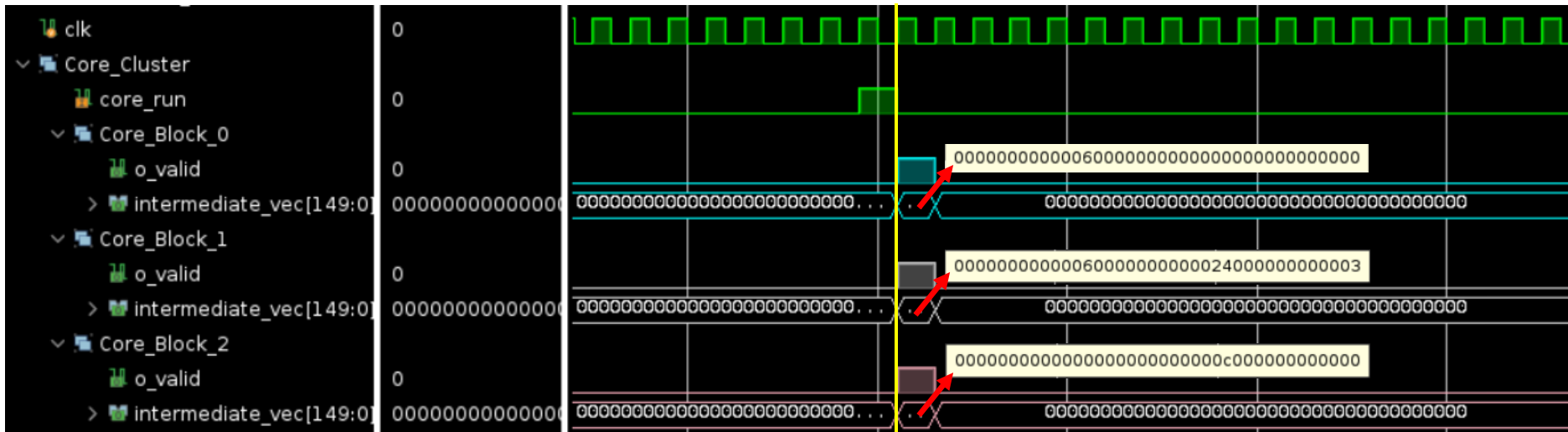




// 그렇기 때문에 core\_run을 한 cycle delay 시켜야 read 타이밍이랑 맞음



# 7. CT\_module: core\_run generator



# 7. CT\_module: ifmap idx

```
// *_*_*_*_*_*_*_* ifmap mem addr *_*_*_*_*_*_*_*
reg [IDX_BITS-1:0] row_idx;
reg [IDX_BITS-1:0] col_idx;
reg [IDX_BITS-1:0] in_channel_idx;
reg [IDX_BITS-1:0] out_channel_idx;
//
always @(posedge clk or negedge reset_n) begin
    if(!reset_n) begin
        row_idx <= {IDX_BITS{1'b0}};
        col_idx <= {IDX_BITS{1'b0}};
        in_channel_idx <= {IDX_BITS{1'b0}};
        out_channel_idx <= {IDX_BITS{1'b0}};
    end
    else if(core_run) begin
        if(col_idx == IW-1) begin
            col_idx <= {IDX_BITS{1'b0}};
            //
            if(row_idx == IH-1) begin
                row_idx <= {IDX_BITS{1'b0}};
                //
                if(out_channel_idx == OUT_C-1) begin
                    out_channel_idx <= {IDX_BITS{1'b0}};
                    in_channel_idx <= in_channel_idx + 1'b1;
                end
                //
                else out_channel_idx <= out_channel_idx + 1'b1;
            end
            else row_idx <= row_idx + 1'b1;
        end
        else col_idx <= col_idx + 1'b1;
    end
end
```

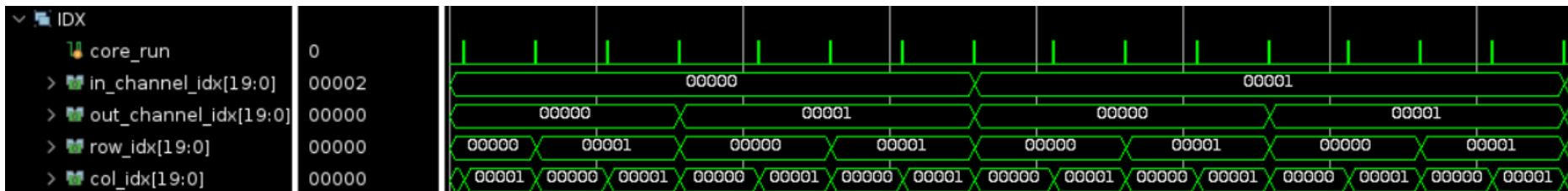
// The Verilog code above performs the same function as the following quadruple for-loop.

```
// C language
int row_idx;
int col_idx;
int in_channel_idx;
int out_channel_idx;

for(in_channel_idx = 0; in_channel_idx < IN_C; in_channel_idx++)
    for(out_channel_idx = 0; out_channel_idx < OUT_C; out_channel_idx++)
        for(row_idx = 0; row_idx < IH; row_idx++)
            for(col_idx = 0; col_idx < IW; col_idx++)
```

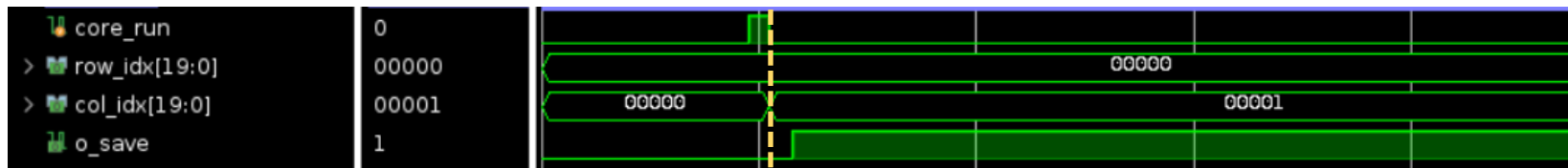
# 7. CT\_module: ifmap idx

```
// *_*_*_*_*_* ifmap mem addr *_*_*_*_*_*
reg [IDX_BITS-1:0] row_idx;
reg [IDX_BITS-1:0] col_idx;
reg [IDX_BITS-1:0] in_channel_idx;
reg [IDX_BITS-1:0] out_channel_idx;
//
always @(posedge clk or negedge reset_n) begin
    if(!reset_n) begin
        row_idx <= {IDX_BITS{1'b0}};
        col_idx <= {IDX_BITS{1'b0}};
        in_channel_idx <= {IDX_BITS{1'b0}};
        out_channel_idx <= {IDX_BITS{1'b0}};
    end
    else if(core_run) begin
        if(col_idx == IW-1) begin
            col_idx <= {IDX_BITS{1'b0}};
            //
            if(row_idx == IH-1) begin
                row_idx <= {IDX_BITS{1'b0}};
                //
                if(out_channel_idx == OUT_C-1) begin
                    out_channel_idx <= {IDX_BITS{1'b0}};
                    in_channel_idx <= in_channel_idx + 1'b1;
                end
                //
                else out_channel_idx <= out_channel_idx + 1'b1;
            end
            else row_idx <= row_idx + 1'b1;
        end
        else col_idx <= col_idx + 1'b1;
    end
end
```



# 7. CT\_module: idx capture

```
// *_*_*_*_*_*_ idx capture *_*_*_*_*_*
reg [DELAY * IDX_BITS - 1 : 0] captured_ifmap_col;
reg [DELAY * IDX_BITS - 1 : 0] captured_ifmap_row;
reg [DELAY * IDX_BITS - 1 : 0] captured_ifmap_in_channel;
reg [DELAY * IDX_BITS - 1 : 0] captured_ifmap_out_channel;
//
always @(posedge clk or negedge reset_n) begin
    if(!reset_n) begin
        captured_ifmap_col <= {DELAY * IDX_BITS{1'b0}};
        captured_ifmap_row <= {DELAY * IDX_BITS{1'b0}};
        captured_ifmap_in_channel <= {DELAY * IDX_BITS{1'b0}};
        captured_ifmap_out_channel <= {DELAY * IDX_BITS{1'b0}};
    end
    else if(i_accum_from_SAVE && mem_access_cnt == (FW * FH - 1)) begin
        if(DELAY == 1) begin
            captured_ifmap_col <= col_idx;
            captured_ifmap_row <= row_idx;
            captured_ifmap_in_channel <= in_channel_idx;
            captured_ifmap_out_channel <= out_channel_idx;
        end
        else begin
            captured_ifmap_col <= {captured_ifmap_col[(DELAY-1) * IDX_BITS - 1 : 0], col_idx};
            captured_ifmap_row <= {captured_ifmap_row[(DELAY-1) * IDX_BITS - 1 : 0], row_idx};
            captured_ifmap_in_channel <= {captured_ifmap_in_channel[(DELAY-1) * IDX_BITS - 1 : 0], in_channel_idx};
            captured_ifmap_out_channel <= {captured_ifmap_out_channel[(DELAY-1) * IDX_BITS - 1 : 0], out_channel_idx};
        end
    end
end
```



- The value of ifmap(0,0) is calculated in the CT\_module.
- However, when the result is saved in the SAVE\_module, col\_idx and row\_idx have already changed.
- To prevent this issue, each index is captured.

# 7. CT\_module: idx capture

```
// *_*_*_*_*_*_ idx capture *_*_*_*_*_*
reg [DELAY * IDX_BITS - 1 : 0] captured_ifmap_col;
reg [DELAY * IDX_BITS - 1 : 0] captured_ifmap_row;
reg [DELAY * IDX_BITS - 1 : 0] captured_ifmap_in_channel;
reg [DELAY * IDX_BITS - 1 : 0] captured_ifmap_out_channel;
//
```

```
always @(posedge clk or negedge reset_n) begin
```

```
  if(!reset_n) begin
```

```
    captured_ifmap_col <= {DELAY * IDX_BITS{1'b0}};
```

```
    captured_ifmap_row <= {DELAY * IDX_BITS{1'b0}};
```

```
    captured_ifmap_in_channel <= {DELAY * IDX_BITS{1'b0}};
```

```
    captured_ifmap_out_channel <= {DELAY * IDX_BITS{1'b0}};
```

```
  end
```

```
  else if(i_accum_from_SAVE && mem_access_cnt == (FW * FH - 1)) begin
```

```
    if(DELAY == 1) begin
```

```
      captured_ifmap_col <= col_idx;
```

```
      captured_ifmap_row <= row_idx;
```

```
      captured_ifmap_in_channel <= in_channel_idx;
```

```
      captured_ifmap_out_channel <= out_channel_idx;
```

```
    end
```

```
  else begin
```

```
    captured_ifmap_col <= {captured_ifmap_col[(DELAY-1) * IDX_BITS - 1 : 0], col_idx};
```

```
    captured_ifmap_row <= {captured_ifmap_row[(DELAY-1) * IDX_BITS - 1 : 0], row_idx};
```

```
    captured_ifmap_in_channel <= {captured_ifmap_in_channel[(DELAY-1) * IDX_BITS - 1 : 0], in_channel_idx};
```

```
    captured_ifmap_out_channel <= {captured_ifmap_out_channel[(DELAY-1) * IDX_BITS - 1 : 0], out_channel_idx};
```

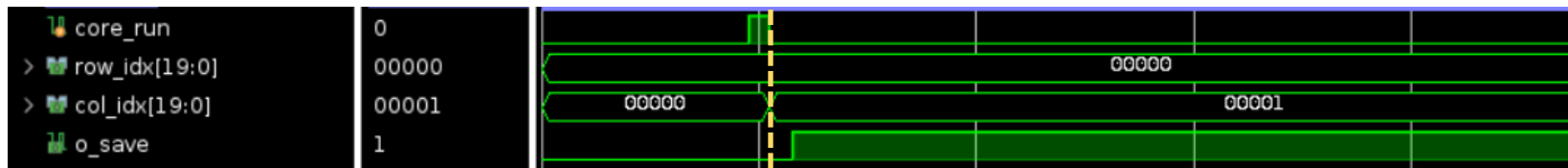
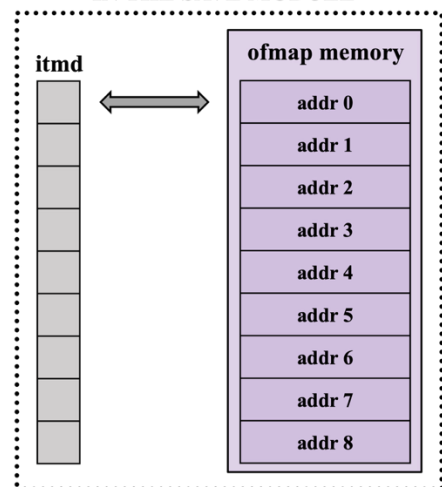
```
  end
```

```
end
```

```
end
```

mem\_access\_cnt = 1

IN THE SAVE MODULE



- The value of ifmap(0,0) is calculated in the CT\_module.
- However, when the result is saved in the SAVE\_module, col\_idx and row\_idx have already changed.
- To prevent this issue, each index is captured.

# 7. CT\_module: idx capture

```
// *_*_*_*_*_*_ idx capture *_*_*_*_*_*
reg [DELAY * IDX_BITS - 1 : 0] captured_ifmap_col;
reg [DELAY * IDX_BITS - 1 : 0] captured_ifmap_row;
reg [DELAY * IDX_BITS - 1 : 0] captured_ifmap_in_channel;
reg [DELAY * IDX_BITS - 1 : 0] captured_ifmap_out_channel;
//
```

```
always @(posedge clk or negedge reset_n) begin
```

```
  if(!reset_n) begin
```

```
    captured_ifmap_col <= {DELAY * IDX_BITS{1'b0}};
```

```
    captured_ifmap_row <= {DELAY * IDX_BITS{1'b0}};
```

```
    captured_ifmap_in_channel <= {DELAY * IDX_BITS{1'b0}};
```

```
    captured_ifmap_out_channel <= {DELAY * IDX_BITS{1'b0}};
```

```
  end
```

```
  else if(i_accum_from_SAVE && mem_access_cnt == (FW * FH - 1)) begin
```

```
    if(DELAY == 1) begin
```

```
      captured_ifmap_col <= col_idx;
```

```
      captured_ifmap_row <= row_idx;
```

```
      captured_ifmap_in_channel <= in_channel_idx;
```

```
      captured_ifmap_out_channel <= out_channel_idx;
```

```
    end
```

```
  else begin
```

```
    captured_ifmap_col <= {captured_ifmap_col[(DELAY-1) * IDX_BITS - 1 : 0], col_idx};
```

```
    captured_ifmap_row <= {captured_ifmap_row[(DELAY-1) * IDX_BITS - 1 : 0], row_idx};
```

```
    captured_ifmap_in_channel <= {captured_ifmap_in_channel[(DELAY-1) * IDX_BITS - 1 : 0], in_channel_idx};
```

```
    captured_ifmap_out_channel <= {captured_ifmap_out_channel[(DELAY-1) * IDX_BITS - 1 : 0], out_channel_idx};
```

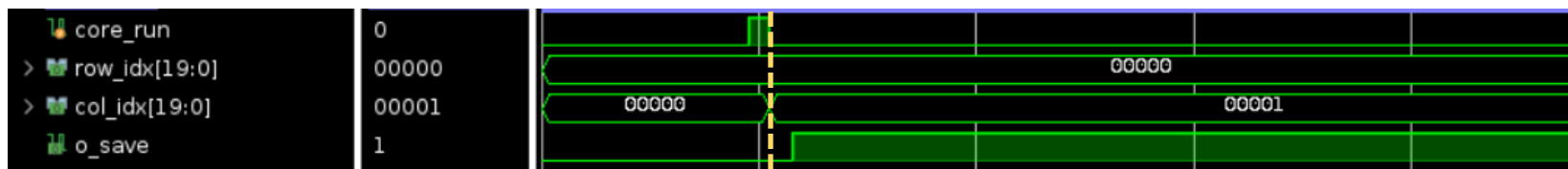
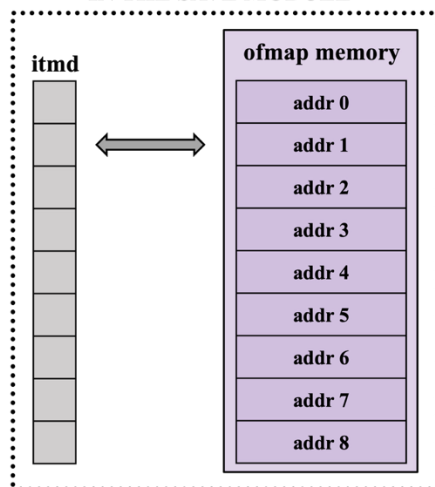
```
  end
```

```
end
```

```
end
```

mem\_access\_cnt = 2

IN THE SAVE MODULE



- The value of ifmap(0,0) is calculated in the CT\_module.
- However, when the result is saved in the SAVE\_module, col\_idx and row\_idx have already changed.
- To prevent this issue, each index is captured.

# 7. CT\_module: idx capture

```
// *_*_*_*_*_*_ idx capture *_*_*_*_*_*
reg [DELAY * IDX_BITS - 1 : 0] captured_ifmap_col;
reg [DELAY * IDX_BITS - 1 : 0] captured_ifmap_row;
reg [DELAY * IDX_BITS - 1 : 0] captured_ifmap_in_channel;
reg [DELAY * IDX_BITS - 1 : 0] captured_ifmap_out_channel;
//
```

```
always @(posedge clk or negedge reset_n) begin
```

```
  if(!reset_n) begin
```

```
    captured_ifmap_col <= {DELAY * IDX_BITS{1'b0}};
```

```
    captured_ifmap_row <= {DELAY * IDX_BITS{1'b0}};
```

```
    captured_ifmap_in_channel <= {DELAY * IDX_BITS{1'b0}};
```

```
    captured_ifmap_out_channel <= {DELAY * IDX_BITS{1'b0}};
```

```
  end
```

```
  else if(i_accum_from_SAVE && mem_access_cnt == (FW * FH - 1)) begin
```

```
    if(DELAY == 1) begin
```

```
      captured_ifmap_col <= col_idx;
```

```
      captured_ifmap_row <= row_idx;
```

```
      captured_ifmap_in_channel <= in_channel_idx;
```

```
      captured_ifmap_out_channel <= out_channel_idx;
```

```
    end
```

```
  else begin
```

```
    captured_ifmap_col <= {captured_ifmap_col[(DELAY-1) * IDX_BITS - 1 : 0], col_idx};
```

```
    captured_ifmap_row <= {captured_ifmap_row[(DELAY-1) * IDX_BITS - 1 : 0], row_idx};
```

```
    captured_ifmap_in_channel <= {captured_ifmap_in_channel[(DELAY-1) * IDX_BITS - 1 : 0], in_channel_idx};
```

```
    captured_ifmap_out_channel <= {captured_ifmap_out_channel[(DELAY-1) * IDX_BITS - 1 : 0], out_channel_idx};
```

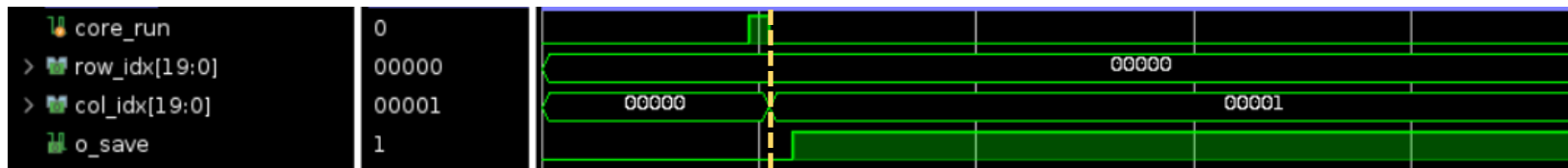
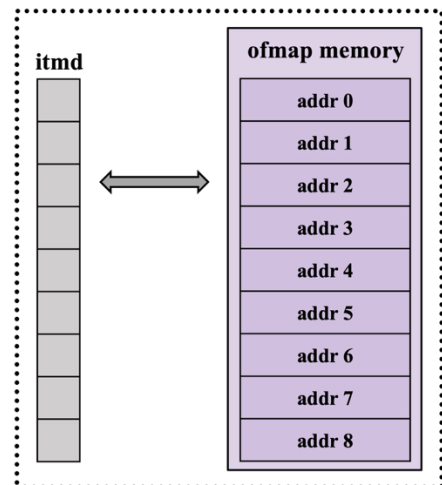
```
  end
```

```
end
```

```
end
```

mem\_access\_cnt = 3

IN THE SAVE MODULE



- The value of ifmap(0,0) is calculated in the CT\_module.
- However, when the result is saved in the SAVE\_module, col\_idx and row\_idx have already changed.
- To prevent this issue, each index is captured.

# 7. CT\_module: idx capture

```
// *_*_*_*_*_*_ idx capture *_*_*_*_*_*
reg [DELAY * IDX_BITS - 1 : 0] captured_ifmap_col;
reg [DELAY * IDX_BITS - 1 : 0] captured_ifmap_row;
reg [DELAY * IDX_BITS - 1 : 0] captured_ifmap_in_channel;
reg [DELAY * IDX_BITS - 1 : 0] captured_ifmap_out_channel;
//
```

```
always @(posedge clk or negedge reset_n) begin
```

```
  if(!reset_n) begin
```

```
    captured_ifmap_col <= {DELAY * IDX_BITS{1'b0}};
```

```
    captured_ifmap_row <= {DELAY * IDX_BITS{1'b0}};
```

```
    captured_ifmap_in_channel <= {DELAY * IDX_BITS{1'b0}};
```

```
    captured_ifmap_out_channel <= {DELAY * IDX_BITS{1'b0}};
```

```
  end
```

```
  else if(i_accum_from_SAVE && mem_access_cnt == (FW * FH - 1)) begin
```

```
    if(DELAY == 1) begin
```

```
      captured_ifmap_col <= col_idx;
```

```
      captured_ifmap_row <= row_idx;
```

```
      captured_ifmap_in_channel <= in_channel_idx;
```

```
      captured_ifmap_out_channel <= out_channel_idx;
```

```
    end
```

```
  else begin
```

```
    captured_ifmap_col <= {captured_ifmap_col[(DELAY-1) * IDX_BITS - 1 : 0], col_idx};
```

```
    captured_ifmap_row <= {captured_ifmap_row[(DELAY-1) * IDX_BITS - 1 : 0], row_idx};
```

```
    captured_ifmap_in_channel <= {captured_ifmap_in_channel[(DELAY-1) * IDX_BITS - 1 : 0], in_channel_idx};
```

```
    captured_ifmap_out_channel <= {captured_ifmap_out_channel[(DELAY-1) * IDX_BITS - 1 : 0], out_channel_idx};
```

```
  end
```

```
end
```

```
end
```

mem\_access\_cnt = 4

IN THE SAVE MODULE

itmd

ofmap memory

addr 0

addr 1

addr 2

addr 3

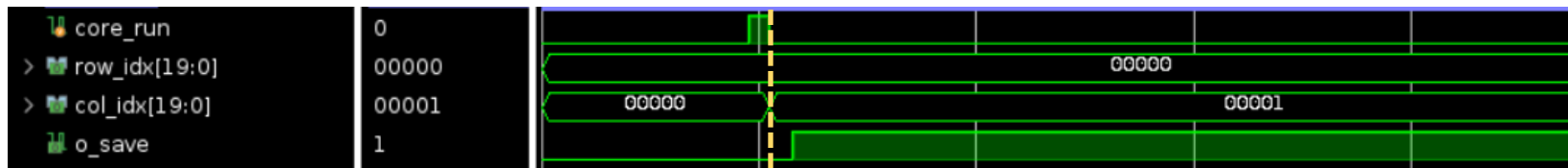
addr 4

addr 5

addr 6

addr 7

addr 8



- The value of ifmap(0,0) is calculated in the CT\_module.
- However, when the result is saved in the SAVE\_module, col\_idx and row\_idx have already changed.
- To prevent this issue, each index is captured.

# 7. CT\_module: idx capture

```
// *_*_*_*_*_*_ idx capture *_*_*_*_*_*
reg [DELAY * IDX_BITS - 1 : 0] captured_ifmap_col;
reg [DELAY * IDX_BITS - 1 : 0] captured_ifmap_row;
reg [DELAY * IDX_BITS - 1 : 0] captured_ifmap_in_channel;
reg [DELAY * IDX_BITS - 1 : 0] captured_ifmap_out_channel;
//
```

```
always @(posedge clk or negedge reset_n) begin
```

```
  if(!reset_n) begin
```

```
    captured_ifmap_col <= {DELAY * IDX_BITS{1'b0}};
```

```
    captured_ifmap_row <= {DELAY * IDX_BITS{1'b0}};
```

```
    captured_ifmap_in_channel <= {DELAY * IDX_BITS{1'b0}};
```

```
    captured_ifmap_out_channel <= {DELAY * IDX_BITS{1'b0}};
```

```
  end
```

```
  else if(i_accum_from_SAVE && mem_access_cnt == (FW * FH - 1)) begin
```

```
    if(DELAY == 1) begin
```

```
      captured_ifmap_col <= col_idx;
```

```
      captured_ifmap_row <= row_idx;
```

```
      captured_ifmap_in_channel <= in_channel_idx;
```

```
      captured_ifmap_out_channel <= out_channel_idx;
```

```
    end
```

```
  else begin
```

```
    captured_ifmap_col <= {captured_ifmap_col[(DELAY-1) * IDX_BITS - 1 : 0], col_idx};
```

```
    captured_ifmap_row <= {captured_ifmap_row[(DELAY-1) * IDX_BITS - 1 : 0], row_idx};
```

```
    captured_ifmap_in_channel <= {captured_ifmap_in_channel[(DELAY-1) * IDX_BITS - 1 : 0], in_channel_idx};
```

```
    captured_ifmap_out_channel <= {captured_ifmap_out_channel[(DELAY-1) * IDX_BITS - 1 : 0], out_channel_idx};
```

```
  end
```

```
end
```

```
end
```

mem\_access\_cnt = 5

IN THE SAVE MODULE

itmd

ofmap memory

addr 0

addr 1

addr 2

addr 3

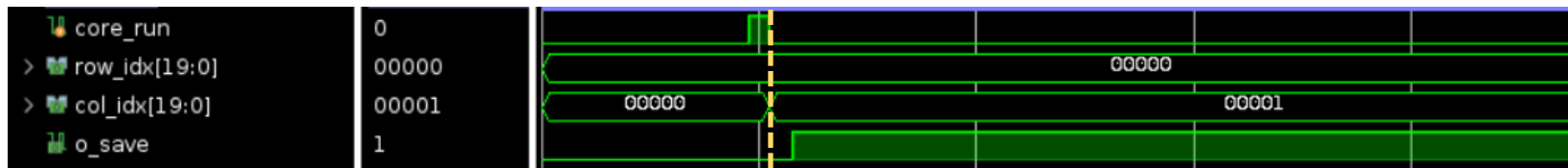
addr 4

addr 5

addr 6

addr 7

addr 8



- The value of ifmap(0,0) is calculated in the CT\_module.
- However, when the result is saved in the SAVE\_module, col\_idx and row\_idx have already changed.
- To prevent this issue, each index is captured.

# 7. CT\_module: idx capture

```
// *_*_*_*_*_*_ idx capture *_*_*_*_*_*_
reg [DELAY * IDX_BITS - 1 : 0] captured_ifmap_col;
reg [DELAY * IDX_BITS - 1 : 0] captured_ifmap_row;
reg [DELAY * IDX_BITS - 1 : 0] captured_ifmap_in_channel;
reg [DELAY * IDX_BITS - 1 : 0] captured_ifmap_out_channel;
//
```

```
always @(posedge clk or negedge reset_n) begin
```

```
  if(!reset_n) begin
```

```
    captured_ifmap_col <= {DELAY * IDX_BITS{1'b0}};
```

```
    captured_ifmap_row <= {DELAY * IDX_BITS{1'b0}};
```

```
    captured_ifmap_in_channel <= {DELAY * IDX_BITS{1'b0}};
```

```
    captured_ifmap_out_channel <= {DELAY * IDX_BITS{1'b0}};
```

```
  end
```

```
  else if(i_accum_from_SAVE && mem_access_cnt == (FW * FH - 1)) begin
```

```
    if(DELAY == 1) begin
```

```
      captured_ifmap_col <= col_idx;
```

```
      captured_ifmap_row <= row_idx;
```

```
      captured_ifmap_in_channel <= in_channel_idx;
```

```
      captured_ifmap_out_channel <= out_channel_idx;
```

```
    end
```

```
  else begin
```

```
    captured_ifmap_col <= {captured_ifmap_col[(DELAY-1) * IDX_BITS - 1 : 0], col_idx};
```

```
    captured_ifmap_row <= {captured_ifmap_row[(DELAY-1) * IDX_BITS - 1 : 0], row_idx};
```

```
    captured_ifmap_in_channel <= {captured_ifmap_in_channel[(DELAY-1) * IDX_BITS - 1 : 0], in_channel_idx};
```

```
    captured_ifmap_out_channel <= {captured_ifmap_out_channel[(DELAY-1) * IDX_BITS - 1 : 0], out_channel_idx};
```

```
  end
```

```
end
```

```
end
```

mem\_access\_cnt = 6

IN THE SAVE MODULE

itmd

ofmap memory

addr 0

addr 1

addr 2

addr 3

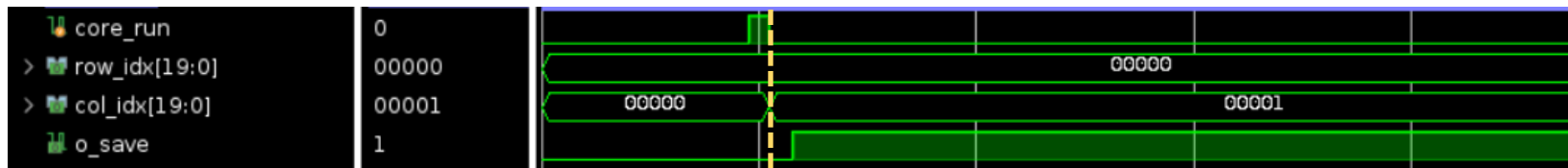
addr 4

addr 5

addr 6

addr 7

addr 8



- The value of ifmap(0,0) is calculated in the CT\_module.
- However, when the result is saved in the SAVE\_module, col\_idx and row\_idx have already changed.
- To prevent this issue, each index is captured.

# 7. CT\_module: idx capture

```
// *_*_*_*_*_*_ idx capture *_*_*_*_*_*_
reg [DELAY * IDX_BITS - 1 : 0] captured_ifmap_col;
reg [DELAY * IDX_BITS - 1 : 0] captured_ifmap_row;
reg [DELAY * IDX_BITS - 1 : 0] captured_ifmap_in_channel;
reg [DELAY * IDX_BITS - 1 : 0] captured_ifmap_out_channel;
//
```

```
always @(posedge clk or negedge reset_n) begin
```

```
if(!reset_n) begin
```

```
captured_ifmap_col <= {DELAY * IDX_BITS{1'b0}};
```

```
captured_ifmap_row <= {DELAY * IDX_BITS{1'b0}};
```

```
captured_ifmap_in_channel <= {DELAY * IDX_BITS{1'b0}};
```

```
captured_ifmap_out_channel <= {DELAY * IDX_BITS{1'b0}};
```

```
end
```

```
else if(i_accum_from_SAVE && mem_access_cnt == (FW * FH - 1)) begin
```

```
if(DELAY == 1) begin
```

```
captured_ifmap_col <= col_idx;
```

```
captured_ifmap_row <= row_idx;
```

```
captured_ifmap_in_channel <= in_channel_idx;
```

```
captured_ifmap_out_channel <= out_channel_idx;
```

```
end
```

```
else begin
```

```
captured_ifmap_col <= {captured_ifmap_col[(DELAY-1) * IDX_BITS - 1 : 0], col_idx};
```

```
captured_ifmap_row <= {captured_ifmap_row[(DELAY-1) * IDX_BITS - 1 : 0], row_idx};
```

```
captured_ifmap_in_channel <= {captured_ifmap_in_channel[(DELAY-1) * IDX_BITS - 1 : 0], in_channel_idx};
```

```
captured_ifmap_out_channel <= {captured_ifmap_out_channel[(DELAY-1) * IDX_BITS - 1 : 0], out_channel_idx};
```

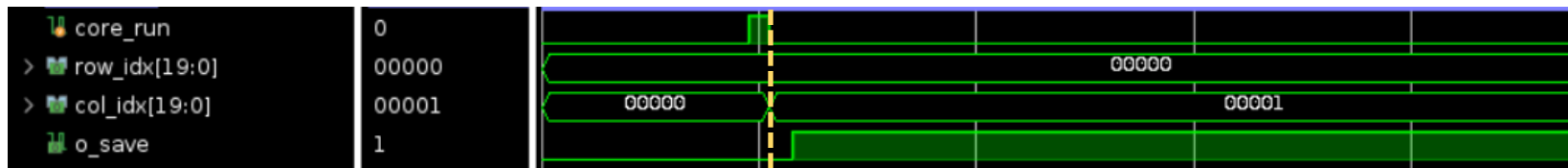
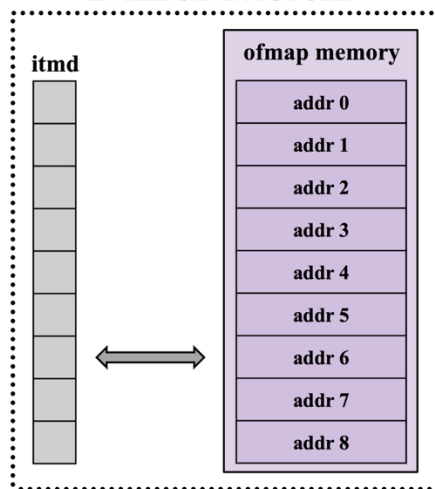
```
end
```

```
end
```

```
end
```

mem\_access\_cnt = 7

IN THE SAVE MODULE



- The value of ifmap(0,0) is calculated in the CT\_module.
- However, when the result is saved in the SAVE\_module, col\_idx and row\_idx have already changed.
- To prevent this issue, each index is captured.

# 7. CT\_module: idx capture

```
// *_*_*_*_*_*_ idx capture *_*_*_*_*_*
reg [DELAY * IDX_BITS - 1 : 0] captured_ifmap_col;
reg [DELAY * IDX_BITS - 1 : 0] captured_ifmap_row;
reg [DELAY * IDX_BITS - 1 : 0] captured_ifmap_in_channel;
reg [DELAY * IDX_BITS - 1 : 0] captured_ifmap_out_channel;
//
```

```
always @(posedge clk or negedge reset_n) begin
```

```
  if(!reset_n) begin
```

```
    captured_ifmap_col <= {DELAY * IDX_BITS{1'b0}};
```

```
    captured_ifmap_row <= {DELAY * IDX_BITS{1'b0}};
```

```
    captured_ifmap_in_channel <= {DELAY * IDX_BITS{1'b0}};
```

```
    captured_ifmap_out_channel <= {DELAY * IDX_BITS{1'b0}};
```

```
  end
```

```
  else if(i_accum_from_SAVE && mem_access_cnt == (FW * FH - 1)) begin
```

```
    if(DELAY == 1) begin
```

```
      captured_ifmap_col <= col_idx;
```

```
      captured_ifmap_row <= row_idx;
```

```
      captured_ifmap_in_channel <= in_channel_idx;
```

```
      captured_ifmap_out_channel <= out_channel_idx;
```

```
    end
```

```
  else begin
```

```
    captured_ifmap_col <= {captured_ifmap_col[(DELAY-1) * IDX_BITS - 1 : 0], col_idx};
```

```
    captured_ifmap_row <= {captured_ifmap_row[(DELAY-1) * IDX_BITS - 1 : 0], row_idx};
```

```
    captured_ifmap_in_channel <= {captured_ifmap_in_channel[(DELAY-1) * IDX_BITS - 1 : 0], in_channel_idx};
```

```
    captured_ifmap_out_channel <= {captured_ifmap_out_channel[(DELAY-1) * IDX_BITS - 1 : 0], out_channel_idx};
```

```
  end
```

```
end
```

```
end
```

mem\_access\_cnt = 8

IN THE SAVE MODULE

itmd

ofmap memory

addr 0

addr 1

addr 2

addr 3

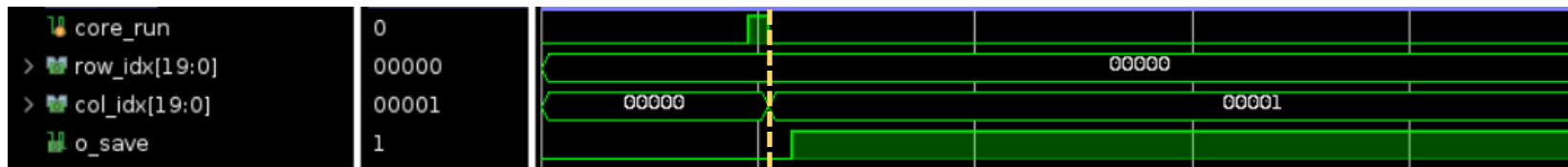
addr 4

addr 5

addr 6

addr 7

addr 8



- The value of ifmap(0,0) is calculated in the CT\_module.
- However, when the result is saved in the SAVE\_module, col\_idx and row\_idx have already changed.
- To prevent this issue, each index is captured.

# 7. CT\_module: idx capture

```
// *_*_*_*_*_ idx capture *_*_*_*_*_
reg [DELAY * IDX_BITS - 1 : 0] captured_ifmap_col;
reg [DELAY * IDX_BITS - 1 : 0] captured_ifmap_row;
reg [DELAY * IDX_BITS - 1 : 0] captured_ifmap_in_channel;
reg [DELAY * IDX_BITS - 1 : 0] captured_ifmap_out_channel;
//
```

```
always @(posedge clk or negedge reset_n) begin
```

```
if(!reset_n) begin
```

```
captured_ifmap_col <= {DELAY * IDX_BITS{1'b0}};
```

```
captured_ifmap_row <= {DELAY * IDX_BITS{1'b0}};
```

```
captured_ifmap_in_channel <= {DELAY * IDX_BITS{1'b0}};
```

```
captured_ifmap_out_channel <= {DELAY * IDX_BITS{1'b0}};
```

```
end
```

```
else if(i_accum_from_SAVE && mem_access_cnt == (FW * FH - 1)) begin
```

```
if(DELAY == 1) begin
```

```
captured_ifmap_col <= col_idx;
```

```
captured_ifmap_row <= row_idx;
```

```
captured_ifmap_in_channel <= in_channel_idx;
```

```
captured_ifmap_out_channel <= out_channel_idx;
```

```
end
```

```
else begin
```

```
captured_ifmap_col <= {captured_ifmap_col[(DELAY-1) * IDX_BITS - 1 : 0], col_idx};
```

```
captured_ifmap_row <= {captured_ifmap_row[(DELAY-1) * IDX_BITS - 1 : 0], row_idx};
```

```
captured_ifmap_in_channel <= {captured_ifmap_in_channel[(DELAY-1) * IDX_BITS - 1 : 0], in_channel_idx};
```

```
captured_ifmap_out_channel <= {captured_ifmap_out_channel[(DELAY-1) * IDX_BITS - 1 : 0], out_channel_idx};
```

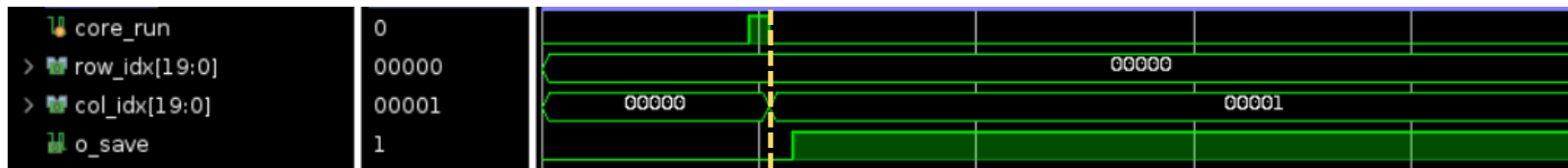
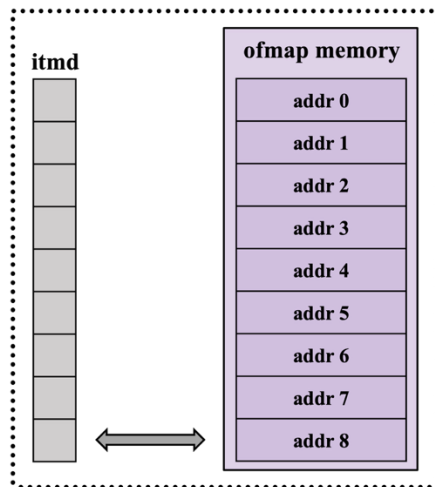
```
end
```

```
end
```

```
end
```

mem\_access\_cnt = 9

IN THE SAVE MODULE



- The value of ifmap(0,0) is calculated in the CT\_module.
- However, when the result is saved in the SAVE\_module, col\_idx and row\_idx have already changed.
- To prevent this issue, each index is captured.



- in\_channel\_idx = 0
- row\_idx = 0
- col\_idx = 0
- ifmap\_addr = 0

Diagram illustrating the mapping of input indices to output indices. The input indices are shown on the left, and the output indices are shown in a grid on the right. The output indices are mapped to the input indices as follows:

| Input Index          | Output Index |
|----------------------|--------------|
| in_channel_idx[19:0] | 00001        |
| row_idx[19:0]        | 00000        |
| col_idx[19:0]        | 00000        |
| ifmap_addr[19:0]     | 00004        |

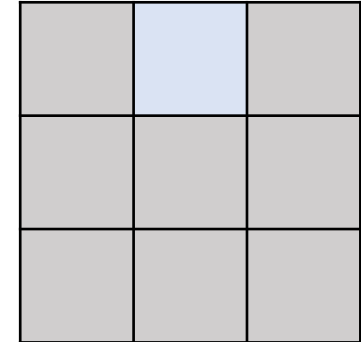


```
// *_*_*_*_*_*_ WEIGHT & IFMAP ADDR *_*_*_*_*_*_
reg [WEIGHT_AWIDTH - 1 : 0] wegt_addr;
reg [IFMAP_AWIDTH - 1 : 0] ifmap_addr;

//
always @(posedge clk or negedge reset_n) begin
    if(!reset_n) wegt_addr <= {IDX_BITS{1'b0}};
    else if(wegt_ce0) wegt_addr <= wegt_addr + 1'b1;
end

//
always @(posedge clk or negedge reset_n) begin
    if(!reset_n) ifmap_addr <= {IDX_BITS{1'b0}};
    else ifmap_addr <= IW * IH * in_channel_idx + IH * r

    // each index increases when the core_run is active h
end
```



- in\_channel\_idx = 0
- row\_idx = 0
- col\_idx = 1
- ifmap\_addr = 1

Diagram illustrating the mapping of input indices to output indices:

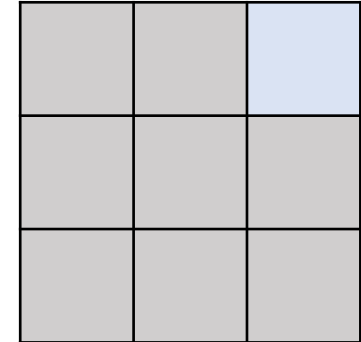
- `in_channel_idx[19:0]` maps to `00001`
- `row_idx[19:0]` maps to `00000`
- `col_idx[19:0]` maps to `00000`
- `ifmap_addr[19:0]` maps to `00004`

The diagram shows a 4x4 grid of output indices, where each cell contains a 20-bit value. The values are:

|       |       |       |       |
|-------|-------|-------|-------|
| 00000 | 00001 | 00002 | 00003 |
| 00000 | 00001 | 00002 | 00003 |
| 00000 | 00001 | 00002 | 00003 |
| 00000 | 00001 | 00002 | 00003 |



end



- in\_channel\_idx = 0
- row\_idx = 0
- col\_idx = 2
- ifmap\_addr = 2

Diagram illustrating the mapping of input indices to output indices:

| Input Index          | Output Index |
|----------------------|--------------|
| in_channel_idx[19:0] | 00001        |
| row_idx[19:0]        | 00000        |
| col_idx[19:0]        | 00000        |
| ifmap_addr[19:0]     | 00004        |

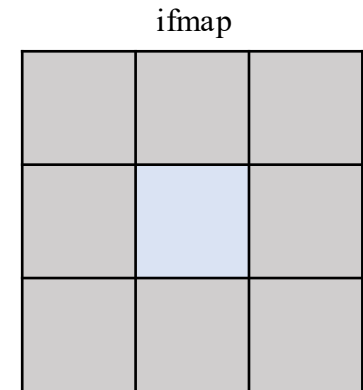


Diagram illustrating the mapping of input indices to output indices. The input indices are shown on the left, and the output indices are shown in a grid on the right.

| Input Index          | Value | Output Index | Value |
|----------------------|-------|--------------|-------|
| in_channel_idx[19:0] | 00001 | 00000        | 00000 |
| row_idx[19:0]        | 00000 | 00000        | 00001 |
| col_idx[19:0]        | 00000 | 00000        | 00001 |
| ifmap_addr[19:0]     | 00004 | 00000        | 00002 |



```
q_addr <= IW * IH * in_channel_idx + IH * row_idx + col_idx;
```



- in\_channel\_idx = 0
- row\_idx = 1
- col\_idx = 1
- ifmap\_addr = 4

| Variable             | Value | Hex   |
|----------------------|-------|-------|
| in_channel_idx[19:0] | 00001 | 00000 |
| row_idx[19:0]        | 00000 | 00001 |
| col_idx[19:0]        | 00000 | 00001 |
| ifmap_addr[19:0]     | 00004 | 00002 |



Diagram illustrating the mapping of input indices to output indices:

| Input Index          | Output Index |
|----------------------|--------------|
| in_channel_idx[19:0] | 00001        |
| row_idx[19:0]        | 00000        |
| col_idx[19:0]        | 00000        |
| ifmap_addr[19:0]     | 00004        |

The diagram also shows a 4x4 grid of output indices, where each cell contains a 4-bit value (0000, 0001, 0002, 0003) representing the output index for a given input index.



- in\_channel\_idx = 0
- row\_idx = 2
- col\_idx = 0
- ifmap\_addr = 6

Diagram illustrating the mapping of input indices to output indices:

| Input Index          | Output Index |
|----------------------|--------------|
| in_channel_idx[19:0] | 00001        |
| row_idx[19:0]        | 00000        |
| col_idx[19:0]        | 00000        |
| ifmap_addr[19:0]     | 00004        |



Diagram illustrating the mapping of input indices to output indices:

| Input Index          | Output Index |
|----------------------|--------------|
| in_channel_idx[19:0] | 00001        |
| row_idx[19:0]        | 00000        |
| col_idx[19:0]        | 00000        |
| ifmap_addr[19:0]     | 00004        |

The diagram also shows a 4x4 grid of output indices, where each cell contains a 4-bit value (0000, 0001, 0002, 0003) representing the output index for a given input index.

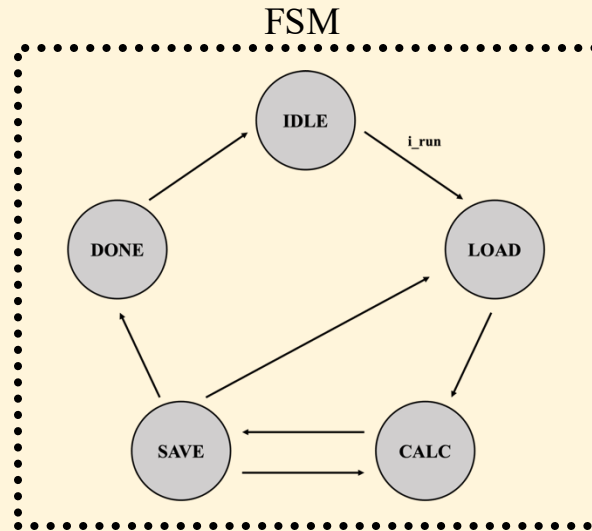


Diagram illustrating the mapping of input indices to output indices. The input indices are shown on the left, and the output indices are shown in a grid on the right.

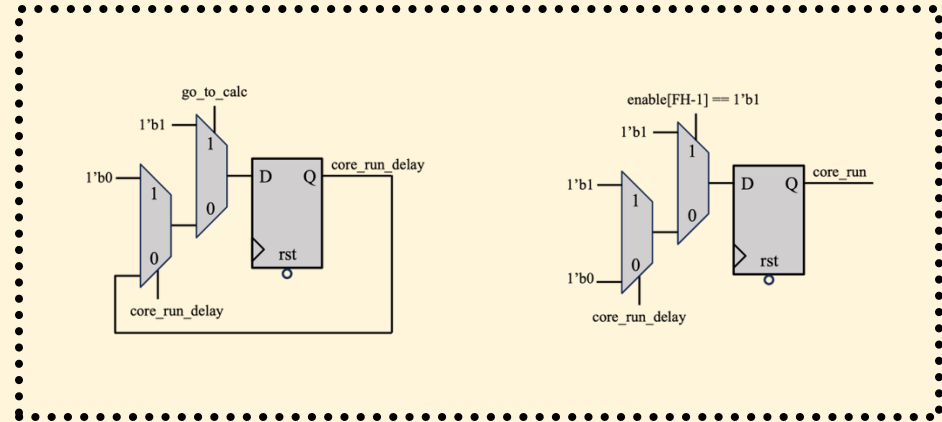
| Input Index          | Value | Output Index | Value |
|----------------------|-------|--------------|-------|
| in_channel_idx[19:0] | 00001 | 00000        | 00000 |
| row_idx[19:0]        | 00000 | 00000        | 00001 |
| col_idx[19:0]        | 00000 | 00000        | 00001 |
| ifmap_addr[19:0]     | 00004 | 00000        | 00003 |

# 7. CT\_module: summarization

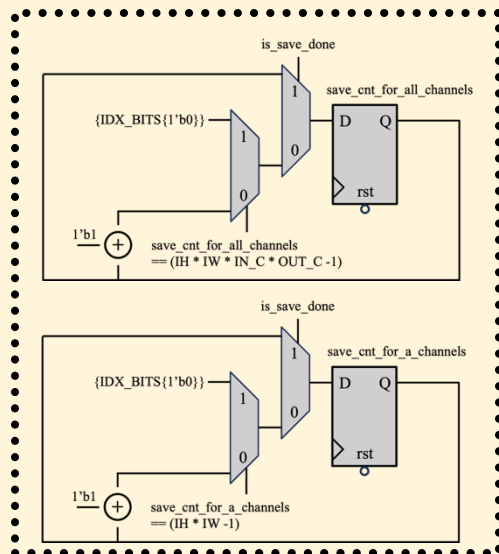
## CT\_module



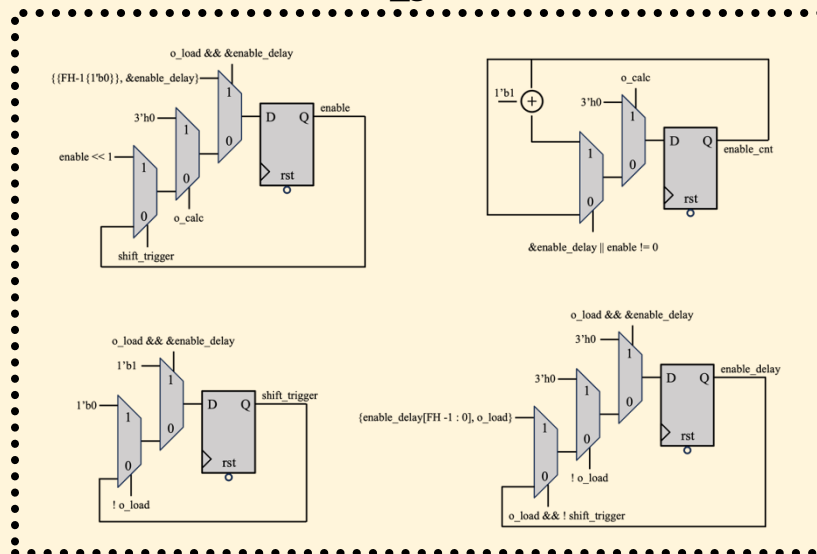
## core\_run\_generator



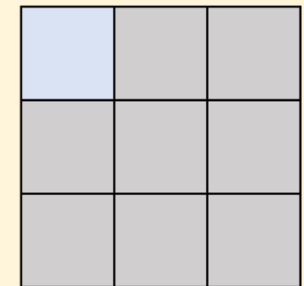
## save\_counter



## enable\_generator

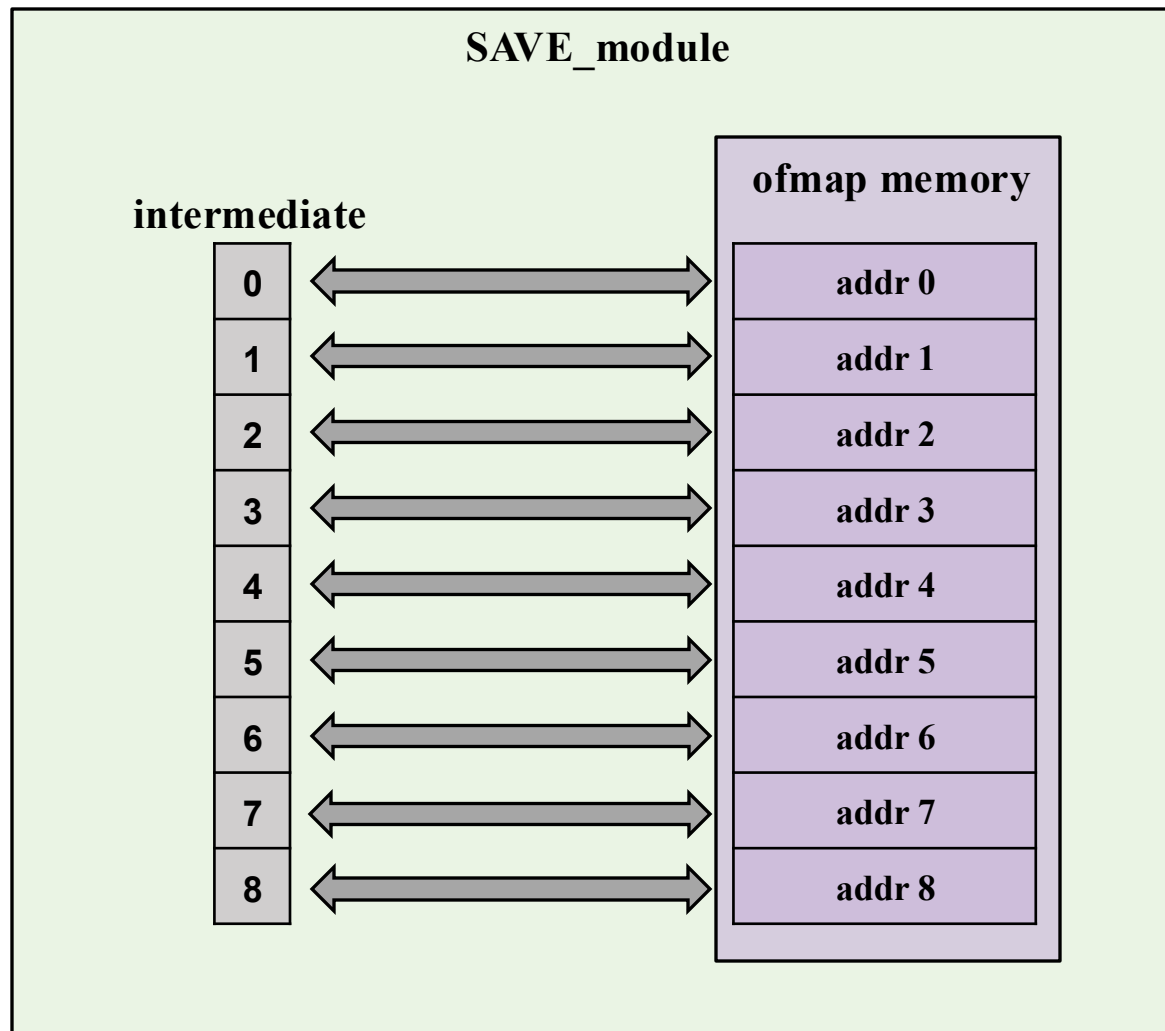


## address controller

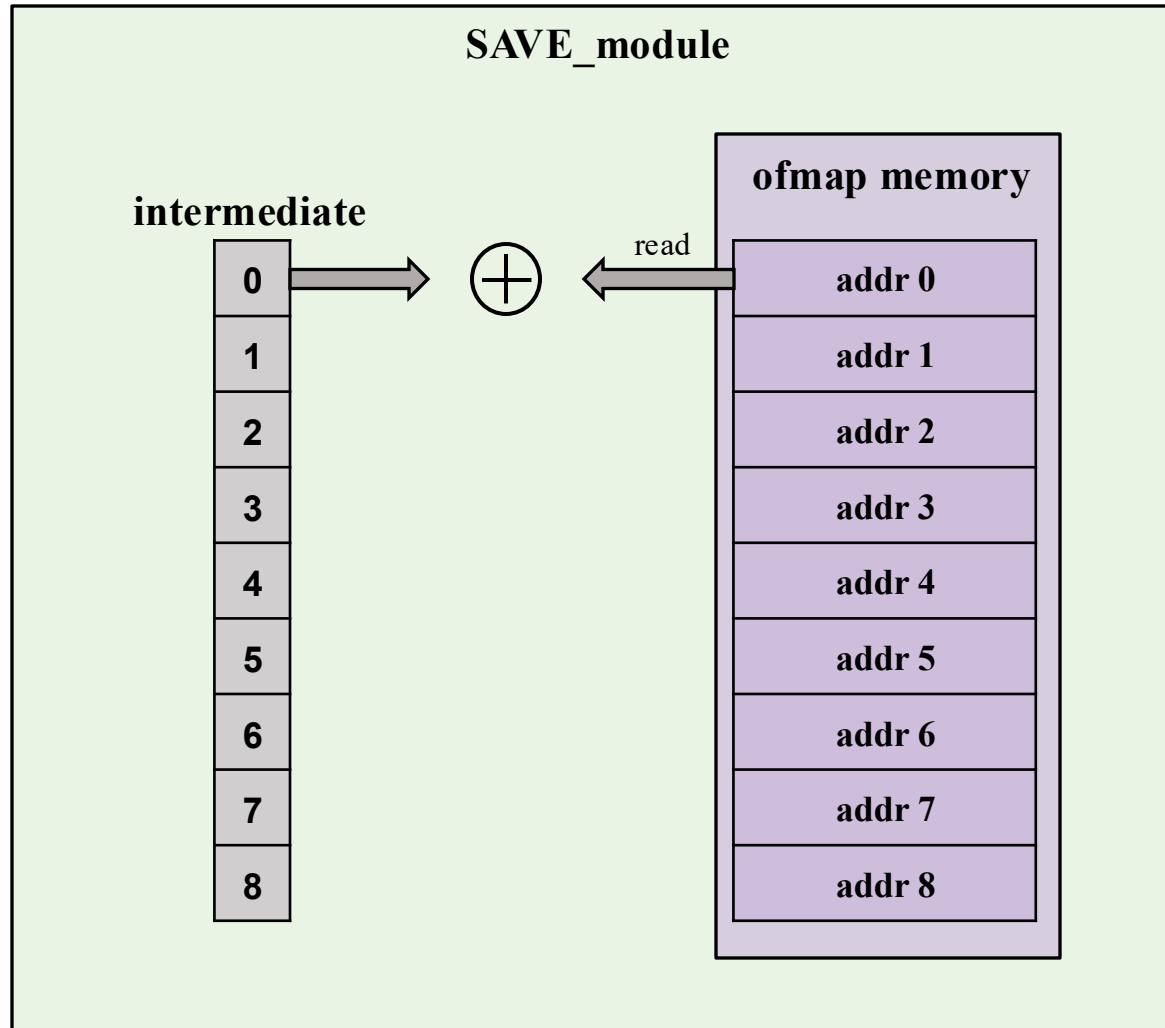


- in\_channel\_idx = 0
- row\_idx = 0
- col\_idx = 0
- ifmap\_addr = 0

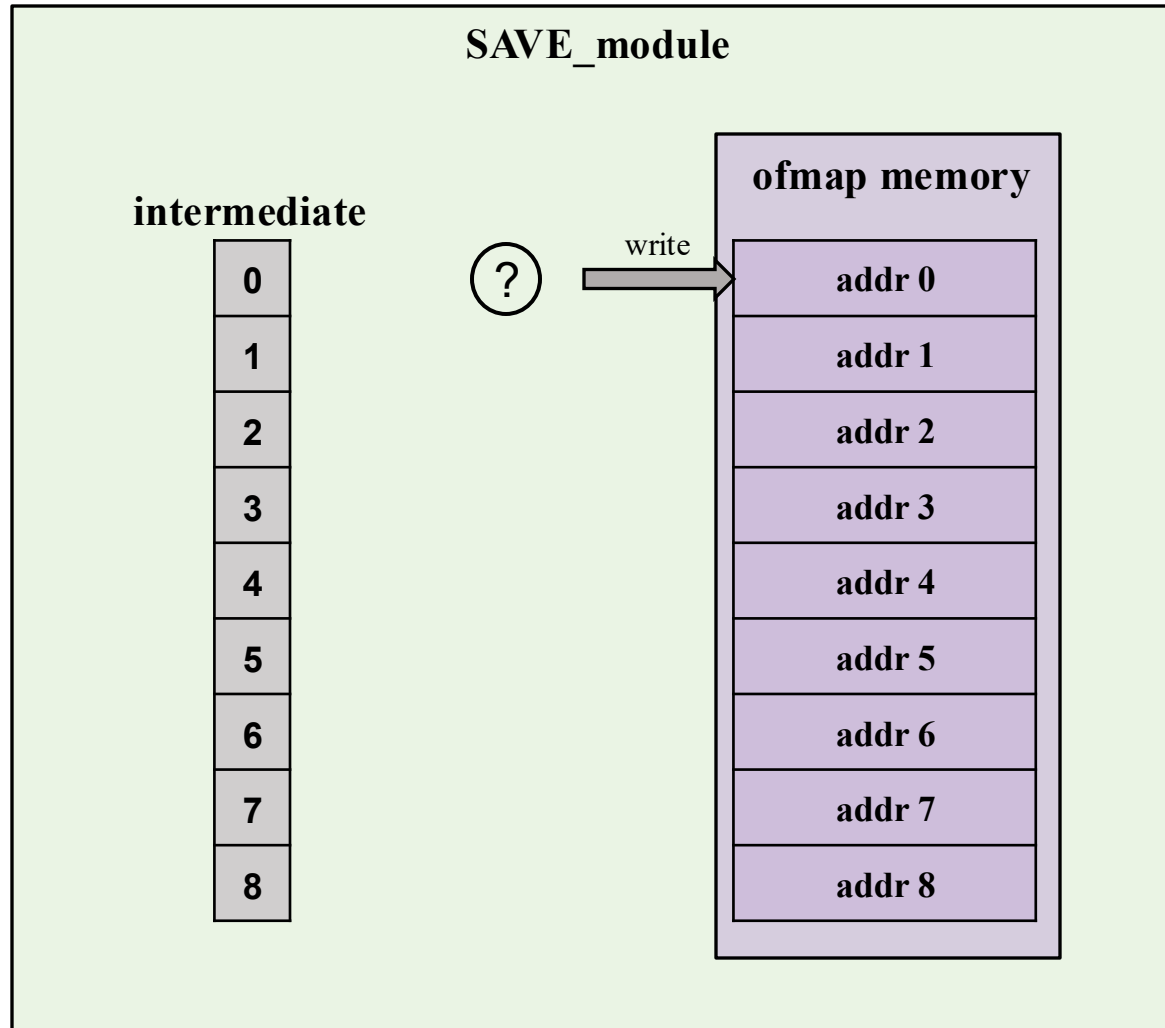
## 8. SAVE\_module: summarization



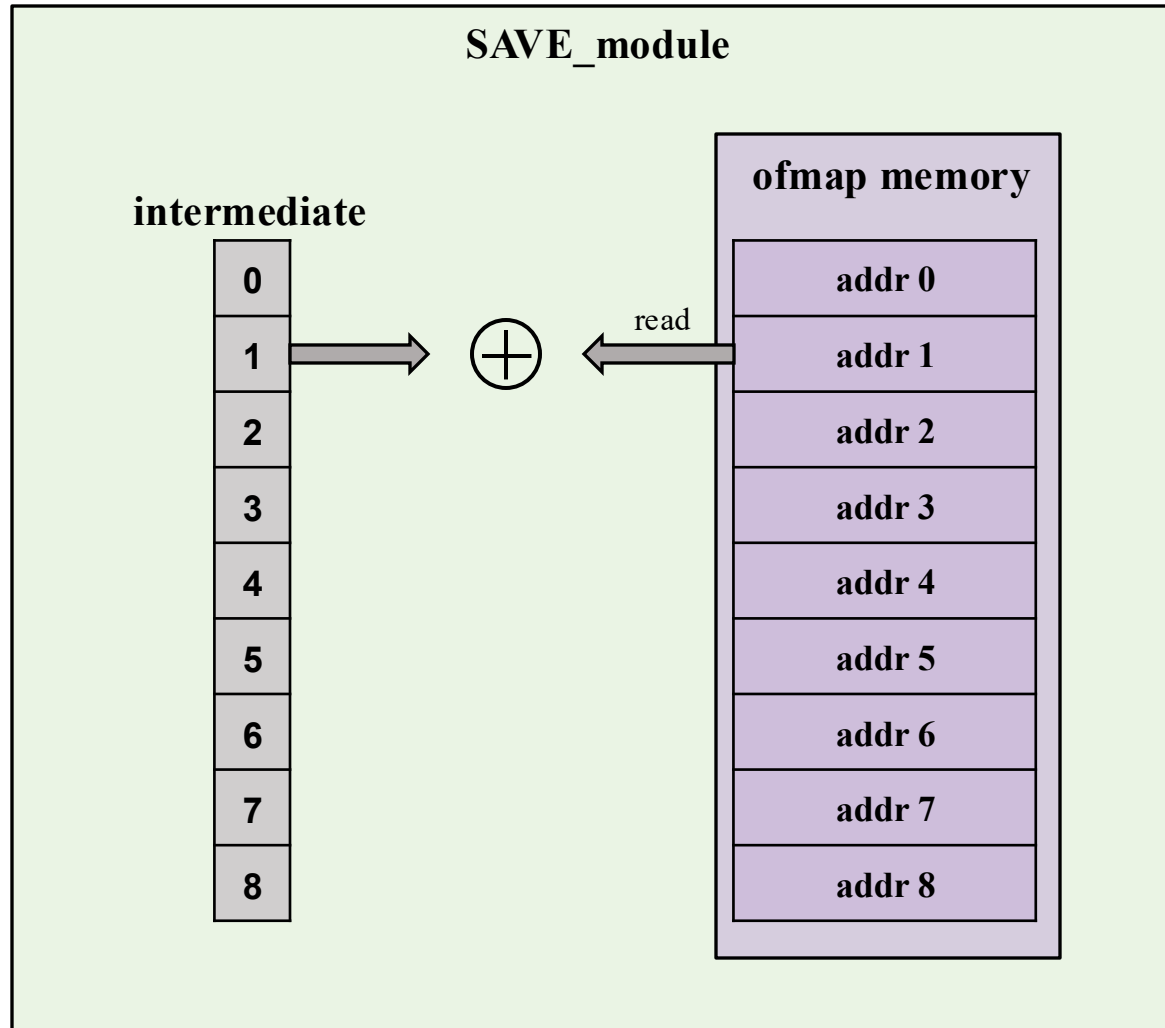
## 8. SAVE\_module: summarization



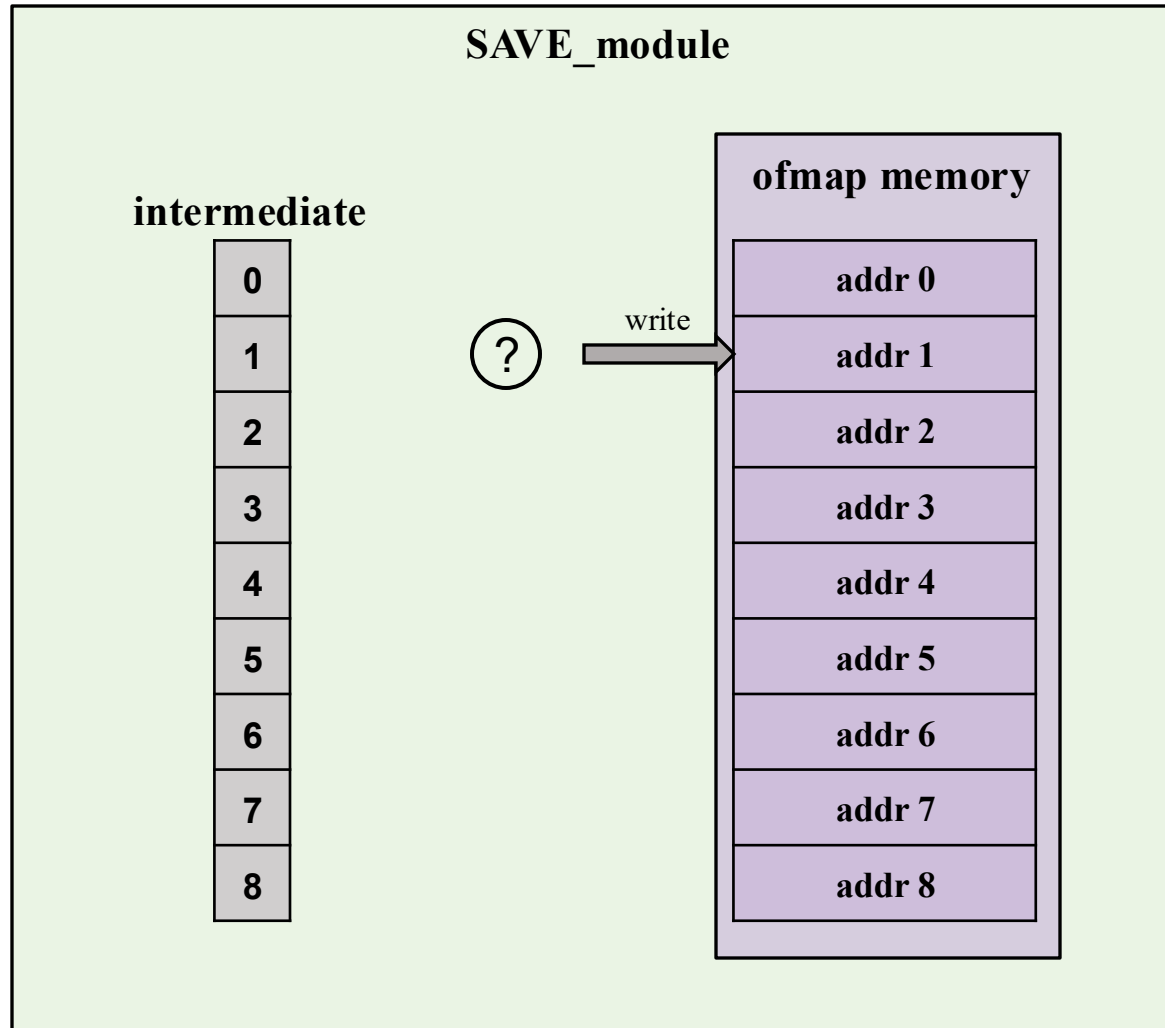
## 8. SAVE\_module: summarization



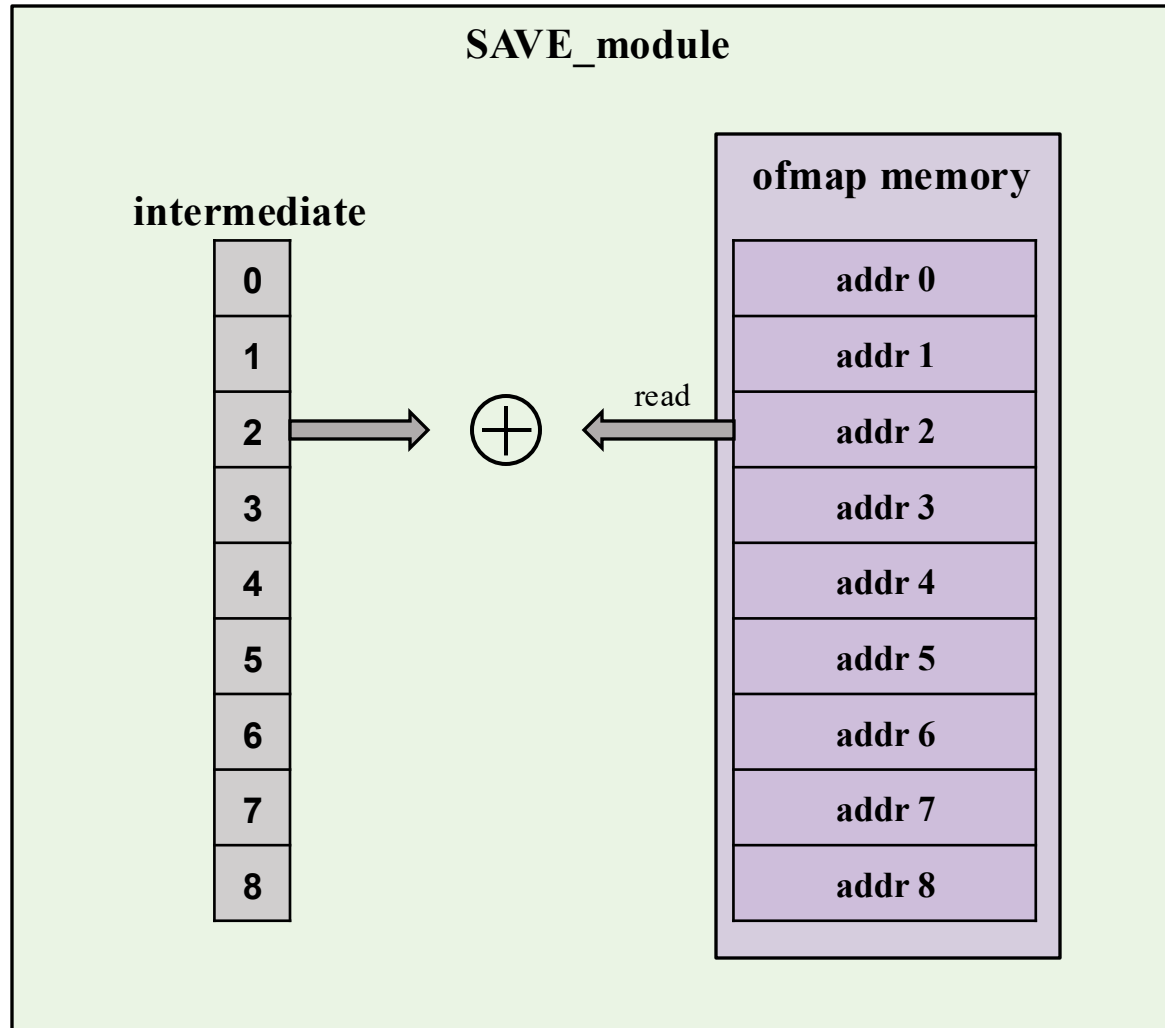
## 8. SAVE\_module: summarization



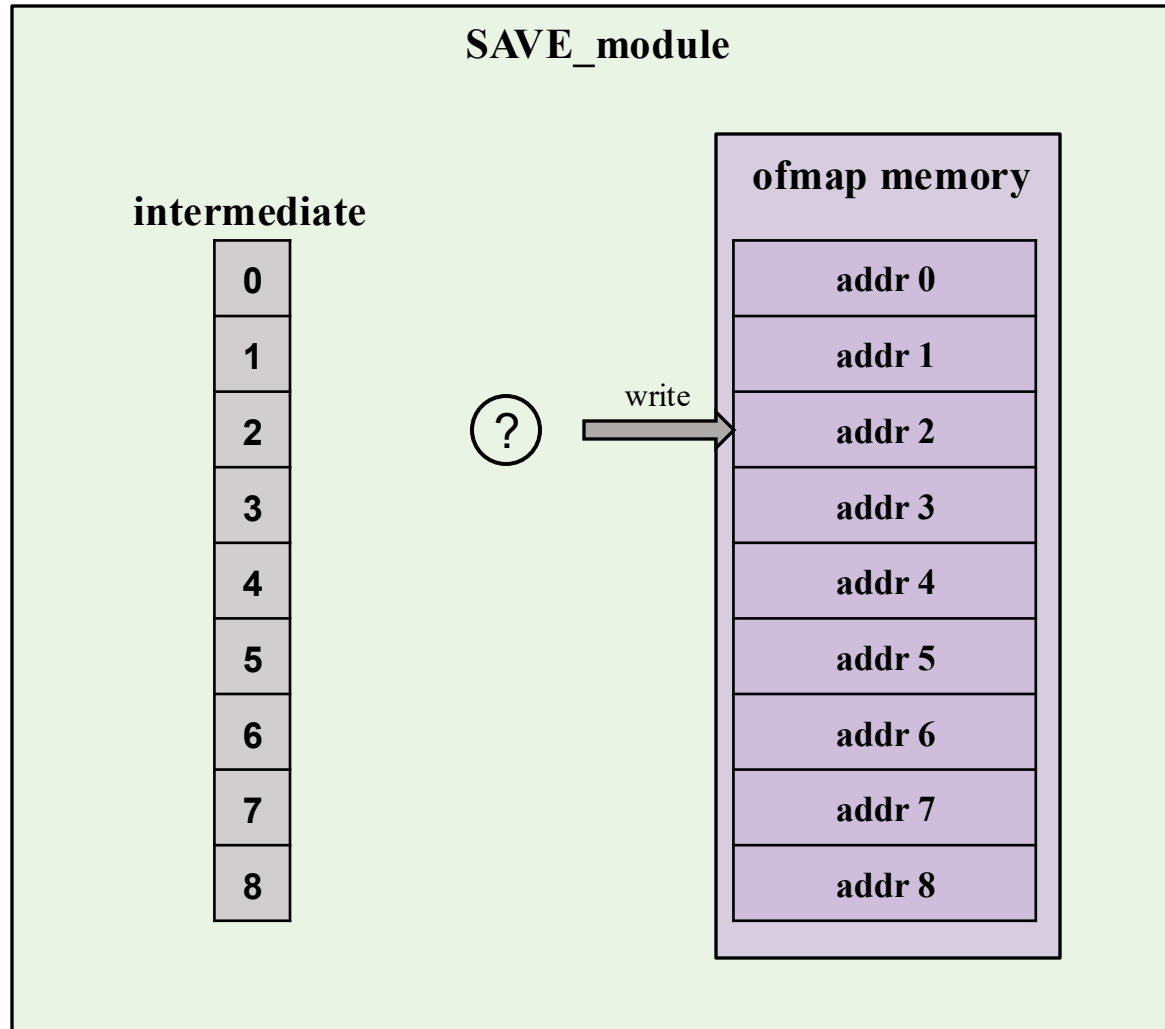
## 8. SAVE\_module: summarization



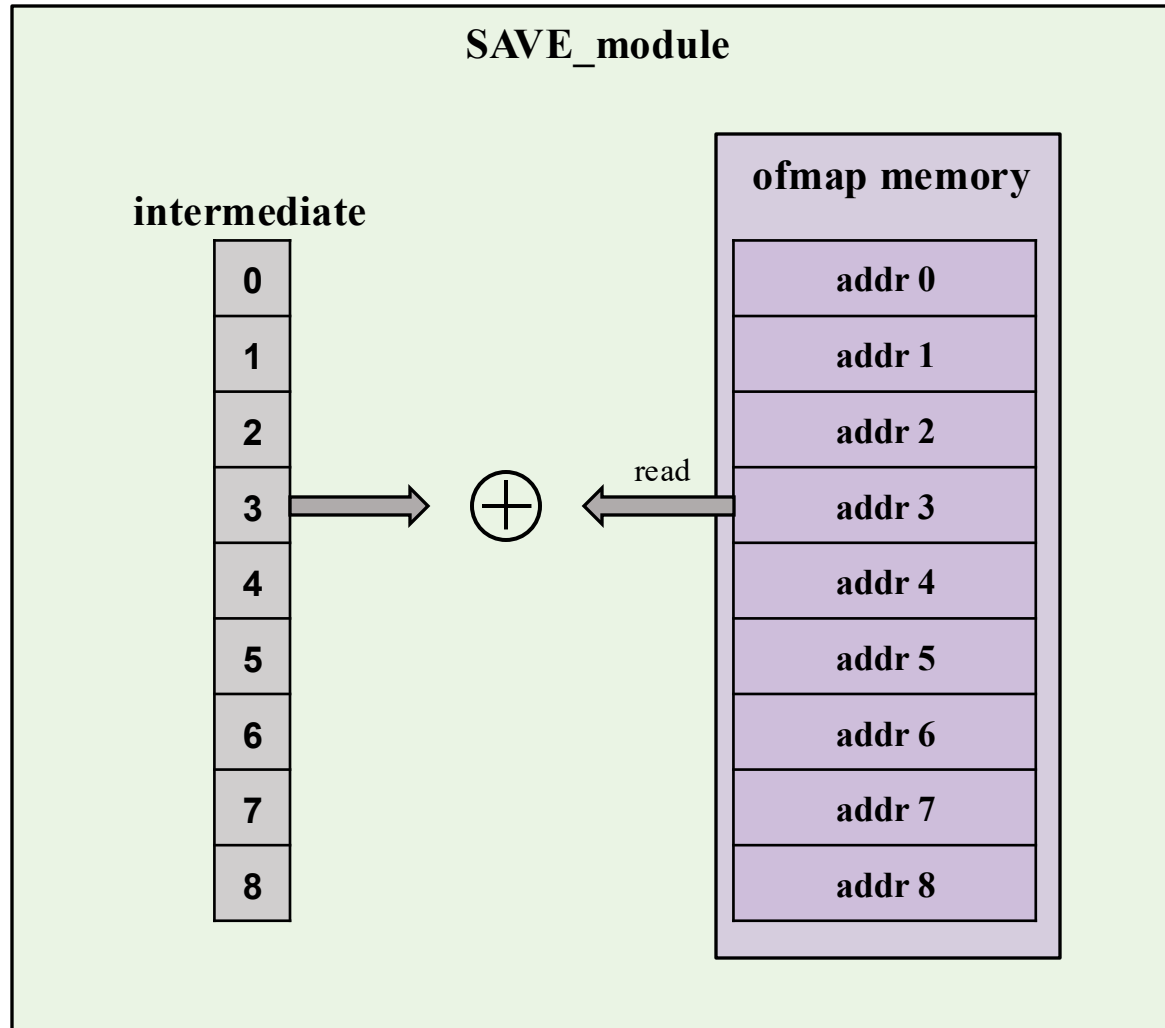
## 8. SAVE\_module: summarization



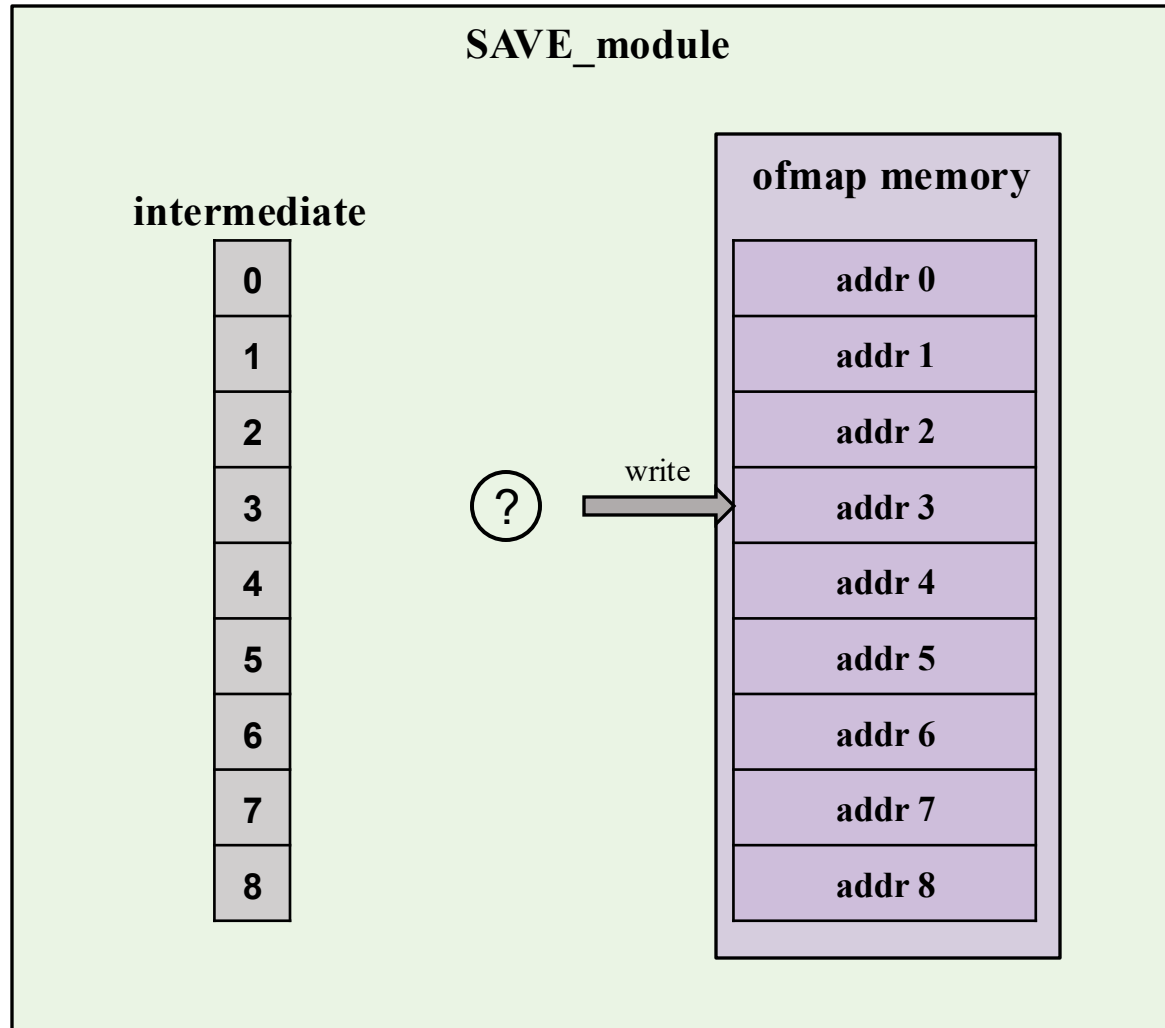
## 8. SAVE\_module: summarization



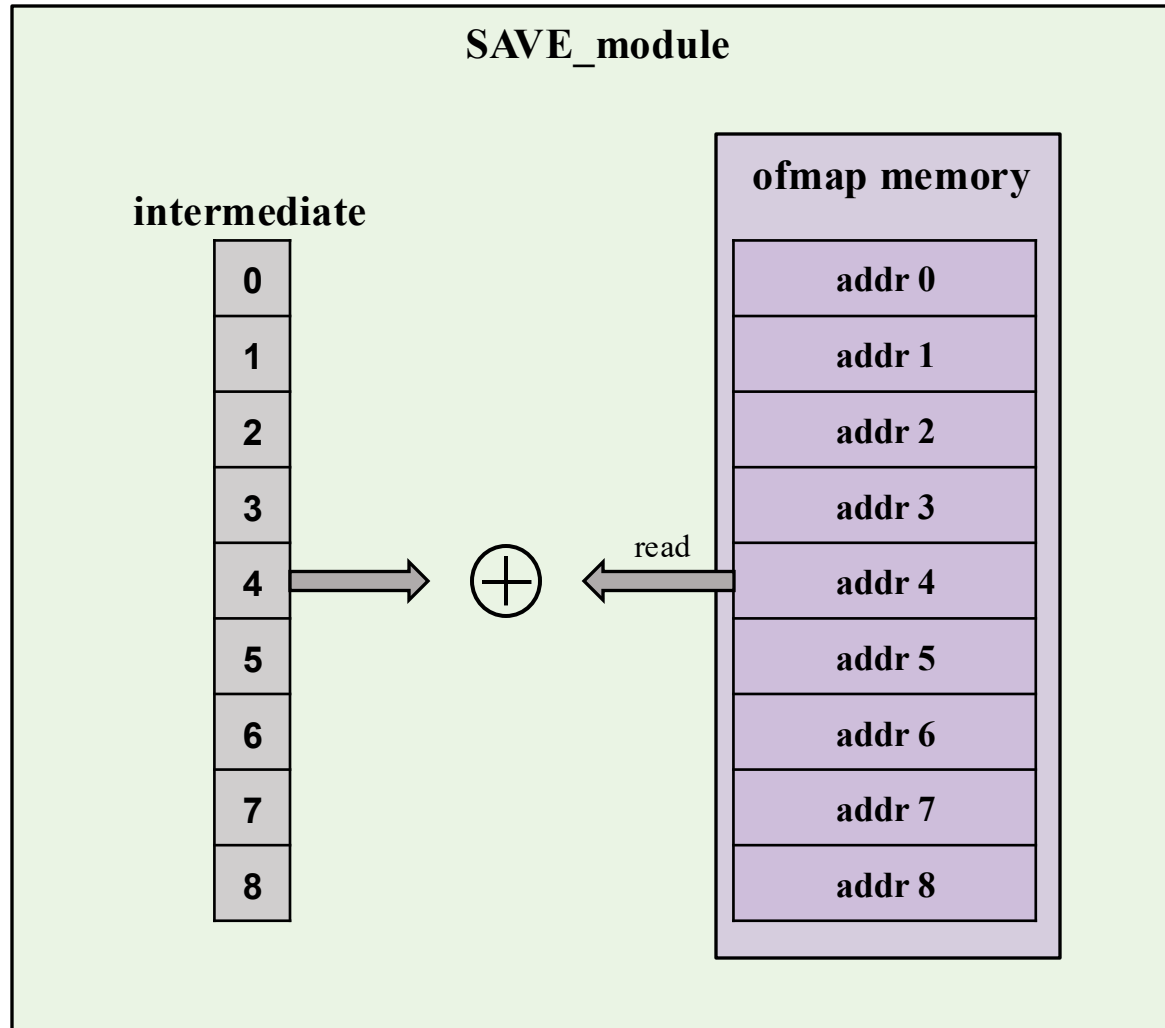
## 8. SAVE\_module: summarization



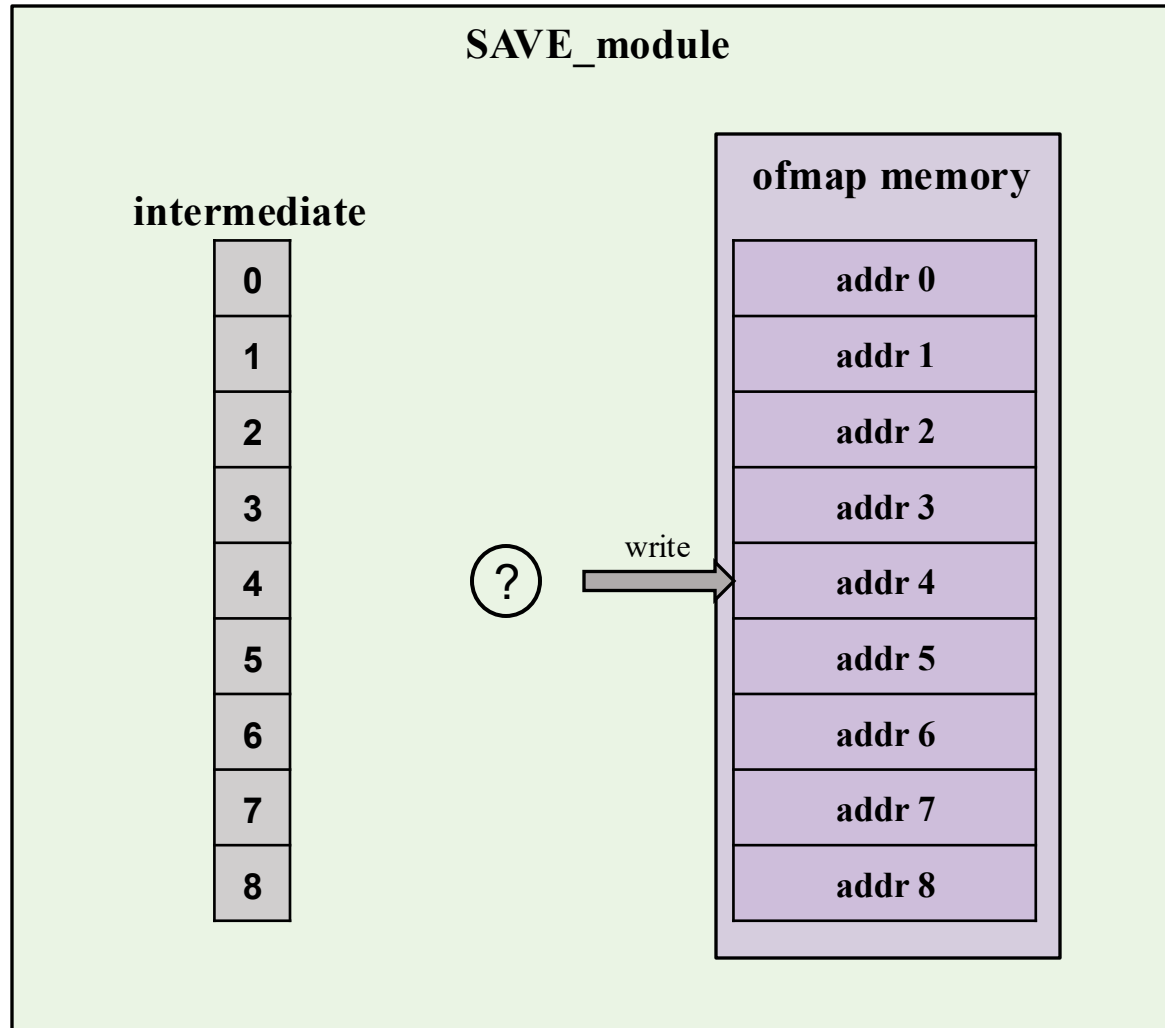
## 8. SAVE\_module: summarization

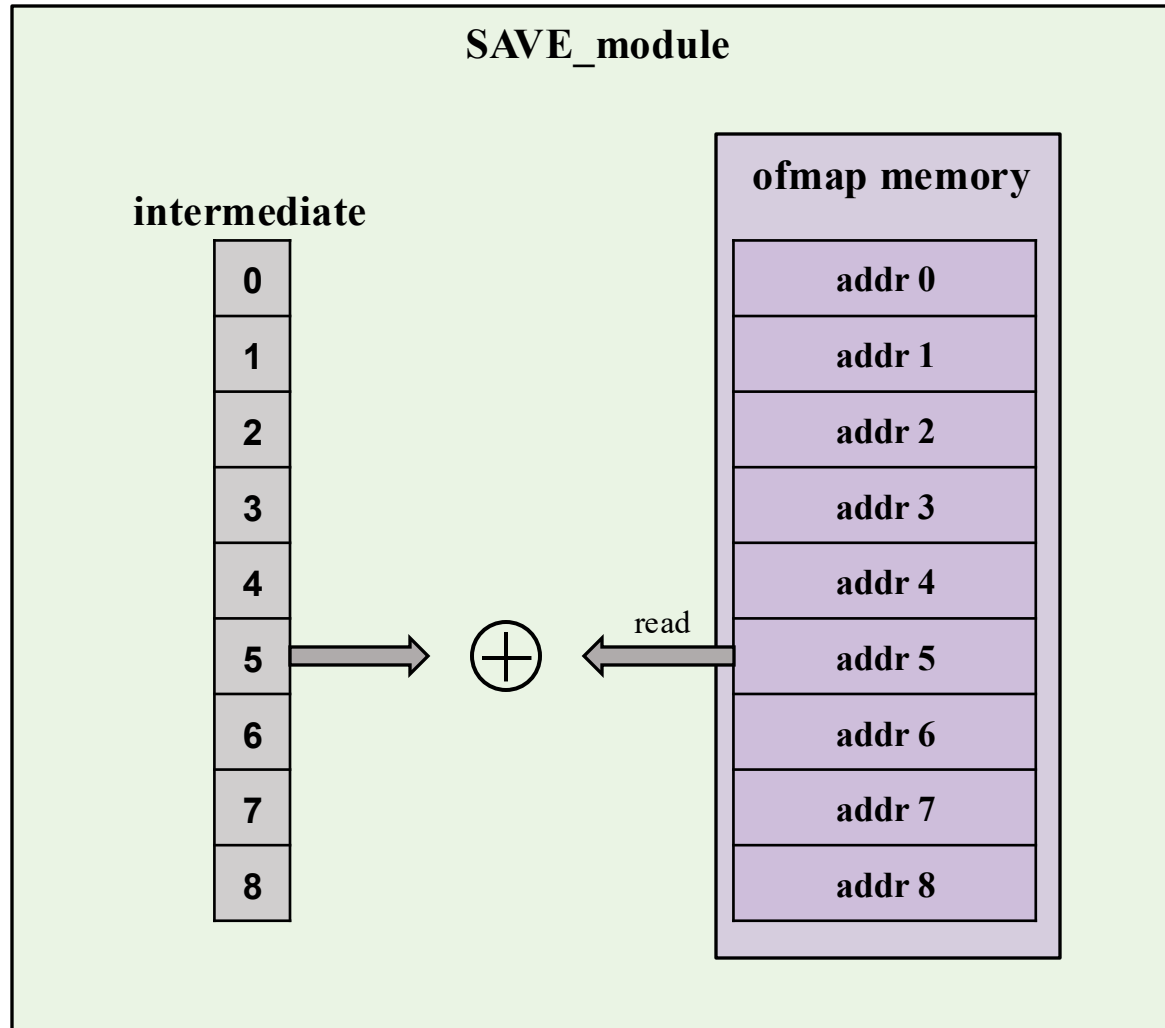


## 8. SAVE\_module: summarization

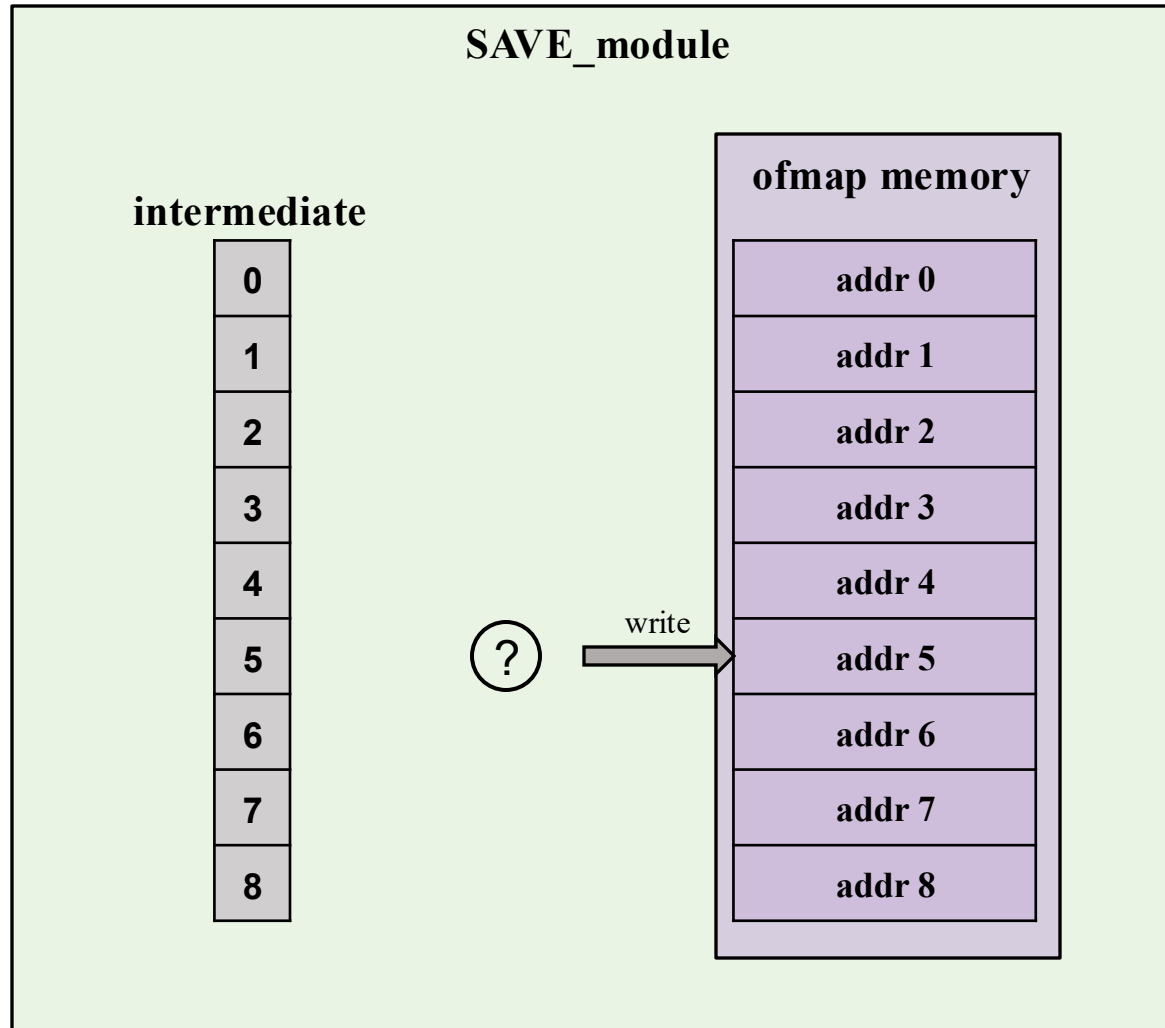


## 8. SAVE\_module: summarization

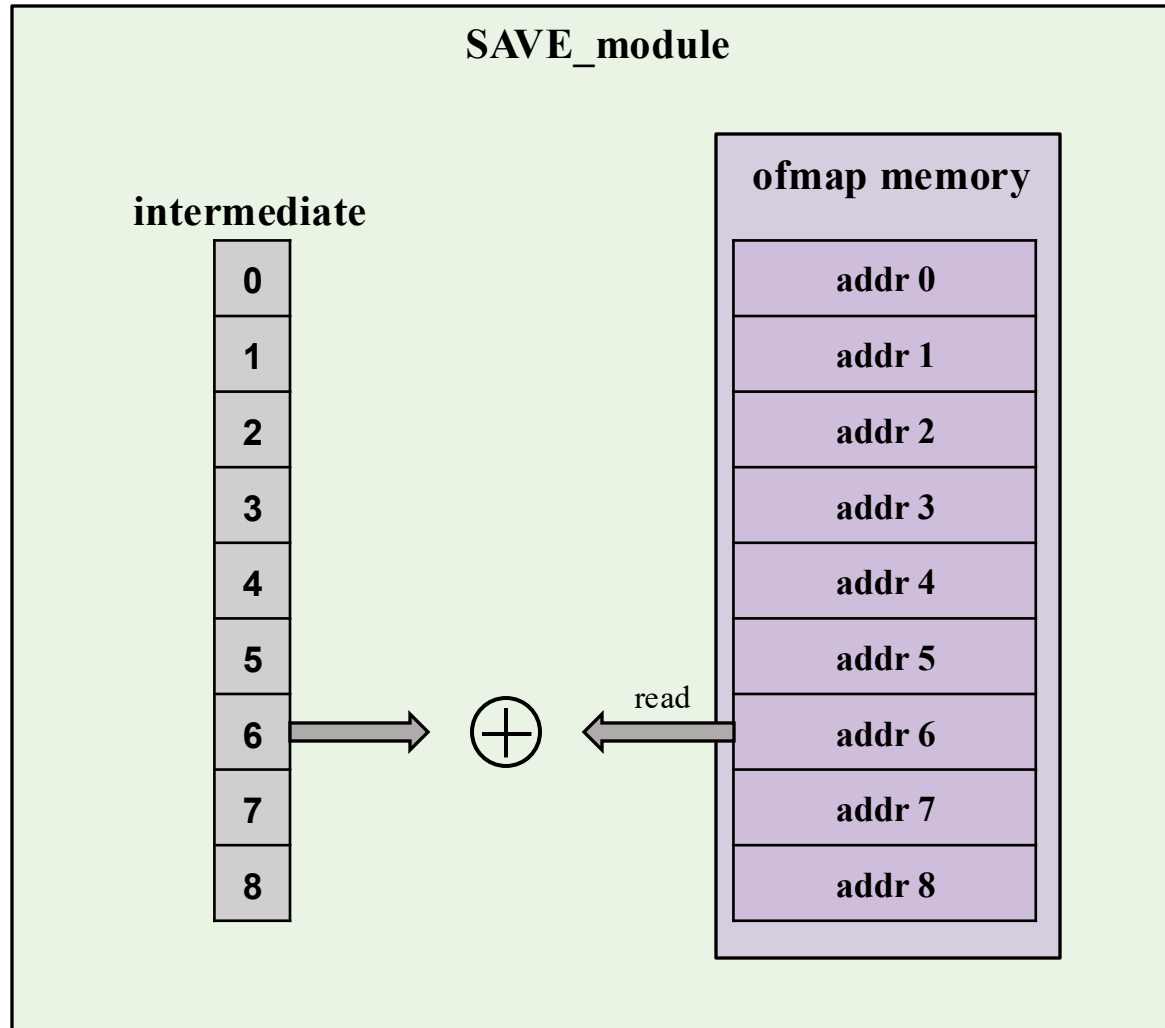




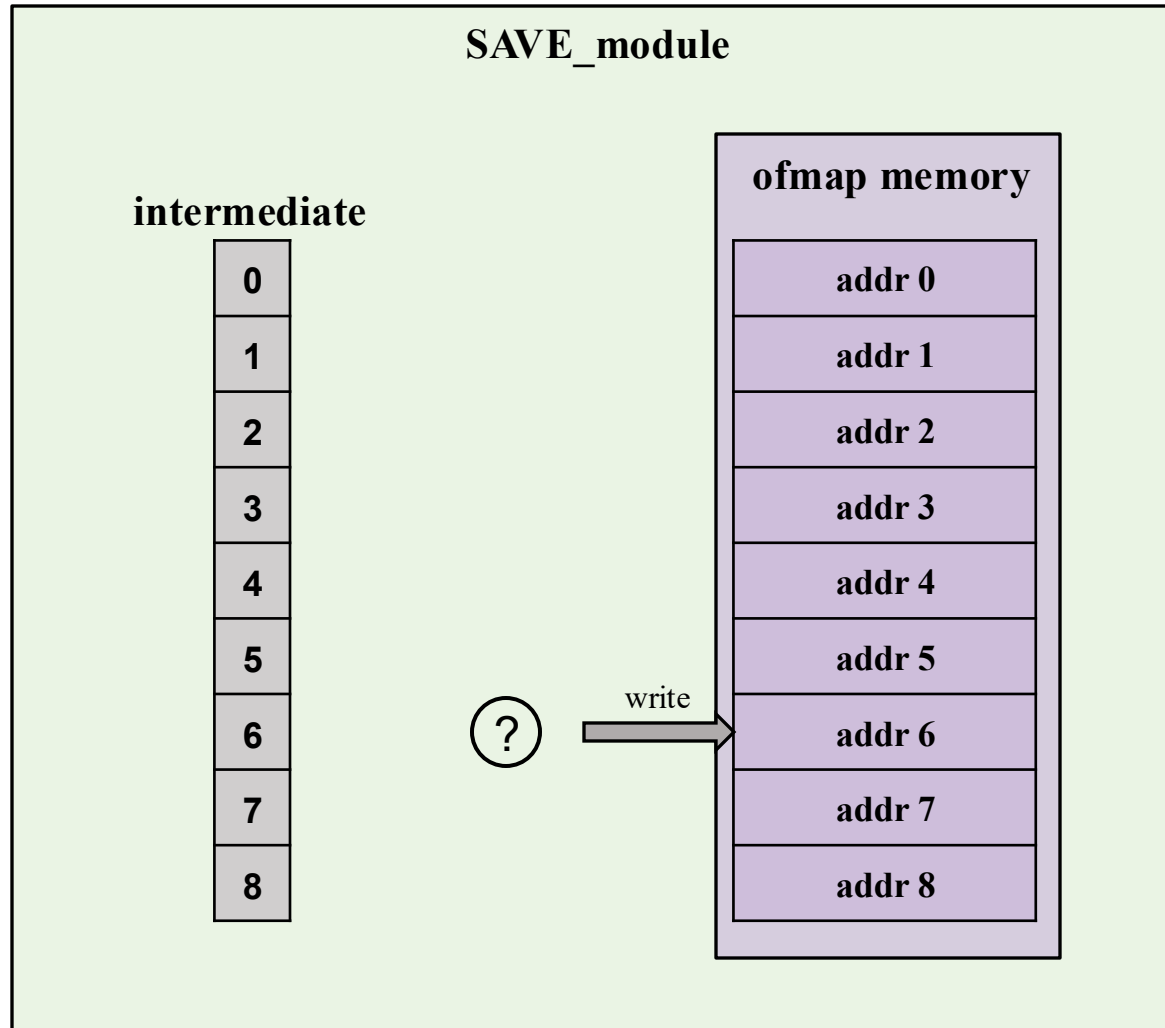
## 8. SAVE\_module: summarization



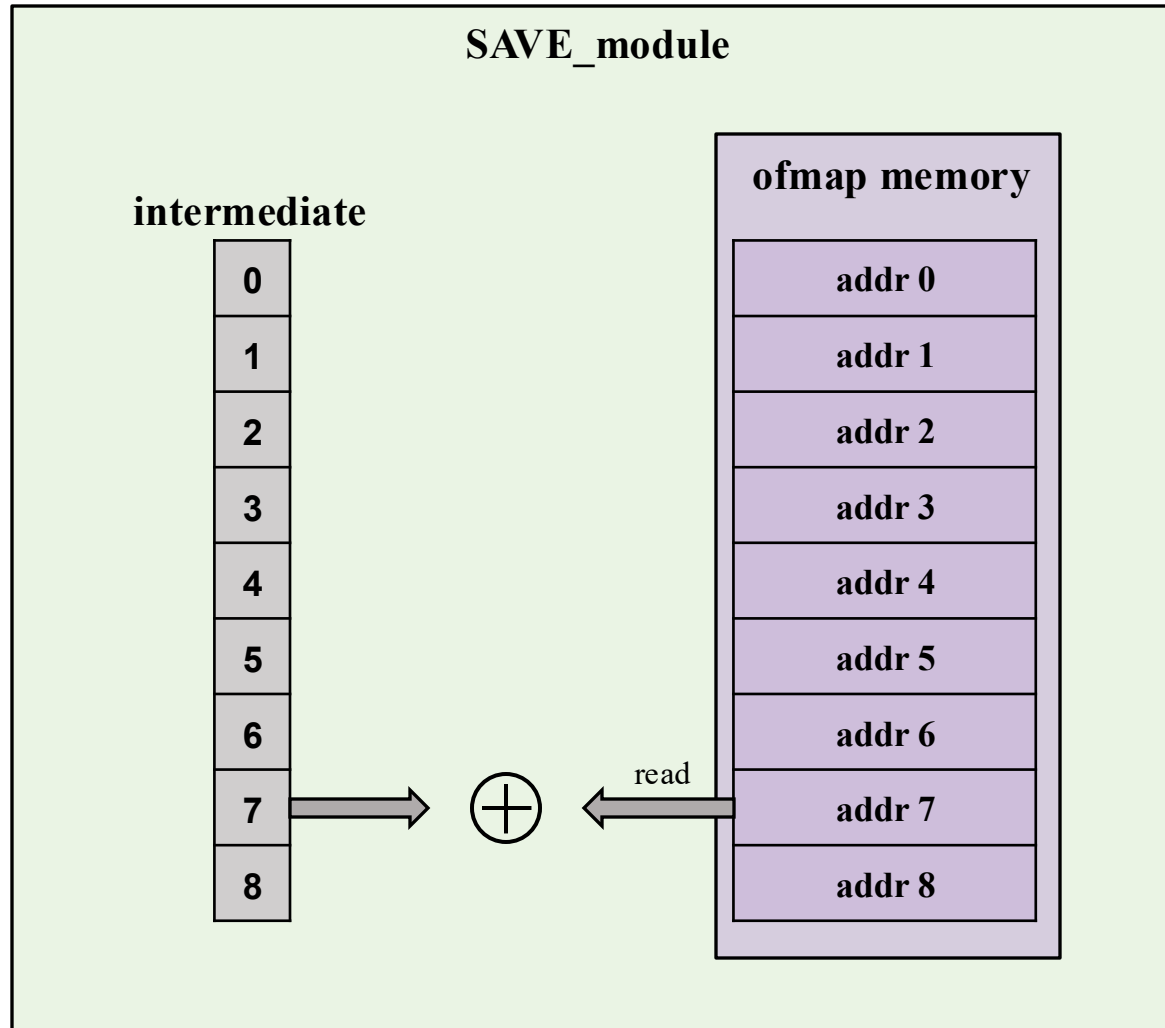
## 8. SAVE\_module: summarization



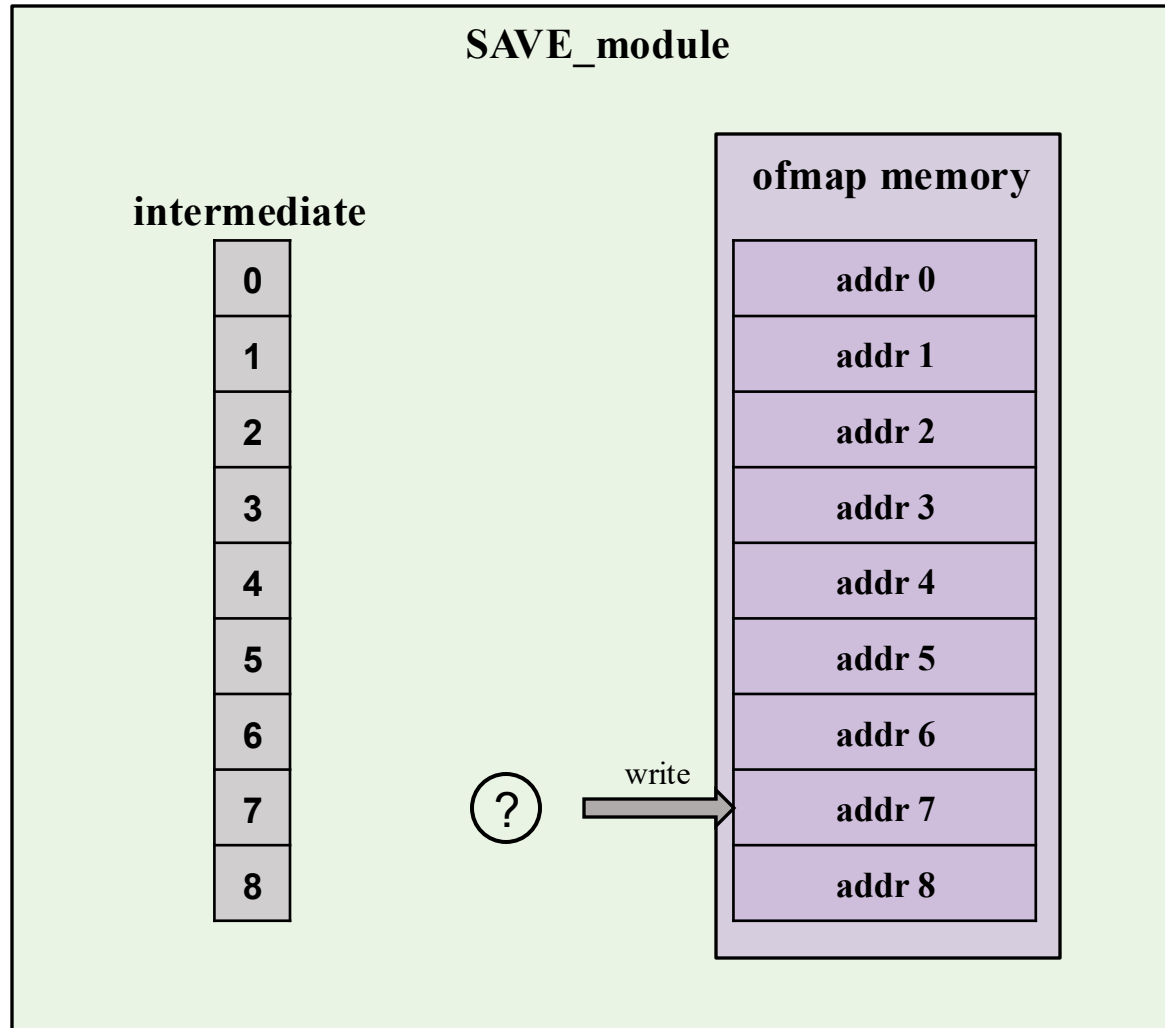
## 8. SAVE\_module: summarization



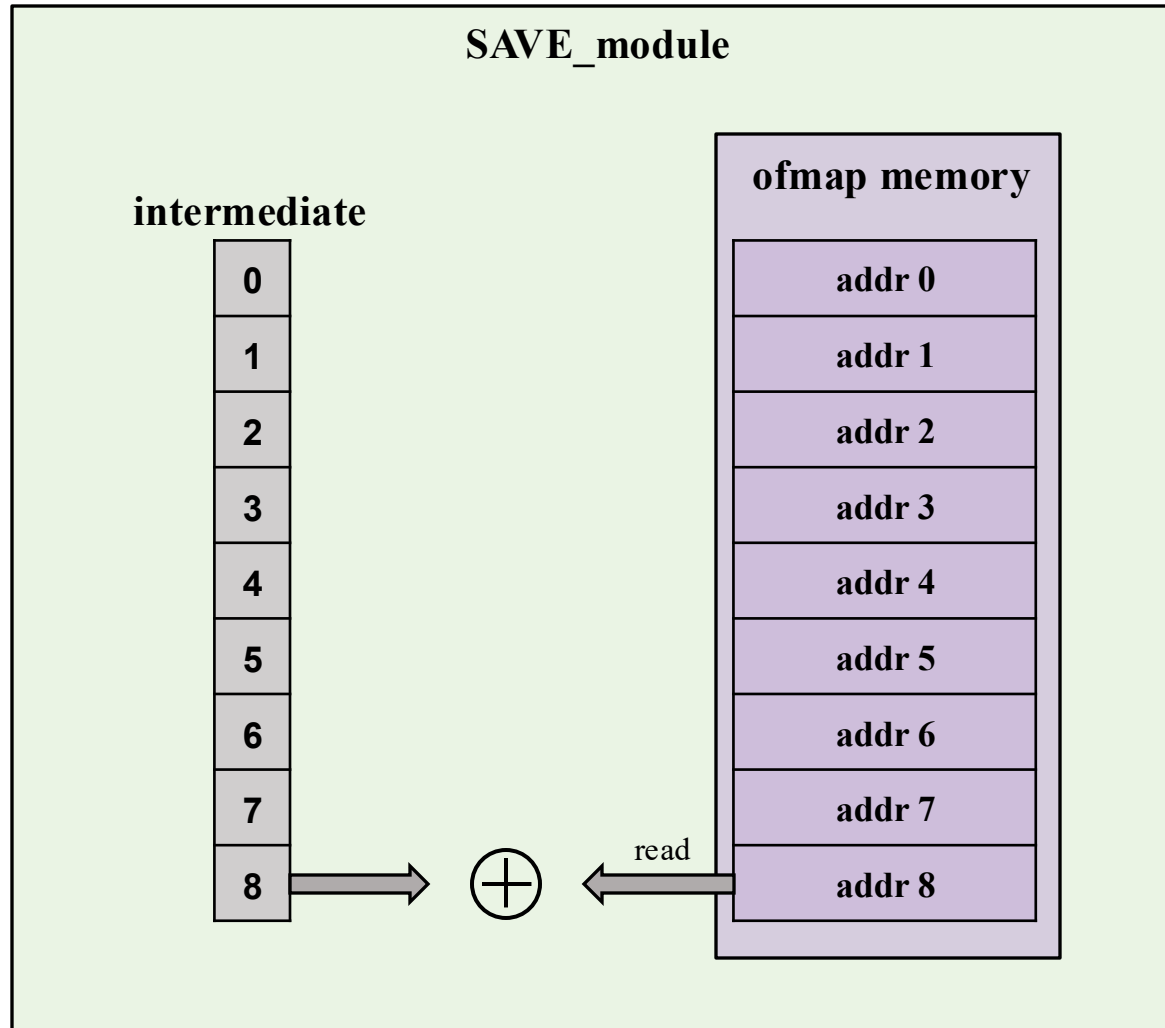
## 8. SAVE\_module: summarization



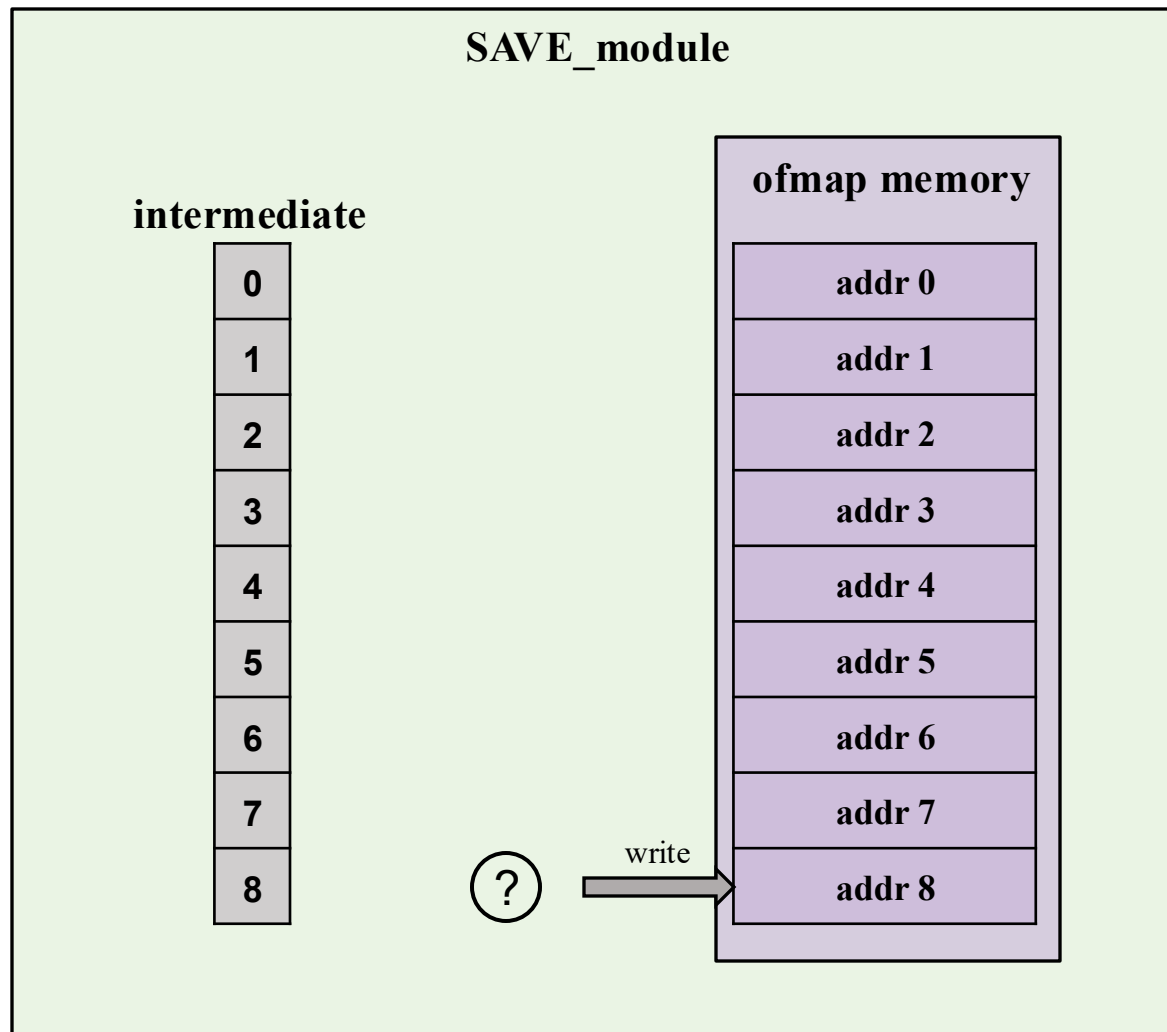
## 8. SAVE\_module: summarization



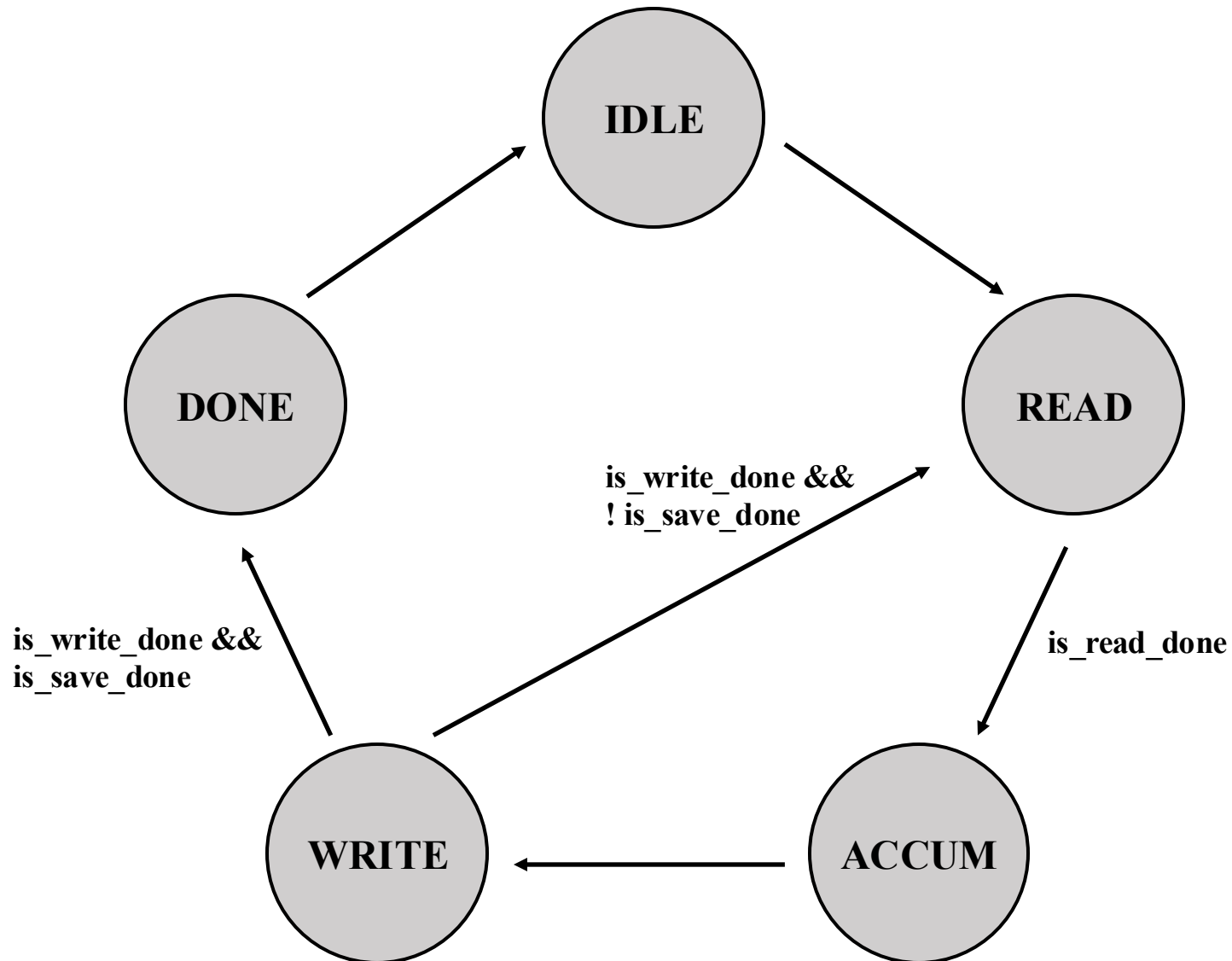
## 8. SAVE\_module: summarization



## 8. SAVE\_module: summarization



## 8. SAVE\_module: FSM

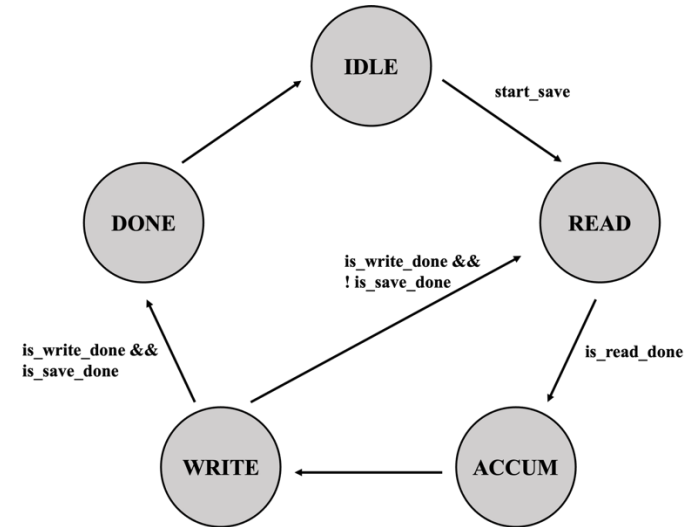


# 8. SAVE\_module: FSM

```
// *_**_**_** FSM *_**_**_**  
localparam IDLE = 3'b000, READ = 3'b001, ACCUM = 3'b010, WRITE = 3'b011, DONE = 3'b100;  
//  
reg [1:0] c_state;  
reg [1:0] n_state;  
//  
always @(posedge clk or negedge reset_n) begin  
    if(!reset_n) c_state <= IDLE;  
    else c_state <= n_state;  
end  
//  
always @(*) begin  
    case(c_state)  
        IDLE : n_state = start_save ? READ : IDLE;  
        READ : n_state = is_read_done ? ACCUM : READ;  
        ACCUM : n_state = WRITE;  
        WRITE : n_state = is_write_done ? (is_save_done ? DONE : READ) : WRITE;  
        DONE : n_state = IDLE;  
    endcase  
end  
  
//  
//  
//
```

```
// *_**_**_** STATE SIGNAL *_**_**_**  
wire o_idle, o_read, o_accum, o_write, o_done;  
assign o_idle = c_state == IDLE;  
assign o_read = c_state == READ;  
assign o_accum = c_state == ACCUM;  
assign o_write = c_state == WRITE;  
assign o_done = c_state == DONE;
```

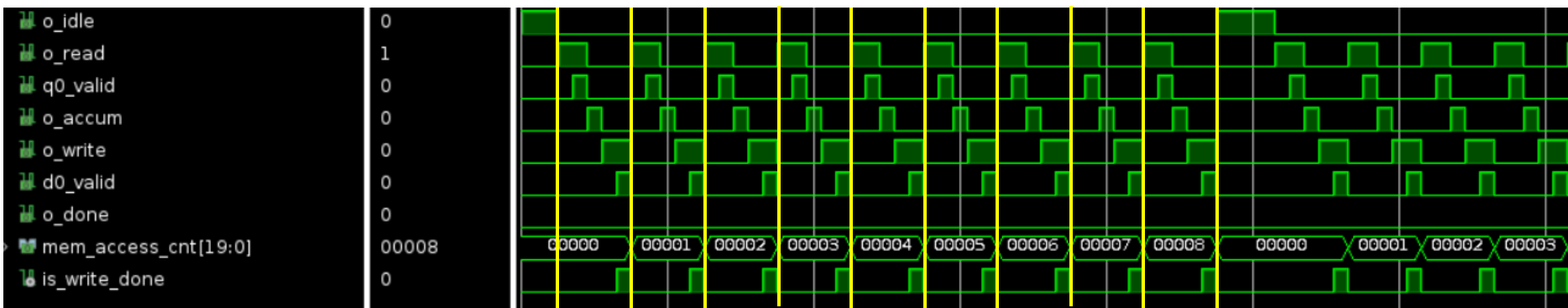
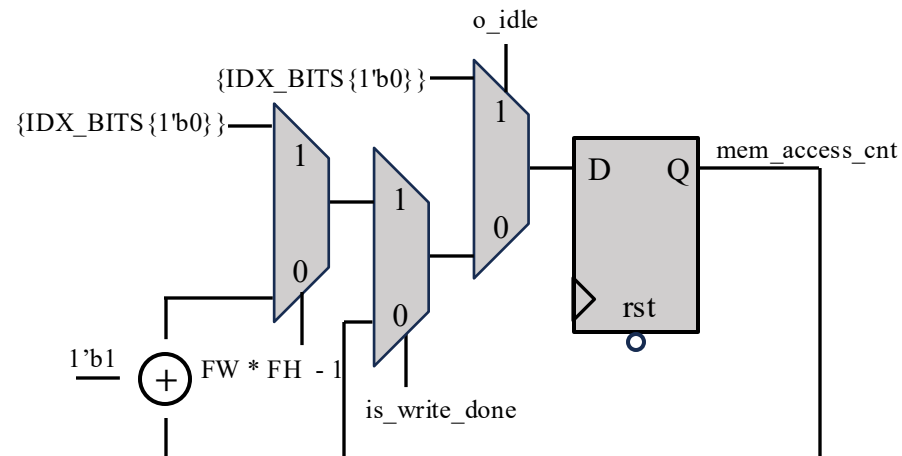
```
// *_**_**_** STATE CONTROL SIGNAL *_**_**_**  
wire start_save;  
wire is_read_done;  
wire is_write_done;  
//  
assign start_save = i_save_from_CT;  
assign is_read_done = o_read && q0_valid;  
assign is_write_done = o_write && d0_valid;  
//  
assign is_save_done = is_write_done && mem_access_cnt == FH * FW - 1;
```



# 8. SAVE\_module: mem access cnt

```
// *_*_*_*_* MEM_ACCESS_CNT *_*_*_*_*
```

```
reg [IDX_BITS-1:0] mem_access_cnt;
//
always @(posedge clk or negedge reset_n) begin
    if(!reset_n) mem_access_cnt <= {IDX_BITS{1'b0}};
    //
    else if(o_idle) mem_access_cnt <= {IDX_BITS{1'b0}};
    //
    else if(is_write_done) begin
        if(mem_access_cnt == FW * FH -1)
            mem_access_cnt <= {IDX_BITS{1'b0}};
        else mem_access_cnt <= mem_access_cnt + 1'b1;
    end
end
```



## 8. SAVE\_module: make itmd idx

```
// *_*_*_*_* MAKE INTERMEDIATE IDX -*_*_*_*_*
reg [IDX_BITS-1 : 0] intermediate_col_idx;
reg [IDX_BITS-1 : 0] intermediate_row_idx;
reg [IDX_BITS-1 : 0] intermediate_channel_idx;
//
always @(posedge clk or negedge reset_n) begin
    if(!reset_n) begin
        intermediate_col_idx <= {IDX_BITS{1'b0}};
        intermediate_row_idx <= {IDX_BITS{1'b0}};
        intermediate_channel_idx <= {IDX_BITS{1'b0}};
    end
    //
    else if(o_read && !is_read_done) begin
        if(intermediate_col_idx == FW-1) begin
            intermediate_col_idx <= {IDX_BITS{1'b0}};
            //
            if(intermediate_row_idx == FH-1) begin
                intermediate_channel_idx <= intermediate_channel_idx + 1'b1;
                intermediate_row_idx <= {IDX_BITS{1'b0}};
            end
            //
            else intermediate_row_idx <= intermediate_row_idx + 1'b1;
        end
        //
        else intermediate_col_idx <= intermediate_col_idx + 1'b1;
    end
end

// The Verilog code above performs the same function as the following triple for-loop.

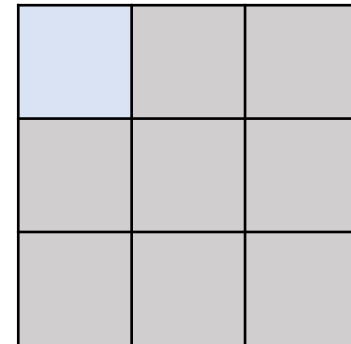
// C language
int itmd_col_idx;
int itmd_row_idx;
int itmd_channel_idx;

for(itmd_channel_idx = 0; itmd_channel_idx < IN_C; itmd_channel_idx++)
    for(itmd_row_idx = 0; itmd_row_idx < FH; itmd_row_idx++)
        for(itmd_col_idx = 0; itmd_col_idx < FW; itmd_col_idx++)
```

## 8. SAVE\_module: make itmd idx

```
// *_*_*_*_* MAKE INTERMEDIATE IDX - *_*_*_*_*
reg [IDX_BITS-1 : 0] intermediate_col_idx;
reg [IDX_BITS-1 : 0] intermediate_row_idx;
reg [IDX_BITS-1 : 0] intermediate_channel_idx;
//
always @(posedge clk or negedge reset_n) begin
    if(!reset_n) begin
        intermediate_col_idx <= {IDX_BITS{1'b0}};
        intermediate_row_idx <= {IDX_BITS{1'b0}};
        intermediate_channel_idx <= {IDX_BITS{1'b0}};
    end
    //
    else if(o_read && !is_read_done) begin
        if(intermediate_col_idx == FW-1) begin
            intermediate_col_idx <= {IDX_BITS{1'b0}};
            //
            if(intermediate_row_idx == FH-1) begin
                intermediate_channel_idx <= intermediate_channel_idx + 1'b1;
                intermediate_row_idx <= {IDX_BITS{1'b0}};
            end
            //
            else intermediate_row_idx <= intermediate_row_idx + 1'b1;
        end
        //
        else intermediate_col_idx <= intermediate_col_idx + 1'b1;
    end
end
```

intermediate

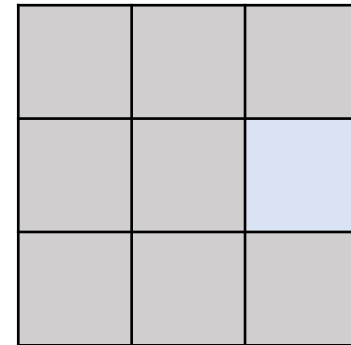


- itmd\_channel\_idx = 0
- itmd\_row\_idx = 0
- itmd\_col\_idx = 0

## 8. SAVE\_module: make itmd idx

```
// *_*_*_*_*_* MAKE INTERMEDIATE IDX - *_*_*_*_*_*
reg [IDX_BITS-1 : 0] intermediate_col_idx;
reg [IDX_BITS-1 : 0] intermediate_row_idx;
reg [IDX_BITS-1 : 0] intermediate_channel_idx;
//
always @(posedge clk or negedge reset_n) begin
    if(!reset_n) begin
        intermediate_col_idx <= {IDX_BITS{1'b0}};
        intermediate_row_idx <= {IDX_BITS{1'b0}};
        intermediate_channel_idx <= {IDX_BITS{1'b0}};
    end
    //
    else if(o_read && !is_read_done) begin
        if(intermediate_col_idx == FW-1) begin
            intermediate_col_idx <= {IDX_BITS{1'b0}};
            //
            if(intermediate_row_idx == FH-1) begin
                intermediate_channel_idx <= intermediate_channel_idx + 1'b1;
                intermediate_row_idx <= {IDX_BITS{1'b0}};
            end
            //
            else intermediate_row_idx <= intermediate_row_idx + 1'b1;
        end
        //
        else intermediate_col_idx <= intermediate_col_idx + 1'b1;
    end
end
```

intermediate

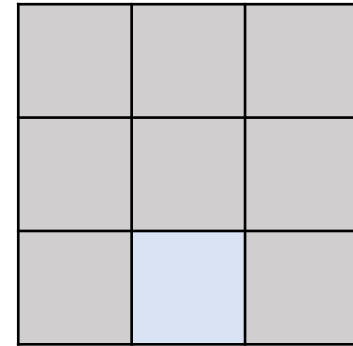


- itmd\_channel\_idx = 0
- itmd\_row\_idx = 1
- itmd\_col\_idx = 2

## 8. SAVE\_module: make itmd idx

```
// *_*_*_*_* MAKE INTERMEDIATE IDX - *_*_*_*_*
reg [IDX_BITS-1 : 0] intermediate_col_idx;
reg [IDX_BITS-1 : 0] intermediate_row_idx;
reg [IDX_BITS-1 : 0] intermediate_channel_idx;
//
always @(posedge clk or negedge reset_n) begin
    if(!reset_n) begin
        intermediate_col_idx <= {IDX_BITS{1'b0}};
        intermediate_row_idx <= {IDX_BITS{1'b0}};
        intermediate_channel_idx <= {IDX_BITS{1'b0}};
    end
    //
    else if(o_read && !is_read_done) begin
        if(intermediate_col_idx == FW-1) begin
            intermediate_col_idx <= {IDX_BITS{1'b0}};
            //
            if(intermediate_row_idx == FH-1) begin
                intermediate_channel_idx <= intermediate_channel_idx + 1'b1;
                intermediate_row_idx <= {IDX_BITS{1'b0}};
            end
            //
            else intermediate_row_idx <= intermediate_row_idx + 1'b1;
        end
        //
        else intermediate_col_idx <= intermediate_col_idx + 1'b1;
    end
end
```

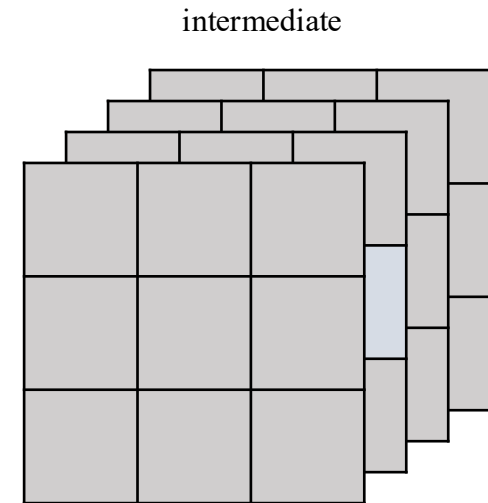
intermediate



- itmd\_channel\_idx = 0
- itmd\_row\_idx = 2
- itmd\_col\_idx = 1

## 8. SAVE\_module: make itmd idx

```
// *_*_*_*_* MAKE INTERMEDIATE IDX -*_*_*_*_*
reg [IDX_BITS-1 : 0] intermediate_col_idx;
reg [IDX_BITS-1 : 0] intermediate_row_idx;
reg [IDX_BITS-1 : 0] intermediate_channel_idx;
//
always @(posedge clk or negedge reset_n) begin
    if(!reset_n) begin
        intermediate_col_idx <= {IDX_BITS{1'b0}};
        intermediate_row_idx <= {IDX_BITS{1'b0}};
        intermediate_channel_idx <= {IDX_BITS{1'b0}};
    end
    //
    else if(o_read && !is_read_done) begin
        if(intermediate_col_idx == FW-1) begin
            intermediate_col_idx <= {IDX_BITS{1'b0}};
            //
            if(intermediate_row_idx == FH-1) begin
                intermediate_channel_idx <= intermediate_channel_idx + 1'b1;
                intermediate_row_idx <= {IDX_BITS{1'b0}};
            end
            //
            else intermediate_row_idx <= intermediate_row_idx + 1'b1;
        end
        //
        else intermediate_col_idx <= intermediate_col_idx + 1'b1;
    end
end
```

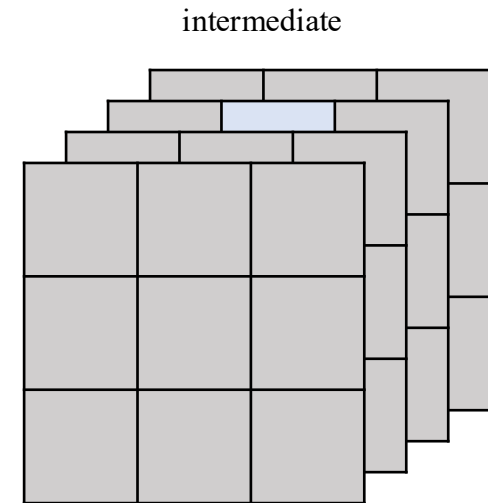


- itmd\_channel\_idx = 1
- itmd\_row\_idx = 2
- itmd\_col\_idx = 2

- In the computation between an ifmap and a kernel for a single channel, the intermediate feature map is generated with the shape (IW \* IH, KH, KW).
- The intermediate channel index indicates which of the IW \* IH intermediate channels is currently being processed.

## 8. SAVE\_module: make itmd idx

```
// *_*_*_*_*_* MAKE INTERMEDIATE IDX - *_*_*_*_*_*
reg [IDX_BITS-1 : 0] intermediate_col_idx;
reg [IDX_BITS-1 : 0] intermediate_row_idx;
reg [IDX_BITS-1 : 0] intermediate_channel_idx;
//
always @(posedge clk or negedge reset_n) begin
    if(!reset_n) begin
        intermediate_col_idx <= {IDX_BITS{1'b0}};
        intermediate_row_idx <= {IDX_BITS{1'b0}};
        intermediate_channel_idx <= {IDX_BITS{1'b0}};
    end
    //
    else if(o_read && !is_read_done) begin
        if(intermediate_col_idx == FW-1) begin
            intermediate_col_idx <= {IDX_BITS{1'b0}};
            //
            if(intermediate_row_idx == FH-1) begin
                intermediate_channel_idx <= intermediate_channel_idx + 1'b1;
                intermediate_row_idx <= {IDX_BITS{1'b0}};
            end
            //
            else intermediate_row_idx <= intermediate_row_idx + 1'b1;
        end
        //
        else intermediate_col_idx <= intermediate_col_idx + 1'b1;
    end
end
```



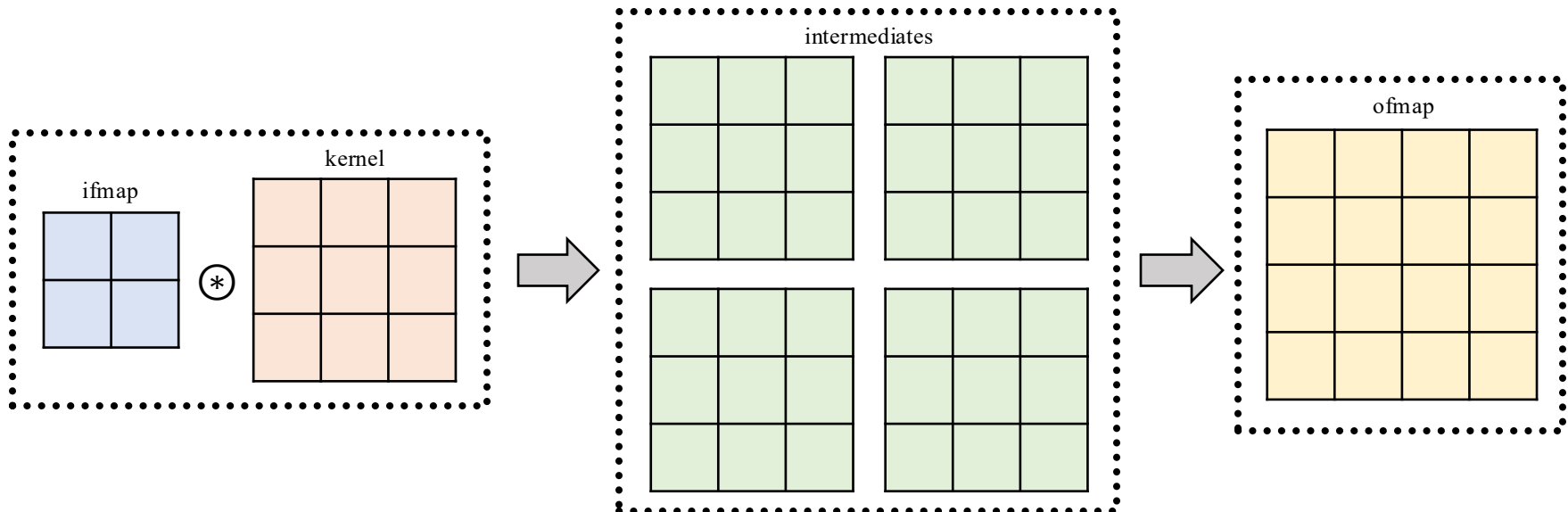
- itmd\_channel\_idx = 2
- itmd\_row\_idx = 0
- itmd\_col\_idx = 1

- In the computation between an ifmap and a kernel for a single channel, the intermediate feature map is generated with the shape (IW \* IH, KH, KW).
- The intermediate channel index indicates which of the IW \* IH intermediate channels is currently being processed.

## 8. SAVE\_module: make ofmap idx

```
// *_*_*_*_*_* MAKE OFMAP IDX *_*_*_*_*_*  
wire [IDX_BITS-1 : 0] orig_ofmap_col_idx;  
wire [IDX_BITS-1 : 0] orig_ofmap_row_idx;  
//  
assign orig_ofmap_col_idx = intermediate_col_idx + S * captured_ifmap_col;  
assign orig_ofmap_row_idx = intermediate_row_idx + S * captured_ifmap_row;
```

Assume)  $S = 1$ ,  $PAD = 0$



# 8. SAVE\_module: make ofmap idx

```
// *_*_*_*_*_* MAKE OFMAP IDX *_*_*_*_*_*  
wire [IDX_BITS-1 : 0] orig_ofmap_col_idx;  
wire [IDX_BITS-1 : 0] orig_ofmap_row_idx;  
//  
assign orig_ofmap_col_idx = intermediate_col_idx + S * captured_ifmap_col;  
assign orig_ofmap_row_idx = intermediate_row_idx + S * captured_ifmap_row;
```

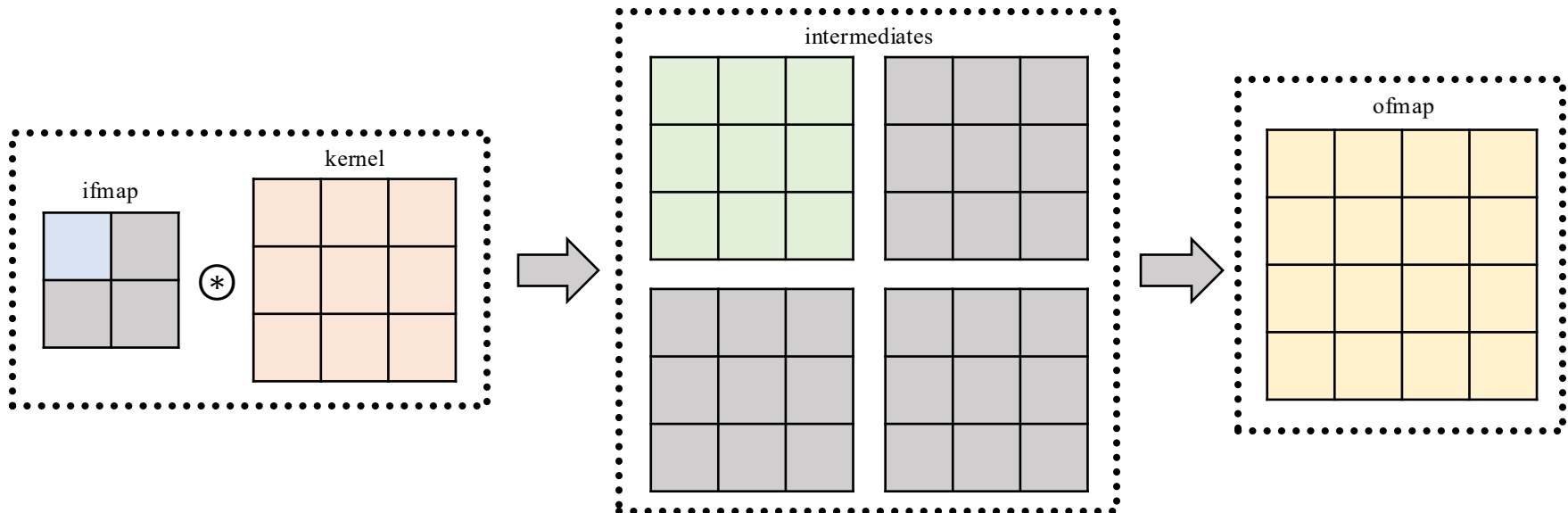
## Process 1) captured\_ifmap\_row = 0 / captured\_ifmap\_col = 0

❖  $\text{orig\_ofmap\_col\_idx}, \text{orig\_ofmap\_row\_idx} \in \mathbb{N}$

$\text{orig\_ofmap\_col\_idx} = \text{intermediate\_col\_idx} + S * 0$

$\text{orig\_ofmap\_row\_idx} = \text{intermediate\_row\_idx} + S * 0$

Assume)  $S = 1$ ,  $\text{PAD} = 0$



# 8. SAVE\_module: make ofmap idx

```
// *_*_*_*_*_* MAKE OFMAP IDX *_*_*_*_*_*  
wire [IDX_BITS-1 : 0] orig_ofmap_col_idx;  
wire [IDX_BITS-1 : 0] orig_ofmap_row_idx;  
//  
assign orig_ofmap_col_idx = intermediate_col_idx + S * captured_ifmap_col;  
assign orig_ofmap_row_idx = intermediate_row_idx + S * captured_ifmap_row;
```

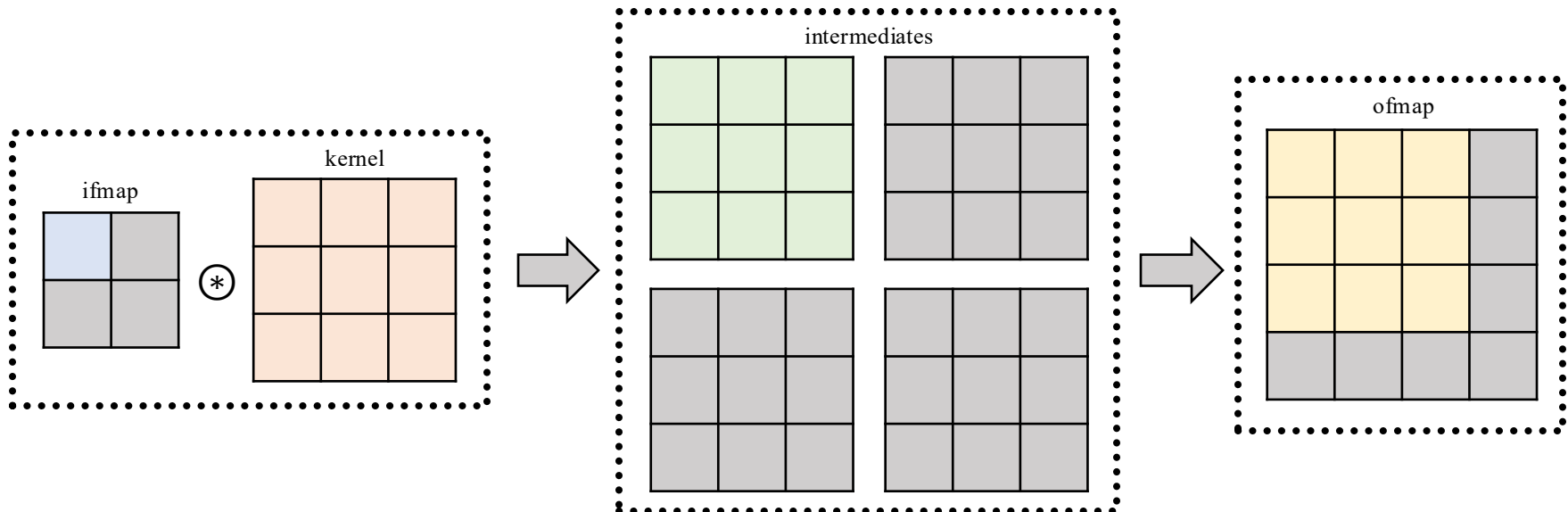
**Process 1) captured\_ifmap\_row = 0 / captured\_ifmap\_col = 0**

❖  $\text{orig\_ofmap\_col\_idx}, \text{orig\_ofmap\_row\_idx} \in \mathbb{N}$

$0 \leq \text{orig\_ofmap\_col\_idx} \leq \text{KW} - 1$

$0 \leq \text{orig\_ofmap\_row\_idx} \leq \text{KW} - 1$

Assume)  $S = 1, \text{PAD} = 0$



# 8. SAVE\_module: make ofmap idx

```
// *_*_*_*_*_* MAKE OFMAP IDX *_*_*_*_*_*  
wire [IDX_BITS-1 : 0] orig_ofmap_col_idx;  
wire [IDX_BITS-1 : 0] orig_ofmap_row_idx;  
//  
assign orig_ofmap_col_idx = intermediate_col_idx + S * captured_ifmap_col;  
assign orig_ofmap_row_idx = intermediate_row_idx + S * captured_ifmap_row;
```

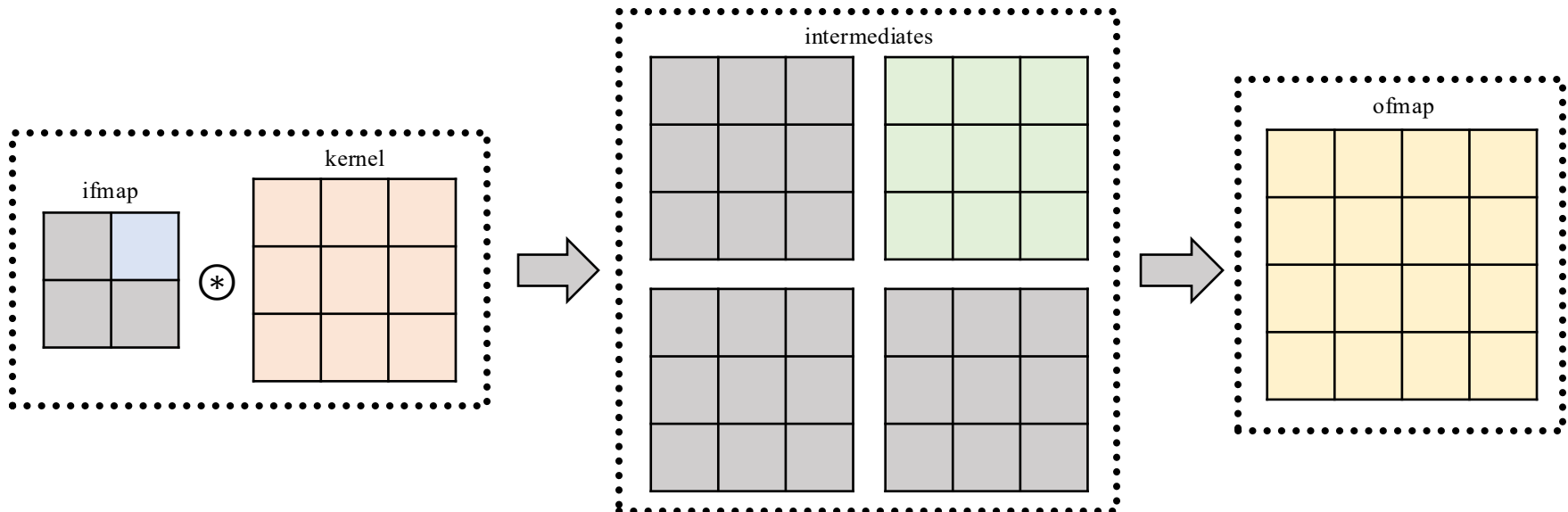
## Process 2) captured\_ifmap\_row = 0 / captured\_ifmap\_col = 1

❖  $\text{orig\_ofmap\_col\_idx}, \text{orig\_ofmap\_row\_idx} \in \mathbb{N}$

$\text{orig\_ofmap\_col\_idx} = \text{intermediate\_col\_idx} + S * 1$

$\text{orig\_ofmap\_row\_idx} = \text{intermediate\_row\_idx} + S * 0$

Assume)  $S = 1$ ,  $\text{PAD} = 0$



# 8. SAVE\_module: make ofmap idx

```
// *_*_*_*_*_* MAKE OFMAP IDX *_*_*_*_*_*  
wire [IDX_BITS-1 : 0] orig_ofmap_col_idx;  
wire [IDX_BITS-1 : 0] orig_ofmap_row_idx;  
//  
assign orig_ofmap_col_idx = intermediate_col_idx + S * captured_ifmap_col;  
assign orig_ofmap_row_idx = intermediate_row_idx + S * captured_ifmap_row;
```

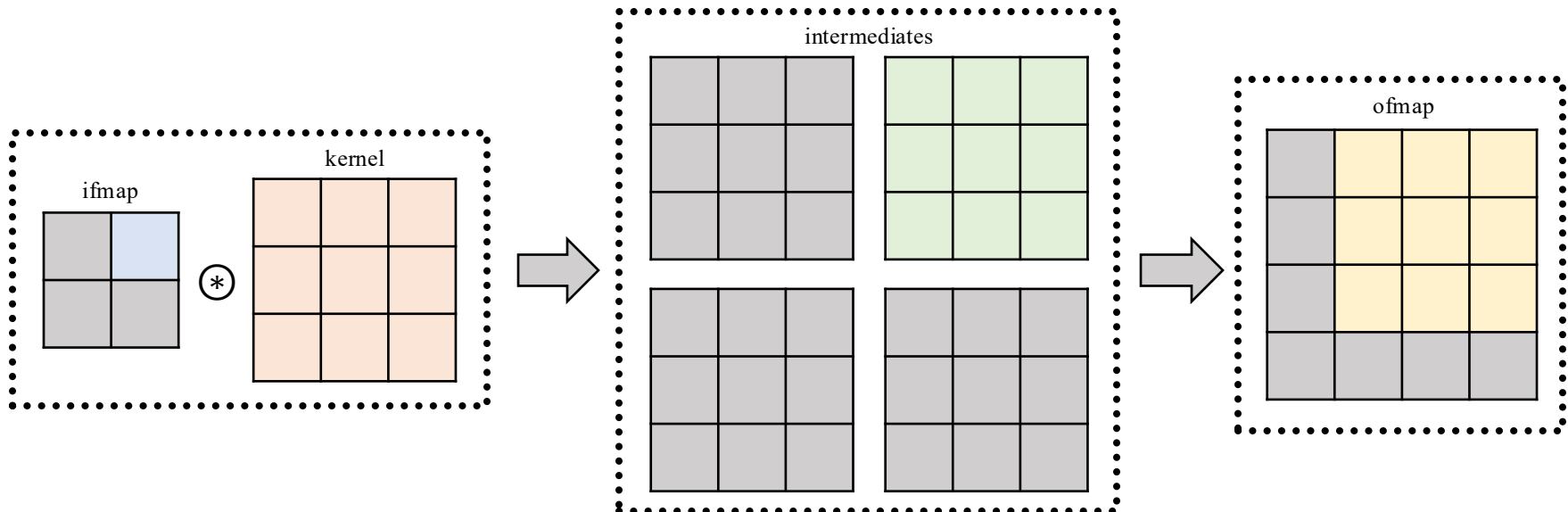
## Process 2) captured\_ifmap\_row = 0 / captured\_ifmap\_col = 1

❖  $\text{orig\_ofmap\_col\_idx}, \text{orig\_ofmap\_row\_idx} \in \mathbb{N}$

$0 + S \leq \text{orig\_ofmap\_col\_idx} \leq KW - 1 + S$

$0 \leq \text{orig\_ofmap\_row\_idx} \leq KW - 1$

Assume)  $S = 1, \text{PAD} = 0$



## 8. SAVE\_module: make ofmap idx

```
// *_*_*_*_*_* MAKE OFMAP IDX *_*_*_*_*_*  
wire [IDX_BITS-1 : 0] orig_ofmap_col_idx;  
wire [IDX_BITS-1 : 0] orig_ofmap_row_idx;  
//  
assign orig_ofmap_col_idx = intermediate_col_idx + S * captured_ifmap_col;  
assign orig_ofmap_row_idx = intermediate_row_idx + S * captured_ifmap_row;
```

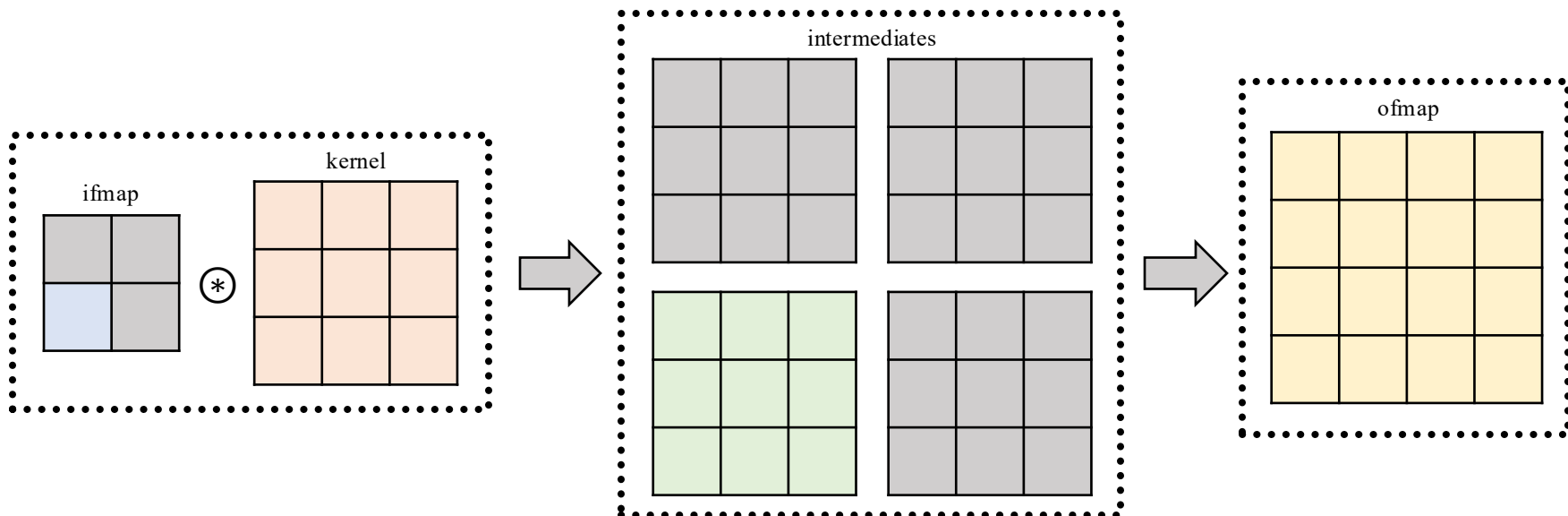
### Process 3) $\text{captured\_ifmap\_row} = 1 / \text{captured\_ifmap\_col} = 0$

❖  $\text{orig\_ofmap\_col\_idx}, \text{orig\_ofmap\_row\_idx} \in \mathbb{N}$

$\text{orig\_ofmap\_col\_idx} = \text{intermediate\_col\_idx} + S * 0$

$\text{orig\_ofmap\_row\_idx} = \text{intermediate\_row\_idx} + S * 1$

Assume)  $S = 1, \text{PAD} = 0$



## 8. SAVE\_module: make ofmap idx

```
// *_*_*_*_*_* MAKE OFMAP IDX *_*_*_*_*_*  
wire [IDX_BITS-1 : 0] orig_ofmap_col_idx;  
wire [IDX_BITS-1 : 0] orig_ofmap_row_idx;  
//  
assign orig_ofmap_col_idx = intermediate_col_idx + S * captured_ifmap_col;  
assign orig_ofmap_row_idx = intermediate_row_idx + S * captured_ifmap_row;
```

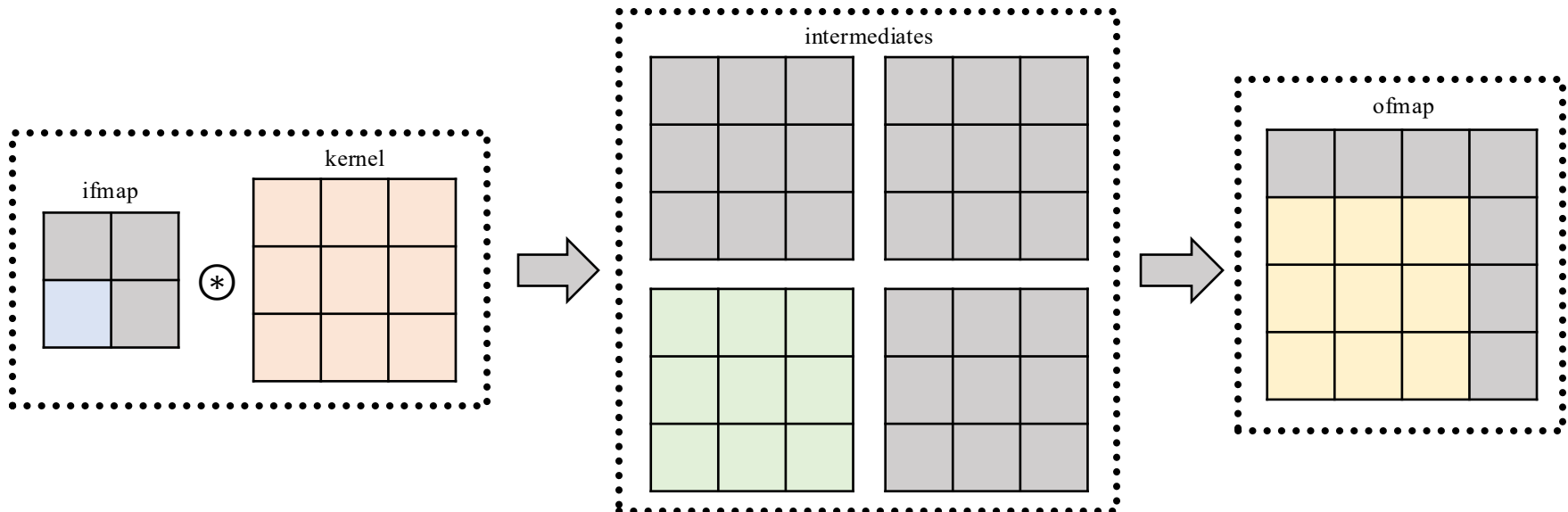
**Process 3) captured\_ifmap\_row = 1 / captured\_ifmap\_col = 0**

❖  $\text{orig\_ofmap\_col\_idx}, \text{orig\_ofmap\_row\_idx} \in \mathbb{N}$

$0 \leq \text{orig\_ofmap\_col\_idx} \leq KW - 1$

$0 + S \leq \text{orig\_ofmap\_row\_idx} \leq KW - 1 + S$

Assume)  $S = 1, \text{PAD} = 0$



# 8. SAVE\_module: make ofmap idx

```
// *_*_*_*_*_* MAKE OFMAP IDX *_*_*_*_*_*  
wire [IDX_BITS-1 : 0] orig_ofmap_col_idx;  
wire [IDX_BITS-1 : 0] orig_ofmap_row_idx;  
//  
assign orig_ofmap_col_idx = intermediate_col_idx + S * captured_ifmap_col;  
assign orig_ofmap_row_idx = intermediate_row_idx + S * captured_ifmap_row;
```

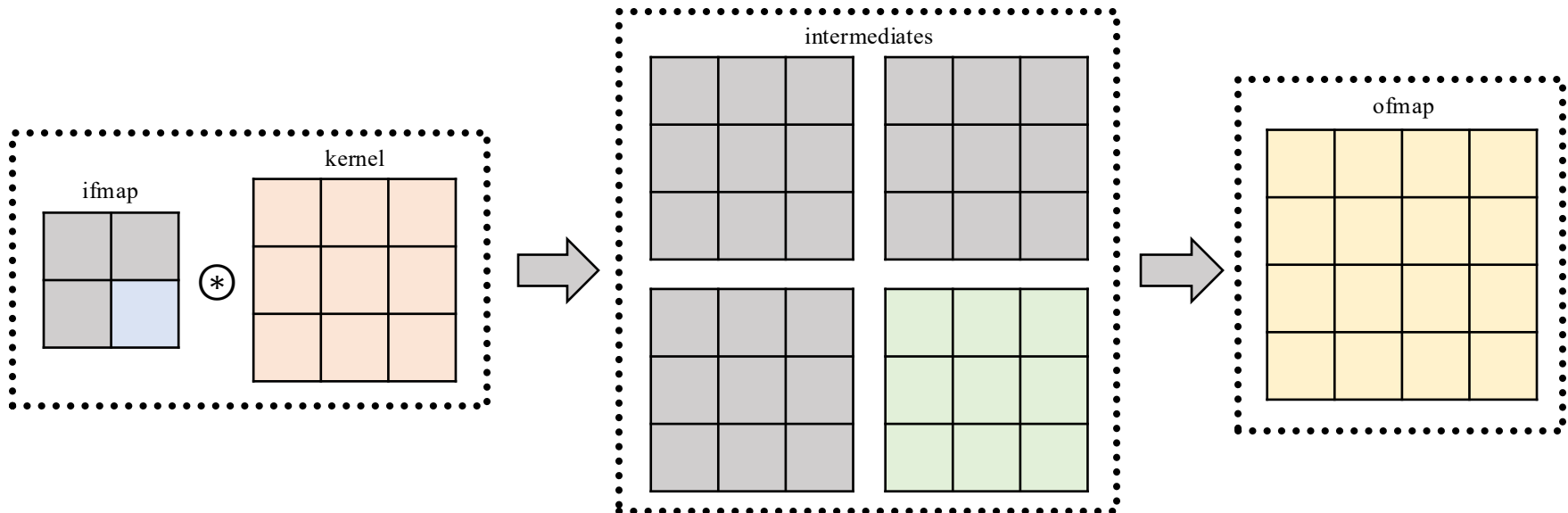
## Process 4) captured\_ifmap\_row = 1 / captured\_ifmap\_col = 1

❖ orig\_ofmap\_col\_idx, orig\_ofmap\_row\_idx  $\in \mathbb{N}$

orig\_ofmap\_col\_idx = intermediate\_col\_idx + S \* 1

orig\_ofmap\_row\_idx = intermediate\_row\_idx + S \* 1

Assume) S = 1, PAD = 0



# 8. SAVE\_module: make ofmap idx

```
// *_*_*_*_*_* MAKE OFMAP IDX *_*_*_*_*_*  
wire [IDX_BITS-1 : 0] orig_ofmap_col_idx;  
wire [IDX_BITS-1 : 0] orig_ofmap_row_idx;  
//  
assign orig_ofmap_col_idx = intermediate_col_idx + S * captured_ifmap_col;  
assign orig_ofmap_row_idx = intermediate_row_idx + S * captured_ifmap_row;
```

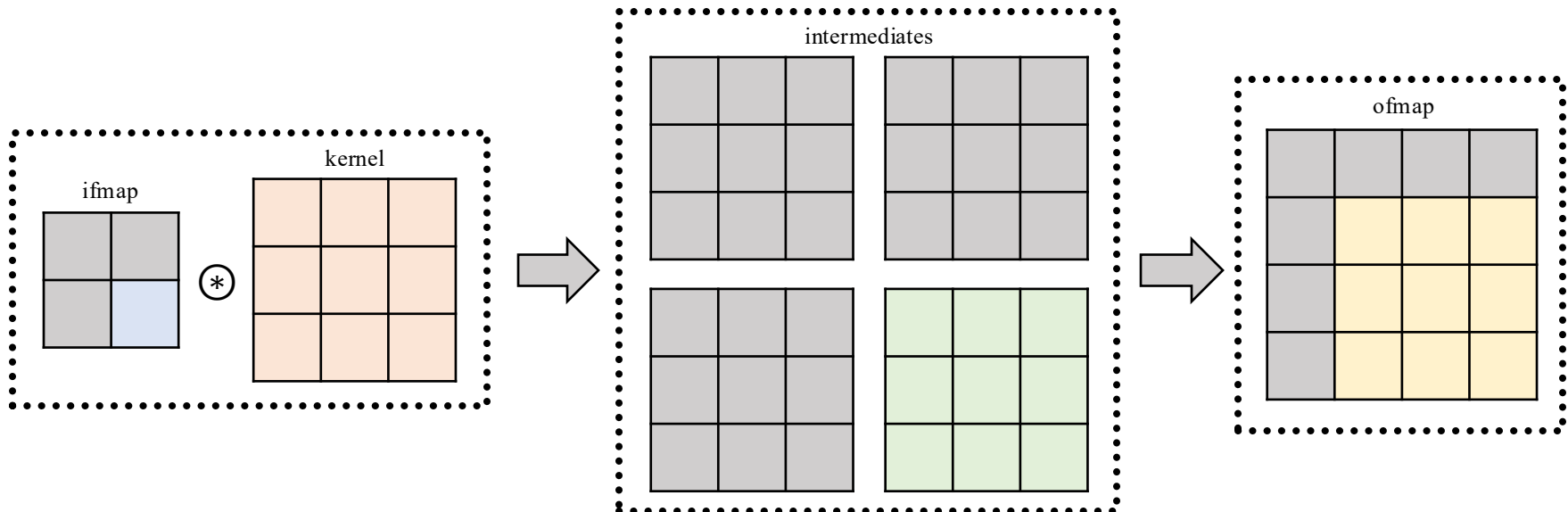
**Process 4) captured\_ifmap\_row = 1 / captured\_ifmap\_col = 1**

❖  $\text{orig\_ofmap\_col\_idx}, \text{orig\_ofmap\_row\_idx} \in \mathbb{N}$

$0 + S \leq \text{orig\_ofmap\_col\_idx} \leq KW - 1 + S$

$0 + S \leq \text{orig\_ofmap\_row\_idx} \leq KW - 1 + S$

Assume)  $S = 1, \text{PAD} = 0$



# 8. SAVE\_module: make ofmap idx

```
// *_*_*_*_*_* MAKE OFMAP IDX *_*_*_*_*_*  
wire [IDX_BITS-1 : 0] orig_ofmap_col_idx;  
wire [IDX_BITS-1 : 0] orig_ofmap_row_idx;  
//  
assign orig_ofmap_col_idx = intermediate_col_idx + S * captured_ifmap_col;  
assign orig_ofmap_row_idx = intermediate_row_idx + S * captured_ifmap_row;
```

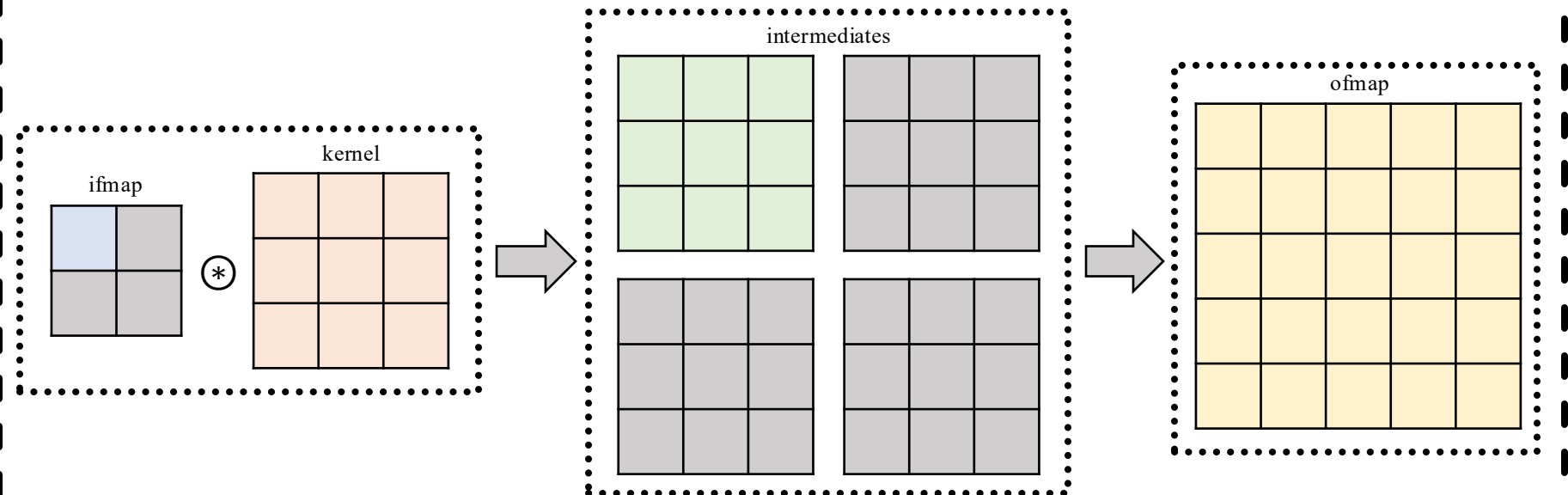
## Process 1) captured\_ifmap\_row = 0 / captured\_ifmap\_col = 0

❖  $\text{orig\_ofmap\_col\_idx}, \text{orig\_ofmap\_row\_idx} \in \mathbb{N}$

$\text{orig\_ofmap\_col\_idx} = \text{intermediate\_col\_idx} + S * 0$

$\text{orig\_ofmap\_row\_idx} = \text{intermediate\_row\_idx} + S * 0$

Assume)  $S = 2$ ,  $\text{PAD} = 0$



# 8. SAVE\_module: make ofmap idx

```
// *_*_*_*_*_* MAKE OFMAP IDX *_*_*_*_*_*  
wire [IDX_BITS-1 : 0] orig_ofmap_col_idx;  
wire [IDX_BITS-1 : 0] orig_ofmap_row_idx;  
//  
assign orig_ofmap_col_idx = intermediate_col_idx + S * captured_ifmap_col;  
assign orig_ofmap_row_idx = intermediate_row_idx + S * captured_ifmap_row;
```

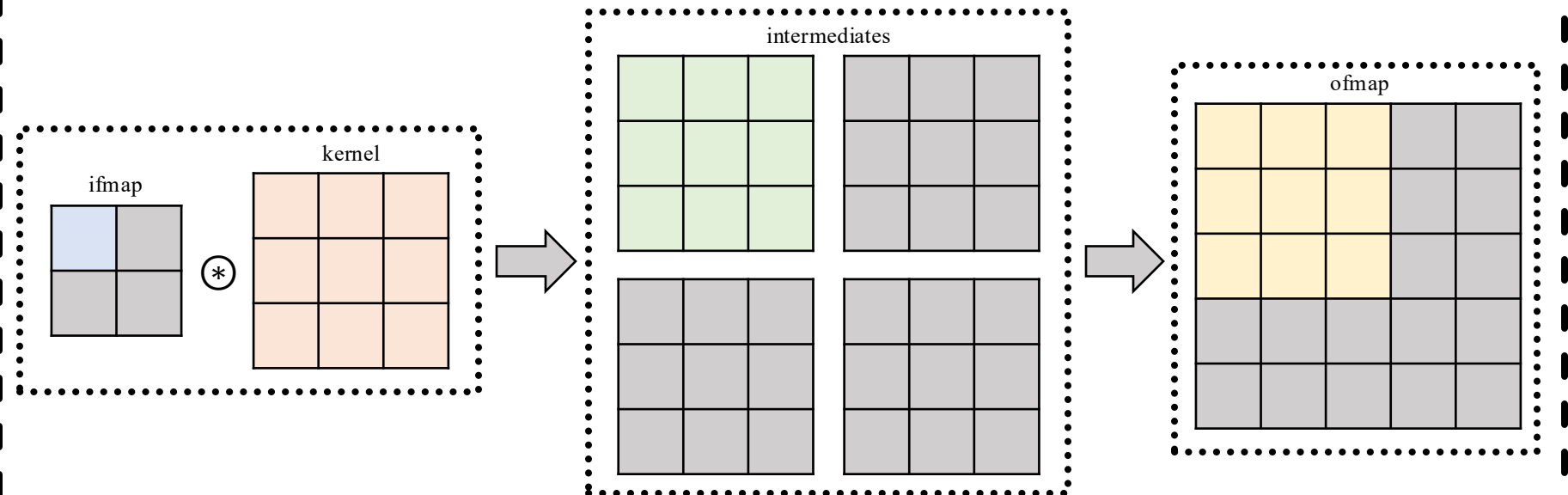
**Process 1) captured\_ifmap\_row = 0 / captured\_ifmap\_col = 0**

❖  $\text{orig\_ofmap\_col\_idx}, \text{orig\_ofmap\_row\_idx} \in \mathbb{N}$

$0 \leq \text{orig\_ofmap\_col\_idx} \leq KW - 1$

$0 \leq \text{orig\_ofmap\_row\_idx} \leq KW - 1$

Assume)  $S = 2$ ,  $PAD = 0$



## 8. SAVE\_module: make ofmap idx

```
// *_*_*_*_*_* MAKE OFMAP IDX *_*_*_*_*_*  
wire [IDX_BITS-1 : 0] orig_ofmap_col_idx;  
wire [IDX_BITS-1 : 0] orig_ofmap_row_idx;  
//  
assign orig_ofmap_col_idx = intermediate_col_idx + S * captured_ifmap_col;  
assign orig_ofmap_row_idx = intermediate_row_idx + S * captured_ifmap_row;
```

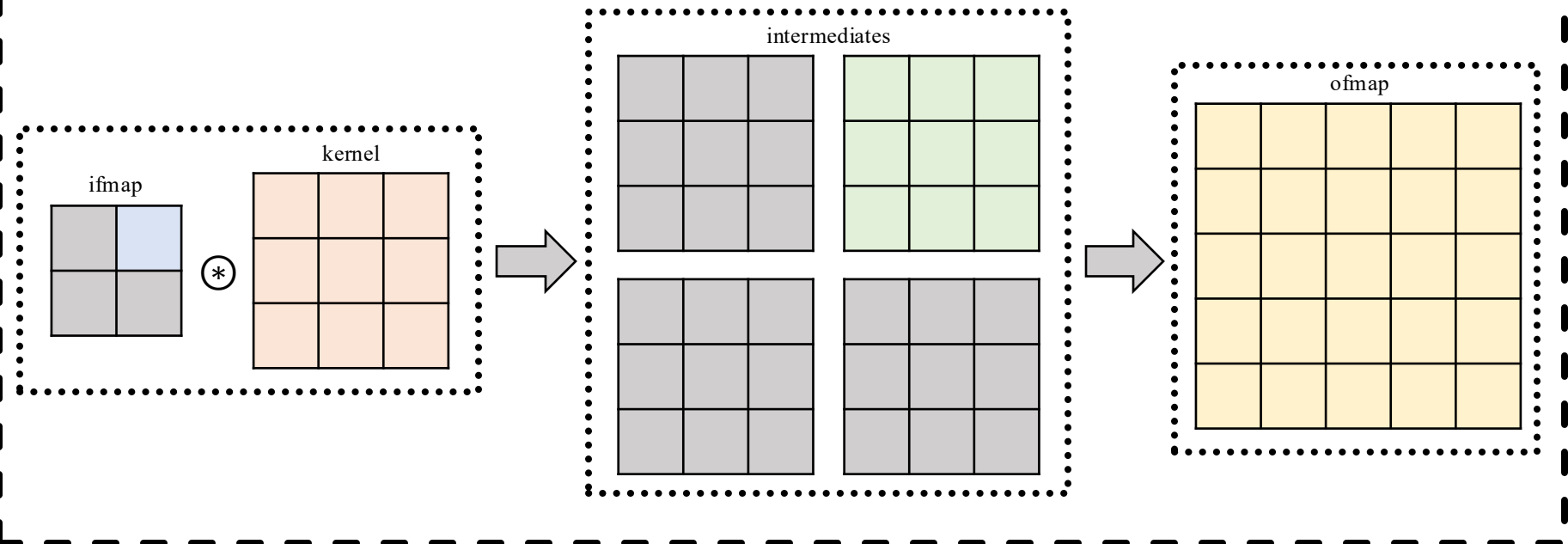
### Process 2) captured\_ifmap\_row = 0 / captured\_ifmap\_col = 1

❖  $\text{orig\_ofmap\_col\_idx}, \text{orig\_ofmap\_row\_idx} \in \mathbb{N}$

$\text{orig\_ofmap\_col\_idx} = \text{intermediate\_col\_idx} + S * 1$

$\text{orig\_ofmap\_row\_idx} = \text{intermediate\_row\_idx} + S * 0$

Assume)  $S = 2$ ,  $\text{PAD} = 0$



# 8. SAVE\_module: make ofmap idx

```
// *_*_*_*_*_* MAKE OFMAP IDX *_*_*_*_*_*  
wire [IDX_BITS-1 : 0] orig_ofmap_col_idx;  
wire [IDX_BITS-1 : 0] orig_ofmap_row_idx;  
//  
assign orig_ofmap_col_idx = intermediate_col_idx + S * captured_ifmap_col;  
assign orig_ofmap_row_idx = intermediate_row_idx + S * captured_ifmap_row;
```

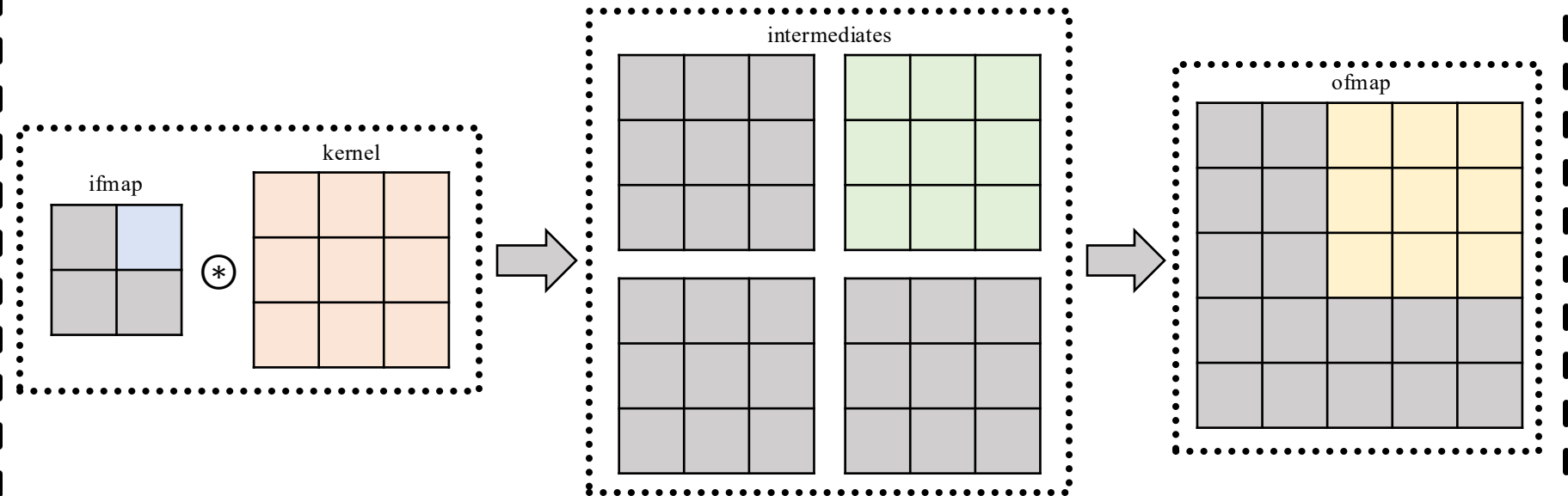
**Process 2) captured\_ifmap\_row = 0 / captured\_ifmap\_col = 1**

❖  $\text{orig\_ofmap\_col\_idx}, \text{orig\_ofmap\_row\_idx} \in \mathbb{N}$

$0 + S \leq \text{orig\_ofmap\_col\_idx} \leq \text{KW} - 1 + S$

$0 \leq \text{orig\_ofmap\_row\_idx} \leq \text{KW} - 1$

Assume)  $S = 2$ ,  $\text{PAD} = 0$



# 8. SAVE\_module: make ofmap idx

```
// *_*_*_*_*_* MAKE OFMAP IDX *_*_*_*_*_*  
wire [IDX_BITS-1 : 0] orig_ofmap_col_idx;  
wire [IDX_BITS-1 : 0] orig_ofmap_row_idx;  
//  
assign orig_ofmap_col_idx = intermediate_col_idx + S * captured_ifmap_col;  
assign orig_ofmap_row_idx = intermediate_row_idx + S * captured_ifmap_row;
```

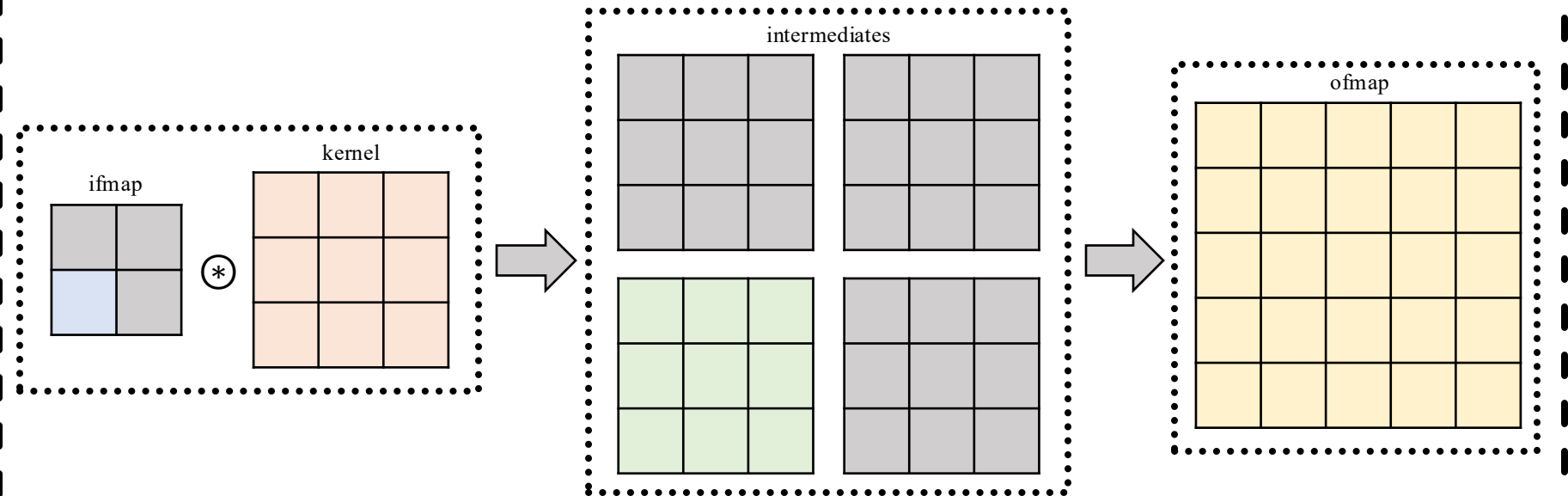
## Process 3) $\text{captured\_ifmap\_row} = 1 / \text{captured\_ifmap\_col} = 0$

❖  $\text{orig\_ofmap\_col\_idx}, \text{orig\_ofmap\_row\_idx} \in \mathbb{N}$

$\text{orig\_ofmap\_col\_idx} = \text{intermediate\_col\_idx} + S * 0$

$\text{orig\_ofmap\_row\_idx} = \text{intermediate\_row\_idx} + S * 1$

Assume)  $S = 2, \text{PAD} = 0$



# 8. SAVE\_module: make ofmap idx

```
// *_*_*_*_*_* MAKE OFMAP IDX *_*_*_*_*_*  
wire [IDX_BITS-1 : 0] orig_ofmap_col_idx;  
wire [IDX_BITS-1 : 0] orig_ofmap_row_idx;  
//  
assign orig_ofmap_col_idx = intermediate_col_idx + S * captured_ifmap_col;  
assign orig_ofmap_row_idx = intermediate_row_idx + S * captured_ifmap_row;
```

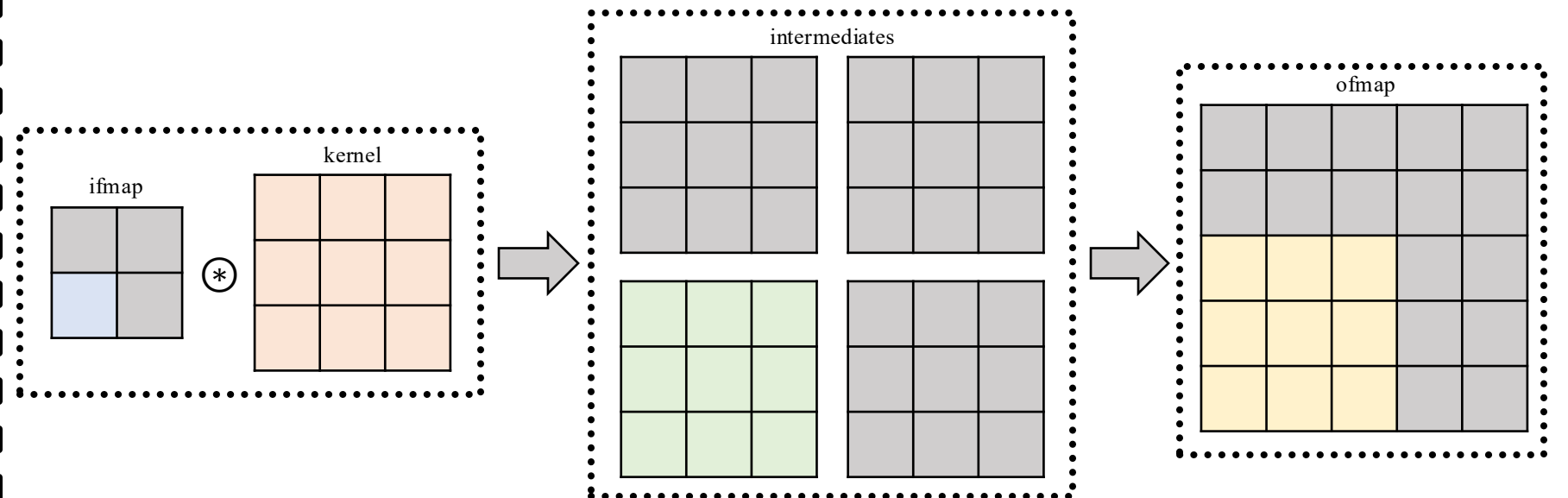
**Process 3) captured\_ifmap\_row = 1 / captured\_ifmap\_col = 0**

❖  $\text{orig\_ofmap\_col\_idx}, \text{orig\_ofmap\_row\_idx} \in \mathbb{N}$

$0 \leq \text{orig\_ofmap\_col\_idx} \leq KW - 1$

$0 + S \leq \text{orig\_ofmap\_row\_idx} \leq KW - 1 + S$

Assume)  $S = 2, \text{PAD} = 0$



# 8. SAVE\_module: make ofmap idx

```
// *_*_*_*_*_* MAKE OFMAP IDX *_*_*_*_*_*  
wire [IDX_BITS-1 : 0] orig_ofmap_col_idx;  
wire [IDX_BITS-1 : 0] orig_ofmap_row_idx;  
//  
assign orig_ofmap_col_idx = intermediate_col_idx + S * captured_ifmap_col;  
assign orig_ofmap_row_idx = intermediate_row_idx + S * captured_ifmap_row;
```

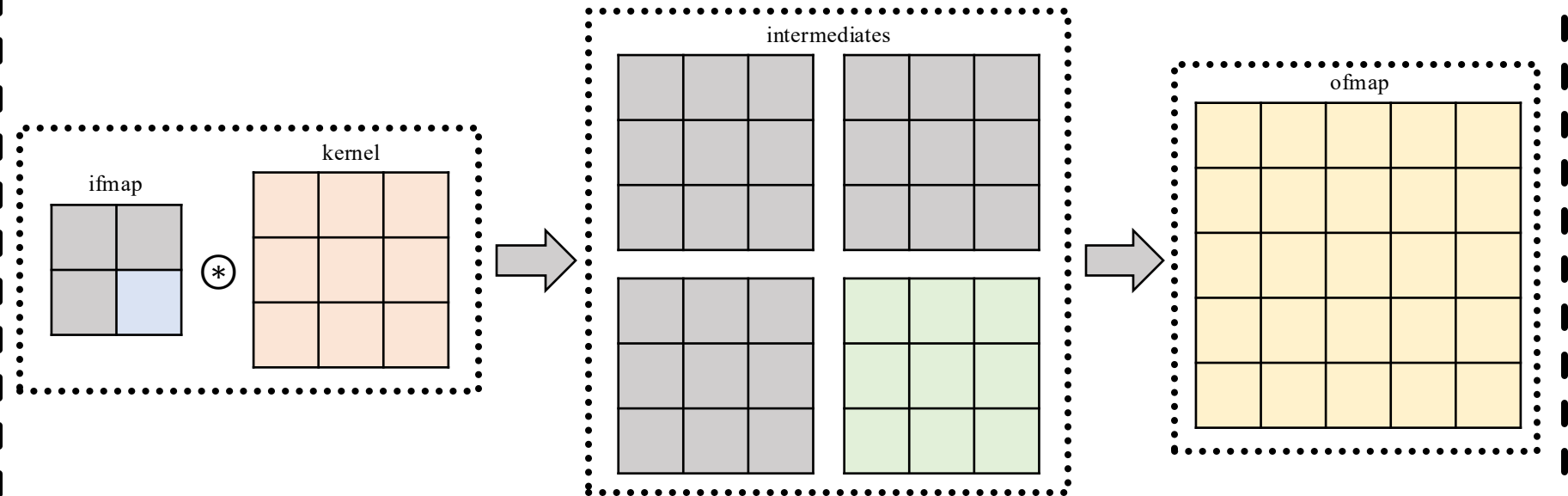
## Process 4) $\text{captured\_ifmap\_row} = 1 / \text{captured\_ifmap\_col} = 1$

❖  $\text{orig\_ofmap\_col\_idx}, \text{orig\_ofmap\_row\_idx} \in \mathbb{N}$

$\text{orig\_ofmap\_col\_idx} = \text{intermediate\_col\_idx} + S * 1$

$\text{orig\_ofmap\_row\_idx} = \text{intermediate\_row\_idx} + S * 1$

Assume)  $S = 2, \text{PAD} = 0$



## 8. SAVE\_module: make ofmap idx

```
// *_*_*_*_*_* MAKE OFMAP IDX *_*_*_*_*_*  
wire [IDX_BITS-1 : 0] orig_ofmap_col_idx;  
wire [IDX_BITS-1 : 0] orig_ofmap_row_idx;  
//  
assign orig_ofmap_col_idx = intermediate_col_idx + S * captured_ifmap_col;  
assign orig_ofmap_row_idx = intermediate_row_idx + S * captured_ifmap_row;
```

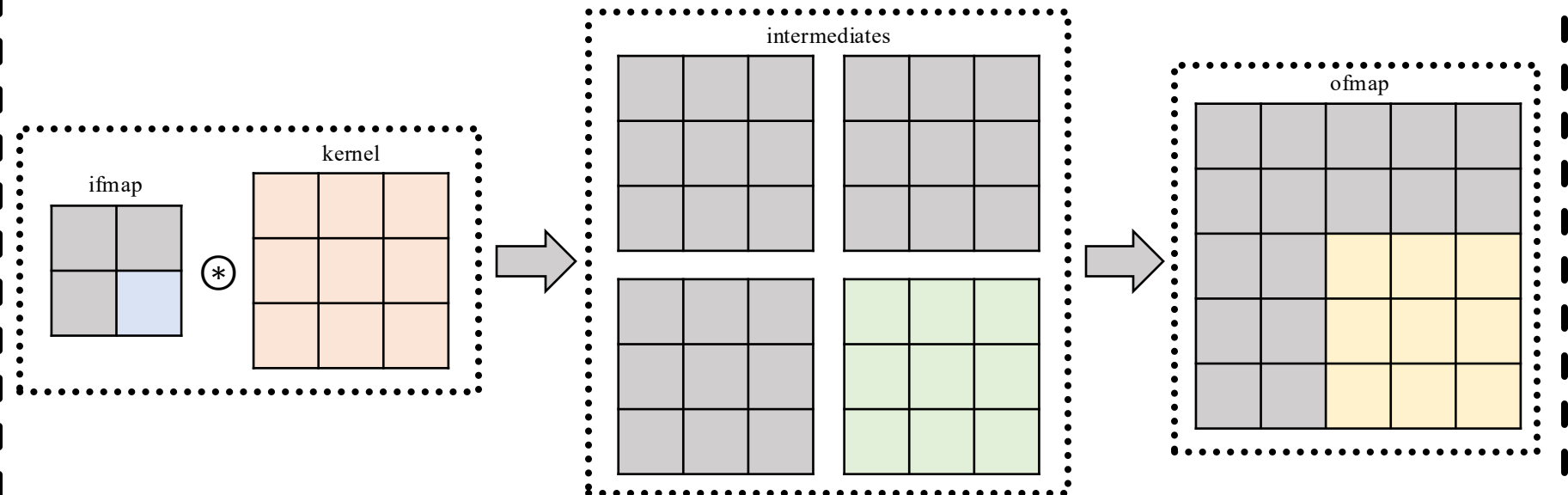
**Process 4) captured\_ifmap\_row = 1 / captured\_ifmap\_col = 1**

❖  $\text{orig\_ofmap\_col\_idx}, \text{orig\_ofmap\_row\_idx} \in \mathbb{N}$

$0 + S \leq \text{orig\_ofmap\_col\_idx} \leq KW - 1 + S$

$0 + S \leq \text{orig\_ofmap\_row\_idx} \leq KW - 1 + S$

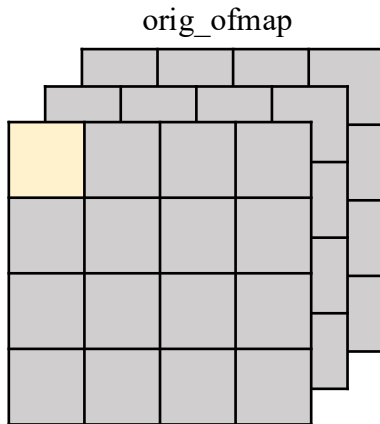
Assume)  $S = 2$ ,  $PAD = 0$



# 8. SAVE\_module: orig ofmap address

```
// *_*_*_*_ OFMAP_ADDR *_*_*_*_
reg [OFMAP_AWIDTH-1 : 0] ofmap_addr;
//
wire in_padding_region = (orig_ofmap_row_idx >= PAD) && (orig_ofmap_row_idx <= PAD + PADDED_OH - 1) &&
    (orig_ofmap_col_idx >= PAD) && (orig_ofmap_col_idx <= PAD + PADDED_OW - 1);
//
always @(posedge clk or negedge reset_n) begin
    if(!reset_n) ofmap_addr <= {OFMAP_AWIDTH{1'b0}};
    else if(is_write_done && !PAD)
        ofmap_addr <= (orig_ofmap_row_idx * ORIG_OW + orig_ofmap_col_idx) + (ORIG_OW * ORIG_OH * captured_ifmap_out_channel);
    else if(is_write_done && PAD) begin
        if(in_padding_region)
            ofmap_addr <= (orig_ofmap_row_idx-PAD) * PADDED_OH + (orig_ofmap_col_idx-PAD) +
                (PADDED_OW * PADDED_OH * captured_ifmap_out_channel);
        else ; // intentionally left blank: no address update for out-of-bound padding
    end
end
end
```

```
`define ORIG_OW ( `FW + `S * (`IW - 1) )
`define ORIG_OH ( `FH + `S * (`IH - 1) )
```

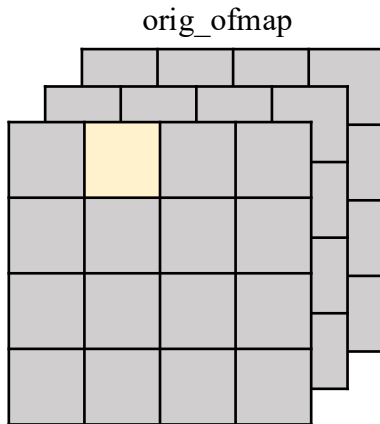


- captured\_ifmap\_out\_channel = 0
- ofmap\_row\_idx = 0
- ofmap\_col\_idx = 0
- ofmap\_addr = 0 + 0 + 0 = 0

# 7. SAVE\_module: orig ofmap address

```
// *_*_*_*_* OFMAP_ADDR *_*_*_*_*
reg [OFMAP_AWIDTH-1 : 0] ofmap_addr;
//
wire in_padding_region = (orig_ofmap_row_idx >= PAD) && (orig_ofmap_row_idx <= PAD + PADDED_OH - 1) &&
    (orig_ofmap_col_idx >= PAD) && (orig_ofmap_col_idx <= PAD + PADDED_OW - 1);
//
always @(posedge clk or negedge reset_n) begin
    if(!reset_n) ofmap_addr <= {OFMAP_AWIDTH{1'b0}};
    else if(is_write_done && !PAD)
        ofmap_addr <= (orig_ofmap_row_idx * ORIG_OW + orig_ofmap_col_idx) + (ORIG_OW * ORIG_OH * captured_ifmap_out_channel);
    else if(is_write_done && PAD) begin
        if(in_padding_region)
            ofmap_addr <= (orig_ofmap_row_idx-PAD) * PADDED_OH + (orig_ofmap_col_idx-PAD) +
                (PADDED_OW * PADDED_OH * captured_ifmap_out_channel);
        else ; // intentionally left blank: no address update for out-of-bound padding
    end
end
end
```

```
`define ORIG_OW ( `FW + `S * (`IW - 1) )
`define ORIG_OH ( `FH + `S * (`IH - 1) )
```

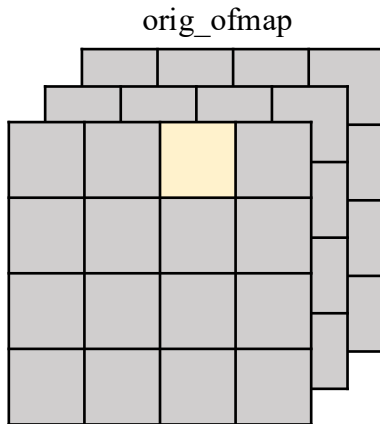


- captured\_ifmap\_out\_channel = 0
- ofmap\_row\_idx = 0
- ofmap\_col\_idx = 1
- ofmap\_addr = 0 + 1 + 0 = 1

# 8. SAVE\_module: orig ofmap address

```
// *_*_*_*_* OFMAP_ADDR *_*_*_*_*
reg [OFMAP_AWIDTH-1 : 0] ofmap_addr;
//
wire in_padding_region = (orig_ofmap_row_idx >= PAD) && (orig_ofmap_row_idx <= PAD + PADDED_OH - 1) &&
    (orig_ofmap_col_idx >= PAD) && (orig_ofmap_col_idx <= PAD + PADDED_OW - 1);
//
always @(posedge clk or negedge reset_n) begin
    if(!reset_n) ofmap_addr <= {OFMAP_AWIDTH{1'b0}};
    else if(is_write_done && !PAD)
        ofmap_addr <= (orig_ofmap_row_idx * ORIG_OW + orig_ofmap_col_idx) + (ORIG_OW * ORIG_OH * captured_ifmap_out_channel);
    else if(is_write_done && PAD) begin
        if(in_padding_region)
            ofmap_addr <= (orig_ofmap_row_idx-PAD) * PADDED_OH + (orig_ofmap_col_idx-PAD) +
                (PADDED_OW * PADDED_OH * captured_ifmap_out_channel);
        else ; // intentionally left blank: no address update for out-of-bound padding
    end
end
end
```

```
`define ORIG_OW ( `FW + `S * (`IW - 1) )
`define ORIG_OH ( `FH + `S * (`IH - 1) )
```

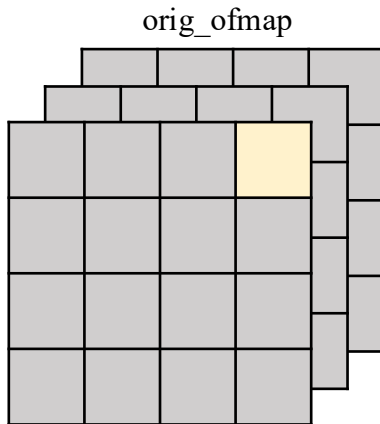


- captured\_ifmap\_out\_channel = 0
- ofmap\_row\_idx = 0
- ofmap\_col\_idx = 2
- ofmap\_addr = 0 + 2 + 0 = 2

# 8. SAVE\_module: orig ofmap address

```
// *_*_*_*_* OFMAP_ADDR *_*_*_*_*  
reg [OFMAP_AWIDTH-1 : 0] ofmap_addr;  
//  
wire in_padding_region = (orig_ofmap_row_idx >= PAD) && (orig_ofmap_row_idx <= PAD + PADDED_OH - 1) &&  
    (orig_ofmap_col_idx >= PAD) && (orig_ofmap_col_idx <= PAD + PADDED_OW - 1);  
//  
always @(posedge clk or negedge reset_n) begin  
    if(!reset_n) ofmap_addr <= {OFMAP_AWIDTH{1'b0}};  
    else if(is_write_done && !PAD)  
        ofmap_addr <= (orig_ofmap_row_idx * ORIG_OW + orig_ofmap_col_idx) + (ORIG_OW * ORIG_OH * captured_ifmap_out_channel);  
    else if(is_write_done && PAD) begin  
        if(in_padding_region)  
            ofmap_addr <= (orig_ofmap_row_idx-PAD) * PADDED_OH + (orig_ofmap_col_idx-PAD) +  
                (PADDED_OW * PADDED_OH * captured_ifmap_out_channel);  
        else ; // intentionally left blank: no address update for out-of-bound padding  
    end  
end
```

```
`define ORIG_OW ( `FW + `S * (`IW - 1) )  
`define ORIG_OH ( `FH + `S * (`IH - 1) )
```

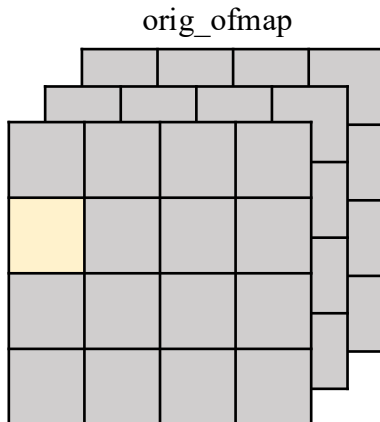


- captured\_ifmap\_out\_channel = 0
- ofmap\_row\_idx = 0
- ofmap\_col\_idx = 3
- ofmap\_addr = 0 + 3 + 0 = 3

# 8. SAVE\_module: orig ofmap address

```
// *_*_*_*_* OFMAP_ADDR *_*_*_*_*
reg [OFMAP_AWIDTH-1 : 0] ofmap_addr;
//
wire in_padding_region = (orig_ofmap_row_idx >= PAD) && (orig_ofmap_row_idx <= PAD + PADDED_OH - 1) &&
    (orig_ofmap_col_idx >= PAD) && (orig_ofmap_col_idx <= PAD + PADDED_OW - 1);
//
always @(posedge clk or negedge reset_n) begin
    if(!reset_n) ofmap_addr <= {OFMAP_AWIDTH{1'b0}};
    else if(is_write_done && !PAD)
        ofmap_addr <= (orig_ofmap_row_idx * ORIG_OW + orig_ofmap_col_idx) + (ORIG_OW * ORIG_OH * captured_ifmap_out_channel);
    else if(is_write_done && PAD) begin
        if(in_padding_region)
            ofmap_addr <= (orig_ofmap_row_idx-PAD) * PADDED_OH + (orig_ofmap_col_idx-PAD) +
                (PADDED_OW * PADDED_OH * captured_ifmap_out_channel);
        else ; // intentionally left blank: no address update for out-of-bound padding
    end
end
end
```

```
`define ORIG_OW ( `FW + `S * (`IW - 1) )
`define ORIG_OH ( `FH + `S * (`IH - 1) )
```



- captured\_ifmap\_out\_channel = 0
- ofmap\_row\_idx = 1
- ofmap\_col\_idx = 0
- ofmap\_addr = 4 + 0 + 0 = 4

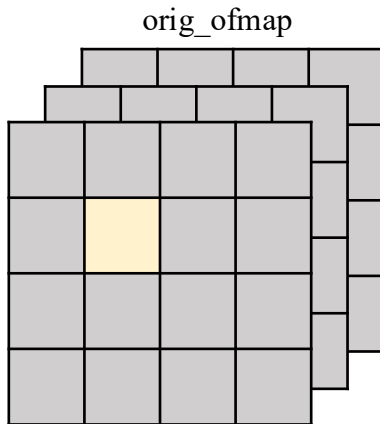
# 8. SAVE\_module: orig ofmap address



Kumoh National Institute of Technology

```
// *_*_*_*_* OFMAP_ADDR *_*_*_*_*
reg [OFMAP_AWIDTH-1 : 0] ofmap_addr;
//
wire in_padding_region = (orig_ofmap_row_idx >= PAD) && (orig_ofmap_row_idx <= PAD + PADDED_OH - 1) &&
    (orig_ofmap_col_idx >= PAD) && (orig_ofmap_col_idx <= PAD + PADDED_OW - 1);
//
always @(posedge clk or negedge reset_n) begin
    if(!reset_n) ofmap_addr <= {OFMAP_AWIDTH{1'b0}};
    else if(is_write_done && !PAD)
        ofmap_addr <= (orig_ofmap_row_idx * ORIG_OW + orig_ofmap_col_idx) + (ORIG_OW * ORIG_OH * captured_ifmap_out_channel);
    else if(is_write_done && PAD) begin
        if(in_padding_region)
            ofmap_addr <= (orig_ofmap_row_idx-PAD) * PADDED_OH + (orig_ofmap_col_idx-PAD) +
                (PADDED_OW * PADDED_OH * captured_ifmap_out_channel);
        else ; // intentionally left blank: no address update for out-of-bound padding
    end
end
end
```

```
`define ORIG_OW ( `FW + `S * (`IW - 1) )
`define ORIG_OH ( `FH + `S * (`IH - 1) )
```

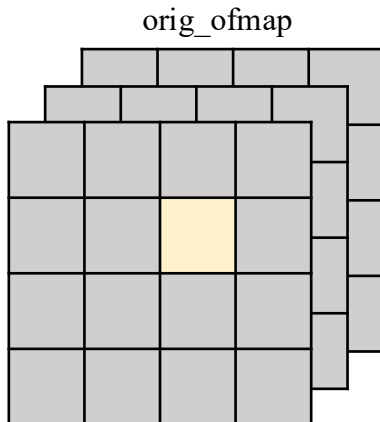


- captured\_ifmap\_out\_channel = 0
- ofmap\_row\_idx = 1
- ofmap\_col\_idx = 1
- ofmap\_addr = 4 + 1 + 0 = 5

# 8. SAVE\_module: orig ofmap address

```
// *_*_*_*_* OFMAP_ADDR *_*_*_*_*
reg [OFMAP_AWIDTH-1 : 0] ofmap_addr;
//
wire in_padding_region = (orig_ofmap_row_idx >= PAD) && (orig_ofmap_row_idx <= PAD + PADDED_OH - 1) &&
    (orig_ofmap_col_idx >= PAD) && (orig_ofmap_col_idx <= PAD + PADDED_OW - 1);
//
always @(posedge clk or negedge reset_n) begin
    if(!reset_n) ofmap_addr <= {OFMAP_AWIDTH{1'b0}};
    else if(is_write_done && !PAD)
        ofmap_addr <= (orig_ofmap_row_idx * ORIG_OW + orig_ofmap_col_idx) + (ORIG_OW * ORIG_OH * captured_ifmap_out_channel);
    else if(is_write_done && PAD) begin
        if(in_padding_region)
            ofmap_addr <= (orig_ofmap_row_idx-PAD) * PADDED_OH + (orig_ofmap_col_idx-PAD) +
                (PADDED_OW * PADDED_OH * captured_ifmap_out_channel);
        else ; // intentionally left blank: no address update for out-of-bound padding
    end
end
end
```

```
`define ORIG_OW ( `FW + `S * (`IW - 1) )
`define ORIG_OH ( `FH + `S * (`IH - 1) )
```

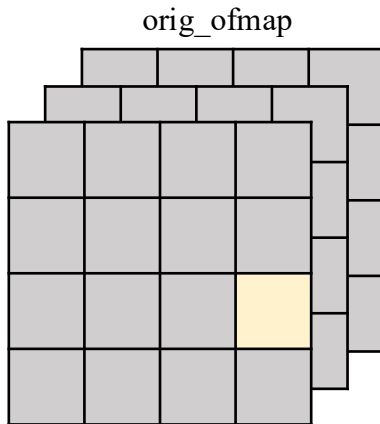


- captured\_ifmap\_out\_channel = 0
- ofmap\_row\_idx = 1
- ofmap\_col\_idx = 2
- ofmap\_addr = 4 + 2 + 0 = 6

# 8. SAVE\_module: orig ofmap address

```
// *_*_*_*_* OFMAP_ADDR *_*_*_*_*
reg [OFMAP_AWIDTH-1 : 0] ofmap_addr;
//
wire in_padding_region = (orig_ofmap_row_idx >= PAD) && (orig_ofmap_row_idx <= PAD + PADDED_OH - 1) &&
    (orig_ofmap_col_idx >= PAD) && (orig_ofmap_col_idx <= PAD + PADDED_OW - 1);
//
always @(posedge clk or negedge reset_n) begin
    if(!reset_n) ofmap_addr <= {OFMAP_AWIDTH{1'b0}};
    else if(is_write_done && !PAD)
        ofmap_addr <= (orig_ofmap_row_idx * ORIG_OW + orig_ofmap_col_idx) + (ORIG_OW * ORIG_OH * captured_ifmap_out_channel);
    else if(is_write_done && PAD) begin
        if(in_padding_region)
            ofmap_addr <= (orig_ofmap_row_idx-PAD) * PADDED_OH + (orig_ofmap_col_idx-PAD) +
                (PADDED_OW * PADDED_OH * captured_ifmap_out_channel);
        else ; // intentionally left blank: no address update for out-of-bound padding
    end
end
end
```

```
`define ORIG_OW ( `FW + `S * (`IW - 1) )
`define ORIG_OH ( `FH + `S * (`IH - 1) )
```

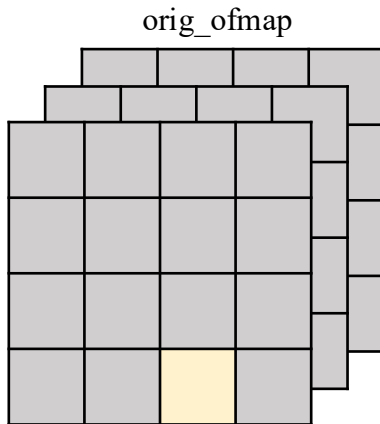


- captured\_ifmap\_out\_channel = 0
- ofmap\_row\_idx = 2
- ofmap\_col\_idx = 3
- ofmap\_addr = 8 + 3 + 0 = 11

# 8. SAVE\_module: orig ofmap address

```
// *_*_*_*_* OFMAP_ADDR *_*_*_*_*
reg [OFMAP_AWIDTH-1 : 0] ofmap_addr;
//
wire in_padding_region = (orig_ofmap_row_idx >= PAD) && (orig_ofmap_row_idx <= PAD + PADDED_OH - 1) &&
    (orig_ofmap_col_idx >= PAD) && (orig_ofmap_col_idx <= PAD + PADDED_OW - 1);
//
always @(posedge clk or negedge reset_n) begin
    if(!reset_n) ofmap_addr <= {OFMAP_AWIDTH{1'b0}};
    else if(is_write_done && !PAD)
        ofmap_addr <= (orig_ofmap_row_idx * ORIG_OW + orig_ofmap_col_idx) + (ORIG_OW * ORIG_OH * captured_ifmap_out_channel);
    else if(is_write_done && PAD) begin
        if(in_padding_region)
            ofmap_addr <= (orig_ofmap_row_idx-PAD) * PADDED_OH + (orig_ofmap_col_idx-PAD) +
                (PADDED_OW * PADDED_OH * captured_ifmap_out_channel);
        else ; // intentionally left blank: no address update for out-of-bound padding
    end
end
end
```

```
`define ORIG_OW ( `FW + `S * (`IW - 1) )
`define ORIG_OH ( `FH + `S * (`IH - 1) )
```

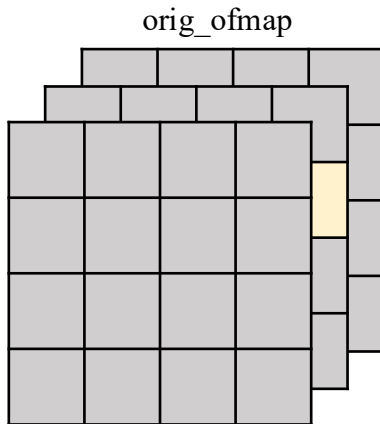


- captured\_ifmap\_out\_channel = 0
- ofmap\_row\_idx = 3
- ofmap\_col\_idx = 2
- ofmap\_addr = 12 + 2 + 0 = 14

# 8. SAVE\_module: orig ofmap address

```
// *_*_*_*_ OFMAP_ADDR *_*_*_*_  
reg [OFMAP_AWIDTH-1 : 0] ofmap_addr;  
//  
wire in_padding_region = (orig_ofmap_row_idx >= PAD) && (orig_ofmap_row_idx <= PAD + PADDED_OH - 1) &&  
    (orig_ofmap_col_idx >= PAD) && (orig_ofmap_col_idx <= PAD + PADDED_OW - 1);  
//  
always @(posedge clk or negedge reset_n) begin  
    if (!reset_n) ofmap_addr <= {OFMAP_AWIDTH{1'b0}};  
    else if (is_write_done && !PAD)  
        ofmap_addr <= (orig_ofmap_row_idx * ORIG_OW + orig_ofmap_col_idx) + (ORIG_OW * ORIG_OH * captured_ifmap_out_channel);  
    else if (is_write_done && PAD) begin  
        if (in_padding_region)  
            ofmap_addr <= (orig_ofmap_row_idx - PAD) * PADDED_OH + (orig_ofmap_col_idx - PAD) +  
                (PADDED_OW * PADDED_OH * captured_ifmap_out_channel);  
        else ; // intentionally left blank: no address update for out-of-bound padding  
    end  
end
```

```
`define ORIG_OW ( `FW + `S * (`IW - 1) )  
`define ORIG_OH ( `FH + `S * (`IH - 1) )
```

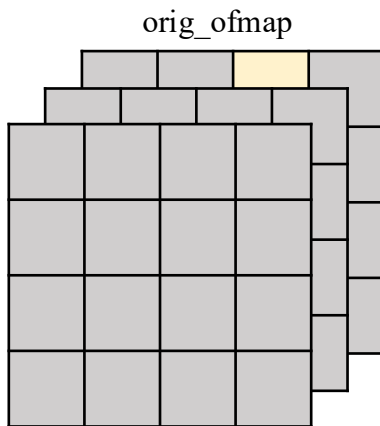


- captured\_ifmap\_out\_channel = 1
- ofmap\_row\_idx = 1
- ofmap\_col\_idx = 3
- ofmap\_addr = 4 + 3 + 16 = 23

# 8. SAVE\_module: orig ofmap address

```
// *_*_*_*_ OFMAP_ADDR *_*_*_*_
reg [OFMAP_AWIDTH-1 : 0] ofmap_addr;
//
wire in_padding_region = (orig_ofmap_row_idx >= PAD) && (orig_ofmap_row_idx <= PAD + PADDED_OH - 1) &&
    (orig_ofmap_col_idx >= PAD) && (orig_ofmap_col_idx <= PAD + PADDED_OW - 1);
//
always @(posedge clk or negedge reset_n) begin
    if(!reset_n) ofmap_addr <= {OFMAP_AWIDTH{1'b0}};
    else if(is_write_done && !PAD)
        ofmap_addr <= (orig_ofmap_row_idx * ORIG_OW + orig_ofmap_col_idx) + (ORIG_OW * ORIG_OH * captured_ifmap_out_channel);
    else if(is_write_done && PAD) begin
        if(in_padding_region)
            ofmap_addr <= (orig_ofmap_row_idx-PAD) * PADDED_OH + (orig_ofmap_col_idx-PAD) +
                (PADDED_OW * PADDED_OH * captured_ifmap_out_channel);
        else ; // intentionally left blank: no address update for out-of-bound padding
    end
end
end
```

```
`define ORIG_OW ( `FW + `S * (`IW - 1) )
`define ORIG_OH ( `FH + `S * (`IH - 1) )
```



- captured\_ifmap\_out\_channel = 2
- ofmap\_row\_idx = 0
- ofmap\_col\_idx = 2
- ofmap\_addr = 0 + 2 + 32 = 34

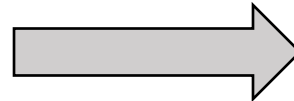
# 8. SAVE\_module: padded ofmap address

```
// *_*_*_*_ OFMAP_ADDR *_*_*_*_  
reg [OFMAP_AWIDTH-1 : 0] ofmap_addr;  
//  
wire in_padding_region = (orig_ofmap_row_idx >= PAD) && (orig_ofmap_row_idx <= PAD + PADDED_OH - 1) &&  
    (orig_ofmap_col_idx >= PAD) && (orig_ofmap_col_idx <= PAD + PADDED_OW - 1);  
//  
always @(posedge clk or negedge reset_n) begin  
    if(!reset_n) ofmap_addr <= {OFMAP_AWIDTH{1'b0}};  
    else if(is_write_done && !PAD)  
        ofmap_addr <= (orig_ofmap_row_idx * ORIG_OW + orig_ofmap_col_idx) + (ORIG_OW * ORIG_OH * captured_ifmap_out_channel);  
    else if(is_write_done && PAD) begin  
        if(in_padding_region)  
            ofmap_addr <= (orig_ofmap_row_idx-PAD) * PADDED_OH + (orig_ofmap_col_idx-PAD) +  
                (PADDED_OW * PADDED_OH * captured_ifmap_out_channel);  
        else ; // intentionally left blank: no address update for out-of-bound padding  
    end  
end
```

```
`define PADDED_OW ( `FW + `S * (`IW - 1) - 2 * `PAD )  
`define PADDED_OH ( `FH + `S * (`IH - 1) - 2 * `PAD )
```

| orig_ofmap |       |       |       |
|------------|-------|-------|-------|
| (0,0)      | (0,1) | (0,2) | (0,3) |
| (1,0)      | (1,1) | (1,2) | (1,3) |
| (2,0)      | (2,1) | (2,2) | (2,3) |
| (3,0)      | (3,1) | (3,2) | (3,3) |

PAD = 1



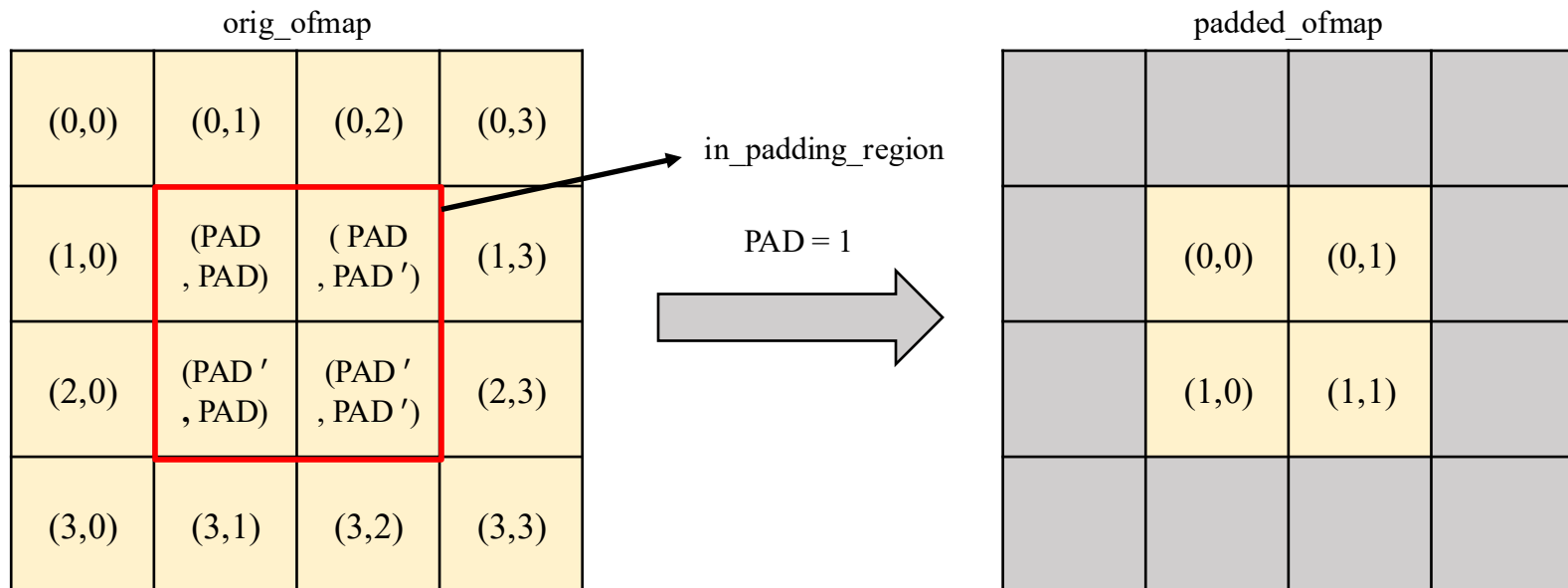
| padded_ofmap |       |       |  |
|--------------|-------|-------|--|
|              |       |       |  |
|              | (0,0) | (0,1) |  |
|              | (1,0) | (1,1) |  |
|              |       |       |  |

# 8. SAVE\_module: padded ofmap address

```
// *_*_*_*_ OFMAP_ADDR *_*_*_*_
reg [OFMAP_AWIDTH-1 : 0] ofmap_addr;
//
wire in_padding_region = (orig_ofmap_row_idx >= PAD) && (orig_ofmap_row_idx <= PAD + PADDED_OH - 1) &&
    (orig_ofmap_col_idx >= PAD) && (orig_ofmap_col_idx <= PAD + PADDED_OW - 1);
//
always @(posedge clk or negedge reset_n) begin
    if(!reset_n) ofmap_addr <= {OFMAP_AWIDTH{1'b0}};
    else if(is_write_done && !PAD)
        ofmap_addr <= (orig_ofmap_row_idx * ORIG_OW + orig_ofmap_col_idx) + (ORIG_OW * ORIG_OH * captured_ifmap_out_channel);
    else if(is_write_done && PAD) begin
        if(in_padding_region)
            ofmap_addr <= (orig_ofmap_row_idx-PAD) * PADDED_OH + (orig_ofmap_col_idx-PAD) +
                (PADDED_OW * PADDED_OH * captured_ifmap_out_channel);
        else ; // intentionally left blank: no address update for out-of-bound padding
    end
end
```

``define PADDED_OW ( `FW + `S * (`IW - 1) - 2 * `PAD )`  
``define PADDED_OH ( `FH + `S * (`IH - 1) - 2 * `PAD )`

$$\diamond \text{ PAD}' = \text{PAD} + \text{PADDED\_OW}(\text{or OH}) - 1$$



# 8. SAVE\_module: padded ofmap address

```
// *_*_*_*_* OFMAP_ADDR *_*_*_*_*_
reg [OFMAP_AWIDTH-1 : 0] ofmap_addr;
//
wire in_padding_region = (orig_ofmap_row_idx >= PAD) && (orig_ofmap_row_idx <= PAD + PADDED_OH - 1) &&
    (orig_ofmap_col_idx >= PAD) && (orig_ofmap_col_idx <= PAD + PADDED_OW - 1);
//
```

```
`define PADDED_OW ( `FW + `S * (`IW - 1) - 2 * `PAD )
`define PADDED_OH ( `FH + `S * (`IH - 1) - 2 * `PAD )
```

|                            |       |       |       |       |       |
|----------------------------|-------|-------|-------|-------|-------|
| > orig_ofmap_row_idx[19:0] | 00001 | 00000 | 00001 | 00002 | 00003 |
| > orig_ofmap_col_idx[19:0] | 00001 | 00001 | 00002 | 00000 | 00001 |
| in_padding_region          | 1     |       |       |       |       |

```
else if (is_write_done && PAD) begin
    if (in_padding_region)
        ofmap_addr <= (orig_ofmap_row_idx - PAD) * PADDED_OH + (orig_ofmap_col_idx - PAD) +
            (PADDED_OW * PADDED_OH * captured_ifmap_out_channel);
    else ; // intentionally left blank: no address update for out-of-bound padding
end
end
```

$$\diamond \text{ PAD}' = \text{PAD} + \text{PADDED\_OW (or OH)} - 1$$

orig\_ofmap

|       |             |              |       |
|-------|-------------|--------------|-------|
| (0,0) | (0,1)       | (0,2)        | (0,3) |
| (1,0) | (PAD, PAD)  | (PAD, PAD')  | (1,3) |
| (2,0) | (PAD', PAD) | (PAD', PAD') | (2,3) |
| (3,0) | (3,1)       | (3,2)        | (3,3) |

in\_padding\_region

PAD = 1

padded\_ofmap

|  |       |       |  |
|--|-------|-------|--|
|  |       |       |  |
|  | (0,0) | (0,1) |  |
|  | (1,0) | (1,1) |  |
|  |       |       |  |

# 8. SAVE\_module: padded ofmap address

```
// *_*_*_*_* OFMAP_ADDR *_*_*_*_*_
reg [OFMAP_AWIDTH-1 : 0] ofmap_addr;
//
wire in_padding_region = (orig_ofmap_row_idx >= PAD) && (orig_ofmap_row_idx <= PAD + PADDED_OH - 1) &&
    (orig_ofmap_col_idx >= PAD) && (orig_ofmap_col_idx <= PAD + PADDED_OW - 1);
```

```
`define PADDED_OW ( `FW + `S * (`IW - 1) - 2 * `PAD )
`define PADDED_OH ( `FH + `S * (`IH - 1) - 2 * `PAD )
```

|                            |       |       |       |       |       |
|----------------------------|-------|-------|-------|-------|-------|
| > orig_ofmap_row_idx[19:0] | 00001 | 00000 | 00001 | 00002 | 00003 |
| > orig_ofmap_col_idx[19:0] | 00001 | 00001 | 00002 | 00000 | 00001 |
| in_padding_region          | 1     |       |       |       |       |

```
else if (is_write_done && PAD) begin
    if (in_padding_region)
        ofmap_addr <= (orig_ofmap_row_idx - PAD) * PADDED_OH + (orig_ofmap_col_idx - PAD) +
            (PADDED_OW * PADDED_OH * captured_ifmap_out_channel);
    else ; // intentionally left blank: no address update for out-of-bound padding
end
end
```

$$\diamond \text{ PAD}' = \text{PAD} + \text{PADDED\_OW (or OH)} - 1$$

orig\_ofmap

|       |             |              |       |
|-------|-------------|--------------|-------|
| (0,0) | (0,1)       | (0,2)        | (0,3) |
| (1,0) | (PAD, PAD)  | (PAD, PAD')  | (1,3) |
| (2,0) | (PAD', PAD) | (PAD', PAD') | (2,3) |
| (3,0) | (3,1)       | (3,2)        | (3,3) |

in\_padding\_region

PAD = 1

padded\_ofmap

|  |       |       |  |
|--|-------|-------|--|
|  |       |       |  |
|  | (0,0) | (0,1) |  |
|  | (1,0) | (1,1) |  |
|  |       |       |  |

# 8. SAVE\_module: padded ofmap address

```
// *_*_*_*_ OFMAP_ADDR *_*_*_*_
reg [OFMAP_AWIDTH-1 : 0] ofmap_addr;
//
wire in_padding_region = (orig_ofmap_row_idx >= PAD) && (orig_ofmap_row_idx <= PAD + PADDED_OH - 1) &&
    (orig_ofmap_col_idx >= PAD) && (orig_ofmap_col_idx <= PAD + PADDED_OW - 1);
//
always @(posedge clk or negedge reset_n) begin
    if(!reset_n) ofmap_addr <= {OFMAP_AWIDTH{1'b0}};
    else if(is_write_done && !PAD)
        ofmap_addr <= (orig_ofmap_row_idx * ORIG_OW + orig_ofmap_col_idx) + (ORIG_OW * ORIG_OH * captured_ifmap_out_channel);
    else if(is_write_done && PAD) begin
        if(in_padding_region)
            ofmap_addr <= (orig_ofmap_row_idx-PAD) * PADDED_OH + (orig_ofmap_col_idx-PAD) +
                (PADDED_OW * PADDED_OH * captured_ifmap_out_channel);
        else; // intentionally left blank: no address update for out-of-bound padding
    end
end
```

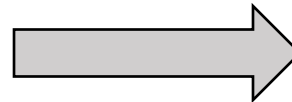
```
`define PADDED_OW ( `FW + `S * (`IW - 1) - 2 * `PAD )
`define PADDED_OH ( `FH + `S * (`IH - 1) - 2 * `PAD )
```

$$\diamond \text{ PAD}' = \text{PAD} + \text{PADDED\_OW}(\text{or OH}) - 1$$

orig\_ofmap

|       |             |              |       |
|-------|-------------|--------------|-------|
| (0,0) | (0,1)       | (0,2)        | (0,3) |
| (1,0) | (PAD, PAD)  | (PAD, PAD')  | (1,3) |
| (2,0) | (PAD', PAD) | (PAD', PAD') | (2,3) |
| (3,0) | (3,1)       | (3,2)        | (3,3) |

PAD = 1



padded\_ofmap

|                                               |                                               |
|-----------------------------------------------|-----------------------------------------------|
| orig_ofmap_col - PAD,<br>orig_ofmap_row - PAD | orig_ofmap_col - PAD,<br>orig_ofmap_row - PAD |
| orig_ofmap_col - PAD,<br>orig_ofmap_row - PAD | orig_ofmap_col - PAD,<br>orig_ofmap_row - PAD |



Kumoh National Institute of Technology

[illegible]



Kumoh National Institute of Technology

Figure 10 illustrates the memory access pattern of the naive implementation. The figure is divided into three main sections:

- Memory Access List:** A table showing the memory access pattern for various variables.
 

| Variable                          | Address      |
|-----------------------------------|--------------|
| o_valid[2:0]                      | 0            |
| intermediate[449:0]               | 000000000000 |
| intermediate_buf[449:0]           | 000000000028 |
| a_pixel_of_intermediate[99:0]     | -4914        |
| a_pixel_of_intermediate_buf[99:0] | -4914        |
- Memory Layout Diagram:** A diagram showing the memory layout of the naive implementation. It consists of a grid of memory blocks. A yellow box highlights a specific memory access pattern, showing a sequence of values: 00000000011f8ffffffffffcc20000000001018ffffffffffb323ffffffffffe70ff... The diagram also shows the memory access pattern for the variables listed in the first section.
- Detailed View of Memory Access:** A detailed view of the memory access pattern for the variable 'a\_pixel\_of\_intermediate\_buf[99:0]'. It shows a sequence of values: -3320, 4120, -4920, -400, -2160, -640, 920, 1520, 4600, -7553, 9373, -111. The values are arranged in a grid, with the highlighted sequence shown in a yellow box.



Kumoh National Institute of Technology

[illegible]

# 8. SAVE\_module: SAVE



Kumoh National Institute of Technology

```
// *_*_*_*_*_*_*_*_*_* IN PADDING REGION BUFFER *_*_*_*_*_*_*_*_*_*
reg [IN_PAD_REGION_DELAY-1 : 0] in_padding_region_buf;

always @(posedge clk or negedge reset_n) begin
    if(!reset_n) in_padding_region_buf <= {IN_PAD_REGION_DELAY{1'b0}};
    else in_padding_region_buf <= {in_padding_region_buf[IN_PAD_REGION_DELAY-2 : 0], in_padding_region};
end

// *_*_*_*_*_*_*_*_*_* ACCUMULATION OF INTERMEDIATE VEC AND OFMAP VEC FROM OFMAP MEM *_*_*_*_*_*_*_*_*_*
reg signed [OFMAP_BITS-1 : 0] write_ofmap_value;
reg accum_signal;

//
always @(posedge clk or negedge reset_n) begin
    if(!reset_n) begin
        write_ofmap_value <= {OFMAP_BITS{1'b0}};
        accum_signal <= 1'b0;
    end
    else if(o_read) write_ofmap_value <= read_ofmap_value;
    else if(o_accum && !PAD) write_ofmap_value <= write_ofmap_value + a_pixel_of_intermediate_buf;
    else if(o_accum && PAD) begin
        if(in_padding_region_buf[IN_PAD_REGION_DELAY-1]) write_ofmap_value <= write_ofmap_value + a_pixel_of_intermediate_buf;
    end
end
```

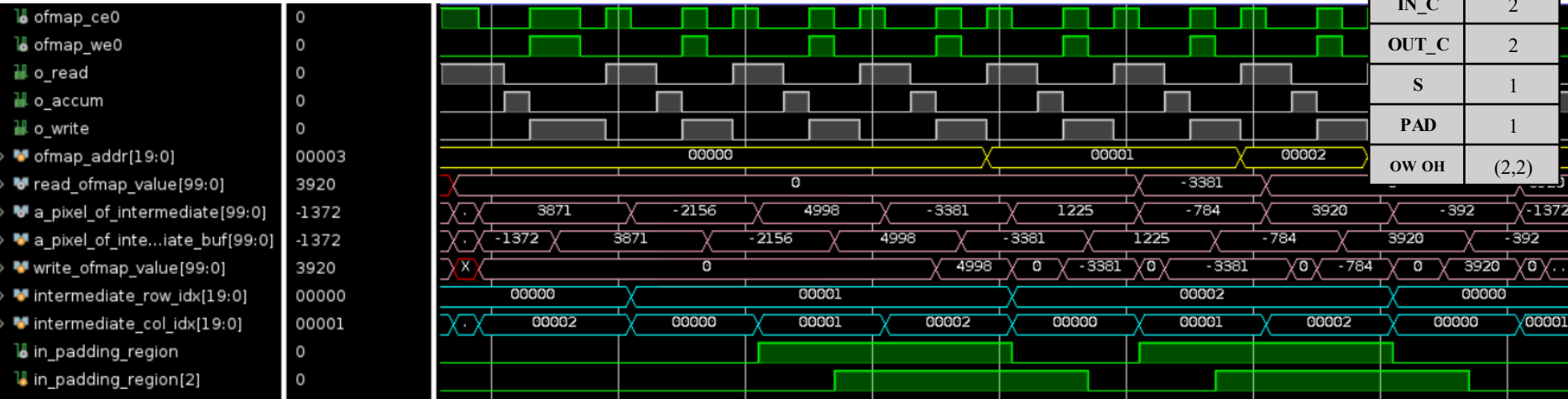
|       |       |
|-------|-------|
| IW    | 2     |
| IH    | 2     |
| FW    | 3     |
| FH    | 3     |
| IN_C  | 2     |
| OUT_C | 2     |
| S     | 1     |
| PAD   | 1     |
| OW OH | (2,2) |

# 8. SAVE\_module: SAVE

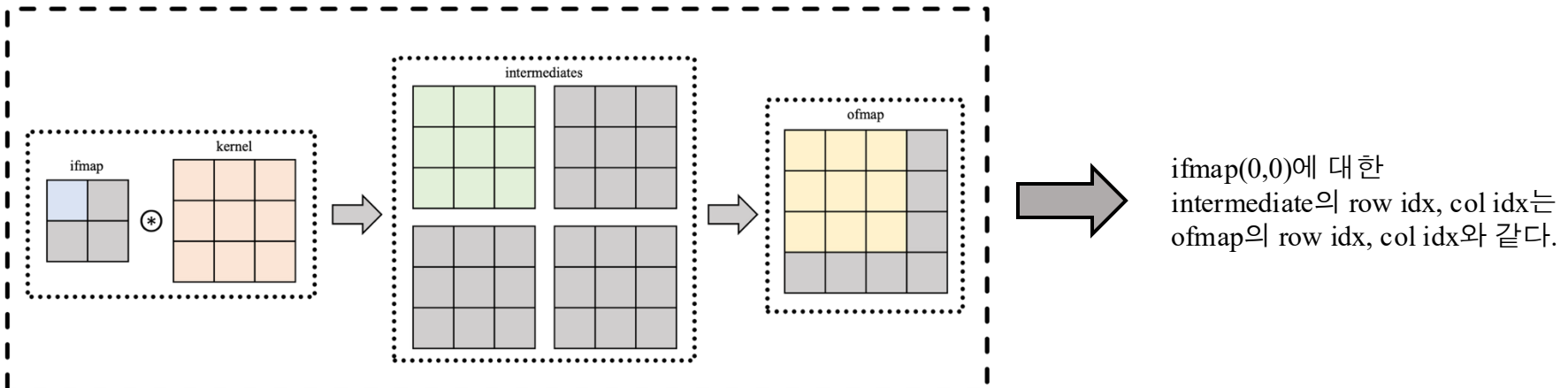
```
// *_*_*_*_*_* IN PADDING REGION BUFFER *_*_*_*_*_*
reg [IN_PAD_REGION_DELAY-1 : 0] in_padding_region_buf;

always @(posedge clk or negedge reset_n) begin
    if(!reset_n) in_padding_region_buf <= {IN_PAD_REGION_DELAY{1'b0}};
    else in_padding_region_buf <= {in_padding_region_buf[IN_PAD_REGION_DELAY-2 : 0], in_padding_region};
end
```

|       |       |
|-------|-------|
| IW    | 2     |
| IH    | 2     |
| FW    | 3     |
| FH    | 3     |
| IN_C  | 2     |
| OUT_C | 2     |
| S     | 1     |
| PAD   | 1     |
| OW OH | (2,2) |



in\_padding\_region을 delay시키는 이유 : (전제: 위 파형은 ifmap(0,0)에 대한 부분)



# 8. SAVE\_module: SAVE

```
// *_*_*_*_*_*_* IN PADDING REGION BUFFER *_*_*_*_*_*_
```

```
reg [IN_PAD_REGION_DELAY-1 : 0] in_padding_region_buf;
```

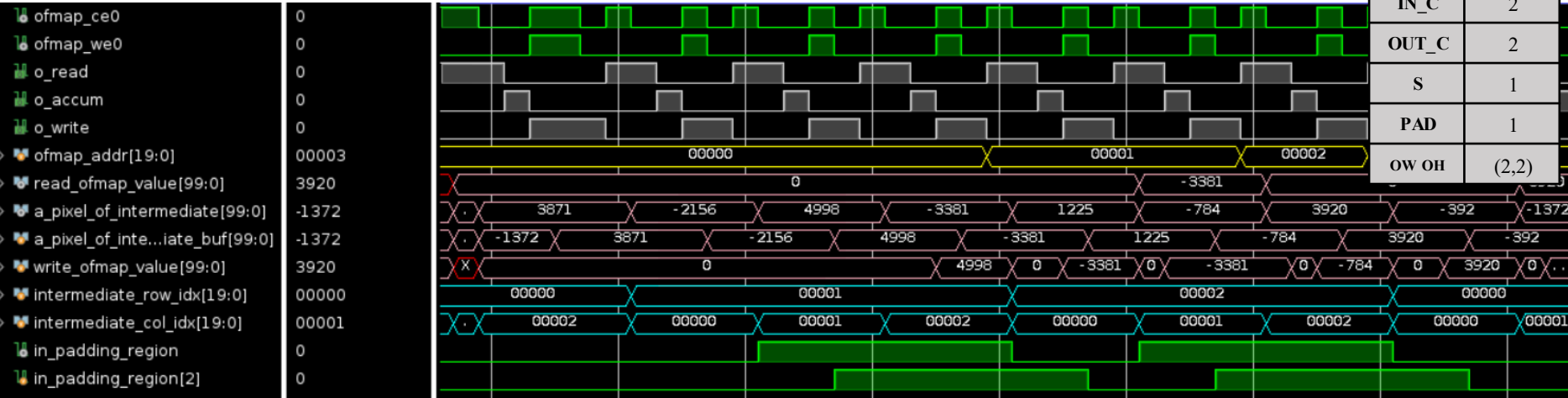
```
always @(posedge clk or negedge reset_n) begin
```

```
  if(!reset_n) in_padding_region_buf <= {IN_PAD_REGION_DELAY{1'b0}};
```

```
  else in_padding_region_buf <= {in_padding_region_buf[IN_PAD_REGION_DELAY-2 : 0], in_padding_region};
```

```
end
```

|       |       |
|-------|-------|
| IW    | 2     |
| IH    | 2     |
| FW    | 3     |
| FH    | 3     |
| IN_C  | 2     |
| OUT_C | 2     |
| S     | 1     |
| PAD   | 1     |
| OW OH | (2,2) |



in\_padding\_region을 delay시키는 이유 : (전제: 위 파형은 ifmap(0,0)에 대한 부분)

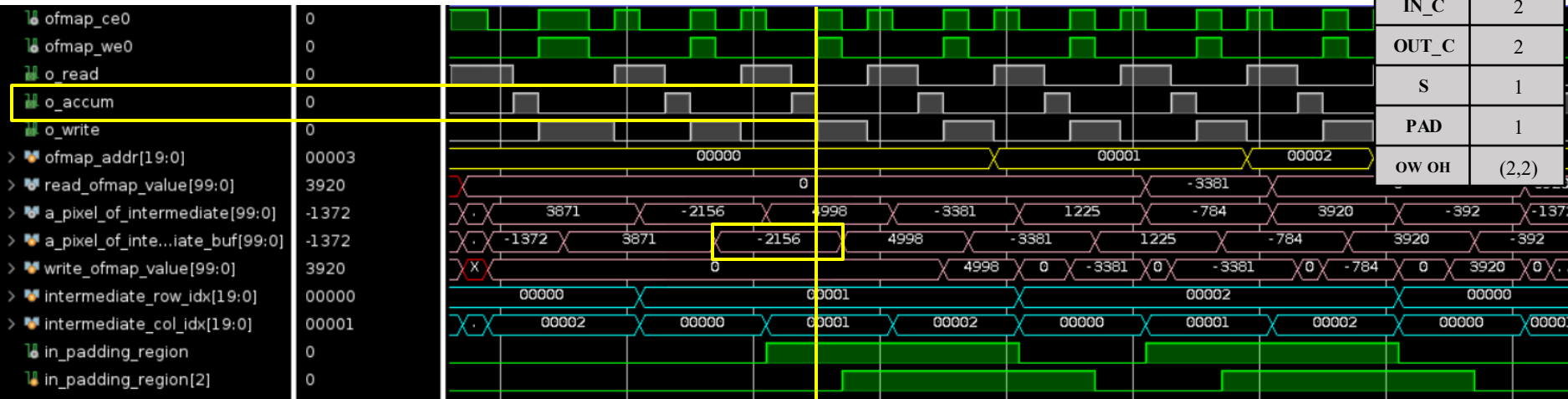
- Expected result: ofmap\_addr 0번지에 (intermediate\_row, intermediate\_col) = (1,1)의 값이 저장되면 좋겠다.

# 8. SAVE\_module: SAVE

```
// *_*_*_*_*_* IN PADDING REGION BUFFER *_*_*_*_*_*
reg [IN_PAD_REGION_DELAY-1 : 0] in_padding_region_buf;

always @(posedge clk or negedge reset_n) begin
    if(!reset_n) in_padding_region_buf <= {IN_PAD_REGION_DELAY{1'b0}};
    else in_padding_region_buf <= {in_padding_region_buf[IN_PAD_REGION_DELAY-2 : 0], in_padding_region};
end
```

|       |       |
|-------|-------|
| IW    | 2     |
| IH    | 2     |
| FW    | 3     |
| FH    | 3     |
| IN_C  | 2     |
| OUT_C | 2     |
| S     | 1     |
| PAD   | 1     |
| OW OH | (2,2) |

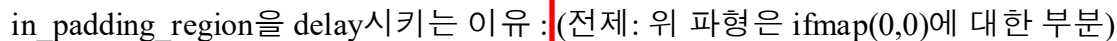


in\_padding\_region을 delay시키는 이유 : (전제: 위 파형은 ifmap(0,0)에 대한 부분)

- Expected result: ofmap\_addr 0번지에 (intermediate\_row, intermediate\_col) = (1,1)의 값이 들어갔으면 좋겠다.
  - in\_padding\_region 기준으로 봤을 때, o\_accum에서의 write\_ofmap\_value값은 -2156



|       |       |
|-------|-------|
| IW    | 2     |
| IH    | 2     |
| FW    | 3     |
| FH    | 3     |
| IN_C  | 2     |
| OUT_C | 2     |
| S     | 1     |
| PAD   | 1     |
| OW OH | (2,2) |



- o accum에서의 write ofmap value값을 확인 하는 이유

```
else if(o_accum && PAD) begin
    if(in_padding_region buff[IN_PAD_REGION_DELAY-1]) a_target <= a_target + a_pixel of intermediate buf; end
```



```
if(in padding region buff[IN PAD REGION DELAY-1]) a target<=a target + a pixel of intermediate buf; end
```

# 8. SAVE\_module: SAVE

```
// *_*_*_*_*_*_* IN PADDING REGION BUFFER *_*_*_*_*_*_
```

```
reg [IN_PAD_REGION_DELAY-1 : 0] in_padding_region_buf;
```

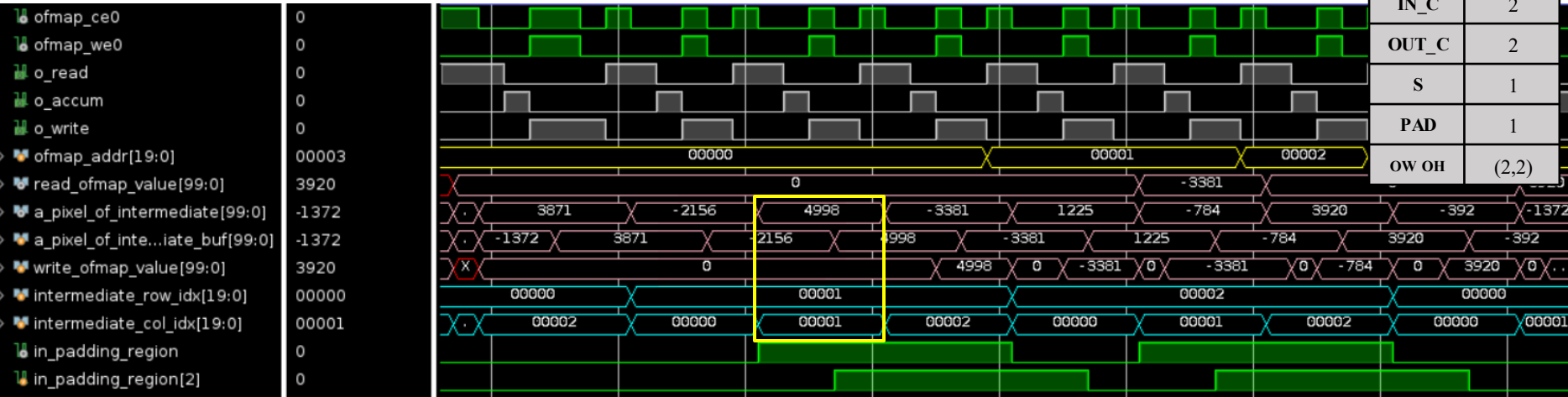
```
always @(posedge clk or negedge reset_n) begin
```

```
  if(!reset_n) in_padding_region_buf <= {IN_PAD_REGION_DELAY{1'b0}};
```

```
  else in_padding_region_buf <= {in_padding_region_buf[IN_PAD_REGION_DELAY-2 : 0], in_padding_region};
```

```
end
```

|       |       |
|-------|-------|
| IW    | 2     |
| IH    | 2     |
| FW    | 3     |
| FH    | 3     |
| IN_C  | 2     |
| OUT_C | 2     |
| S     | 1     |
| PAD   | 1     |
| OW OH | (2,2) |



in\_padding\_region을 delay시키는 이유 : (전제: 위 파형은 ifmap(0,0)에 대한 부분)

- Expected result: ofmap\_addr 0번지에 (intermediate\_row, intermediate\_col) = (1,1)의 값이 들어갔으면 좋겠다.
  - in\_padding\_region 기준으로 봤을 때, o\_accum에서의 write\_ofmap\_value값은 -2156
  - 하지만, a\_pixel\_of\_intermediate와 intermediate\_idx를 확인해보면 -2156은 (intermediate\_row, intermediate\_col) = (1,0)의 값
  - in\_padding\_region[2] 기준으로 봤을 때, o\_accum에서의 write\_ofmap\_value값은 4998
  - a\_pixel\_of\_intermediate와 intermediate\_idx를 확인해보면 -2156은 (intermediate\_row, intermediate\_col) = (1,1)의 값
  - in\_padding\_region을 delay시켜야 기대하는 결과를 얻을 수 있음.

o\_accum에서의 write\_ofmap\_value값을 확인하는 이유

```
else if(o_accum && PAD) begin
```

```
  if(in_padding_region_buf[IN_PAD_REGION_DELAY-1]) a_target <= a_target + a_pixel_of_intermediate_buf; end
```

# 8. SAVE\_module: SAVE

```
// *_*_*_*_*_* ACCUMULATION OF INTERMEDIATE VEC AND OFMAP VEC FROM OFMAP MEM *_*_*_*_*_*
```

```
reg signed [OFMAP_BITS - 1 : 0] write_ofmap_value;
```

```
reg accum_signal;
```

```
//
```

```
always @(posedge clk or negedge reset_n) begin
```

```
  if(!reset_n) begin
```

```
    write_ofmap_value <= {OFMAP_BITS{1'b0}};
```

```
    accum_signal <= 1'b0;
```

```
  end
```

```
  //
```

```
  else if(o_read) write_ofmap_value <= read_ofmap_value;
```

```
  //
```

```
  else if(o_accum && !PAD) write_ofmap_value <= write_ofmap_value + a_pixel_of_intermediate_buf;
```

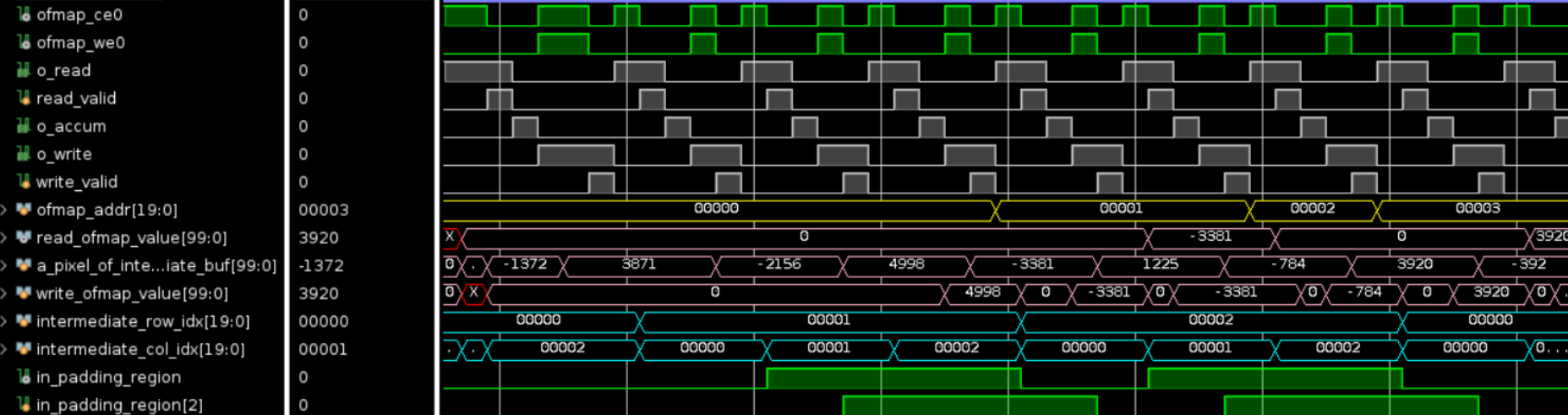
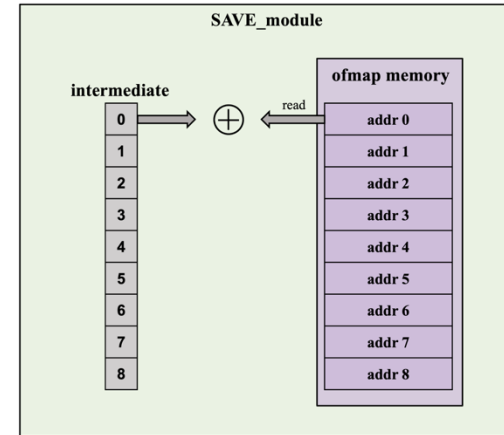
```
  //
```

```
  else if(o_accum && PAD) begin
```

```
    if(in_padding_region_buf[IN_PAD_REGION_DELAY-1]) write_ofmap_value <= write_ofmap_value + a_pixel_of_intermediate_buf;
```

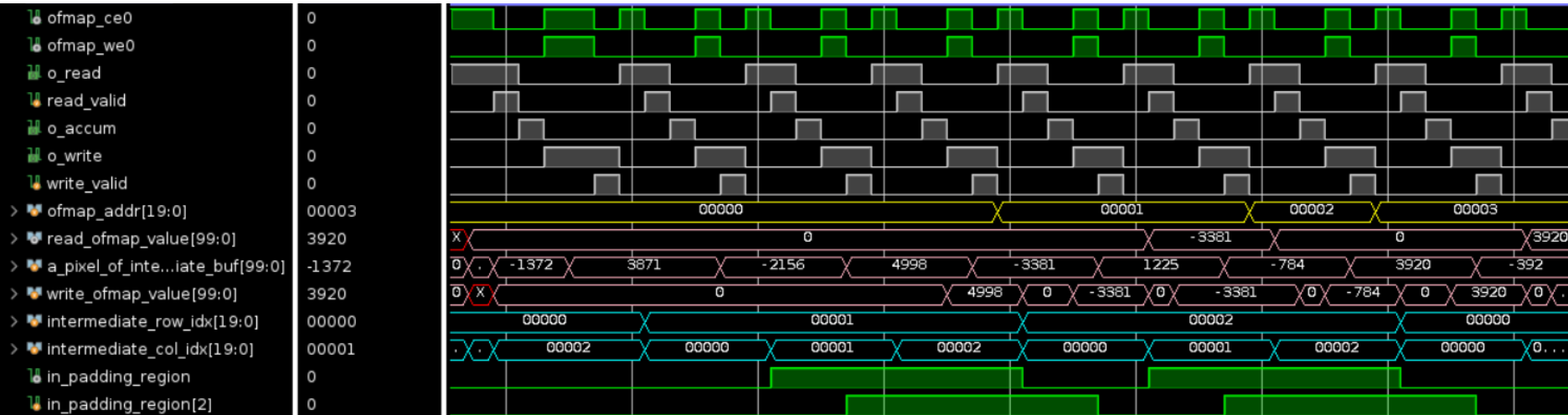
```
  end
```

```
end
```





The diagram illustrates the SAVE module's internal data flow. On the left, a vertical column of boxes labeled 'intermediate' contains the values 0 through 8. An arrow labeled 'write' points from a circle containing a question mark to a vertical column of boxes labeled 'ofmap memory'. The 'ofmap memory' column contains addresses 'addr 0' through 'addr 8'. The 'write' arrow points to the 'addr 0' box.



# 8. SAVE\_module: SAVE

```
// *_*_*_*_*_* ACCUMULATION OF INTERMEDIATE VEC AND OFMAP VEC FROM OFMAP MEM *_*_*_*_*_*
```

```
reg signed [OFMAP_BITS -1 : 0] write_ofmap_value;
```

```
reg accum_signal;
```

```
//
```

```
always @(posedge clk or negedge reset_n) begin
```

```
  if(!reset_n) begin
```

```
    write_ofmap_value <= {OFMAP_BITS{1'b0}};
```

```
    accum_signal <= 1'b0;
```

```
  end
```

```
  //
```

```
  else if(o_read) write_ofmap_value <= read_ofmap_value;
```

```
  //
```

```
  else if(o_accum && !PAD) write_ofmap_value <= write_ofmap_value + a_pixel_of_intermediate_buf;
```

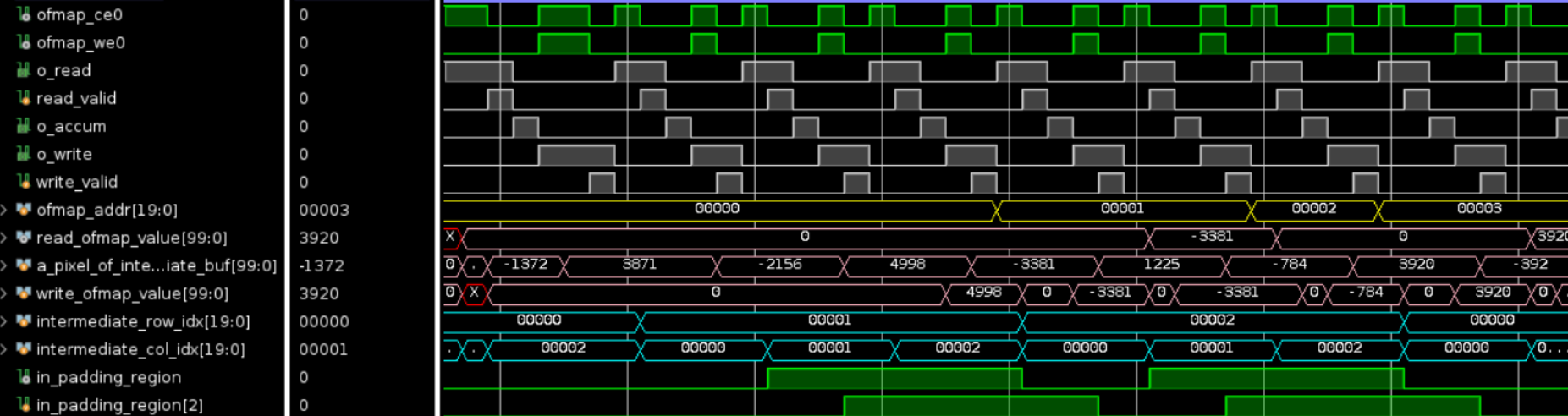
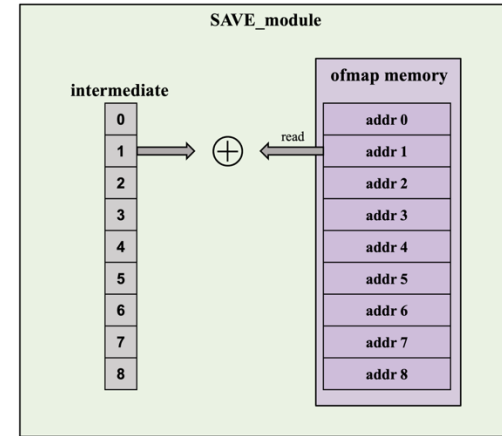
```
  //
```

```
  else if(o_accum && PAD) begin
```

```
    if(in_padding_region_buf[IN_PAD_REGION_DELAY-1]) write_ofmap_value <= write_ofmap_value + a_pixel_of_intermediate_buf;
```

```
  end
```

```
end
```



# 8. SAVE\_module: SAVE

```
// *_*_*_*_*_* ACCUMULATION OF INTERMEDIATE VEC AND OFMAP VEC FROM OFMAP MEM *_*_*_*_*_*
```

```
reg signed [OFMAP_BITS -1 : 0] write_ofmap_value;
```

```
reg accum_signal;
```

```
//
```

```
always @(posedge clk or negedge reset_n) begin
```

```
  if(!reset_n) begin
```

```
    write_ofmap_value <= {OFMAP_BITS{1'b0}};
```

```
    accum_signal <= 1'b0;
```

```
  end
```

```
  //
```

```
  else if(o_read) write_ofmap_value <= read_ofmap_value;
```

```
  //
```

```
  else if(o_accum && !PAD) write_ofmap_value <= write_ofmap_value + a_pixel_of_intermediate_buf;
```

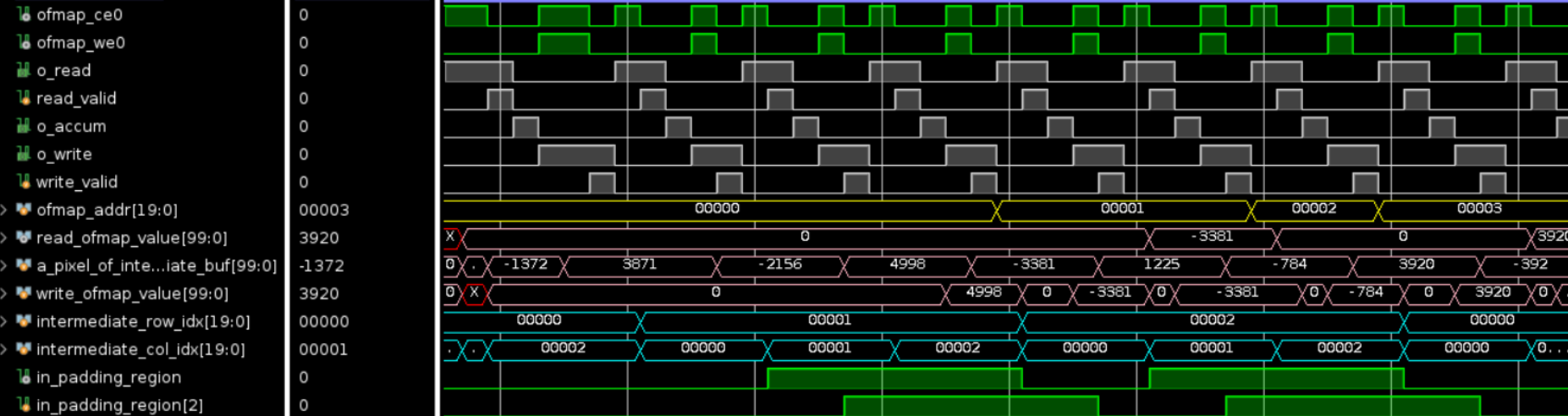
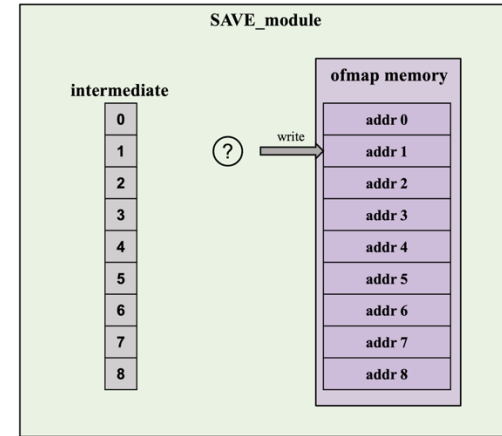
```
  //
```

```
  else if(o_accum && PAD) begin
```

```
    if(in_padding_region_buf[IN_PAD_REGION_DELAY-1]) write_ofmap_value <= write_ofmap_value + a_pixel_of_intermediate_buf;
```

```
  end
```

```
end
```



# 8. SAVE\_module: SAVE

```
// *_*_*_*_*_* ACCUMULATION OF INTERMEDIATE VEC AND OFMAP VEC FROM OFMAP MEM *_*_*_*_*_*
```

```
reg signed [OFMAP_BITS -1 : 0] write_ofmap_value;
```

```
reg accum_signal;
```

```
//
```

```
always @(posedge clk or negedge reset_n) begin
```

```
  if(!reset_n) begin
```

```
    write_ofmap_value <= {OFMAP_BITS{1'b0}};
```

```
    accum_signal <= 1'b0;
```

```
  end
```

```
  //
```

```
  else if(o_read) write_ofmap_value <= read_ofmap_value;
```

```
  //
```

```
  else if(o_accum && !PAD) write_ofmap_value <= write_ofmap_value + a_pixel_of_intermediate_buf;
```

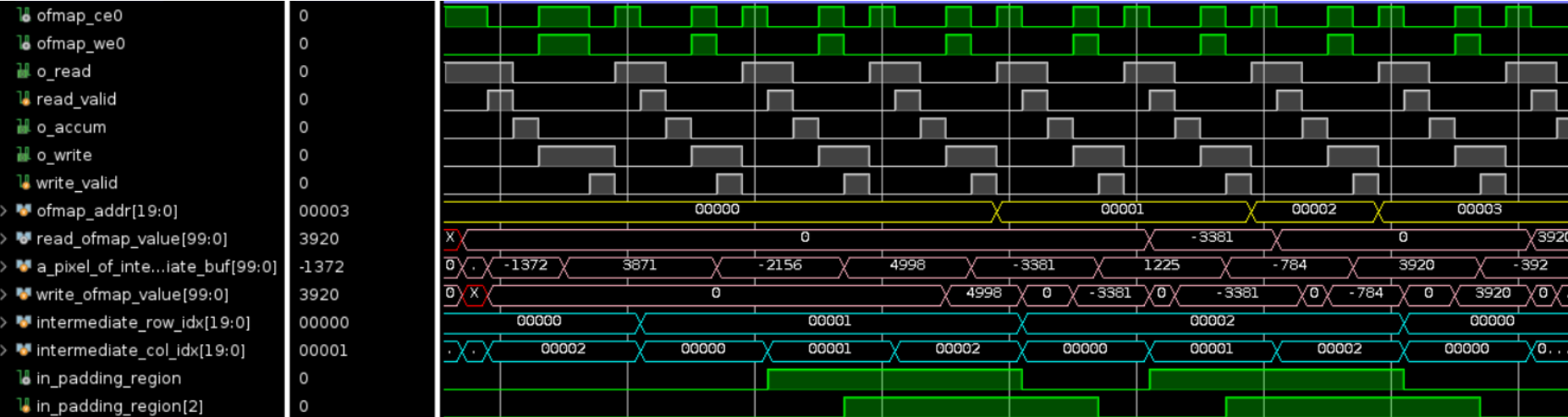
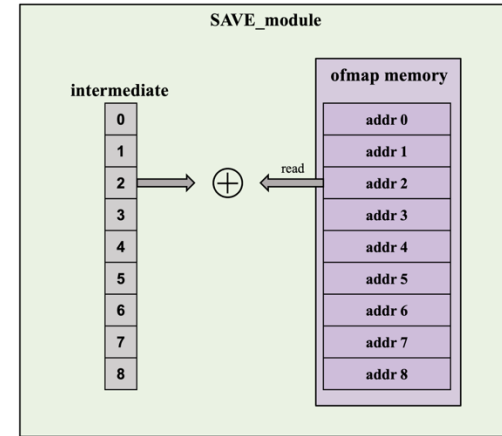
```
  //
```

```
  else if(o_accum && PAD) begin
```

```
    if(in_padding_region_buf[IN_PAD_REGION_DELAY-1]) write_ofmap_value <= write_ofmap_value + a_pixel_of_intermediate_buf;
```

```
  end
```

```
end
```



# 8. SAVE\_module: SAVE

```
// *_*_*_*_*_* ACCUMULATION OF INTERMEDIATE VEC AND OFMAP VEC FROM OFMAP MEM *_*_*_*_*_*
```

```
reg signed [OFMAP_BITS -1 : 0] write_ofmap_value;
```

```
reg accum_signal;
```

```
//
```

```
always @(posedge clk or negedge reset_n) begin
```

```
  if(!reset_n) begin
```

```
    write_ofmap_value <= {OFMAP_BITS{1'b0}};
```

```
    accum_signal <= 1'b0;
```

```
  end
```

```
  //
```

```
  else if(o_read) write_ofmap_value <= read_ofmap_value;
```

```
  //
```

```
  else if(o_accum && !PAD) write_ofmap_value <= write_ofmap_value + a_pixel_of_intermediate_buf;
```

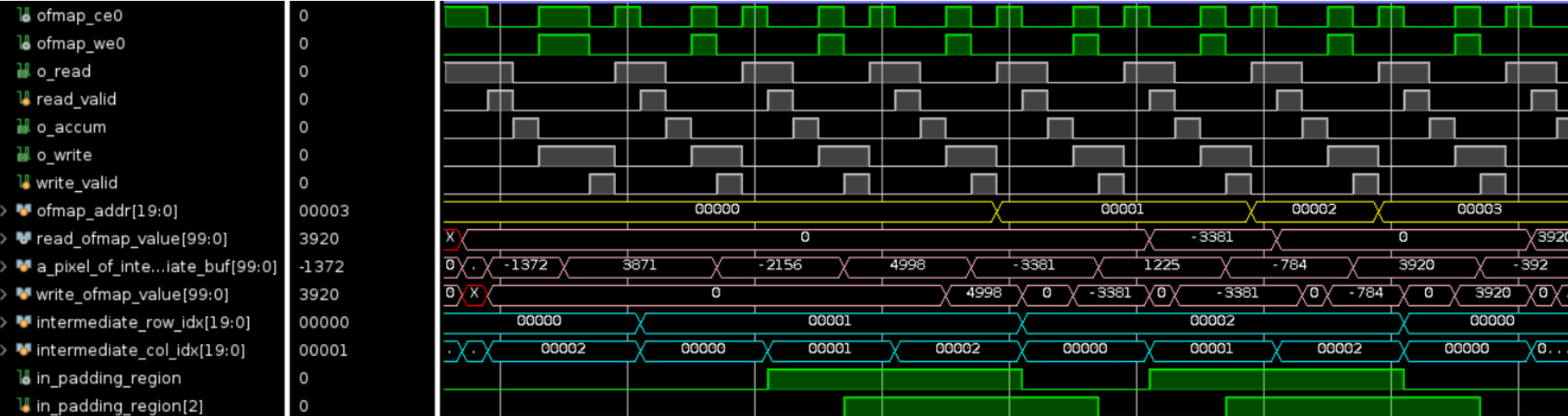
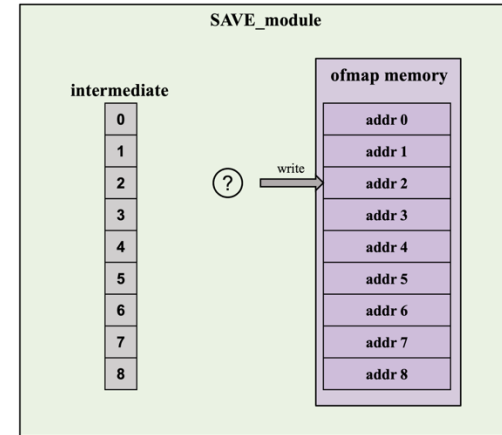
```
  //
```

```
  else if(o_accum && PAD) begin
```

```
    if(in_padding_region_buf[IN_PAD_REGION_DELAY-1]) write_ofmap_value <= write_ofmap_value + a_pixel_of_intermediate_buf;
```

```
  end
```

```
end
```

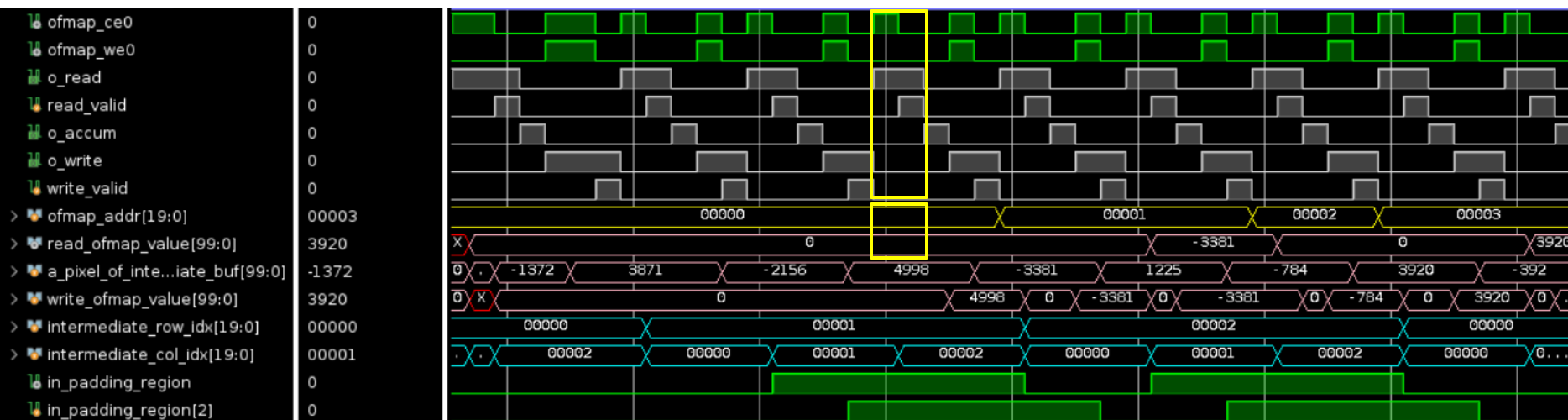




Kumoh National Institute of Technology

end

|              |       |
|--------------|-------|
| <b>IW</b>    | 2     |
| <b>IH</b>    | 2     |
| <b>FW</b>    | 3     |
| <b>FH</b>    | 3     |
| <b>IN_C</b>  | 2     |
| <b>OUT_C</b> | 2     |
| <b>S</b>     | 1     |
| <b>PAD</b>   | 1     |
| <b>OW OH</b> | (2,2) |



- ifmap(0,0)에 대한 intermediate에 대한 read/write 과정
- Read : ofmap addr 0의 초기값을 0으로 reset 시켜두었기 때문에 read ofmap value = 0

# 8. SAVE\_module: SAVE

// \*\_\*\_\*\_\*\_\*\_\* ACCUMULATION OF INTERMEDIATE VEC AND OFMAP VEC FROM OFMAP MEM \*\_\*\_\*\_\*\_\*\_\*

reg signed [OFMAP\_BITS-1 : 0] write\_ofmap\_value;

reg accum\_signal;

//

always @(posedge clk or negedge reset\_n) begin

if(!reset\_n) begin

write\_ofmap\_value <= {OFMAP\_BITS{1'b0}};

accum\_signal <= 1'b0;

end

//

else if(o\_read) write\_ofmap\_value <= read\_ofmap\_value;

//

else if(o\_accum && !PAD) write\_ofmap\_value <= write\_ofmap\_value + a\_pixel\_of\_intermediate\_buf;

//

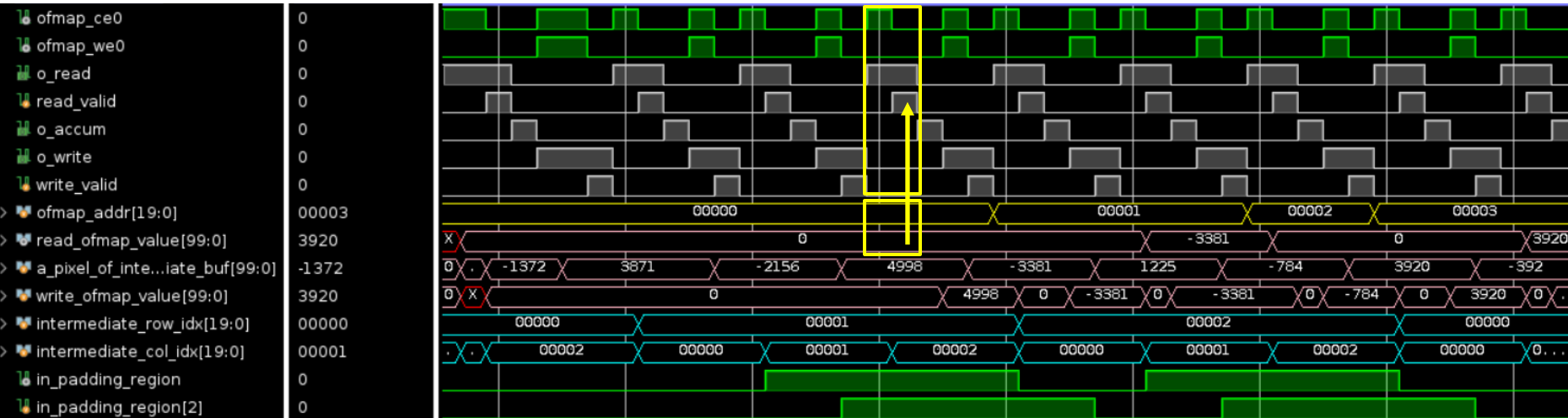
else if(o\_accum && PAD) begin

if(in\_padding\_region\_buf[IN\_PAD\_REGION\_DELAY-1]) write\_ofmap\_value <= write\_ofmap\_value + a\_pixel\_of\_intermediate\_buf;

end

end

|       |       |
|-------|-------|
| IW    | 2     |
| IH    | 2     |
| FW    | 3     |
| FH    | 3     |
| IN_C  | 2     |
| OUT_C | 2     |
| S     | 1     |
| PAD   | 1     |
| OW OH | (2,2) |



- ifmap(0,0)에 대한 intermediate에 대한 read/write 과정
- Read : ofmap\_addr 0의 초기값을 0으로 reset 시켜두었기 때문에 read\_ofmap\_value = 0 / read\_valid check



- ifmap(0,0)에 대한 intermediate에 대한 read/write 과정
- Read : ofmap\_addr 0의 초기값을 0으로 reset 시켜두었기 때문에 read\_ofmap\_value = 0 / read\_valid check
- Accum : write\_ofmap\_value <= write\_ofmap\_value + a pixel of intermediate buf;

# 8. SAVE\_module: SAVE

// \*\_\*\_\*\_\*\_\*\_\* ACCUMULATION OF INTERMEDIATE VEC AND OFMAP VEC FROM OFMAP MEM \*\_\*\_\*\_\*\_\*\_\*

reg signed [OFMAP\_BITS-1 : 0] write\_ofmap\_value;

reg accum\_signal;

//

always @(posedge clk or negedge reset\_n) begin

if(!reset\_n) begin

write\_ofmap\_value <= {OFMAP\_BITS{1'b0}};

accum\_signal <= 1'b0;

end

//

else if(o\_read) write\_ofmap\_value <= read\_ofmap\_value;

//

else if(o\_accum && !PAD) write\_ofmap\_value <= write\_ofmap\_value + a\_pixel\_of\_intermediate\_buf;

//

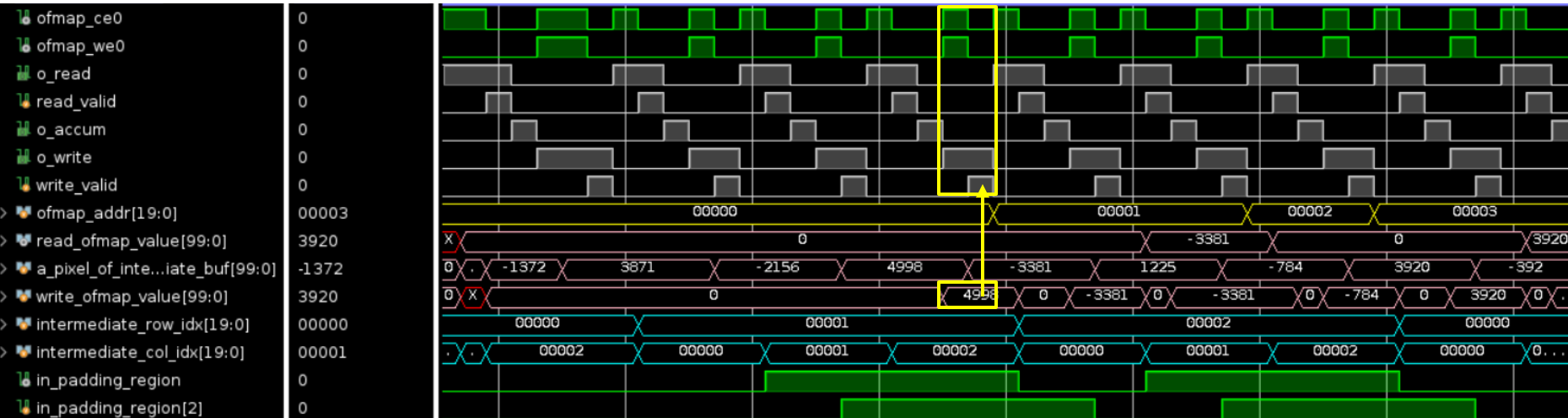
else if(o\_accum && PAD) begin

if(in\_padding\_region\_buf[IN\_PAD\_REGION\_DELAY-1]) write\_ofmap\_value <= write\_ofmap\_value + a\_pixel\_of\_intermediate\_buf;

end

end

|       |       |
|-------|-------|
| IW    | 2     |
| IH    | 2     |
| FW    | 3     |
| FH    | 3     |
| IN_C  | 2     |
| OUT_C | 2     |
| S     | 1     |
| PAD   | 1     |
| OW OH | (2,2) |



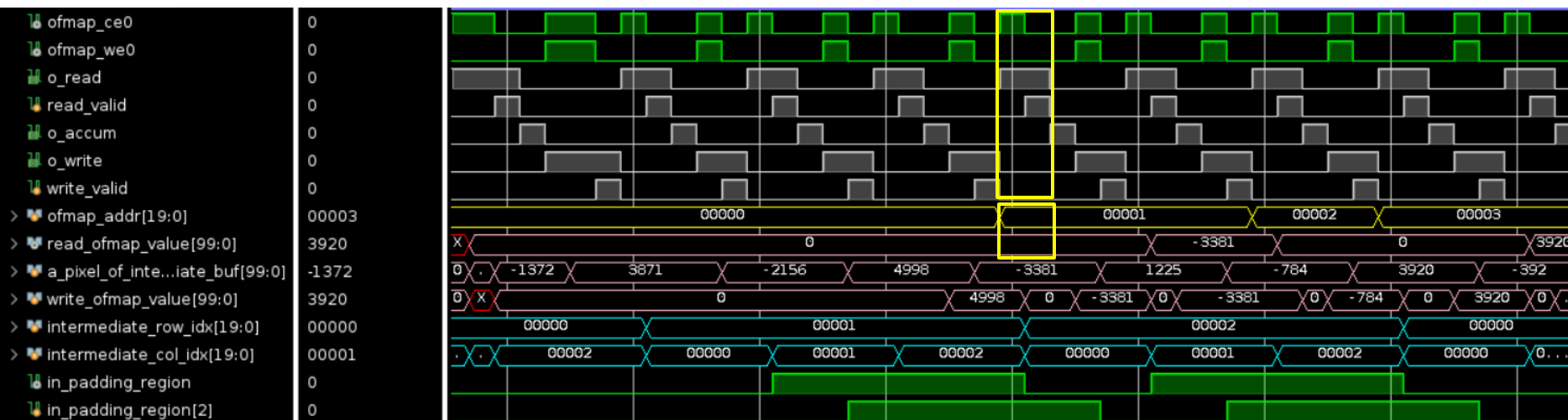
- ifmap(0,0)에 대한 intermediate에 대한 read/write 과정
- Read : ofmap\_addr 0의 초기값을 0으로 reset 시켜두었기 때문에 read\_ofmap\_value = 0 / read\_valid check
- Accum : write\_ofmap\_value <= write\_ofmap\_value + a\_pixel\_of\_intermediate\_buf;
- Write : write\_valid check



Kumoh National Institute of Technology

end

|              |       |
|--------------|-------|
| <b>IW</b>    | 2     |
| <b>IH</b>    | 2     |
| <b>FW</b>    | 3     |
| <b>FH</b>    | 3     |
| <b>IN_C</b>  | 2     |
| <b>OUT_C</b> | 2     |
| <b>S</b>     | 1     |
| <b>PAD</b>   | 1     |
| <b>OW OH</b> | (2,2) |



- ifmap(0,0)에 대한 intermediate에 대한 read/write 과정
- Read : ofmap addr 1의 초기값을 0으로 reset 시켜두었기 때문에 read ofmap value = 0

# 8. SAVE\_module: SAVE

// \*\_\*\_\*\_\*\_\*\_\* ACCUMULATION OF INTERMEDIATE VEC AND OFMAP VEC FROM OFMAP MEM \*\_\*\_\*\_\*\_\*\_\*

reg signed [OFMAP\_BITS-1 : 0] write\_ofmap\_value;

reg accum\_signal;

//

always @(posedge clk or negedge reset\_n) begin

if(!reset\_n) begin

write\_ofmap\_value <= {OFMAP\_BITS{1'b0}};

accum\_signal <= 1'b0;

end

//

else if(o\_read) write\_ofmap\_value <= read\_ofmap\_value;

//

else if(o\_accum && !PAD) write\_ofmap\_value <= write\_ofmap\_value + a\_pixel\_of\_intermediate\_buf;

//

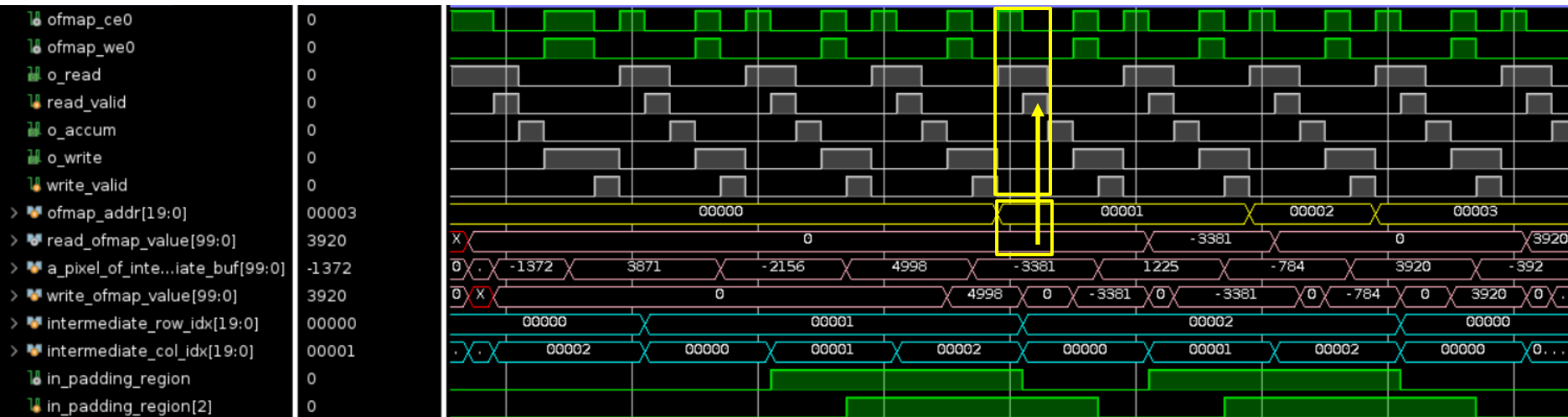
else if(o\_accum && PAD) begin

if(in\_padding\_region\_buf[IN\_PAD\_REGION\_DELAY-1]) write\_ofmap\_value <= write\_ofmap\_value + a\_pixel\_of\_intermediate\_buf;

end

end

|       |       |
|-------|-------|
| IW    | 2     |
| IH    | 2     |
| FW    | 3     |
| FH    | 3     |
| IN_C  | 2     |
| OUT_C | 2     |
| S     | 1     |
| PAD   | 1     |
| OW OH | (2,2) |



- ifmap(0,0)에 대한 intermediate에 대한 read/write 과정
- Read : ofmap\_addr 1의 초기값을 0으로 reset 시켜두었기 때문에 read\_ofmap\_value = 0 / read\_valid check

# 8. SAVE\_module: SAVE

```
// *_*_*_*_*_* ACCUMULATION OF INTERMEDIATE VEC AND OFMAP VEC FROM OFMAP MEM *_*_*_*_*_*
```

```
reg signed [OFMAP_BITS-1 : 0] write_ofmap_value;
```

```
reg accum_signal;
```

```
//
```

```
always @(posedge clk or negedge reset_n) begin
```

```
  if(!reset_n) begin
```

```
    write_ofmap_value <= {OFMAP_BITS{1'b0}};
```

```
    accum_signal <= 1'b0;
```

```
  end
```

```
  //
```

```
  else if(o_read) write_ofmap_value <= read_ofmap_value;
```

```
  //
```

```
  else if(o_accum && !PAD) write_ofmap_value <= write_ofmap_value + a_pixel_of_intermediate_buf;
```

```
  //
```

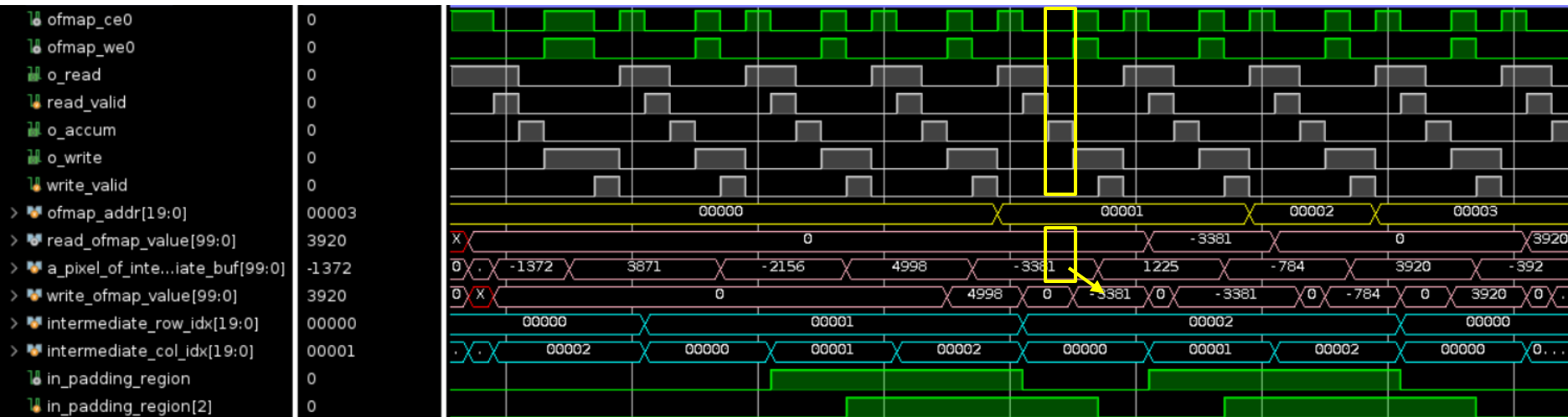
```
  else if(o_accum && PAD) begin
```

```
    if(in_padding_region_buf[IN_PAD_REGION_DELAY-1]) write_ofmap_value <= write_ofmap_value + a_pixel_of_intermediate_buf;
```

```
  end
```

```
end
```

|       |       |
|-------|-------|
| IW    | 2     |
| IH    | 2     |
| FW    | 3     |
| FH    | 3     |
| IN_C  | 2     |
| OUT_C | 2     |
| S     | 1     |
| PAD   | 1     |
| OW OH | (2,2) |



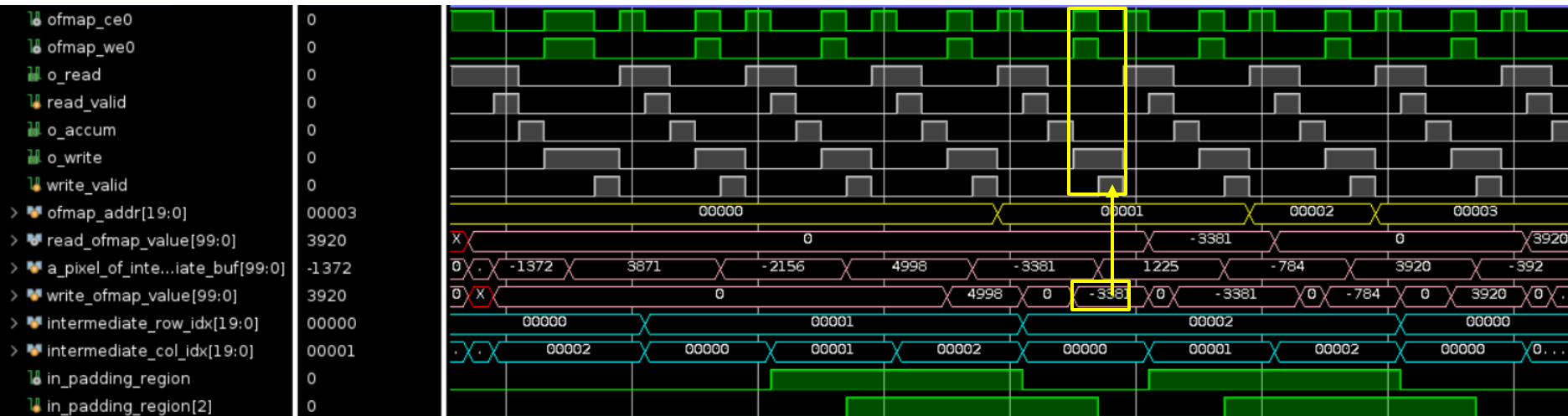
- ifmap(0,0)에 대한 intermediate에 대한 read/write 과정
- Read : ofmap\_addr 1의 초기값을 0으로 reset 시켜두었기 때문에 read\_ofmap\_value = 0 / read\_valid check
- Accum : write\_ofmap\_value <= write\_ofmap\_value + a\_pixel\_of\_intermediate\_buf;

# 8. SAVE\_module: SAVE

// \*\_\*\_\*\_\*\_\*\_\* ACCUMULATION OF INTERMEDIATE VEC AND OFMAP VEC FROM OFMAP MEM \*\_\*\_\*\_\*\_\*\_\*

```
reg signed [OFMAP_BITS-1 : 0] write_ofmap_value;
reg accum_signal;
//
always @(posedge clk or negedge reset_n) begin
    if(!reset_n) begin
        write_ofmap_value <= {OFMAP_BITS{1'b0}};
        accum_signal <= 1'b0;
    end
    //
    else if(o_read) write_ofmap_value <= read_ofmap_value;
    //
    else if(o_accum && !PAD) write_ofmap_value <= write_ofmap_value + a_pixel_of_intermediate_buf;
    //
    else if(o_accum && PAD) begin
        if(in_padding_region_buf[IN_PAD_REGION_DELAY-1]) write_ofmap_value <= write_ofmap_value + a_pixel_of_intermediate_buf;
    end
end
```

|       |       |
|-------|-------|
| IW    | 2     |
| IH    | 2     |
| FW    | 3     |
| FH    | 3     |
| IN_C  | 2     |
| OUT_C | 2     |
| S     | 1     |
| PAD   | 1     |
| OW OH | (2,2) |



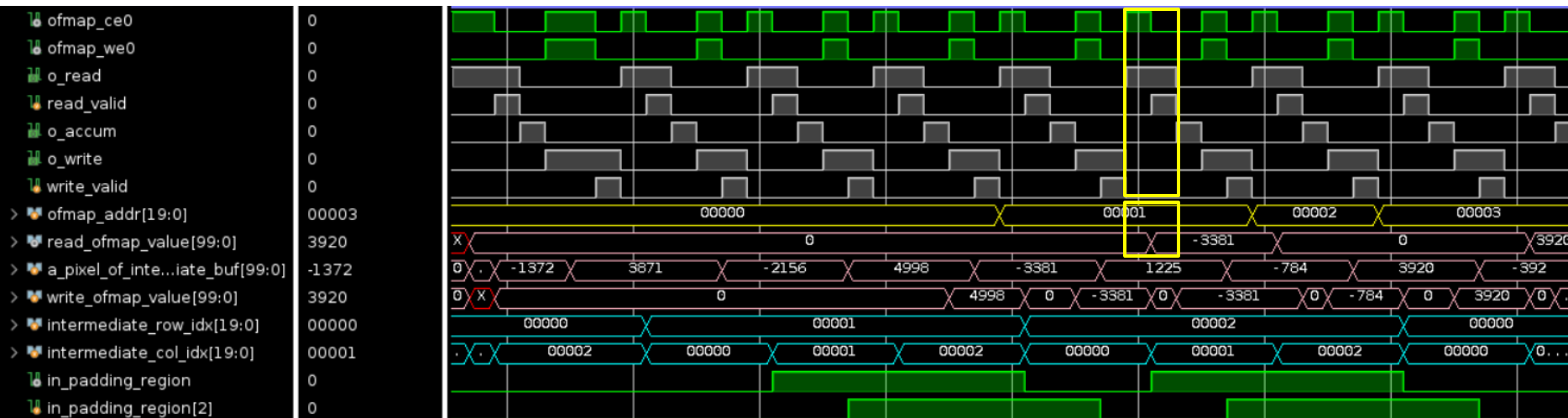
- ifmap(0,0)에 대한 intermediate에 대한 read/write 과정
- Read : ofmap\_addr 1의 초기값을 0으로 reset 시켜두었기 때문에 read\_ofmap\_value = 0 / read\_valid check
- Accum : write\_ofmap\_value <= write\_ofmap\_value + a\_pixel\_of\_intermediate\_buf;
- Write : write\_valid check



Kumoh National Institute of Technology

end

|              |       |
|--------------|-------|
| <b>IW</b>    | 2     |
| <b>IH</b>    | 2     |
| <b>FW</b>    | 3     |
| <b>FH</b>    | 3     |
| <b>IN_C</b>  | 2     |
| <b>OUT_C</b> | 2     |
| <b>S</b>     | 1     |
| <b>PAD</b>   | 1     |
| <b>OW OH</b> | (2,2) |



- ifmap(0,0)에 대한 intermediate에 대한 read/write 과정
- Read : ofmap addr 1에 -3381이 write 되었기 때문에 -3381을 read하는 모습

# 8. SAVE\_module: SAVE

```
// *_*_*_*_*_* ACCUMULATION OF INTERMEDIATE VEC AND OFMAP VEC FROM OFMAP MEM *_*_*_*_*_*
```

```
reg signed [OFMAP_BITS-1:0] write_ofmap_value;
```

```
reg accum_signal;
```

```
//
```

```
always @(posedge clk or negedge reset_n) begin
```

```
  if(!reset_n) begin
```

```
    write_ofmap_value <= {OFMAP_BITS{1'b0}};
```

```
    accum_signal <= 1'b0;
```

```
  end
```

```
  //
```

```
  else if(o_read) write_ofmap_value <= read_ofmap_value;
```

```
  //
```

```
  else if(o_accum && !PAD) write_ofmap_value <= write_ofmap_value + a_pixel_of_intermediate_buf;
```

```
  //
```

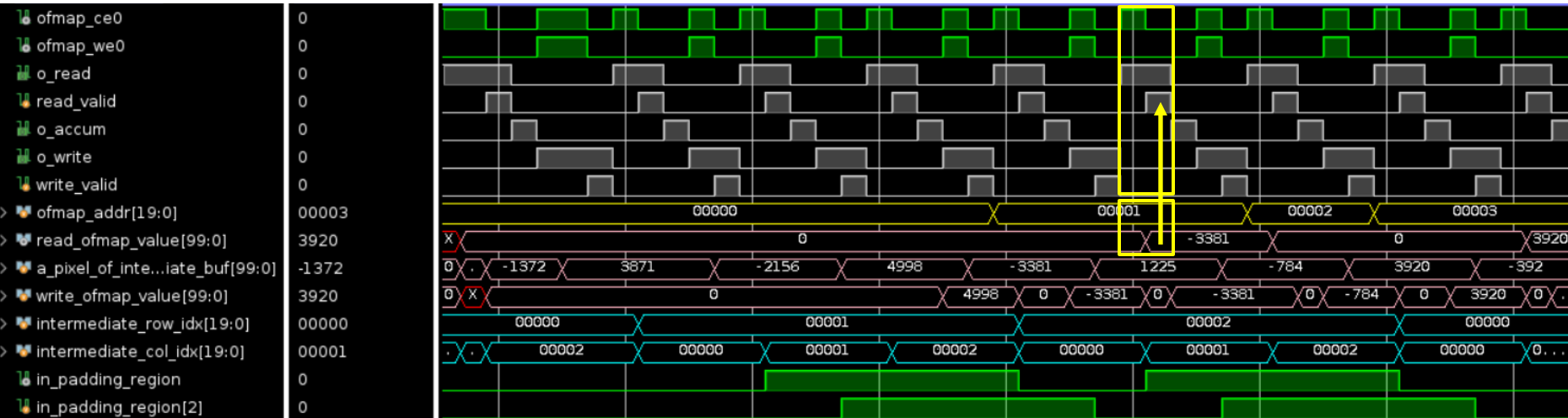
```
  else if(o_accum && PAD) begin
```

```
    if(in_padding_region_buf[IN_PAD_REGION_DELAY-1]) write_ofmap_value <= write_ofmap_value + a_pixel_of_intermediate_buf;
```

```
  end
```

```
end
```

|       |       |
|-------|-------|
| IW    | 2     |
| IH    | 2     |
| FW    | 3     |
| FH    | 3     |
| IN_C  | 2     |
| OUT_C | 2     |
| S     | 1     |
| PAD   | 1     |
| OW OH | (2,2) |



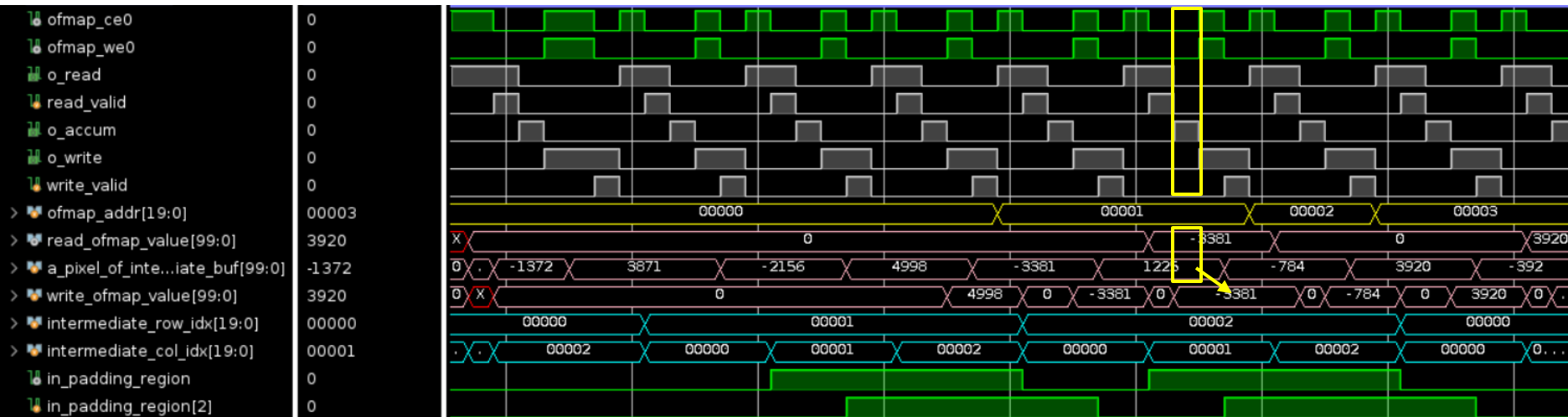
- ifmap(0,0)에 대한 intermediate에 대한 read/write 과정
- Read : ofmap\_addr 1에 -3381이 write 되었기 때문에 -3381을 read하는 모습 / read\_valid check

# 8. SAVE\_module: SAVE

// \*\_\*\_\*\_\*\_\*\_\* ACCUMULATION OF INTERMEDIATE VEC AND OFMAP VEC FROM OFMAP MEM \*\_\*\_\*\_\*\_\*\_

```
reg signed [OFMAP_BITS-1 : 0] write_ofmap_value;
reg accum_signal;
//
always @(posedge clk or negedge reset_n) begin
    if(!reset_n) begin
        write_ofmap_value <= {OFMAP_BITS{1'b0}};
        accum_signal <= 1'b0;
    end
    //
    else if(o_read) write_ofmap_value <= read_ofmap_value;
    //
    else if(o_accum && !PAD) write_ofmap_value <= write_ofmap_value + a_pixel_of_intermediate_buf;
    //
    else if(o_accum && PAD) begin
        if(in_padding_region_buf[IN_PAD_REGION_DELAY-1]) write_ofmap_value <= write_ofmap_value + a_pixel_of_intermediate_buf;
    end
end
```

|       |       |
|-------|-------|
| IW    | 2     |
| IH    | 2     |
| FW    | 3     |
| FH    | 3     |
| IN_C  | 2     |
| OUT_C | 2     |
| S     | 1     |
| PAD   | 1     |
| OW OH | (2,2) |



- ifmap(0,0)에 대한 intermediate에 대한 read/write 과정
- Read : ofmap\_addr 1에 -3381이 write 되었기 때문에 -3381을 read하는 모습 / read\_valid check
- Accum : in\_padding\_region[2]가 0이므로 write\_ofmap\_value <= write\_ofmap\_value + a\_pixel\_of\_intermediate\_buf; 실행 안됨

# 8. SAVE\_module: SAVE

// \*\_\*\_\*\_\*\_\*\_\* ACCUMULATION OF INTERMEDIATE VEC AND OFMAP VEC FROM OFMAP MEM \*\_\*\_\*\_\*\_\*\_\*

reg signed [OFMAP\_BITS-1 : 0] write\_ofmap\_value;

reg accum\_signal;

//

always @(posedge clk or negedge reset\_n) begin

if(!reset\_n) begin

write\_ofmap\_value <= {OFMAP\_BITS{1'b0}};

accum\_signal <= 1'b0;

end

//

else if(o\_read) write\_ofmap\_value <= read\_ofmap\_value;

//

else if(o\_accum && !PAD) write\_ofmap\_value <= write\_ofmap\_value + a\_pixel\_of\_intermediate\_buf;

//

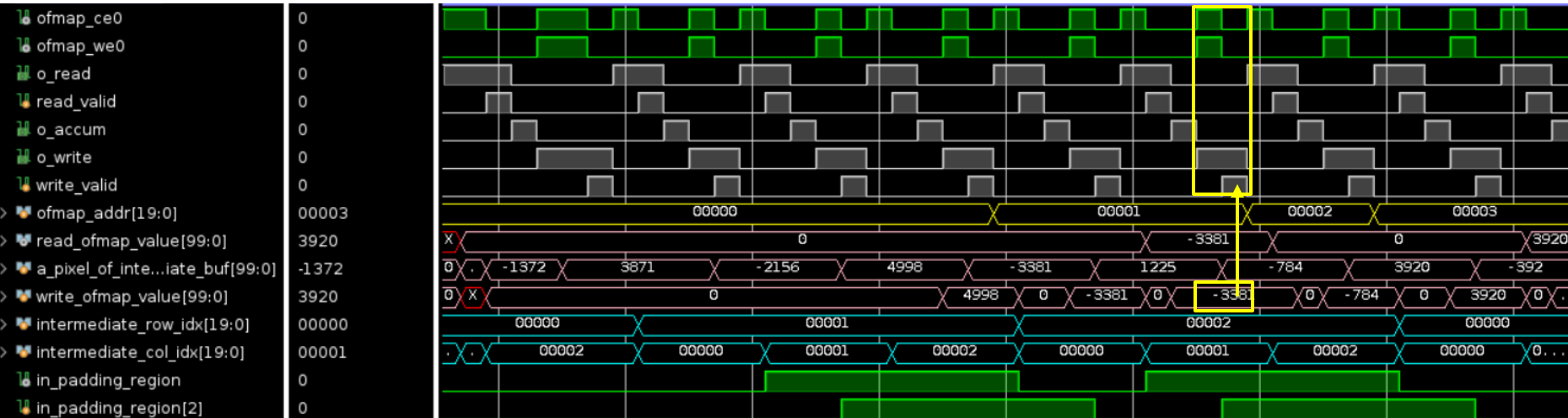
else if(o\_accum && PAD) begin

if(in\_padding\_region\_buf[IN\_PAD\_REGION\_DELAY-1]) write\_ofmap\_value <= write\_ofmap\_value + a\_pixel\_of\_intermediate\_buf;

end

end

|       |       |
|-------|-------|
| IW    | 2     |
| IH    | 2     |
| FW    | 3     |
| FH    | 3     |
| IN_C  | 2     |
| OUT_C | 2     |
| S     | 1     |
| PAD   | 1     |
| OW OH | (2,2) |



- ifmap(0,0)에 대한 intermediate에 대한 read/write 과정
- Read : ofmap\_addr 1에 -3381이 write 되었기 때문에 -3381을 read하는 모습 / read\_valid check
- Accum : in\_padding\_region[2]가 0이므로 write\_ofmap\_value <= write\_ofmap\_value + a\_pixel\_of\_intermediate\_buf; 실행 안됨
- Write : 다시 -3381 write / write\_valid check

# 8. SAVE\_module: SAVE

// \*\_\*\_\*\_\*\_\*\_\* ACCUMULATION OF INTERMEDIATE VEC AND OFMAP VEC FROM OFMAP MEM \*\_\*\_\*\_\*\_\*\_\*

reg signed [OFMAP\_BITS-1 : 0] write\_ofmap\_value;

reg accum\_signal;

//

always @(posedge clk or negedge reset\_n) begin

if(!reset\_n) begin

write\_ofmap\_value <= {OFMAP\_BITS{1'b0}};

accum\_signal <= 1'b0;

end

//

else if(o\_read) write\_ofmap\_value <= read\_ofmap\_value;

//

else if(o\_accum && !PAD) write\_ofmap\_value <= write\_ofmap\_value + a\_pixel\_of\_intermediate\_buf;

//

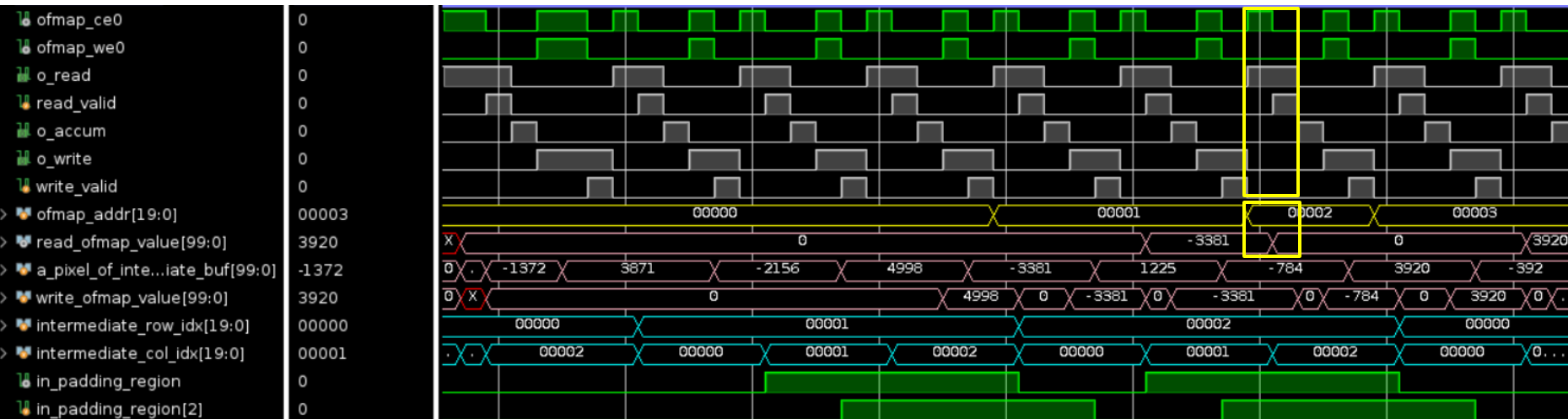
else if(o\_accum && PAD) begin

if(in\_padding\_region\_buf[IN\_PAD\_REGION\_DELAY-1]) write\_ofmap\_value <= write\_ofmap\_value + a\_pixel\_of\_intermediate\_buf;

end

end

|       |       |
|-------|-------|
| IW    | 2     |
| IH    | 2     |
| FW    | 3     |
| FH    | 3     |
| IN_C  | 2     |
| OUT_C | 2     |
| S     | 1     |
| PAD   | 1     |
| OW OH | (2,2) |



- ifmap(0,0)에 대한 intermediate에 대한 read/write 과정
- Read : ofmap\_addr 2의 초기값을 0으로 reset 시켜두었기 때문에 read\_ofmap\_value = 0

# 8. SAVE\_module: SAVE

```
// *_*_*_*_*_* ACCUMULATION OF INTERMEDIATE VEC AND OFMAP VEC FROM OFMAP MEM *_*_*_*_*_*
```

```
reg signed [OFMAP_BITS-1:0] write_ofmap_value;
```

```
reg accum_signal;
```

```
//
```

```
always @(posedge clk or negedge reset_n) begin
```

```
  if(!reset_n) begin
```

```
    write_ofmap_value <= {OFMAP_BITS{1'b0}};
```

```
    accum_signal <= 1'b0;
```

```
  end
```

```
  //
```

```
  else if(o_read) write_ofmap_value <= read_ofmap_value;
```

```
  //
```

```
  else if(o_accum && !PAD) write_ofmap_value <= write_ofmap_value + a_pixel_of_intermediate_buf;
```

```
  //
```

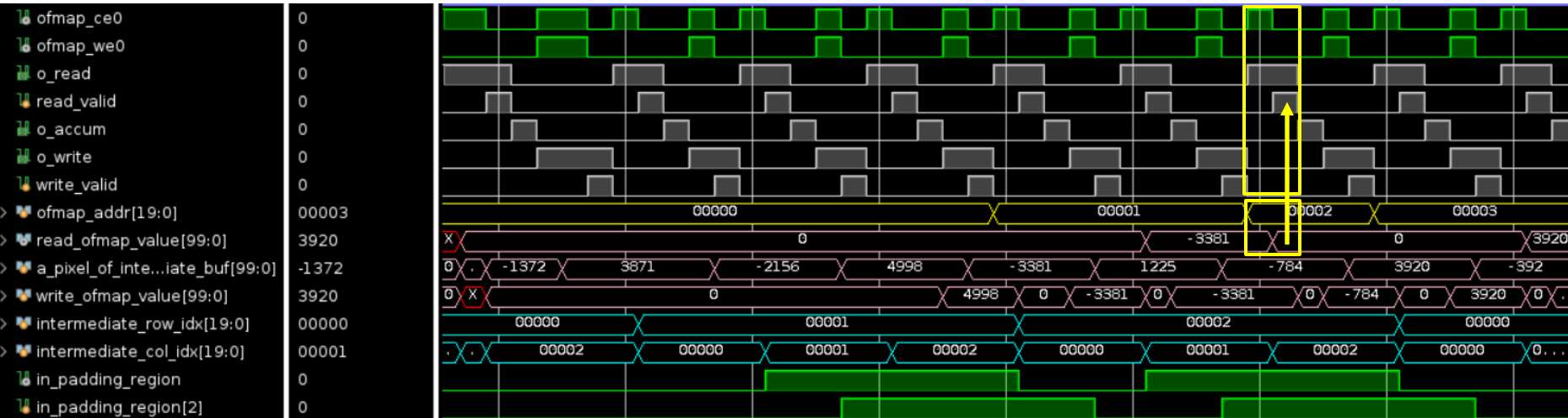
```
  else if(o_accum && PAD) begin
```

```
    if(in_padding_region_buf[IN_PAD_REGION_DELAY-1]) write_ofmap_value <= write_ofmap_value + a_pixel_of_intermediate_buf;
```

```
  end
```

```
end
```

|       |       |
|-------|-------|
| IW    | 2     |
| IH    | 2     |
| FW    | 3     |
| FH    | 3     |
| IN_C  | 2     |
| OUT_C | 2     |
| S     | 1     |
| PAD   | 1     |
| OW OH | (2,2) |



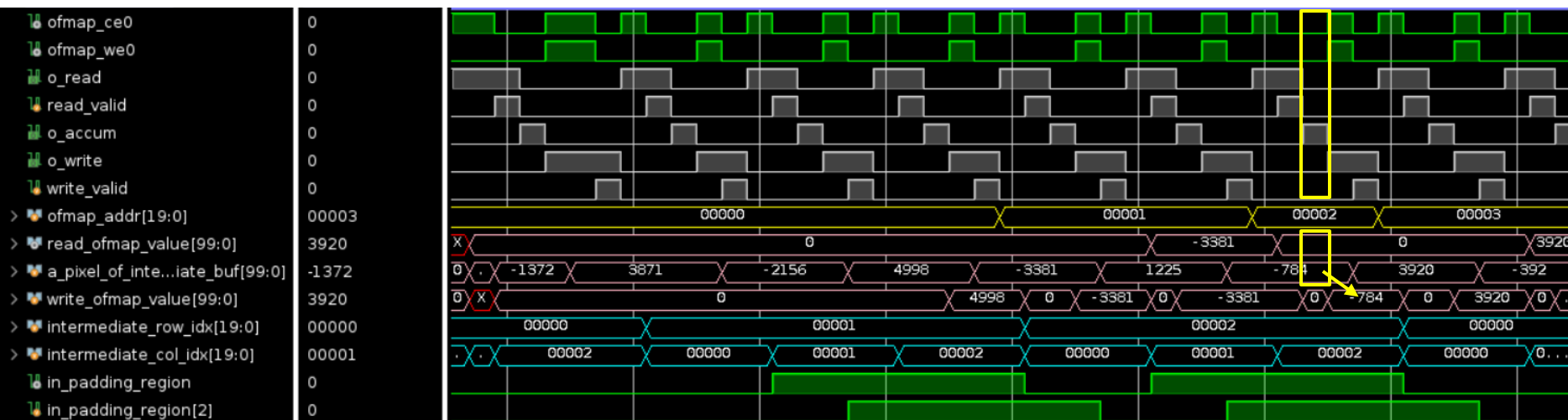
- ifmap(0,0)에 대한 intermediate에 대한 read/write 과정
- Read : ofmap\_addr 2의 초기값을 0으로 reset 시켜두었기 때문에 read\_ofmap\_value = 0 / read\_valid check



Kumoh National Institute of Technology

end

|              |       |
|--------------|-------|
| <b>IW</b>    | 2     |
| <b>IH</b>    | 2     |
| <b>FW</b>    | 3     |
| <b>FH</b>    | 3     |
| <b>IN_C</b>  | 2     |
| <b>OUT_C</b> | 2     |
| <b>S</b>     | 1     |
| <b>PAD</b>   | 1     |
| <b>OW OH</b> | (2,2) |



- ifmap(0,0)에 대한 intermediate에 대한 read/write 과정
- Read : ofmap\_addr 2의 초기값을 0으로 reset 시켜두었기 때문에 read\_ofmap\_value = 0 / read\_valid check
- Accum : write\_ofmap\_value <= write\_ofmap\_value + a pixel of intermediate buf;

# 8. SAVE\_module: SAVE

```
// *_*_*_*_*_* ACCUMULATION OF INTERMEDIATE VEC AND OFMAP VEC FROM OFMAP MEM *_*_*_*_*_*
```

```
reg signed [OFMAP_BITS-1 : 0] write_ofmap_value;
```

```
reg accum_signal;
```

```
//
```

```
always @(posedge clk or negedge reset_n) begin
```

```
  if(!reset_n) begin
```

```
    write_ofmap_value <= {OFMAP_BITS{1'b0}};
```

```
    accum_signal <= 1'b0;
```

```
  end
```

```
  //
```

```
  else if(o_read) write_ofmap_value <= read_ofmap_value;
```

```
  //
```

```
  else if(o_accum && !PAD) write_ofmap_value <= write_ofmap_value + a_pixel_of_intermediate_buf;
```

```
  //
```

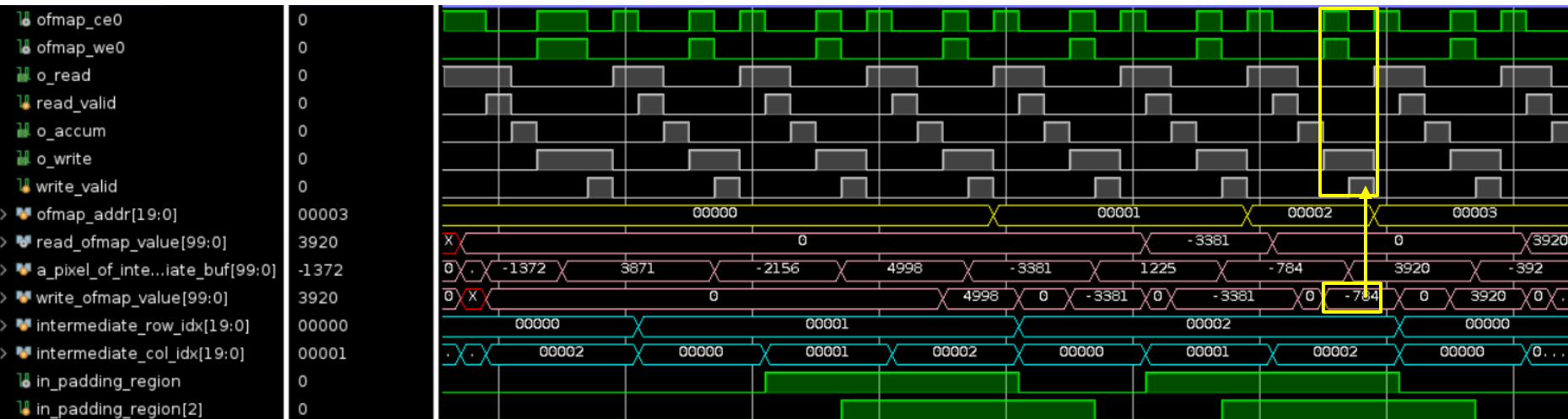
```
  else if(o_accum && PAD) begin
```

```
    if(in_padding_region_buf[IN_PAD_REGION_DELAY-1]) write_ofmap_value <= write_ofmap_value + a_pixel_of_intermediate_buf;
```

```
  end
```

```
end
```

|       |       |
|-------|-------|
| IW    | 2     |
| IH    | 2     |
| FW    | 3     |
| FH    | 3     |
| IN_C  | 2     |
| OUT_C | 2     |
| S     | 1     |
| PAD   | 1     |
| OW OH | (2,2) |



- ifmap(0,0)에 대한 intermediate에 대한 read/write 과정
- Read : ofmap\_addr 2의 초기값을 0으로 reset 시켜두었기 때문에 read\_ofmap\_value = 0 / read\_valid check
- Accum : write\_ofmap\_value <= write\_ofmap\_value + a\_pixel\_of\_intermediate\_buf;
- Write : write\_valid check

# 8. SAVE\_module: SAVE

// \*\_\*\_\*\_\*\_\*\_\* ACCUMULATION OF INTERMEDIATE VEC AND OFMAP VEC FROM OFMAP MEM \*\_\*\_\*\_\*\_\*\_\*

reg signed [OFMAP\_BITS-1 : 0] write\_ofmap\_value;

reg accum\_signal;

//

always @(posedge clk or negedge reset\_n) begin

if(!reset\_n) begin

write\_ofmap\_value <= {OFMAP\_BITS{1'b0}};

accum\_signal <= 1'b0;

end

//

else if(o\_read) write\_ofmap\_value <= read\_ofmap\_value;

//

else if(o\_accum && !PAD) write\_ofmap\_value <= write\_ofmap\_value + a\_pixel\_of\_intermediate\_buf;

//

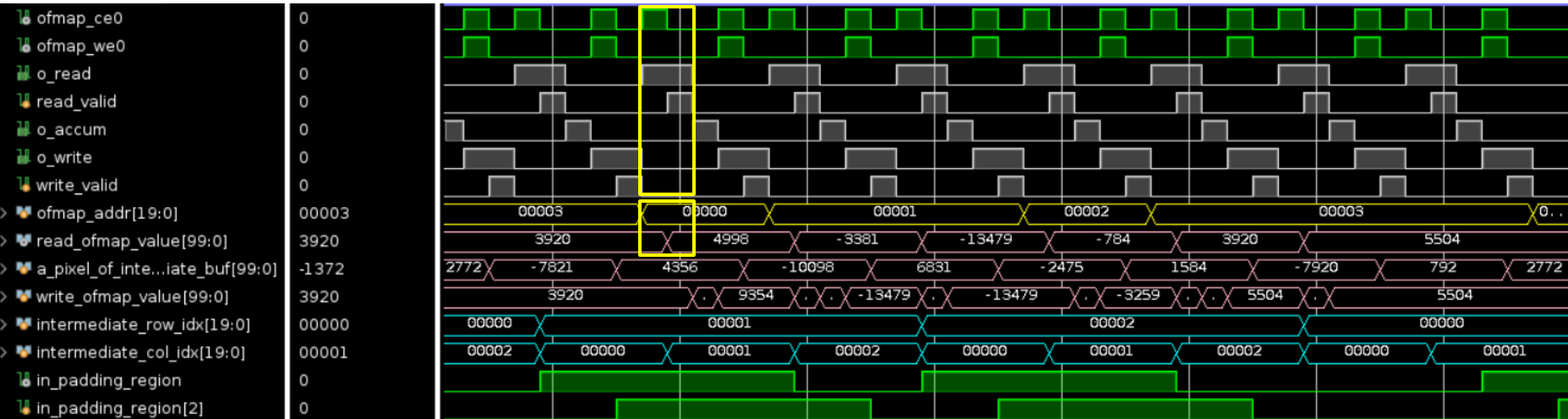
else if(o\_accum && PAD) begin

if(in\_padding\_region\_buf[IN\_PAD\_REGION\_DELAY-1]) write\_ofmap\_value <= write\_ofmap\_value + a\_pixel\_of\_intermediate\_buf;

end

end

|       |       |
|-------|-------|
| IW    | 2     |
| IH    | 2     |
| FW    | 3     |
| FH    | 3     |
| IN_C  | 2     |
| OUT_C | 2     |
| S     | 1     |
| PAD   | 1     |
| OW OH | (2,2) |



- ifmap(0,1)에 대한 intermediate에 대한 read/write 과정
- Read : 이전에 ofmap\_addr 0에 4998이 write되었기 때문에 4998을 read

# 8. SAVE\_module: SAVE

// \*\_\*\_\*\_\*\_\*\_\* ACCUMULATION OF INTERMEDIATE VEC AND OFMAP VEC FROM OFMAP MEM \*\_\*\_\*\_\*\_\*\_\*

reg signed [OFMAP\_BITS-1 : 0] write\_ofmap\_value;

reg accum\_signal;

//

always @(posedge clk or negedge reset\_n) begin

if(!reset\_n) begin

write\_ofmap\_value <= {OFMAP\_BITS{1'b0}};

accum\_signal <= 1'b0;

end

//

else if(o\_read) write\_ofmap\_value <= read\_ofmap\_value;

//

else if(o\_accum && !PAD) write\_ofmap\_value <= write\_ofmap\_value + a\_pixel\_of\_intermediate\_buf;

//

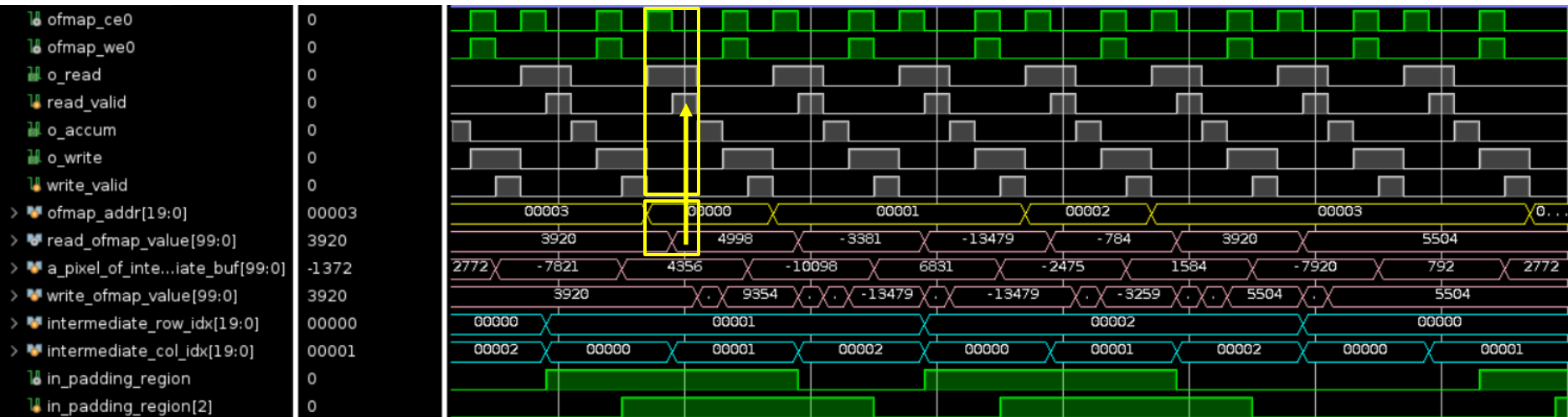
else if(o\_accum && PAD) begin

if(in\_padding\_region\_buf[IN\_PAD\_REGION\_DELAY-1]) write\_ofmap\_value <= write\_ofmap\_value + a\_pixel\_of\_intermediate\_buf;

end

end

|       |       |
|-------|-------|
| IW    | 2     |
| IH    | 2     |
| FW    | 3     |
| FH    | 3     |
| IN_C  | 2     |
| OUT_C | 2     |
| S     | 1     |
| PAD   | 1     |
| OW OH | (2,2) |



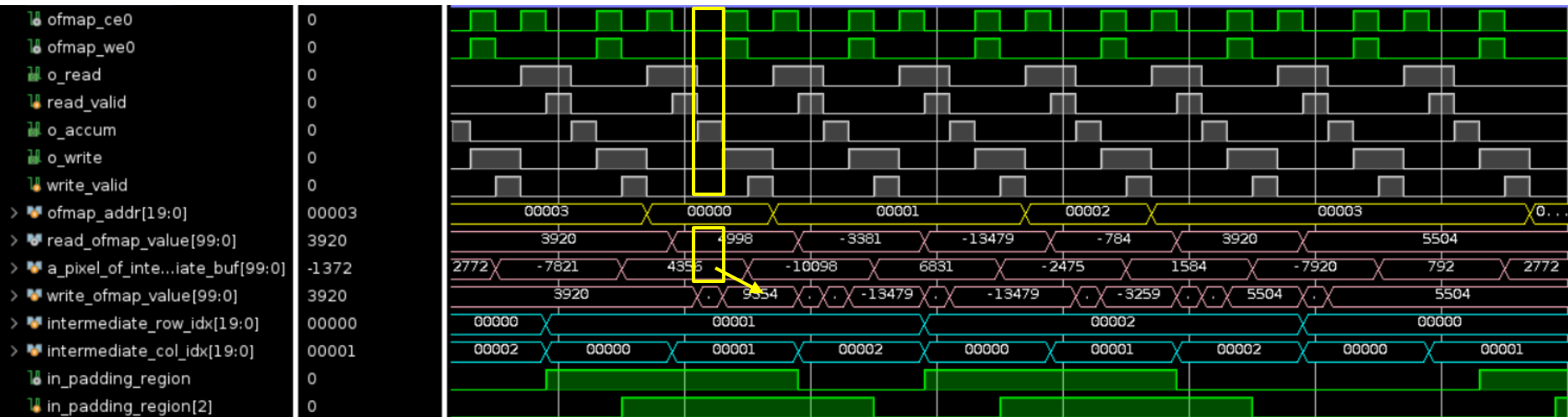
- ifmap(0,1)에 대한 intermediate에 대한 read/write 과정
- Read : 이전에 ofmap\_addr 0에 4998이 write되었기 때문에 4998을 read / read\_valid check

# 8. SAVE\_module: SAVE

// \*\_\*\_\*\_\*\_\*\_\* ACCUMULATION OF INTERMEDIATE VEC AND OFMAP VEC FROM OFMAP MEM \*\_\*\_\*\_\*\_\*\_\*

```
reg signed [OFMAP_BITS-1 : 0] write_ofmap_value;
reg accum_signal;
//
always @(posedge clk or negedge reset_n) begin
    if(!reset_n) begin
        write_ofmap_value <= {OFMAP_BITS{1'b0}};
        accum_signal <= 1'b0;
    end
    //
    else if(o_read) write_ofmap_value <= read_ofmap_value;
    //
    else if(o_accum && !PAD) write_ofmap_value <= write_ofmap_value + a_pixel_of_intermediate_buf;
    //
    else if(o_accum && PAD) begin
        if(in_padding_region_buf[IN_PAD_REGION_DELAY-1]) write_ofmap_value <= write_ofmap_value + a_pixel_of_intermediate_buf;
    end
end
```

|       |       |
|-------|-------|
| IW    | 2     |
| IH    | 2     |
| FW    | 3     |
| FH    | 3     |
| IN_C  | 2     |
| OUT_C | 2     |
| S     | 1     |
| PAD   | 1     |
| OW OH | (2,2) |



- Read : ofmap\_addr 2의 초기값을 0으로 reset 시켜두었기 때문에 read\_ofmap\_value = 0 / read\_valid check
- Accum : write\_ofmap\_value <= write\_ofmap\_value + a\_pixel\_of\_intermediate\_buf;

# 8. SAVE\_module: SAVE

// \*\_\*\_\*\_\*\_\*\_\* ACCUMULATION OF INTERMEDIATE VEC AND OFMAP VEC FROM OFMAP MEM \*\_\*\_\*\_\*\_\*\_\*

reg signed [OFMAP\_BITS-1 : 0] write\_ofmap\_value;

reg accum\_signal;

//

always @(posedge clk or negedge reset\_n) begin

if(!reset\_n) begin

write\_ofmap\_value <= {OFMAP\_BITS{1'b0}};

accum\_signal <= 1'b0;

end

//

else if(o\_read) write\_ofmap\_value <= read\_ofmap\_value;

//

else if(o\_accum && !PAD) write\_ofmap\_value <= write\_ofmap\_value + a\_pixel\_of\_intermediate\_buf;

//

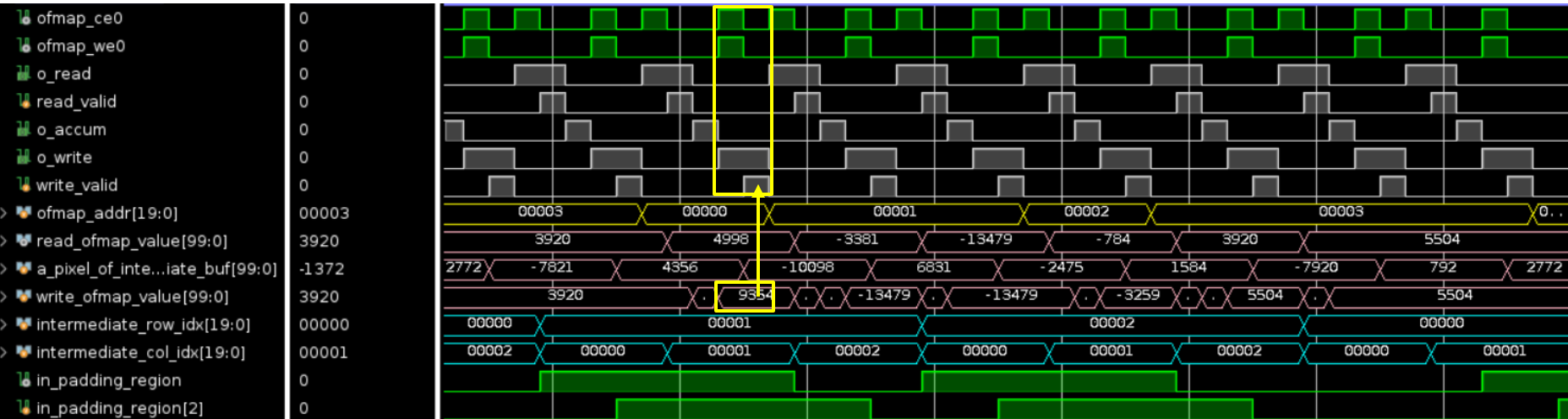
else if(o\_accum && PAD) begin

if(in\_padding\_region\_buf[IN\_PAD\_REGION\_DELAY-1]) write\_ofmap\_value <= write\_ofmap\_value + a\_pixel\_of\_intermediate\_buf;

end

end

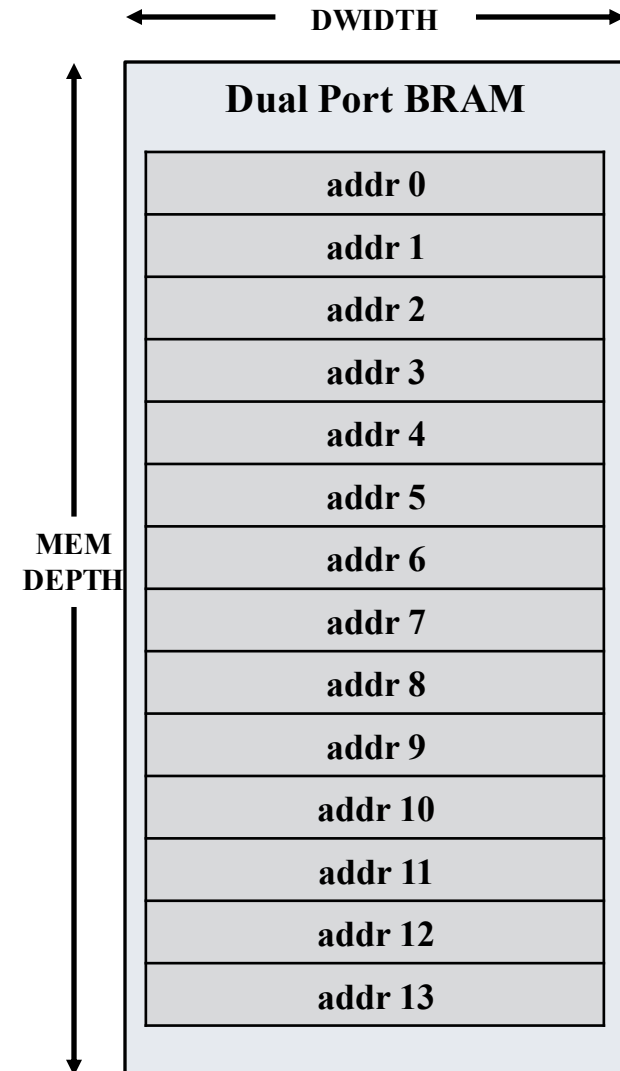
|       |       |
|-------|-------|
| IW    | 2     |
| IH    | 2     |
| FW    | 3     |
| FH    | 3     |
| IN_C  | 2     |
| OUT_C | 2     |
| S     | 1     |
| PAD   | 1     |
| OW OH | (2,2) |



- Read : ofmap\_addr 2의 초기값을 0으로 reset 시켜두었기 때문에 read\_ofmap\_value = 0 / read\_valid check
- Accum : write\_ofmap\_value <= write\_ofmap\_value + a\_pixel\_of\_intermediate\_buf;
- Write : write\_valid check

# 9. BRAM

```
// *_*_*_*_* Dual Port BRAM *_*_*_*_*  
(* ram_style = "block" *) reg [DWIDTH-1:0] ram[0:MEM_DEPTH-1];  
  
integer ram_index;  
generate  
  initial  
    for(ram_index = 0; ram_index < MEM_DEPTH; ram_index = ram_index + 1)  
      ram[ram_index] = {DWIDTH{1'b0}};  
endgenerate  
  
// Normal Operation  
always @(posedge clk) begin  
  if (ce0) begin  
    if (we0) begin  
      ram[addr0] <= d0;  
    end  
  
    else begin  
      q0 <= ram[addr0];  
    end  
  end  
end
```



# 9. BRAM : ifmap, kernel, ofmap

**DWIDTH:** 8bit  
**MEM\_DEPTH:**  $IN\_C * IH * IW$

**DWIDTH:**  $8bit * KW$   
**MEM\_DEPTH:**  $OUT\_C * IN\_C * KH$

**DWIDTH:** OFMAP\_BIT bit  
**MEM\_DEPTH:**  $OUT\_C * OH * OW$

**ifmap mem**

|           |
|-----------|
| (0, 0, 0) |
| (0, 0, 1) |
| (0, 1, 0) |
| (0, 1, 1) |
| (1, 0, 0) |
| (1, 0, 1) |
| (1, 1, 0) |
| (1, 0, 1) |

(IN\_C, H, W)

**kernel mem**

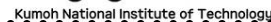
|           |           |           |
|-----------|-----------|-----------|
| (0,0,0,0) | (0,0,0,1) | (0,0,0,2) |
| (0,0,1,0) | (0,0,1,1) | (0,0,1,2) |
| (0,0,2,0) | (0,0,2,1) | (0,0,2,2) |
| (0,1,0,0) | (0,1,0,1) | (0,1,0,2) |
| (0,1,1,0) | (0,1,1,1) | (0,1,1,2) |
| (0,1,2,0) | (0,1,2,1) | (0,1,2,2) |
| (1,0,0,0) | (1,0,0,1) | (1,0,0,2) |
| (1,0,1,0) | (1,0,1,1) | (1,0,1,2) |
| (1,0,2,0) | (1,0,2,1) | (1,0,2,2) |
| (1,1,0,0) | (1,1,0,1) | (1,1,0,2) |
| (1,1,1,0) | (1,1,1,1) | (1,1,1,2) |
| (1,1,2,0) | (1,1,2,1) | (1,1,2,2) |

(IN\_C, OUT\_C, H, W)

**ofmap mem**

|         |         |
|---------|---------|
| addr 0  | addr 16 |
| addr 1  | addr 17 |
| addr 2  | addr 18 |
| addr 3  | addr 19 |
| addr 4  | addr 20 |
| addr 5  | addr 21 |
| addr 6  | addr 22 |
| addr 7  | addr 23 |
| addr 8  | addr 24 |
| addr 9  | addr 25 |
| addr 10 | addr 26 |
| addr 11 | addr 27 |
| addr 12 | addr 28 |
| addr 13 | addr 29 |
| addr 14 | addr 30 |
| addr 15 | addr 31 |

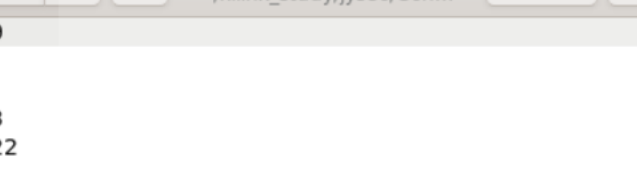
|         |         |
|---------|---------|
| IW      | 2       |
| IH      | 2       |
| FW      | 3       |
| FH      | 3       |
| IN_C    | 2       |
| OUT_C   | 2       |
| S       | 1       |
| PAD     | 0       |
| C OW OH | (2,4,4) |



**MEM\_DEPTH: IN\_C \* IH \* IW**

| ifmap mem |           |
|-----------|-----------|
|           | (0, 0, 0) |
|           | (0, 0, 1) |
|           | (0, 1, 0) |
|           | (0, 1, 1) |
|           | (1, 0, 0) |
|           | (1, 0, 1) |
|           | (1, 1, 0) |
|           | (1, 0, 1) |

(IN\_C, H, W)



ifmap.txt  
~/xilinx\_study/jyseo/Con...

Open Save Close

```
1 |-49  
2 99  
3 52  
4 -43  
5 -122  
6 17  
7 49  
8 126
```

Plain Text Tab Width: 8 Ln 1, Col 1 INS

```
// *- *- *- *- *- IFMAP READ *- *- *- *- *-
integer ifmap_idx;

initial begin
    ifmap_fd = $fopen(`IFMAP_PATH, "r"); // ?? ??
    if (ifmap_fd == 0) begin
        $display("FAIL");
        $finish;
    end
    //
    for(ifmap_idx = 0; ifmap_idx < `IH * `IW * `IN_C; ifmap_idx = ifmap_idx + 1) begin
        r1 = $fscanf(ifmap_fd, "%d\n", ifmap_dpbram.ram[ifmap_idx]);
    end
    #10;
    $fclose(ifmap_fd); // ?? ??
end
```

# 9. BRAM : kernel

DWIDTH: 8bit \* KW  
MEM\_DEPTH: OUT\_C \* IN\_C \* KH

kernel mem

|           |           |           |
|-----------|-----------|-----------|
| (0,0,0,0) | (0,0,0,1) | (0,0,0,2) |
| (0,0,1,0) | (0,0,1,1) | (0,0,1,2) |
| (0,0,2,0) | (0,0,2,1) | (0,0,2,2) |
| (0,1,0,0) | (0,1,0,1) | (0,1,0,2) |
| (0,1,1,0) | (0,1,1,1) | (0,1,1,2) |
| (0,1,2,0) | (0,1,2,1) | (0,1,2,2) |
| (1,0,0,0) | (1,0,0,1) | (1,0,0,2) |
| (1,0,1,0) | (1,0,1,1) | (1,0,1,2) |
| (1,0,2,0) | (1,0,2,1) | (1,0,2,2) |
| (1,1,0,0) | (1,1,0,1) | (1,1,0,2) |
| (1,1,1,0) | (1,1,1,1) | (1,1,1,2) |
| (1,1,2,0) | (1,1,2,1) | (1,1,2,2) |

(IN\_C, OUT\_C, H, W)

```

kernel.txt
~/xilinx_study/jyseo/Con...
Open Save
1 8
2 28
3 -79
4 44
5 -102
6 69
7 -25
8 16
9 -80
10 -93
11 -79
12 41
13 -33
14 -92
15 -36
16 -116
17 50
Plain Text Tab Width: 8 Ln 1, Col 2 INS

```

```

// ***** KERNEL READ *****
integer kernel_out_channel_idx;
integer kernel_in_channel_idx;
integer kernel_row_idx;
integer kernel_col_idx;

initial begin
    kernel_fd = $fopen("KERNEL_PATH", "r"); // ?? ??
    if (kernel_fd == 0) begin
        $display("FAIL");
        $finish;
    end
    //
    for(kernel_out_channel_idx = 0; kernel_out_channel_idx < "OUT_C"; kernel_out_channel_idx = kernel_out_channel_idx + 1) begin
        for(kernel_in_channel_idx = 0; kernel_in_channel_idx < "IN_C"; kernel_in_channel_idx = kernel_in_channel_idx + 1) begin
            for(kernel_row_idx = 0; kernel_row_idx < "FH"; kernel_row_idx = kernel_row_idx + 1) begin
                for(kernel_col_idx = 0; kernel_col_idx < "FW"; kernel_col_idx = kernel_col_idx + 1) begin
                    r2 = $fscanf(kernel_fd, "%d\n",
                        kernel[(kernel_col_idx + kernel_row_idx * "FW" + kernel_in_channel_idx * "FW" + "FH" + kernel_out_channel_idx * "FW" + "FH" * "IN_C") * "BITS" +: "BITS"]);
                    weight_dpbram.ram[kernel_row_idx + kernel_in_channel_idx * "FH" + kernel_out_channel_idx * "FH" * "IN_C"]
                        <= kernel[(kernel_row_idx + kernel_in_channel_idx * "FH" + kernel_out_channel_idx * "FH" * "IN_C") * "BITS" +: "BITS" * "FW"];
                end
            end
        end
    end
    //
    $fclose(kernel_fd); // ?? ??
end

```

# 9. BRAM : kernel

DWIDTH: 8bit \* KW  
MEM\_DEPTH: OUT\_C \* IN\_C \* KH

kernel mem

|           |           |           |
|-----------|-----------|-----------|
| (0,0,0,0) | (0,0,0,1) | (0,0,0,2) |
| (0,0,1,0) | (0,0,1,1) | (0,0,1,2) |
| (0,0,2,0) | (0,0,2,1) | (0,0,2,2) |
| (0,1,0,0) | (0,1,0,1) | (0,1,0,2) |
| (0,1,1,0) | (0,1,1,1) | (0,1,1,2) |
| (0,1,2,0) | (0,1,2,1) | (0,1,2,2) |
| (1,0,0,0) | (1,0,0,1) | (1,0,0,2) |
| (1,0,1,0) | (1,0,1,1) | (1,0,1,2) |
| (1,0,2,0) | (1,0,2,1) | (1,0,2,2) |
| (1,1,0,0) | (1,1,0,1) | (1,1,0,2) |
| (1,1,1,0) | (1,1,1,1) | (1,1,1,2) |
| (1,1,2,0) | (1,1,2,1) | (1,1,2,2) |

(IN\_C, OUT\_C, H, W)

```
// WEGT_MEM_WRITE_VERIFICATION
#include <stdio.h>
#include <stdlib.h>

#define FW 3
#define FH 3
#define C 3

int main()
{
    int kernel[C][FH][FW];
    int WEGT_MEM[C * FH][FW];
    int FLATTEN_WEGT[FW * FH * C];
    int PREDICT[FW * FH * C];
    int concat[FW];

    srand(1); // 난수 초기화

    printf("Flattened Weight \n");
    for (int n = 0; n < FW * FH * C; n++)
    {
        int random = rand() % 256;
        FLATTEN_WEGT[n] = random;
        printf("index: %d, value: %d\n", n, FLATTEN_WEGT[n]);
    }

    printf("\nWeight MEM\n");
    for (int i = 0; i < C; i++)
    {
        for (int j = 0; j < FH; j++)
        {
            for (int k = 0; k < FW; k++)
            {
                concat[k] = FLATTEN_WEGT[k+FH*j+FW*FH*i];
            }
            for (int z = 0; z < 3; z++)
            {
                WEGT_MEM[j+C*i][z] = concat[z];
                printf("row: %d, col: %d, value: %d\n", j+C*i, z, WEGT_MEM[j+C*i][z]);
                PREDICT[z+FH*j+FW*FH*i] = WEGT_MEM[j+C*i][z];
            }
        }
    }

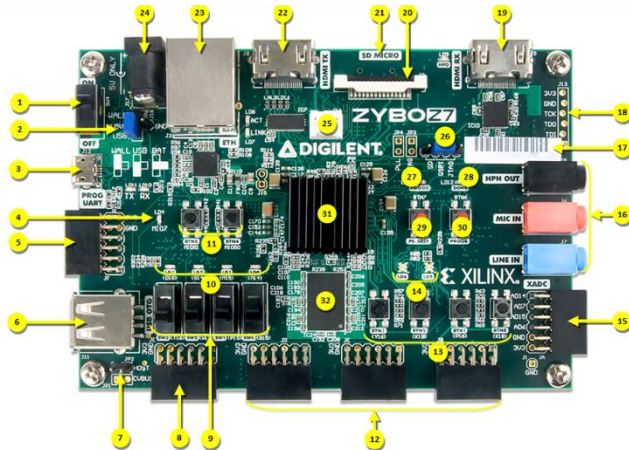
    int count = 0;
    for (int i=0; i < FW * FH * C; i++)
    {
        if(PREDICT[i] == FLATTEN_WEGT[i])
            count += 1;
    }

    if(count == FW * FH * C)
        printf("WEGT_MEM_VERIFICATION SUCCESS\n");
    else
        printf("WEGT_MEM_VERIFICATION FAILED");

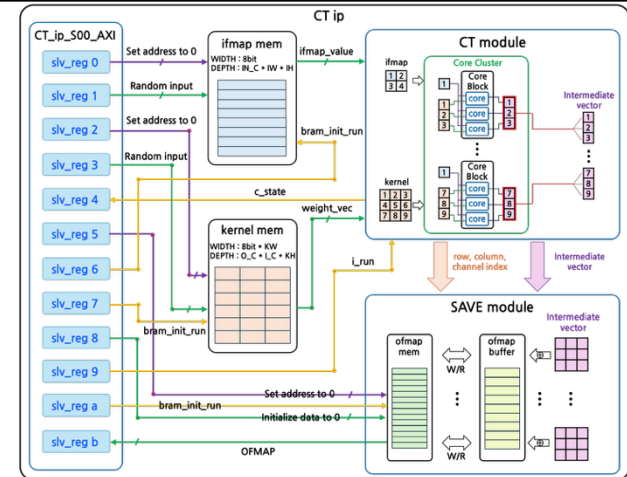
    return 0;
}
```

# 10. FPGA : PS --- PL

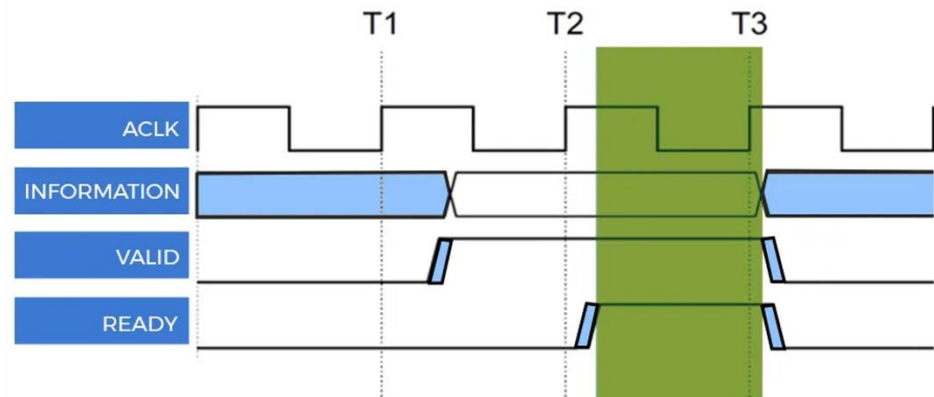
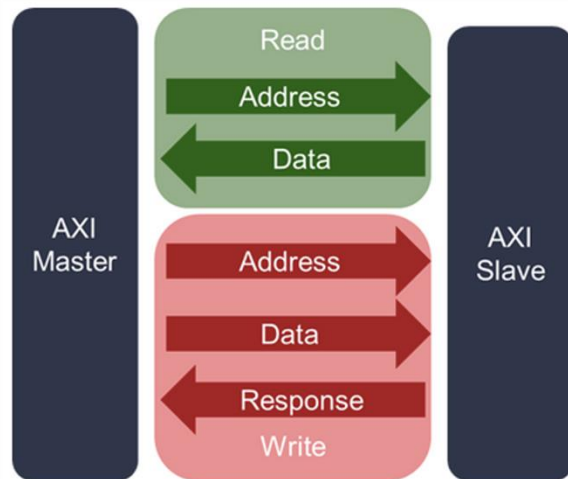
PS: Zybo-Z7-20

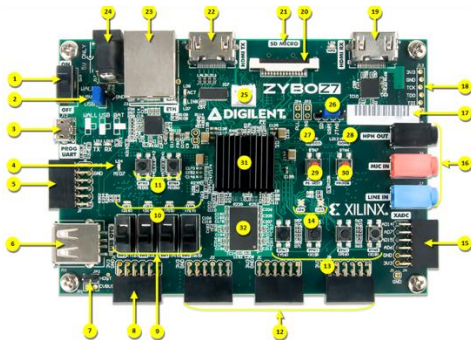


PL: ConvTranspose2D ip



AXI4-Lite





The diagram illustrates the architecture of the CT module and its interaction with the CT\_ip\_S00\_AXI and the SAVE module.

**CT\_ip\_S00\_AXI:** This block contains a series of slave registers (slv\_reg 0 to slv\_reg b). It provides control signals (Set address to 0, Random input, bram\_init\_run) and data signals (ifmap\_value, weight\_vec, i\_run, OFMAP) to the CT module.

**CT module:** This module contains a Core Cluster. The Core Cluster consists of multiple Core Blocks. Each Core Block contains a core and a set of registers (1, 2, 3). The Core Cluster receives ifmap and kernel data from the CT\_ip\_S00\_AXI and outputs intermediate vectors. The Core Cluster also receives control signals (Set address to 0, Random input, bram\_init\_run) and data signals (ifmap\_value, weight\_vec, i\_run) from the CT\_ip\_S00\_AXI.

**SAVE module:** This module contains an ofmap mem and an ofmap buffer. The ofmap mem receives OFMAP data from the CT module and outputs it to the ofmap buffer. The ofmap buffer outputs the final OFMAP data to the CT\_ip\_S00\_AXI. The ofmap mem also receives control signals (Set address to 0, Initialize data to 0) and data signals (OFMAP) from the CT\_ip\_S00\_AXI.

**Data Flow:** The data flow starts with the CT\_ip\_S00\_AXI providing ifmap and kernel data to the CT module. The CT module processes this data and outputs intermediate vectors. These intermediate vectors are then processed by the SAVE module, which outputs the final OFMAP data back to the CT\_ip\_S00\_AXI.

# 10. FPGA : Register Address

| Register Num | Reg Address (HEX) | AXI Addr Bus Bits |
|--------------|-------------------|-------------------|
| 0            | 0x00 = 0000_0000  | 6 bits            |
| 1            | 0x04 = 0000_0100  |                   |
| 2            | 0x08 = 0000_1000  |                   |
| 3            | 0x0C = 0000_1100  |                   |
| 4            | 0x10 = 0001_0000  |                   |
| 5            | 0x14 = 0001_0100  |                   |
| 6            | 0x18 = 0001_1000  |                   |
| 7            | 0x1C = 0001_1100  |                   |
| 8            | 0x20 = 0010_0000  |                   |
| 9            | 0x24 = 0010_0100  |                   |
| 10           | 0x28 = 0010_1000  |                   |
| 11           | 0x2C = 0010_1100  |                   |

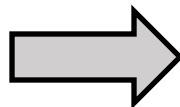
# 10. FPGA : Register Address

| Register Num | Reg Address (HEX) | AXI Addr Bus Bits |
|--------------|-------------------|-------------------|
| 0            | 0x00 = 0000_0000  | 6 bits            |
| 1            | 0x04 = 0000_0100  |                   |
| 2            | 0x08 = 0000_1000  |                   |
| 3            | 0x0C = 0000_1100  |                   |
| 4            | 0x10 = 0001_0000  |                   |
| 5            | 0x14 = 0001_0100  |                   |
| 6            | 0x18 = 0001_1000  |                   |
| 7            | 0x1C = 0001_1100  |                   |
| 8            | 0x20 = 0010_0000  |                   |
| 9            | 0x24 = 0010_0100  |                   |
| 10           | 0x28 = 0010_1000  |                   |
| 11           | 0x2C = 0010_1100  |                   |

하위 2비트는 모두 0이므로 고려하지 않음

# 10. FPGA : Register Address

| Register Num | Reg Address (HEX) | AXI Addr Bus Bits |
|--------------|-------------------|-------------------|
| 0            | 0x00 = 0000_0000  | 6 bits            |
| 1            | 0x04 = 0000_0100  |                   |
| 2            | 0x08 = 0000_1000  |                   |
| 3            | 0x0C = 0000_1100  |                   |
| 4            | 0x10 = 0001_0000  |                   |
| 5            | 0x14 = 0001_0100  |                   |
| 6            | 0x18 = 0001_1000  |                   |
| 7            | 0x1C = 0001_1100  |                   |
| 8            | 0x20 = 0010_0000  |                   |
| 9            | 0x24 = 0010_0100  |                   |
| 10           | 0x28 = 0010_1000  |                   |
| 11           | 0x2C = 0010_1100  |                   |



| Register Num | Reg Address (HEX)   | AXI Addr Bus Bits |
|--------------|---------------------|-------------------|
| 0            | 0 = 0000_00 (6bit)  | 6 bits            |
| 1            | 1 = 0000_01 (6bit)  |                   |
| 2            | 2 = 0000_10 (6bit)  |                   |
| 3            | 3 = 0000_11 (6bit)  |                   |
| 4            | 4 = 0001_00 (6bit)  |                   |
| 5            | 5 = 0001_01 (6bit)  |                   |
| 6            | 6 = 0001_10 (6bit)  |                   |
| 7            | 7 = 0001_11 (6bit)  |                   |
| 8            | 8 = 0010_00 (6bit)  |                   |
| 9            | 9 = 0010_01 (6bit)  |                   |
| 10           | 10 = 0010_10 (6bit) |                   |
| 11           | 11 = 0010_11 (6bit) |                   |

# 10. FPGA : Register Map

| Register Num | Reg Address (HEX) | AXI Addr Bus Bits | Default value | Mode | Description                                                  |
|--------------|-------------------|-------------------|---------------|------|--------------------------------------------------------------|
| 0            | 0x00              | 6 bits            | 32'b0         | R/W  | To reset the ifmap bram addr to zero                         |
| 1            | 0x04              |                   | 32'b0         | R/W  | To initialize the data of the ifmap bram with 8bit rand int  |
| 2            | 0x08              |                   | 32'b0         | R/W  | To reset the weight bram addr to zero                        |
| 3            | 0x0C              |                   | 32'b0         | R/W  | To initialize the data of the weight bram with 8bit rand int |
| 4            | 0x10              |                   | 32'bx         | R    | To check the current state of the CT module                  |
| 5            | 0x14              |                   | 32'b0         | R/W  | To reset the ofmap bram addr to zero                         |
| 6            | 0x18              |                   | 32'b0         | R/W  | To generate ifmap bram init run signal                       |
| 7            | 0x1C              |                   | 32'b0         | R/W  | To generate weight bram init run signal                      |
| 8            | 0x20              |                   | 32'b0         | R/W  | To initialize the data of the ofmap bram with zero           |
| 9            | 0x24              |                   | 32'b0         | R/W  | To generate i_run signal for CT module                       |
| 10           | 0x28              |                   | 32'b0         | R/W  | To generate ofmap bram init run signal                       |
| 11           | 0x2C              |                   | 32'b0         | R    | To read the data of the ofmap bram                           |

# 10. FPGA : random data generation using C

| Register Num | Reg Address (HEX) | AXI Addr Bus Bits | Default value | Mode | Description                          |
|--------------|-------------------|-------------------|---------------|------|--------------------------------------|
| 0            | 0x00              | 6 bits            | 32'b0         | R/W  | To reset the ifmap bram addr to zero |

## [VITIS Code]

```
int *write_buf_ifmap; // 8bit signed
write_buf_ifmap = (int *) malloc(sizeof(int) * IFMAP_MEM_DEPTH);

int *write_buf_weight; // 32bit = 8bit signed * 4?
write_buf_weight = (int *) malloc(sizeof(int) * WEIGHT_MEM_DEPTH);

if (case_num == DATA_GEN)
{
    scanf("%d", &seed);
    srand(seed);

    for(i=0; i< IFMAP_MEM_DEPTH ; i++)
        write_buf_ifmap[i] = (rand() % 256) - 128; // signed 8bit

    for(i=0; i< WEIGHT_MEM_DEPTH ; i++)
    {
        write_buf_weight[i] = 0; // init
        for (core = 0; core < FW; core++)
        {
            IN_WEGT[core] = (rand() % 256) - 128; // signed 8bit
        }
        write_buf_weight[i] = ((IN_WEGT[3] & 0xFF) << 24) |
            ((IN_WEGT[2] & 0xFF) << 16) |
            ((IN_WEGT[1] & 0xFF) << 8) |
            ((IN_WEGT[0] & 0xFF));
    }
}
```

Kernel size가 3x3인 경우 write\_buf\_weight[i]는 signed 8bit \* 3 = 27bit 요구됨.

그러나 C에서 27비트 자료형은 선언할 수 없기 때문에 kernel size 4x4로 진행

# 10. FPGA : Code

| Register Num | Reg Address (HEX) | AXI Addr Bus Bits | Default value | Mode | Description                          |
|--------------|-------------------|-------------------|---------------|------|--------------------------------------|
| 0            | 0x00              | 6 bits            | 32'b0         | R/W  | To reset the ifmap bram addr to zero |

## [VITIS Code]

Xil\_Out32((XPAR\_CT\_IP\_O\_BASEADDR) + (REG\_IFMAP\_BRAM\_ADDR\_RESET\_TO\_ZERO \* AXI\_DATA\_BYTE), (u32)(0x00000000));

## [Verilog Code]

```

always @(posedge S_AXI_ACLK) begin
    if (S_AXI_ARESETN == 1'b0) begin
        slv_reg0 <= 0;
    end
    else begin
        if (slv_reg_wren) begin
            case (axi_awaddr[ADDR_LSB+OPT_MEM_ADDR_BITS:ADDR_LSB]) // 하위 2비트를 제외한 6비트 Register addr
                4'h0: // axi_awaddr[ADDR_LSB+OPT_MEM_ADDR_BITS:ADDR_LSB]이 0000_00이면,
                    for (byte_index = 0; byte_index <= (C_S_AXI_DATA_WIDTH/8)-1; byte_index = byte_index+1)
                        if (S_AXI_WSTRB[byte_index] == 1) begin
                            slv_reg0[(byte_index*8) +: 8] <= S_AXI_WDATA[(byte_index*8) +: 8]; // slv_reg0에 (u32)(0x00000000) 대입
                        end
                    end
            end
        end
    end

    wire [C_S_AXI_ADDR_WIDTH-1:0] reg_0_ifmap_bram_addr_clr = 'h00;
    wire ifmap_bram_addr_write_hit = slv_reg_wren &&
        (axi_awaddr[ADDR_LSB+OPT_MEM_ADDR_BITS:ADDR_LSB] == reg_0_ifmap_bram_addr_clr[ADDR_LSB+OPT_MEM_ADDR_BITS:ADDR_LSB]);
    // slv_reg_wren는 Xil_Out32함수가 실행되면 AXI 내부에서 알아서 1로 만들어 줌.

    wire [IFMAP_AWIDTH-1:0] ifmap_bram_addr_reg = slv_reg0[IFMAP_AWIDTH-1:0]; // slv_reg0은 32bit이므로 [IFMAP_AWIDTH-1:0]만큼 사용

```

# 10. FPGA : Code

| Register Num | Reg Address (HEX) | AXI Addr Bus Bits | Default value | Mode | Description                                                 |
|--------------|-------------------|-------------------|---------------|------|-------------------------------------------------------------|
| 1            | 0x04              | 6 bits            | 32'b0         | R/W  | To initialize the data of the ifmap bram with 8bit rand int |

## [VITIS Code]

```
for(i=0; i< IFMAP_MEM_DEPTH ; i++)
{
    Xil_Out32((XPAR_CT_IP_0_BASEADDR) + (REG_IFMAP_BRAM_DATA_WRITE * AXI_DATA_BYTE), write_buf_ifmap[i]);
}
```

## [Verilog Code]

```
always @(posedge S_AXI_ACLK) begin
    if (S_AXI_ARESETN == 1'b0) begin
        slv_reg1 <= 0;
    end
    else begin
        if (slv_reg_wren) begin
            case (axi_awaddr[ADDR_LSB+OPT_MEM_ADDR_BITS:ADDR_LSB]) // 하위 2비트를 제외한 6비트 Register addr
                4'h1: // axi_awaddr[ADDR_LSB+OPT_MEM_ADDR_BITS:ADDR_LSB]이 0000_01이면,
                    for (byte_index = 0; byte_index <= (C_S_AXI_DATA_WIDTH/8)-1; byte_index = byte_index+1 )
                        if (S_AXI_WSTRB[byte_index] == 1) begin
                            slv_reg1[(byte_index*8) +: 8] <= S_AXI_WDATA[(byte_index*8) +: 8]; // slv_reg1에 write_buf_ifmap 대입
                        end
            end
        end
    end
end

wire [C_S_AXI_ADDR_WIDTH-1:0] reg_1_ifmap_bram_data_push = 'h04;
wire ifmap_bram_data_write_hit = slv_reg_wren
&& (axi_awaddr[ADDR_LSB+OPT_MEM_ADDR_BITS:ADDR_LSB] == reg_1_ifmap_bram_data_push[ADDR_LSB+OPT_MEM_ADDR_BITS:ADDR_LSB]);
//
wire signed [IFMAP_DWIDTH-1:0] ifmap_bram_data_reg = slv_reg1[IFMAP_DWIDTH-1:0]; // slv_reg1은 32bit이므로 [IFMAP_DWIDTH-1:0]만큼 사용
assign ifmap_bram_d1 = ifmap_bram_data_reg;
```

# 10. FPGA : Code

| Register Num | Reg Address (HEX) | AXI Addr Bus Bits | Default value | Mode | Description                           |
|--------------|-------------------|-------------------|---------------|------|---------------------------------------|
| 2            | 0x08              | 6 bits            | 32'b0         | R/W  | To reset the weight bram addr to zero |

## [VITIS Code]

```
Xil_Out32((XPAR_CT_IP_O_BASEADDR) + (REG_WEIGHT_BRAM_ADDR_RESET_TO_ZERO * AXI_DATA_BYTE), (u32)(0x00000000));
```

## [Verilog Code]

```
always @(posedge S_AXI_ACLK) begin
    if (S_AXI_ARESETN == 1'b0) begin
        slv_reg2 <= 0;
    end
    else begin
        if (slv_reg_wren) begin
            case (axi_awaddr[ADDR_LSB+OPT_MEM_ADDR_BITS:ADDR_LSB]) // 하위 2비트를 제외한 6비트 Register addr
                4'h2: // axi_awaddr[ADDR_LSB+OPT_MEM_ADDR_BITS:ADDR_LSB]이 0000_10이면,
                    for (byte_index = 0; byte_index <= (C_S_AXI_DATA_WIDTH/8)-1; byte_index = byte_index+1)
                        if (S_AXI_WSTRB[byte_index] == 1) begin
                            slv_reg2[(byte_index*8) +: 8] <= S_AXI_WDATA[(byte_index*8) +: 8]; // slv_reg2에 (u32)(0x00000000) 대입
                        end
                    end
            end
        end
    end

    wire [C_S_AXI_ADDR_WIDTH-1:0] reg_2_weight_bram_addr_clr = 'h08;
    wire weight_bram_addr_write_hit = slv_reg_wren &&
        (axi_awaddr[ADDR_LSB+OPT_MEM_ADDR_BITS:ADDR_LSB] == reg_2_weight_bram_addr_clr[ADDR_LSB+OPT_MEM_ADDR_BITS:ADDR_LSB]);
    // slv_reg_wren는 Xil_Out32함수가 실행되면 AXI 내부에서 알아서 1로 만들어 줌.

    wire [WEIGHT_AWIDTH-1:0] weight_bram_addr_reg = slv_reg2[WEIGHT_AWIDTH-1:0]; // slv_reg2는 32bit이므로 [WEIGHT_AWIDTH-1:0]만큼 사용
```

# 10. FPGA : Code

| Register Num | Reg Address (HEX) | AXI Addr Bus Bits | Default value | Mode | Description                                                  |
|--------------|-------------------|-------------------|---------------|------|--------------------------------------------------------------|
| 3            | 0x0C              | 6 bits            | 32'b0         | R/W  | To initialize the data of the weight bram with 8bit rand int |

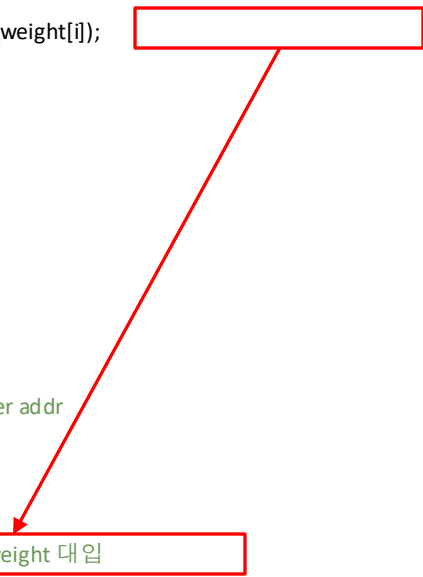
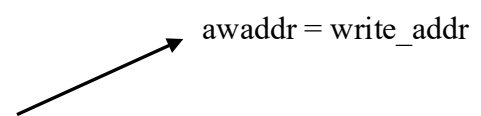
## [VITIS Code]

```
for(i=0; i< WEIGHT_MEM_DEPTH ; i++)
{
    Xil_Out32((XPAR_CT_IP_0_BASEADDR) + (REG_WEIGHT_BRAM_DATA_WRITE * AXI_DATA_BYTE), write_buf_weight[i]);
}
```

## [Verilog Code]

```
always @(posedge S_AXI_ACLK) begin
    if (S_AXI_ARESETN == 1'b0) begin
        slv_reg3 <= 0;
    end
    else begin
        if (slv_reg_wren) begin
            case (axi_awaddr[ADDR_LSB+OPT_MEM_ADDR_BITS:ADDR_LSB]) // 하위 2비트를 제외한 6비트 Register addr
                4'h3: // axi_awaddr[ADDR_LSB+OPT_MEM_ADDR_BITS:ADDR_LSB]이 0000_11이면,
                    for (byte_index = 0; byte_index <= (C_S_AXI_DATA_WIDTH/8)-1; byte_index = byte_index+1 )
                        if (S_AXI_WSTRB[byte_index] == 1) begin
                            slv_reg3[(byte_index*8) +: 8] <= S_AXI_WDATA[(byte_index*8) +: 8]; // slv_reg3에 write_buf_weight 대입
                        end
                    end
            end
        end
    end
end

wire [C_S_AXI_ADDR_WIDTH-1:0] reg_3_weight_bram_data_push = 'h0c;
wire weight_bram_data_write_hit = slv_reg_wren
&& (axi_awaddr[ADDR_LSB+OPT_MEM_ADDR_BITS:ADDR_LSB] == reg_3_weight_bram_data_push[ADDR_LSB+OPT_MEM_ADDR_BITS:ADDR_LSB]);
//
wire signed [WEIGHT_DWIDTH-1:0] weight_bram_data_reg = slv_reg3[WEIGHT_DWIDTH-1:0]; // slv_reg3은 32bit이므로 [WEIGHT_DWIDTH-1:0]
assign weight_bram_d1 = weight_bram_data_reg;
```



# 10. FPGA : Code

| Register Num | Reg Address (HEX) | AXI Addr Bus Bits | Default value | Mode | Description                                 |
|--------------|-------------------|-------------------|---------------|------|---------------------------------------------|
| 4            | 0x10              | 6 bits            | 32'bx         | R    | To check the current state of the CT module |

## [VITIS Code]

```
do
{
    read_data = Xil_In32((XPAR_CT_IP_0_BASEADDR) + (REG_STATE_CHECK * AXI_DATA_BYTE));
} while((read_data & IDLE) != IDLE); // wait until IDLE, IDLE == 1
```

## [Verilog Code]

```
always @(posedge S_AXI_ACLK) begin
    if ( S_AXI_ARESETN == 1'b0 ) begin
        slv_reg4 <= {32{1'b0}};
    end
    else begin
        slv_reg4[0] <= CT_idle; // from CT module
        slv_reg4[1] <= CT_load; // from CT module
        slv_reg4[2] <= CT_calc; // from CT module
        slv_reg4[3] <= CT_save; // from CT module
        slv_reg4[4] <= CT_done; // from CT module
        slv_reg4[5] <= are_all_channel_calcs_done;
        // no use [31:3]
    end
end
end
```

```
araddr = read_addr

always @(*) begin
    // Address decoding for reading registers
    case ( axi_araddr[ADDR_LSB+OPT_MEM_ADDR_BITS:ADDR_LSB] )
        4'h4 : reg_data_out <= slv_reg4;
        default : reg_data_out <= 0;
    endcase
end

always @( posedge S_AXI_ACLK ) begin
    if ( S_AXI_ARESETN == 1'b0 ) begin
        axi_rdata <= 0;
    end
    else begin
        if (slv_reg_rden_d) begin
            axi_rdata <= reg_data_out;
        end
    end
end
end
```

# 10. FPGA : Code

| Register Num | Reg Address (HEX) | AXI Addr Bus Bits | Default value | Mode | Description                                 |
|--------------|-------------------|-------------------|---------------|------|---------------------------------------------|
| 4            | 0x10              | 6 bits            | 32'bx         | R    | To check the current state of the CT module |

## [VITIS Code]

```
do
{
  read_data = Xil_In32((XPAR_CT_IP_0_BASEADDR) + (REG_STATE_CHECK * AXI_DATA_BYTE));
} while((read_data & IDLE) != IDLE); // wait until IDLE, IDLE == 1
```

## [Verilog Code]

```
always @(posedge S_AXI_ACLK) begin
  if (S_AXI_ARESETN == 1'b0) begin
    slv_reg4 <= {32{1'b0}};
  end
  else begin
    slv_reg4[0] <= CT_idle; // from CT module
    slv_reg4[1] <= CT_load; // from CT module
    slv_reg4[2] <= CT_calc; // from CT module
    slv_reg4[3] <= CT_save; // from CT module
    slv_reg4[4] <= CT_done; // from CT module
    slv_reg4[5] <= are_all_channel_calcs_done;
    // no use [31:3]
  end
end
```

```
always @(*) begin
  // Address decoding for reading registers
  case (axi_araddr[ADDR_LSB+OPT_MEM_ADDR_BITS:ADDR_LSB])
    4'h4 : reg_data_out <= slv_reg4;
    default : reg_data_out <= 0;
  endcase
end

always @(posedge S_AXI_ACLK) begin
  if (S_AXI_ARESETN == 1'b0) begin
    axi_rdata <= 0;
  end
  else begin
    if (slv_reg_rden_d) begin
      axi_rdata <= reg_data_out;
    end
  end
end
```

araddr = read\_addr

# 10. FPGA : Code

| Register Num | Reg Address (HEX) | AXI Addr Bus Bits | Default value | Mode | Description                          |
|--------------|-------------------|-------------------|---------------|------|--------------------------------------|
| 5            | 0x14              | 6 bits            | 32'b0         | R/W  | To reset the ofmap bram addr to zero |

## [VITIS Code]

```
Xil_Out32((XPAR_CT_IP_O_BASEADDR) + (REG_OFMAP_BRAM_ADDR_RESET_TO_ZERO * AXI_DATA_BYTE), (u32)(0x00000000));
```

## [Verilog Code]

```
always @(posedge S_AXI_ACLK) begin
    if (S_AXI_ARESETN == 1'b0) begin
        slv_reg5 <= 0;
    end
    else begin
        if (slv_reg_wren) begin
            case (axi_awaddr[ADDR_LSB+OPT_MEM_ADDR_BITS:ADDR_LSB]) // 하위 2비트를 제외한 6비트 Register addr
                4'h5: // axi_awaddr[ADDR_LSB+OPT_MEM_ADDR_BITS:ADDR_LSB]이 0001_01이면,
                    for (byte_index = 0; byte_index <= (C_S_AXI_DATA_WIDTH/8)-1; byte_index = byte_index+1)
                        if (S_AXI_WSTRB[byte_index] == 1) begin
                            slv_reg5[(byte_index*8) +: 8] <= S_AXI_WDATA[(byte_index*8) +: 8]; // slv_reg5에 (u32)(0x00000000) 대입
                        end
            end
        end
    end
end

wire [C_S_AXI_ADDR_WIDTH-1:0] reg_5_ofmap_bram_addr_clr = 'h14;
wire ofmap_bram_addr_write_hit = slv_reg_wren &&
    (axi_awaddr[ADDR_LSB+OPT_MEM_ADDR_BITS:ADDR_LSB] == reg_5_ofmap_bram_addr_clr[ADDR_LSB+OPT_MEM_ADDR_BITS:ADDR_LSB]);
// slv_reg_wren는 Xil_Out32 함수가 실행되면 AXI 내부에서 알아서 1로 만들어 줌.

wire [OFMAP_AWIDTH-1:0] ofmap_bram_addr_reg = slv_reg5[OFMAP_AWIDTH-1:0]; // slv_reg5는 32bit이므로 [OFMAP_AWIDTH-1:0]만큼 사용
```

# 10. FPGA : Code

| Register Num | Reg Address (HEX) | AXI Addr Bus Bits | Default value | Mode | Description                                        |
|--------------|-------------------|-------------------|---------------|------|----------------------------------------------------|
| 8            | 0x20              | 6 bits            | 32'b0         | R/W  | To initialize the data of the ofmap bram with zero |

## [VITIS Code]

```
for(i=0; i< IFMAP_MEM_DEPTH ; i++)
{
    Xil_Out32((XPAR_CT_IP_0_BASEADDR) + (REG_OFMAP_BRAM_DATA_WRITE * AXI_DATA_BYTE), (u32)(0x00000000));
}
```

## [Verilog Code]

```
always @(posedge S_AXI_ACLK) begin
    if (S_AXI_ARESETN == 1'b0) begin
        slv_reg8 <= 0;
    end
    else begin
        if (slv_reg_wren) begin
            case (axi_awaddr[ADDR_LSB+OPT_MEM_ADDR_BITS:ADDR_LSB]) // 하위 2비트를 제외한 6비트 Register addr
                4'h8: // axi_awaddr[ADDR_LSB+OPT_MEM_ADDR_BITS:ADDR_LSB]이 0010_00이면,
                    for (byte_index = 0; byte_index <= (C_S_AXI_DATA_WIDTH/8)-1; byte_index = byte_index+1 )
                        if (S_AXI_WSTRB[byte_index] == 1) begin
                            slv_reg8[(byte_index*8) +: 8] <= S_AXI_WDATA[(byte_index*8) +: 8]; // slv_reg8에 (u32)(0x00000000) 대입
                        end
            end
        end
    end
end

wire [C_S_AXI_ADDR_WIDTH-1:0] reg_8_ofmap_bram_data_push = 'h20;
wire ofmap_bram_data_write_hit = slv_reg_wren
&& (axi_awaddr[ADDR_LSB+OPT_MEM_ADDR_BITS:ADDR_LSB] == reg_8_ofmap_bram_data_push[ADDR_LSB+OPT_MEM_ADDR_BITS:ADDR_LSB]);
//
wire signed [OFMAP_DWIDTH-1:0] ofmap_bram_data_reg = slv_reg8[OFMAP_DWIDTH-1:0]; // slv_reg8은 32bit이므로 [OFMAP_DWIDTH-1:0]만큼 사용
assign ofmap_bram_d1 = ofmap_bram_data_reg;
```

# 10. FPGA : Code

| Register Num | Reg Address (HEX) | AXI Addr Bus Bits | Default value | Mode | Description                            |
|--------------|-------------------|-------------------|---------------|------|----------------------------------------|
| 6            | 0x18              | 6 bits            | 32'b0         | R/W  | To generate ifmap bram init run signal |

## [VITIS Code]

```
Xil_Out32((XPAR_CT_IP_0_BASEADDR) + (GEN_IFMAP_BRAM_INIT_RUN * AXI_DATA_BYTE), 1);
```

## [Verilog Code]

```
always @(posedge S_AXI_ACLK) begin
    if (S_AXI_ARESETN == 1'b0) begin
        slv_reg6 <= 0;
    end
    else begin
        if (slv_reg_wren) begin
            case (axi_awaddr[ADDR_LSB+OPT_MEM_ADDR_BITS:ADDR_LSB]) // 하위 2비트를 제외한 6비트 Register addr
                4'h6: // axi_awaddr[ADDR_LSB+OPT_MEM_ADDR_BITS:ADDR_LSB]이 0001_10이면,
                    for (byte_index = 0; byte_index <= (C_S_AXI_DATA_WIDTH/8)-1; byte_index = byte_index+1)
                        if (S_AXI_WSTRB[byte_index] == 1) begin
                            slv_reg6[(byte_index*8) +: 8] <= S_AXI_WDATA[(byte_index*8) +: 8]; // slv_reg6에 1대입
                        end
            end
        end
    end
end
```



# 10. FPGA : Code

| Register Num | Reg Address (HEX) | AXI Addr Bus Bits | Default value | Mode | Description                            |
|--------------|-------------------|-------------------|---------------|------|----------------------------------------|
| 6            | 0x18              | 6 bits            | 32'b0         | R/W  | To generate ifmap bram init run signal |

## [VITIS Code]

```
Xil_Out32((XPAR_CT_IP_0_BASEADDR) + (GEN_IFMAP_BRAM_INIT_RUN * AXI_DATA_BYTE), 1);
```

## [Verilog Code]

```
wire [C_S_AXI_ADDR_WIDTH-1:0] reg_6_ifmap_bram_init_run = 'h18;
always @(posedge clk or negedge reset_n) begin
    if(!reset_n) begin
        ifmap_addr_cnt <= 0;
        ifmap_bram_init_run <= 1'b0;
        ifmap_addr_cnt_trig <= 1'b0;
    end
    else if (ifmap_bram_addr_write_hit) begin
        ifmap_addr_cnt <= ifmap_bram_addr_reg; // ifmap_bram_addr_reg = slv_reg0[IFMAP_AWIDTH-1:0]에 의해 0으로 초기화
    end
    else if(axi_awaddr[ADDR_LSB+OPT_MEM_ADDR_BITS:ADDR_LSB] == reg_6_ifmap_bram_init_run[ADDR_LSB+OPT_MEM_ADDR_BITS:ADDR_LSB])begin
        ifmap_bram_init_run <= slv_reg6;
    end
    else if (ifmap_bram_init_run && ifmap_addr_cnt == IFMAP_MEM_DEPTH) begin
        ifmap_bram_init_run <= 1'b0; // addr이 IFMAP_MEM의 마지막에 도달하면 init_run을 0으로 reset
    end
    else if (ifmap_bram_data_write_hit && ifmap_bram_init_run) begin // init_run이 0이 되기 직전까지 ifmap_addr 증가
        ifmap_addr_cnt <= ifmap_addr_cnt + 1; // 즉, IFMAP_MEM의 마지막에 도달할 때 까지 증가
    end
end

assign ifmap_bram_addr = ifmap_addr_cnt[IFMAP_AWIDTH-1:0];
```

# 10. FPGA : Code

| Register Num | Reg Address (HEX) | AXI Addr Bus Bits | Default value | Mode | Description                             |
|--------------|-------------------|-------------------|---------------|------|-----------------------------------------|
| 7            | 0x1C              | 6 bits            | 32'b0         | R/W  | To generate weight bram init run signal |

## [VITIS Code]

```
Xil_Out32((XPAR_CT_IP_0_BASEADDR) + (GEN_WEIGHT_BRAM_INIT_RUN * AXI_DATA_BYTE), 1);
```

## [Verilog Code]

```
always @(posedge S_AXI_ACLK) begin
    if (S_AXI_ARESETN == 1'b0) begin
        slv_reg7 <= 0;
    end
    else begin
        if (slv_reg_wren) begin
            case (axi_awaddr[ADDR_LSB+OPT_MEM_ADDR_BITS:ADDR_LSB]) // 하위 2비트를 제외한 6비트 Register addr
                4'h7: // axi_awaddr[ADDR_LSB+OPT_MEM_ADDR_BITS:ADDR_LSB]이 0001_11이면,
                    for (byte_index = 0; byte_index <= (C_S_AXI_DATA_WIDTH/8)-1; byte_index = byte_index+1)
                        if (S_AXI_WSTRB[byte_index] == 1) begin
                            slv_reg7[(byte_index*8) +: 8] <= S_AXI_WDATA[(byte_index*8) +: 8]; // slv_reg7에 1대입
                        end
            end
        end
    end
end
```



# 10. FPGA : Code

| Register Num | Reg Address (HEX) | AXI Addr Bus Bits | Default value | Mode | Description                             |
|--------------|-------------------|-------------------|---------------|------|-----------------------------------------|
| 7            | 0x1C              | 6 bits            | 32'b0         | R/W  | To generate weight bram init run signal |

## [VITIS Code]

```
Xil_Out32((XPAR_CT_IP_0_BASEADDR) + (GEN_WEIGHT_BRAM_INIT_RUN * AXI_DATA_BYTE), 1);
```

## [Verilog Code]

```
wire [C_S_AXI_ADDR_WIDTH-1:0] reg_7_weight_bram_init_run = 'h1c;
always @(posedge clk or negedge reset_n) begin
    if(!reset_n) begin
        weight_addr_cnt <= 0;
        weight_bram_init_run <= 1'b0;
        weight_addr_cnt_trig <= 1'b0;
    end
    else if (ifmap_bram_addr_write_hit) begin
        weight_addr_cnt <= weight_bram_addr_reg; // weight_bram_addr_reg = slv_reg2[IFMAP_AWIDTH-1:0]에 의해 0으로 초기화
    end
    else if(axi_awaddr[ADDR_LSB+OPT_MEM_ADDR_BITS:ADDR_LSB] == reg_7_weight_bram_init_run[ADDR_LSB+OPT_MEM_ADDR_BITS:ADDR_LSB])begin
        weight_bram_init_run <= slv_reg7;
    end
    else if (weight_bram_init_run && weight_addr_cnt == WEIGHT_MEM_DEPTH) begin
        weight_bram_init_run <= 1'b0; // addr이 WEIGHT_MEM의 마지막에 도달하면 init_run을 0으로 reset
    end
    else if (ifmap_bram_data_write_hit && ifmap_bram_init_run) begin // init_run이 0이 되기 직전까지 ifmap_addr 증가
        weight_addr_cnt <= weight_addr_cnt + 1; // 즉, WEIGHT_MEM의 마지막에 도달할 때 까지 증가
    end
end

assign weight_bram_addr = weight_addr_cnt[WEIGHT_AWIDTH-1:0];
```

# 10. FPGA : Code

| Register Num | Reg Address (HEX) | AXI Addr Bus Bits | Default value | Mode | Description                            |
|--------------|-------------------|-------------------|---------------|------|----------------------------------------|
| 10           | 0x28              | 6 bits            | 32'b0         | R/W  | To generate ofmap bram init run signal |

## [VITIS Code]

```
Xil_Out32((XPAR_CT_IP_0_BASEADDR) + (GEN_OFMAP_BRAM_INIT_RUN * AXI_DATA_BYTE), 1);
```

## [Verilog Code]

```
always @(posedge S_AXI_ACLK) begin
    if (S_AXI_ARESETN == 1'b0) begin
        slv_rega <= 0;
    end
    else begin
        if (slv_reg_wren) begin
            case (axi_awaddr[ADDR_LSB+OPT_MEM_ADDR_BITS:ADDR_LSB]) // 하위 2비트를 제외한 6비트 Register addr
                4'h0 : // axi_awaddr[ADDR_LSB+OPT_MEM_ADDR_BITS:ADDR_LSB]이 0010_10이면,
                    for (byte_index = 0; byte_index <= (C_S_AXI_DATA_WIDTH/8)-1; byte_index = byte_index+1)
                        if (S_AXI_WSTRB[byte_index] == 1) begin
                            slv_rega[(byte_index*8) +: 8] <= S_AXI_WDATA[(byte_index*8) +: 8]; // slv_rega에 1대입
                        end
                    end
            end
        end
    end
end
```



# 10. FPGA : Code

| Register Num | Reg Address (HEX) | AXI Addr Bus Bits | Default value | Mode | Description                            |
|--------------|-------------------|-------------------|---------------|------|----------------------------------------|
| 10           | 0x28              | 6 bits            | 32'b0         | R/W  | To generate ofmap bram init run signal |

## [VITIS Code]

```
Xil_Out32((XPAR_CT_IP_0_BASEADDR) + (GEN_OFMAP_BRAM_INIT_RUN * AXI_DATA_BYTE), 1);
```

## [Verilog Code]

// 당시에 원하는 동작이 안 되었어서 코드를 아래와 같이 작성했는데, 코드 문제가 아니었었음. ifmap/weight init run처럼 작성해도 무관

```
reg [OFMAP_AWIDTH-1:0] ofmap_addr_cnt;
```

```
always @(posedge clk or negedge reset_n) begin
```

```
    if(!reset_n) ofmap_addr_cnt <= {OFMAP_AWIDTH{1'b0}};
```

```
    else if(ofmap_bram_addr_write_hit) ofmap_addr_cnt <= ofmap_bram_addr_reg; // slv_reg5[IFMAP_AWIDTH-1:0]에 의해 0으로 초기화
```

```
    else if(ofmap_addr_cnt == OFMAP_MEM_DEPTH) ofmap_addr_cnt <= {OFMAP_AWIDTH{1'b0}}; // OFMAP_MEM 마지막에 도달하면 addr 0으로 초기화
```

```
    else if(ofmap_bram_data_write_hit && ofmap_bram_init_run) ofmap_addr_cnt <= ofmap_addr_cnt + 1; // data write할 때 마다 addr 증가
```

```
    else if(ofmap_bram_data_read_hit && ofmap_read_run) ofmap_addr_cnt <= ofmap_addr_cnt + 1; // data read할 때 마다 addr 증가
```

```
end
```

```
reg ofmap_bram_init_run_trig;
```

```
always @(posedge clk or negedge reset_n) begin
```

```
    if(!reset_n) ofmap_bram_init_run_trig <= 1'b0;
```

```
    else if(axi_awaddr[ADDR_LSB+OPT_MEM_ADDR_BITS:ADDR_LSB] == reg_a_ofmap_bram_init_run[ADDR_LSB+OPT_MEM_ADDR_BITS:ADDR_LSB])
```

```
        ofmap_bram_init_run_trig <= 1'b1; // Xil_Out32가 호출되면 trig <= 1'b1;
```

```
    else if(!ofmap_bram_init_run) ofmap_bram_init_run_trig <= 1'b0; // init_run이 0이면 trig도 0
```

```
end
```

```
wire ofmap_read_run = CT_done; // CT module의 동작이 끝났으면 read를 시작함
```

```
wire ofmap_bram_init_run = ofmap_bram_init_run_trig && (ofmap_addr_cnt < OFMAP_MEM_DEPTH) ? 1'b1 : 1'b0;
```

```
// OFMAP_MEM의 마지막에 도달하면 trig <= 0;
```

# 10. FPGA : Code

| Register Num | Reg Address (HEX) | AXI Addr Bus Bits | Default value | Mode | Description                            |
|--------------|-------------------|-------------------|---------------|------|----------------------------------------|
| 9            | 0x24              | 6 bits            | 32'b0         | R/W  | To generate i_run signal for CT module |

## [VITIS Code]

```
Xil_Out32((XPAR_CT_IP_O_BASEADDR) + (GEN_I_RUN * AXI_DATA_BYTE), 1);
```

## [Verilog Code]

```
always @(posedge S_AXI_ACLK) begin
```

```
  if (S_AXI_ARESETN == 1'b0) begin
```

```
    slv_reg9 <= 0;
```

```
  end
```

```
  else begin
```

```
    if (slv_reg_wren) begin
```

```
      case (axi_awaddr[ADDR_LSB+OPT_MEM_ADDR_BITS:ADDR_LSB]) // 하위 2비트를 제외한 6비트 Register addr
```

```
        4'h9: // axi_awaddr[ADDR_LSB+OPT_MEM_ADDR_BITS:ADDR_LSB]이 0010_01이면,
```

```
          for (byte_index = 0; byte_index <= (C_S_AXI_DATA_WIDTH/8)-1; byte_index = byte_index+1 )
```

```
            if (S_AXI_WSTRB[byte_index] == 1) begin
```

```
              slv_reg9[(byte_index*8) +: 8] <= S_AXI_WDATA[(byte_index*8) +: 8]; // slv_reg9에 1 대입
```

```
            end
```

```
          end
```

```
        end
```

```
wire [C_S_AXI_ADDR_WIDTH-1:0] reg_9_i_run_generation = 'h24;
```

```
always @(posedge clk or negedge reset_n) begin
```

```
  if (!reset_n) i_run_cnt <= 1'b0;
```

```
  else if (axi_awaddr[ADDR_LSB+OPT_MEM_ADDR_BITS:ADDR_LSB] == reg_9_i_run_generation[ADDR_LSB+OPT_MEM_ADDR_BITS:ADDR_LSB])
```

```
    i_run_cnt <= slv_reg9; end // Xil_Out32((XPAR_CT_IP_O_BASEADDR) + (GEN_I_RUN * AXI_DATA_BYTE), 1)에 의해 i_run_cnt에 1대입
```

```
assign i_run = ((axi_awaddr[ADDR_LSB+OPT_MEM_ADDR_BITS:ADDR_LSB] == reg_9_i_run_generation[ADDR_LSB+OPT_MEM_ADDR_BITS:ADDR_LSB]) && i_run_cnt == 0) ? 1'b1 : 1'b0; // Xil_Out32
```

함수가 호출된 후 한 cycle 동안만 1이 유지됨

# 10. FPGA : Code

| Register Num | Reg Address (HEX) | AXI Addr Bus Bits | Default value | Mode | Description                        |
|--------------|-------------------|-------------------|---------------|------|------------------------------------|
| 11           | 0x2C              | 6 bits            | 32'b0         | R/W  | To read the data of the ofmap bram |

## [VITIS Code]

```
do
{
    read_data = Xil_In32((XPAR_CT_IP_0_BASEADDR) + (REG_STATE_CHECK * AXI_DATA_BYTE));
} while((read_data & DONE) != DONE); // wait until DONE, DONE 1 << 4

for(int i = 0; i < OFMAP_MEM_DEPTH; i++)
    ofmap[i] = Xil_In32((XPAR_CT_IP_0_BASEADDR) + (READ_OFMAP * AXI_DATA_BYTE));
```

## [Verilog Code]

```
always @(posedge S_AXI_ACLK) begin
    if (S_AXI_ARESETN == 1'b0) begin
        slv_reg4 <= {32{1'b0}};
    end
    else begin
        slv_reg4[0] <= CT_idle; // from CT module
        slv_reg4[1] <= CT_load; // from CT module
        slv_reg4[2] <= CT_calc; // from CT module
        slv_reg4[3] <= CT_save; // from CT module
        slv_reg4[4] <= CT_done; // from CT module
        slv_reg4[5] <= are_all_channel_calcs_done;
        // no use [31:3]
    end
end
end
```

```
always @(*) begin
    case (axi_araddr[ADDR_LSB+OPT_MEM_ADDR_BITS:ADDR_LSB])
        4'hb : reg_data_out <= ofmap_bram_q1[OFMAP_DWIDTH-1:0];
        default : reg_data_out <= 0;
    endcase
end

always @(posedge S_AXI_ACLK) begin
    if (S_AXI_ARESETN == 1'b0) begin
        axi_rdata <= 0;
    end
    else begin
        if (slv_reg_rden_d) begin
            axi_rdata <= reg_data_out;
        end
    end
end
end
```

# 10. FPGA : Why is the init\_run signal needed?

```
wire [IFMAP_AWIDTH - 1 : 0] ifmap_addr_ = ifmap_bram_init_run ? ifmap_bram_addr - 1: ifmap_addr;
wire ifmap_ce1_ = ifmap_bram_init_run ? 1'b1 : ifmap_ce0;
wire ifmap_we1_ = ifmap_bram_init_run ? 1'b1 : 1'b0;
wire signed [IFMAP_DWIDTH - 1 : 0] ifmap_d1_ = ifmap_bram_init_run ? ifmap_bram_d1 : 0;
assign ifmap_value = ifmap_q1_;
//
wire [WEIGHT_AWIDTH - 1 : 0] weight_addr_ = weight_bram_init_run ? weight_bram_addr - 1: wegt_addr;
wire weight_ce1_ = weight_bram_init_run ? 1'b1 : wegt_ce0;
wire weight_we1_ = weight_bram_init_run ? 1'b1 : 1'b0;
wire signed [WEIGHT_DWIDTH - 1 : 0] weight_d1_ = weight_bram_init_run ? weight_bram_d1 : 0;
assign weight_vec = weight_q1_;
//
assign ofmap_addr_ = ofmap_bram_init_run ? (ofmap_bram_addr) : (i_done_from_CT ? ofmap_bram_addr : ofmap_addr);
assign ofmap_ce0_ = ofmap_bram_init_run ? ofmap_bram_ce1 : (i_done_from_CT ? ofmap_bram_ce1 : ofmap_ce0);
assign ofmap_we0_ = ofmap_bram_init_run ? ofmap_bram_we1 : (i_done_from_CT ? 1'b0 : ofmap_we0);
assign ofmap_d0_ = ofmap_bram_init_run ? ofmap_bram_d1 : write_ofmap_value;
```

```
dpbram#
(
    .AWIDTH (IFMAP_AWIDTH),
    .DWIDTH (IFMAP_DWIDTH),
    .MEM_DEPTH (IFMAP_MEM_DEPTH)
) ifmap_dpbram
(
    .clk (s00_axi_aclk),
    //
    .addr0 (ifmap_addr_),
    .ce0 (ifmap_ce1_),
    .we0 (ifmap_we1_),
    //
    .d0 (ifmap_d1_),
    .q0 (ifmap_q1_)
);
```

```
dpbram#
(
    .AWIDTH (WEIGHT_AWIDTH),
    .DWIDTH (WEIGHT_DWIDTH),
    .MEM_DEPTH (WEIGHT_MEM_DEPTH)
) weight_dpbram
(
    .clk (s00_axi_aclk),
    //
    .addr0 (weight_addr_),
    .ce0 (weight_ce1_),
    .we0 (weight_we1_),
    //
    .d0 (weight_d1_),
    .q0 (weight_q1_)
);
```

```
dpbram#
(
    .AWIDTH (OFMAP_AWIDTH),
    .DWIDTH (OFMAP_DWIDTH),
    .MEM_DEPTH (OFMAP_MEM_DEPTH)
) ofmap_memory
(
    .clk (clk),
    //
    .addr0 (ofmap_addr_),
    .ce0 (ofmap_ce0_),
    .we0 (ofmap_we0_),
    //
    .d0 (ofmap_d0_),
    .q0 (read_ofmap_value)
);
```

# 10. FPGA : Why subtract 1 from addr?

```
wire [IFMAP_AWIDTH - 1 : 0] ifmap_addr_ = ifmap_bram_init_run ? ifmap_bram_addr - 1: ifmap_addr;  
wire [WEIGHT_AWIDTH - 1 : 0] weight_addr_ = weight_bram_init_run ? weight_bram_addr - 1: wegt_addr;
```

bram에 data write 할 때, 2 cycle이 소모되기 때문에 addr에서 -1

## [Verilog Code]

```
always @(posedge S_AXI_ACLK) begin  
    if (S_AXI_ARESETN == 1'b0) begin  
        slv_reg1 <= 0;  
        slv_reg3 <= 0;  
    end  
    else begin  
        if (slv_reg_wren) begin  
            case (axi_awaddr[ADDR_LSB+OPT_MEM_ADDR_BITS:ADDR_LSB])  
                4'h1:  
                    for (byte_index = 0; byte_index <= (C_S_AXI_DATA_WIDTH/8)-1; byte_index = byte_index+1 )  
                        if (S_AXI_WSTRB[byte_index] == 1) begin  
                            slv_reg0[(byte_index*8) +: 8] <= S_AXI_WDATA[(byte_index*8) +: 8];  
                        end  
                4'h3:  
                    for (byte_index = 0; byte_index <= (C_S_AXI_DATA_WIDTH/8)-1; byte_index = byte_index+1 )  
                        if (S_AXI_WSTRB[byte_index] == 1) begin  
                            slv_reg2[(byte_index*8) +: 8] <= S_AXI_WDATA[(byte_index*8) +: 8];  
                        end  
            endcase  
        end  
    end  
end
```

→ (1) slv\_reg에 write하는 데 1 cycle

# 10. FPGA : Why subtract 1 from addr?

```
wire [IFMAP_AWIDTH - 1 : 0] ifmap_addr_ = ifmap_bram_init_run ? ifmap_bram_addr - 1: ifmap_addr;  
wire [WEIGHT_AWIDTH - 1 : 0] weight_addr_ = weight_bram_init_run ? weight_bram_addr - 1: wegt_addr;
```

bram에 data write 할 때, 2 cycle이 소모되기 때문에 addr에서 -1

## [Verilog Code]

```
dpbram#  
(  
  .AWIDTH (IFMAP_AWIDTH),  
  .DWIDTH (IFMAP_DWIDTH),  
  .MEM_DEPTH (IFMAP_MEM_DEPTH)  
) ifmap_dpbram  
(  
  .clk (s00_axi_aclk),  
  //  
  .addr0 (ifmap_addr_),  
  .ce0 (ifmap_ce1_),  
  .we0 (ifmap_we1_),  
  //  
  .d0 (ifmap_d1_),  
  .q0 (ifmap_q1_)  
);
```

```
dpbram#  
(  
  .AWIDTH (WEIGHT_AWIDTH),  
  .DWIDTH (WEIGHT_DWIDTH),  
  .MEM_DEPTH (WEIGHT_MEM_DEPTH)  
) weight_dpbram  
(  
  .clk (s00_axi_aclk),  
  //  
  .addr0 (weight_addr_),  
  .ce0 (weight_ce1_),  
  .we0 (weight_we1_),  
  //  
  .d0 (weight_d1_),  
  .q0 (weight_q1_)  
);
```

→ (2) slv\_reg의 값이 bram에 write되는 데 1 cycle

# 10. FPGA : Result

**Performance Table**

| TIME(us) @ CLK FREQ (MHz)     | 1      | 50    | 100  | 135  |
|-------------------------------|--------|-------|------|------|
| IFMAP BRAM INIT TIME          | 77.96  | 2.42  | 1.65 | 1.49 |
| WEIGHT BRAM INIT TIME         | 78.09  | 2.26  | 1.53 | 1.31 |
| OFMAP BRAM CLR TIME           | 143.21 | 4.04  | 2.62 | 2.18 |
| CT MODULE OPERATION DONE TIME | 373.39 | 7.85  | 4.00 | 3.03 |
| TOTAL OPERATION TIME          | 672.65 | 16.57 | 9.8  | 8.01 |
| PS (C language)               | 18.19  |       |      |      |

**Configuration**

| FW | FH | IW | IH | S | PAD | IN_C | OUT_C | OW | OH |
|----|----|----|----|---|-----|------|-------|----|----|
| 4  | 4  | 2  | 2  | 1 | 1   | 1    | 1     | 3  | 3  |

# 10. FPGA : Result

